

Algorithms and Complexity

COMP 314

What is an algorithm?

Any well-defined computational procedure that takes some value, or set of values, as input and produces some value, or set of values, as output.

What is an algorithm?

Any well-defined computational procedure that takes some value, or set of values, as input and produces some value, or set of values, as output.

Study of algorithms includes

1. How to devise algorithms
2. How to validate algorithms
3. How to analyse algorithms
4. How to test a program

Objectives

- Develop a broad understanding of **standard algorithm design strategies**
- Be familiar with **intermediate and advanced data structures** and their common uses
- Be able to analyze the **asymptotic performance** of a variety of algorithms
- Be able to **experimentally test** the performance of a particular algorithm in a particular context
- Develop a degree of fluency in the mathematical techniques used to demonstrate **correctness**
- Develop and implement algorithms needed in disciplined **problem solving**
- Develop an understanding of **NP-complete (hard) problems** and **approximation algorithms**

Prerequisites

- Basic data structures and algorithms
- Basic programming concepts (loops, functions, objects, recursion etc.)
- Some high level programming languages like C, C++, Python or Java
- Mathematics

Syllabus (Updated)

Chapters

1. Introduction to algorithms - 8 hrs
 - a. Mathematical preliminaries, analysis of sorting algorithms
2. Some Data structures- 10 hrs
 - a. Basic data structures, efficient binary search trees, multi-way search trees, priority queues, graph data structure
3. Algorithm strategies - 12 hrs
 - a. Brute force, greedy, divide-and-conquer, backtracking, branch-and-bound
4. Dynamic Programming - 4 hrs
5. Probabilistic and Parallel algorithms - 8 hrs
6. NP-Completeness - 3 hrs

Evaluation

Internal: 50

- Internal exams and quizzes: 20
- Assignments: 5
- Lab works: 5
- Viva: 5
- Presentation: 5
- Lab exam / Mini project: 10

External: 50

Text Books

1. Cormen, Thomas H., Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. Introduction to algorithms. The MIT press, 2022.
2. Horowitz, Ellis, Sahni Sartaj, and Rajasekaran Sanguthevar. Fundamentals of Computer Algorithms, Second Edition.
3. Goodrich, Michael T., Roberto Tamassia, and Michael H. Goldwasser. Data structures and algorithms in Python. John Wiley & Sons Ltd, 2013.
4. Necaie, Rance D. Data Structures and Algorithms Using Python. Wiley Publishing, 2010.

Reference Books

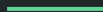
1. Aho, Alfred V., and John E. Hopcroft. The design and analysis of computer algorithms, Fourth Indian Reprint. Pearson Education India, 2001.
2. Goodman, Seymour E., and Stephen T. Hedetniemi. Introduction to the Design and Analysis of Algorithms. Fifth Printing, 1988.
3. Sahni, Sartaj. Data Structures, Algorithms and Applications in Java. Second Edition.
4. Horowitz, Ellis, Sartaj Sahni, and Susan Anderson-Freed. Fundamentals of data structures in C. Second Edition.

Chapter 1:

Introduction to Algorithms

Contents

- Introduction to algorithms
- Characteristics of an algorithm
- Asymptotic notations



Algorithms

The word ‘Algorithm’ comes from the name of a Persian author, Abu Ja’ar Mohammed ibn Musa al Khwarizmi (c. 825 A.D.), Latinized Algoritmi.

He wrote an Arabic language treatise on the Hindu–Arabic numeral system, which was translated into Latin during the 12th century under the title *Algoritmi de numero Indorum* (meaning “Algoritmi on the numbers of the Indians”).

The Latin name *algoritmi* was altered to *algorismus*, *algorithmus*, *algorithme* (in French), and finally to *algorithm* (in English).

Algorithms

- Any well-defined computational procedure that takes some value, or set of values, as input and produces some value, or set of values, as output.
- All algorithms must satisfy the following criteria:
 1. **Input:** Zero or more externally supplied quantities
 2. **Output:** Produce at least one quantity
 3. **Definiteness:** Clear and unambiguous instructions
 4. **Finiteness:** Terminate after a finite number of steps
 5. **Effectiveness:** Feasible instructions

Algorithms

- Any well-defined computational procedure that takes some value, or set of values, as input and produces some value, or set of values, as output.
- All algorithms must satisfy the following criteria:
 1. **Input:** Zero or more externally supplied quantities
 2. **Output:** Produce at least one quantity
 3. **Definiteness:** Clear and unambiguous instructions
 4. **Finiteness:** Terminate after a finite number of steps
 5. **Effectiveness:** Feasible instructions
- Algorithm vs program:

Program = The expression of an algorithm in a programming language

Program does not need to satisfy criterion #4.

Algorithms

Study of algorithms includes

1. How to devise algorithms

- a. Different techniques/design strategies, e.g. divide and conquer, branch and bound, dynamic programming etc.

2. How to validate algorithms

- a. Show that the algorithm computes the correct answer for all possible legal inputs
 - b. 2 phases:
 - i. Algorithm validation
 - ii. Program proving or program verification

Algorithms

3. How to analyse algorithms

- a. Performance analysis
- b. Determining the time and storage needs of an algorithm
- c. To make quantitative judgements about the value of one algorithm over another
- d. To predict whether the software will meet any efficiency constraints that exist

4. How to test a program

- a. Debugging: Executing programs on sample data to determine whether faulty results occur
- b. Profiling: Executing a correct program on data sets and measuring the time and space it takes to compute the results

Algorithm Specification

Algorithms can be described in many ways.

1. Using a natural language like English
2. Using flow charts
3. Using pseudocodes

Example

Devise an algorithm to find the largest element in an array.

Example

Algorithm: Find the largest element in an array

Input: Array A of size n

Output: The maximum element in A

Steps:

1. Set Result to the first element of A
2. Set i to 1
3. If $A[i]$ is greater than Result, then set Result to $A[i]$
4. Increase i by 1
5. If i is less than n, go to Step 3
6. Return Result



Example

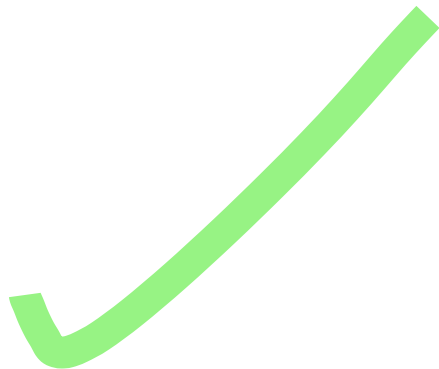
Algorithm: Find the largest element in an array

Input: Array A of size n

Output: The maximum element in A

Steps:

1. Result $\leftarrow$ A[0]
2. i $\leftarrow$ 1
3. Repeat
 - a. If A[i] > Result
 - i. Result $\leftarrow$ A[i]
 - b. Endif
4. Until i $\geq$ n
5. Return Result



Example

Algorithm: Find the largest element in an array

Input: Array A of size n

Output: The maximum element in A

Steps:

1. Result $\leftarrow$ A[0]
 2. For i $\leftarrow$ 1 to n-1 do
 - a. If A[i] > Result
 - i. Result $\leftarrow$ A[i]
 - b. Endif
 3. Endfor
 4. Return Result
- 

Pseudocode conventions

- `//` or `#` for comments
- Braces `{ }` for blocks such as a collection of simple statements
- Assignments of values to variables using `=`, `:=` or `←`
- Loops:

```
while <condition> do  
{  
    <statement 1>  
    ⋮  
    <statement 2>  
}
```

Pseudocode conventions

- Loops:

```
for variable = value1 to valuen step <step> do  
{  
    <statement 1>  
    ⋮  
    <statement 2>  
}
```

Pseudocode conventions

- Loops:

repeat

 <statement 1>

 ⋮

 <statement 2>

until <condition>

Example

```
1. Algorithm Max(A, n)
2. // A is an array of size n
3. {
4.     Result = A[0];
5.     for i = 1 to n - 1 do
6.         if A[i] > Result then Result = A[i];
7.     return Result;
8. }
```

Exercise

1. Devise an algorithm to add two matrices, A and B.

Why analyze an algorithm?

- Classify problems and algorithms by difficulty
- Predict performance, compare algorithms, tune parameters
- Better understand and improve implementations and algorithms
- Intellectual challenge

Performance analysis

Two main resources to consider when writing a program:

1. Memory
2. Time

Performance analysis focuses on obtaining **machine-independent estimates** of time and space

Space complexity

The amount of memory the algorithm needs to run to completion, $S(P)$
= fixed space requirements, c + variable space requirements, $S_p(I)$

Fixed space requirements, c , include space needed to store the code, simple variables, fixed-sized structured variables, and **constants**.

Variable space requirements, $S_p(I)$, include

- Space needed by structured variables whose size depends on the particular instance I of the program
- Additional space required when a function uses **recursion**.

Time complexity

The amount of computer time the algorithm needs to run to completion
= compile time + execution time T_p

Compile time does not depend on the instance characteristics. So, we are concerned only with the program's execution time, T_p

Instance characteristics include the number, size, and values of the inputs and outputs associated with I .

For example, if the input is an array containing n numbers, then n is an instance characteristic.

Time complexity

The more instructions the program contains, the more time it will take to complete.

Counting the number of operations the program performs, we can get a machine-independent estimate of its execution time.

Program step:

A syntactically or semantically meaningful program segment whose execution time is independent of the instance characteristics (i.e. the number, size, and values of inputs and outputs associated with a program instance)

Example

Count program steps

```
1. float sum(float a, float b) {  
2. {  
3.     return a+b;  
4. }
```

Example

Count program steps

```
1. Algorithm Sum(A, B, m, n) {  
2. // Algorithm to add two matrices  
   // A and B of size m x n.  
3. {  
4.     C = A  
5.     for i = 0 to m-1 do  
6.         for j = 0 to n-1 do  
7.             C[i,j] += B[i,j]  
8.     return C  
9. }
```

Order of growth

Algorithm analysis is about understanding the growth in resource consumption as the amount of data increases

As the amount of data gets bigger, how much more resource will my algorithm require?

The order of growth of the running time of an algorithm

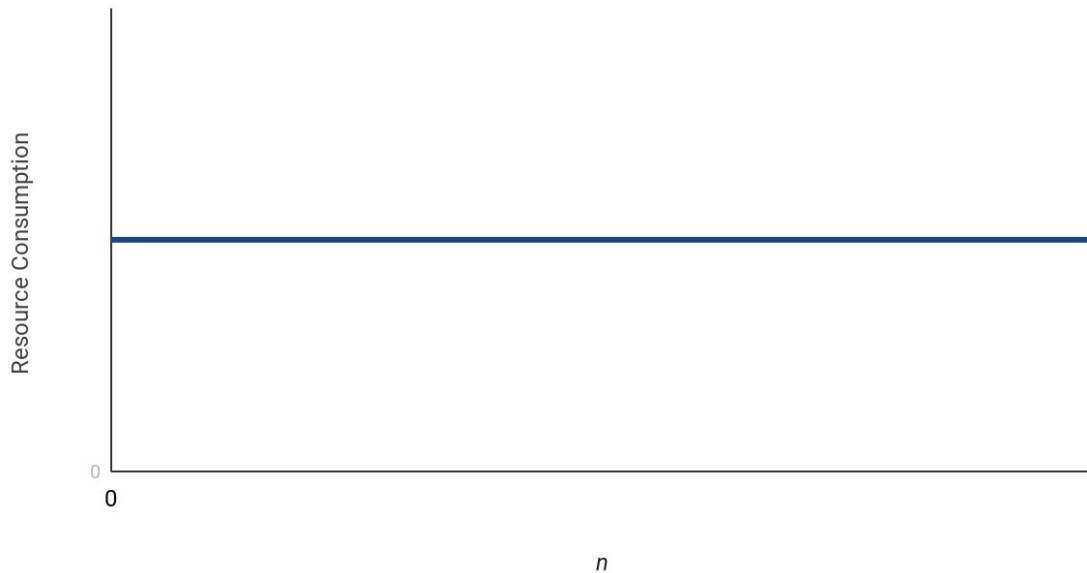
1. Gives a simple characterization of the **algorithm's efficiency**
2. Allows us to **compare the relative performance** of alternative algorithms

Constant growth rate

The resource need does not grow.

Processing 1 piece of data takes the same amount of resource as processing 1 million pieces of data

Constant growth rate

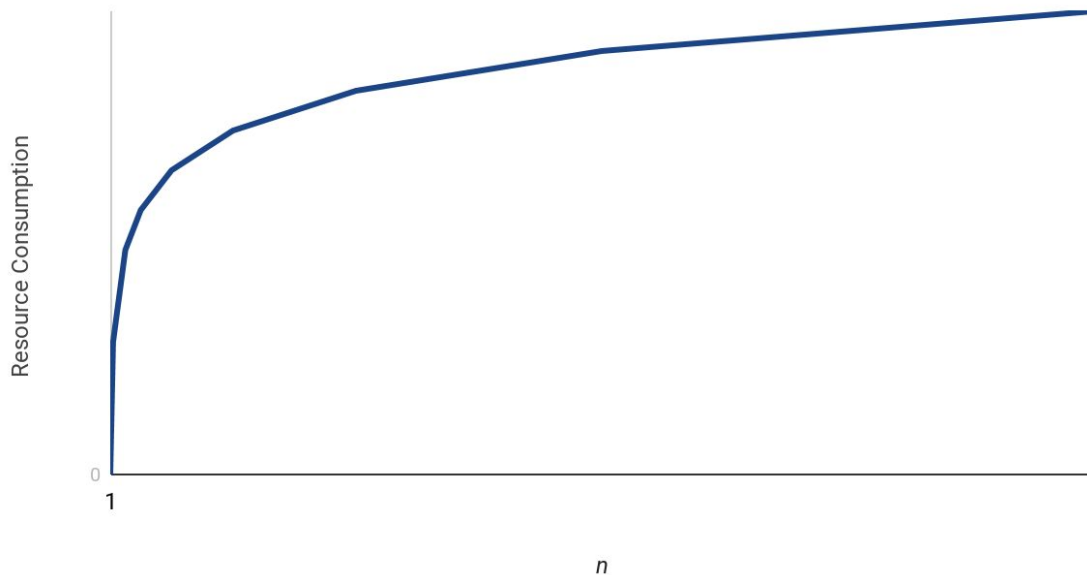


Logarithmic growth rate

The resource needs grow by one unit each time the data is doubled.

As the amount of data gets bigger, the curve gets flatter.

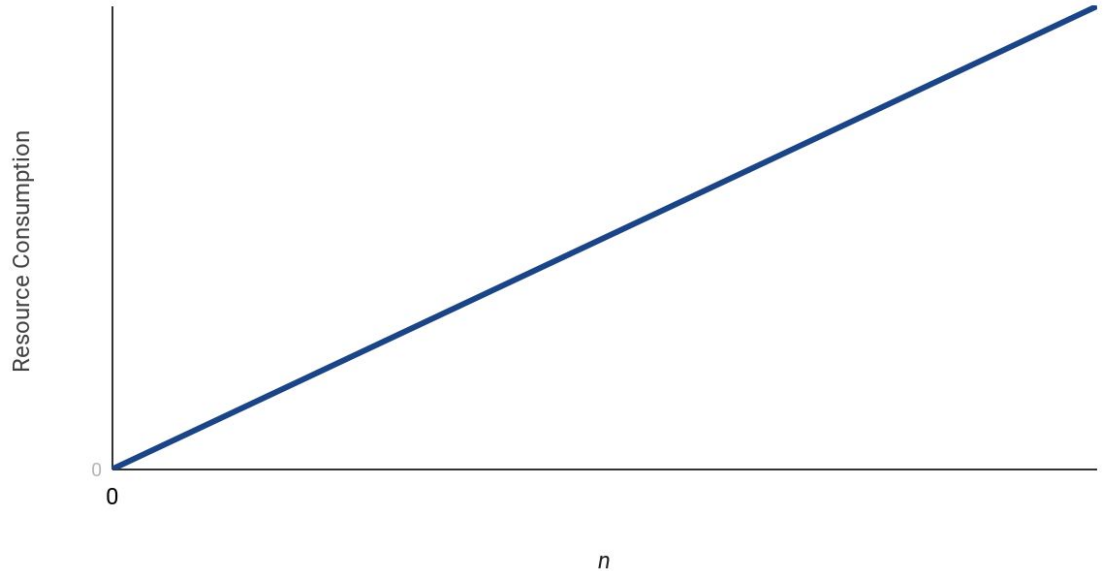
Logarithmic growth rate



Linear growth rate

The resource needs and the amount of data is directly proportional to each other.

Linear Growth Rate

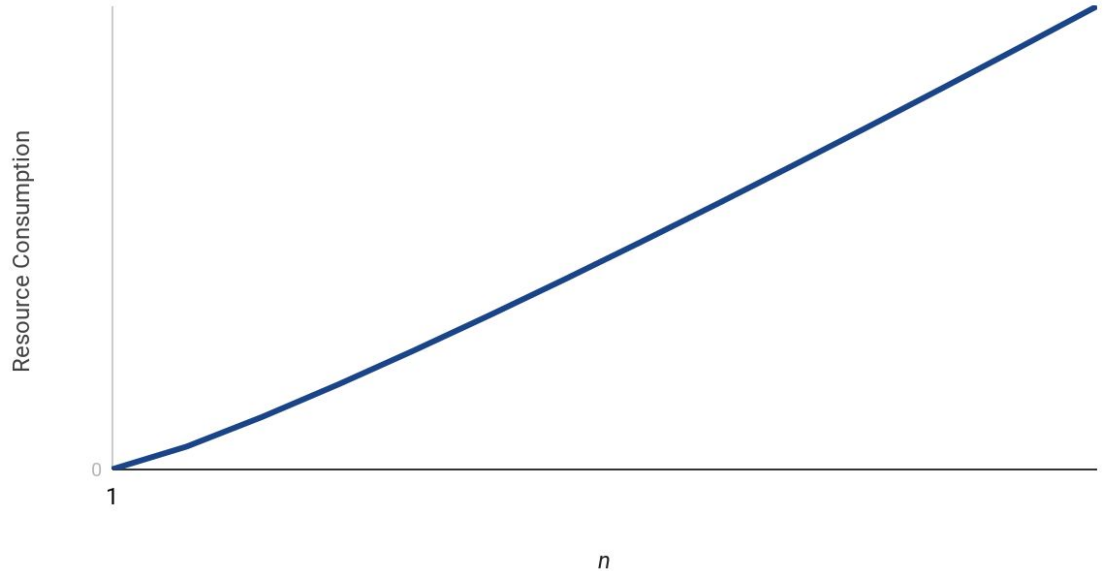


Log linear growth rate

Slightly curved line.

The curve is more pronounced for lower values than higher.

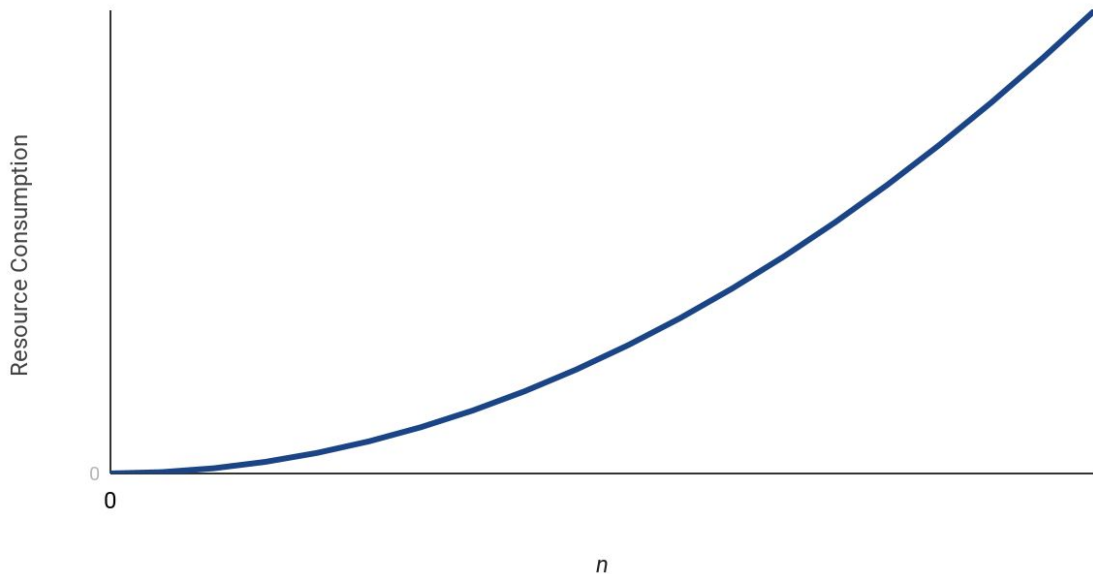
Log Linear Growth Rate



Quadratic growth rate

The resource needs is proportional to the square of the amount of data.

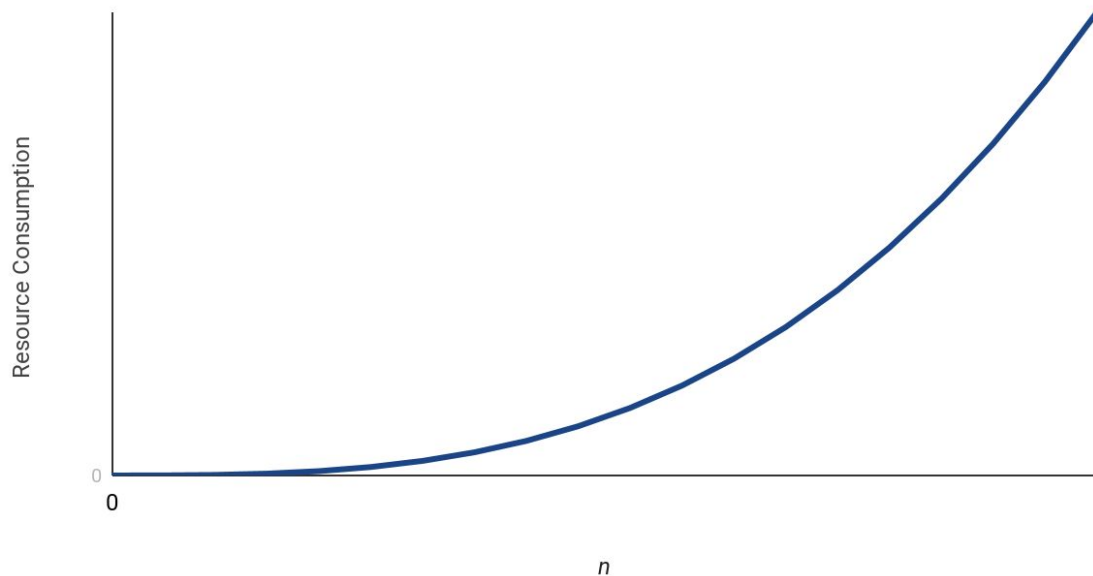
Quadratic growth rate



Cubic growth rate

Similar to quadratic but grows significantly faster.

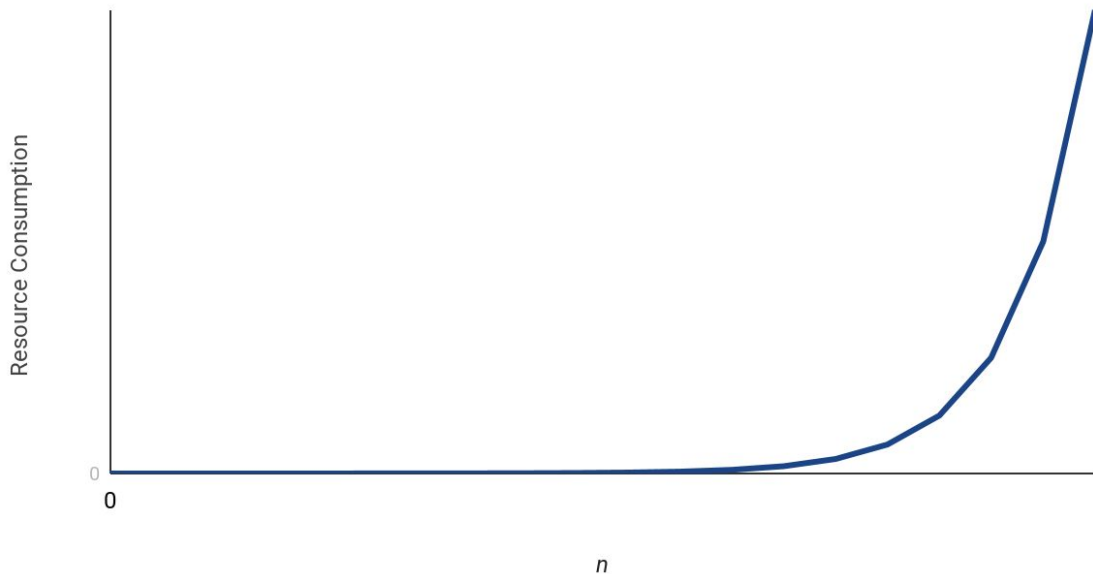
Cubic growth rate



Exponential growth rate

Each extra unit of data requires a doubling of resources. (2^n)

Exponential growth rate



Asymptotic analysis

“How does the running time scale with the size of the input?”

The idea is that we ignore low-order terms and constant factors, focusing instead on the shape of the running time curve.

Typical notation:

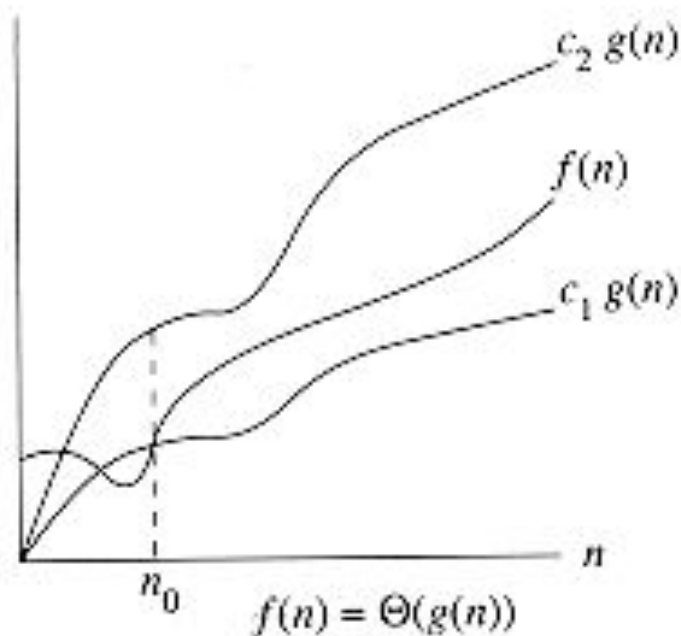
- n = The size of the input
- $T(n)$ = The running time of our algorithm on an input of size n .

Θ -notation

For a given function $g(n)$, we denote by $\Theta(g(n))$ the set of functions

$\Theta(g(n)) = \{f(n) : \text{there exist positive constants } c_1, c_2, \text{ and } n_0 \text{ such that}$
 $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for all } n \geq n_0\}$

Informally, " $T(n)$ is $\Theta(f(n))$ " basically means that the function $f(n)$ describes the exact **(asymptotically tight)** bound for $T(n)$.

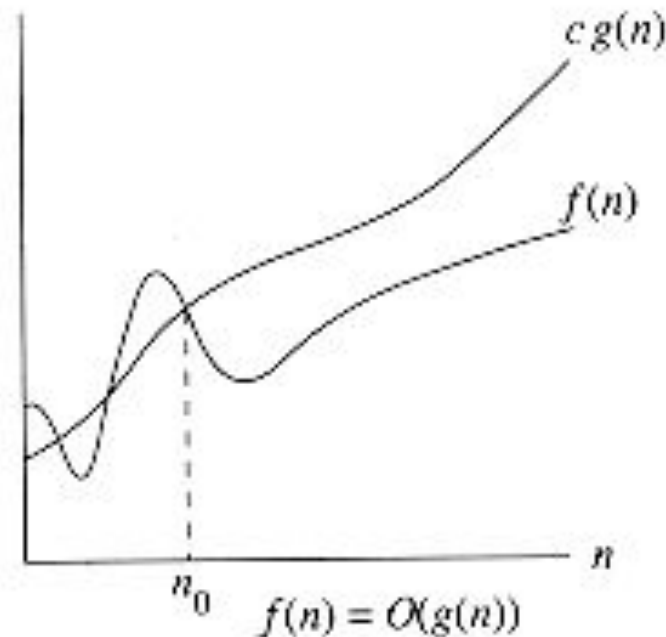


O-notation

Provides an **asymptotic upper bound** on a function.

For a given function $g(n)$, we denote by $O(g(n))$ the set of functions

$O(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0\}.$

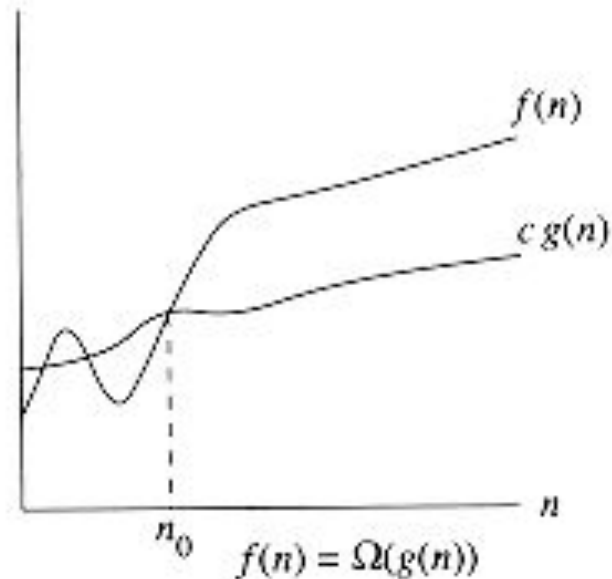


Ω -notation

Provides an **asymptotic lower bound** on a function.

For a given function $g(n)$, we denote by $\Omega(g(n))$ the set of functions

$\Omega(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq c \cdot g(n) \leq f(n) \text{ for all } n \geq n_0\}.$



o-notation and ω -notation

o-notation provides an upper bound that is **not** asymptotically tight.

$o(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq f(n) < c \cdot g(n) \text{ for all } n \geq n_0\}.$

ω -notation provides a lower bound that is **not** asymptotically tight.

$\omega(g(n)) = \{f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq c \cdot g(n) < f(n) \text{ for all } n \geq n_0\}.$

Readings

Chapter 1 from Horowitz et al.

Chapter 1:

Introduction to Algorithms

Part II

Contents

- Algorithms (Part I)
- Mathematical preliminaries
 - Order of growth (Part I)
 - Summation
 - Recurrence
- Proof of correctness (Part III)
- Analysis of sorting algorithms (Part IV)

Mathematical preliminaries: Summation and recurrence

Summation

Arithmetic series:

A sequence of numbers with a fixed difference between consecutive numbers

Examples:

1, 2, 3, 4, ...

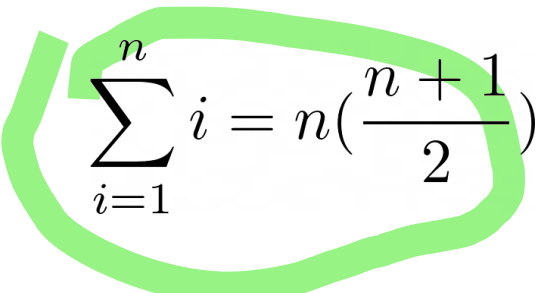
0, 2, 4, 6, ...

Summation

To sum an arithmetic series, multiply the length of the series by the average of the first and the last numbers.

$$1 + 2 + 3 + 4 + 5 = 5 \times \left(\frac{1 + 5}{2}\right) = 15$$

$$2 + 4 + 6 + 8 = 4 \times \left(\frac{2 + 8}{2}\right) = 20$$


$$\sum_{i=1}^n i = n \left(\frac{n + 1}{2}\right)$$

Summation

Linearity:

For any real number c and any finite sequences $a_1, a_2, \dots, a_n$ and $b_1, b_2, \dots, b_n$

$$\sum_{k=1}^n (c \cdot a_k + b_k) = c \sum_{k=1}^n a_k + \sum_{k=1}^n b_k$$

Summation

We can exploit the linearity property to manipulate summations incorporating asymptotic notation. For example,

$$\sum_{k=1}^n \theta(f(k)) = \theta \left(\sum_{k=1}^n f(k) \right)$$

Summation

Geometric series:

A series with a constant ratio between successive terms.

Examples

2, 4, 8, 16, ...

$\frac{1}{2}$, $\frac{1}{4}$, $\frac{1}{8}$, ...

Summation

Geometric series:

A series with a constant ratio between successive terms.

Examples

2, 4, 8, 16, ...

$\frac{1}{2}$, $\frac{1}{4}$, $\frac{1}{8}$, ...

$$\sum_{k=0}^n x^k = \frac{x^{n+1} - 1}{x - 1}$$

When the summation is infinite and $|x| < 1$

$$\sum_{j=0}^{\infty} x^j = \frac{1}{1-x}$$

Analysis of Selection Sort

Basic idea behind Selection sort:

- Given a list of data to be sorted,
 - Select the smallest item and place it in a sorted list
 - Repeat until all of the data are sorted

Selection Sort

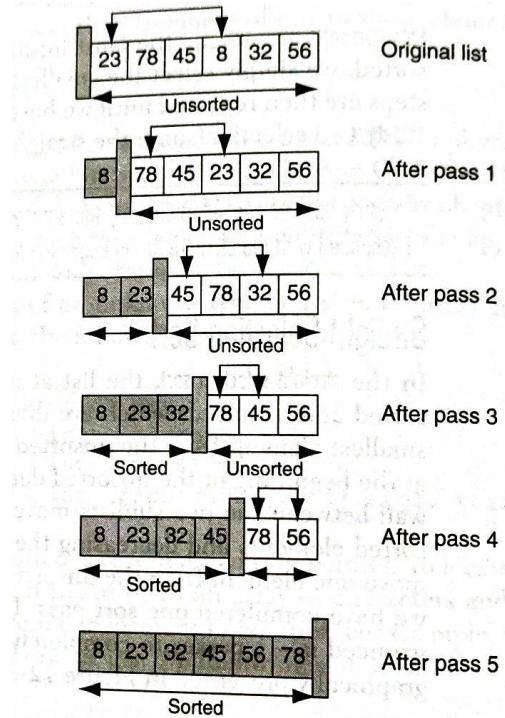
The list at any moment is divided into two sublists, sorted and unsorted, which are divided by an imaginary wall

Steps:

1. Select the smallest element from the unsorted sublist and exchange it with the element at the beginning of the unsorted data |
2. Repeat Step 1 until there is no element in the unsorted sublist

Each time we move one element from the unsorted sublist to the sorted sublist, we say that we have completed one **sort pass**. Therefore, a list of n elements need $n-1$ passes to completely rearrange the data

Selection sort



Selection sort

SelectionSort (A) // An unordered array A of size n. Assuming the index of elements starts from 1.

```
1.  for (i = 1 to n-1)
2.      m = i
3.      for (j = i + 1 to n)  // Find the minimum element in the unsorted sublist
4.          if ( A[ j ] < A[m] )
5.              m = j
6.          endif
7.      endfor
8.      temp = A[i]
9.      A[i] = A[m]
10.     A[m] = temp
11. endfor
```


Analysis of selection sort

Let's assume each execution of the i^{th} line takes time c_i , where c_i is a constant.

	cost	times
1. for (i = 1 to n-1)	c1	
2. m = i	c2	
3. for (j = i + 1 to n)	c3	
4. if (A[j] < A[m])	c4	
5. m = j	c5	
6. temp = A[i]	c6	
7. A[i] = A[m]	c7	
8. A[m] = temp	c8	

Analysis of selection sort

	cost	times
1. for (i = 1 to n-1)	c1	$\sum_{i=1}^n 1 = n$
2. m = i	c2	$\sum_{i=1}^{n-1} 1 = n - 1$
3. for (j = i + 1 to n)	c3	$\sum_{i=1}^{n-1} \sum_{j=i+1}^{n+1} 1 = \frac{(n-1)(n+2)}{2}$
4. if (A[j] < A[m])	c4	$\sum_{i=1}^{n-1} \sum_{j=i+1}^n 1 = \frac{n(n-1)}{2}$
5. m = j	c5	Min. 0 times, max. ? times
6. temp = A[i]	c6	$n - 1$
7. A[i] = A[m]	c7	$n - 1$
8. A[m] = temp	c8	$n - 1$

Analysis of selection sort

The running time of the algorithm is the sum of running times for each statement executed.

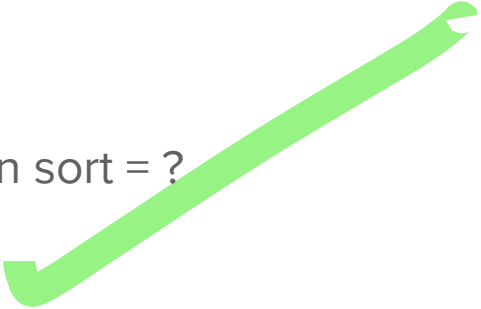
$$\begin{aligned} T(n) &= c_1.n + (c_2 + c_6 + c_7 + c_8)(n - 1) + c_3 \frac{(n - 1)(n + 2)}{2} + (c_4 + c_5) \frac{n(n - 1)}{2} \\ &= \left(\frac{c_3 + c_4 + c_5}{2} \right) n^2 + \left(c_1 + c_2 + c_3 - \left(\frac{c_4 + c_5}{2} \right) + c_6 + c_7 + c_8 \right) n - (c_2 + c_3 + c_6 + c_7 + c_8) \\ &= a.n^2 + b.n + c \text{ for constants } a, b, \text{ and } c \end{aligned}$$

Analysis of selection sort

When the list is already sorted in ascending order

$$\begin{aligned}T(n) &= c_1.n + (c_2 + c_6 + c_7 + c_8)(n - 1) + c_3 \frac{(n - 1)(n + 2)}{2} + c_4 \frac{n(n - 1)}{2} \\&= \left(\frac{c_3 + c_4}{2} \right) n^2 + \left(c_1 + c_2 + c_3 - \frac{c_4}{2} + c_6 + c_7 + c_8 \right) n - (c_2 + c_3 + c_6 + c_7 + c_8) \\&= a.n^2 + b.n + c \text{ for constants } a, b, \text{ and } c\end{aligned}$$

Time complexity of selection sort = ?



Merge sort

Merge sort uses **divide-and-conquer** strategy

- The original problem is partitioned into simpler sub-problems, each sub problem is considered independently.
- Subdivision continues until sub problems obtained are simple.

Steps:

1. **Divide**: partition the list into two roughly equal parts, S1 and S2, called the left and the right sublists
2. **Conquer**: recursively sort S1 and S2
3. **Combine**: merge the sorted sublists.

Merge sort

Sort the elements in the subarray $A[p..r]$

MERGE-SORT(A, p, r)

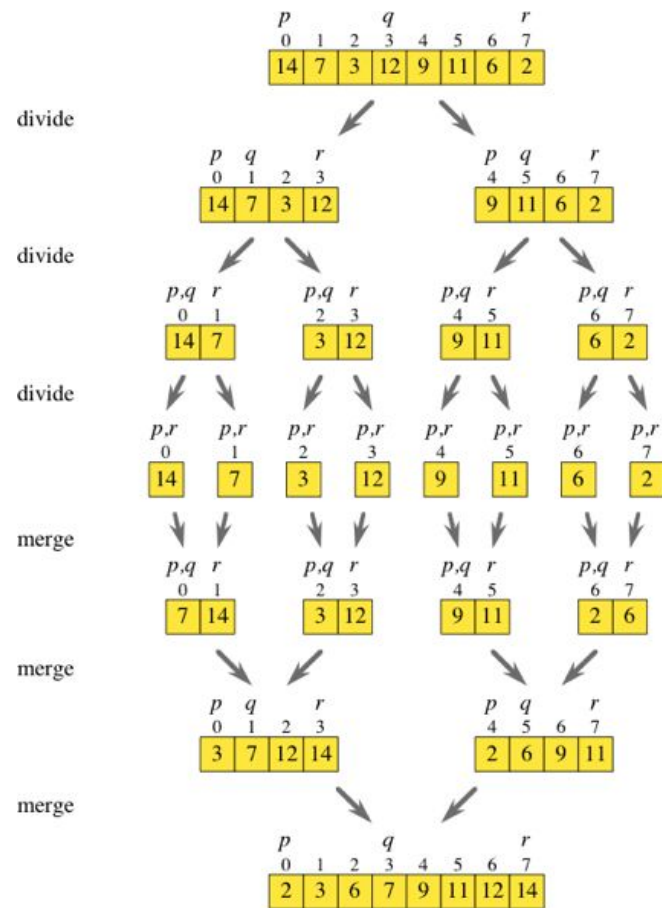
```
1  if  $p < r$   
2       $q = \lfloor (p + r) / 2 \rfloor$   
3      MERGE-SORT( $A, p, q$ )  
4      MERGE-SORT( $A, q + 1, r$ )  
5      MERGE( $A, p, q, r$ )
```

Merge sort

Sort the elements in the subarray $A[p..r]$

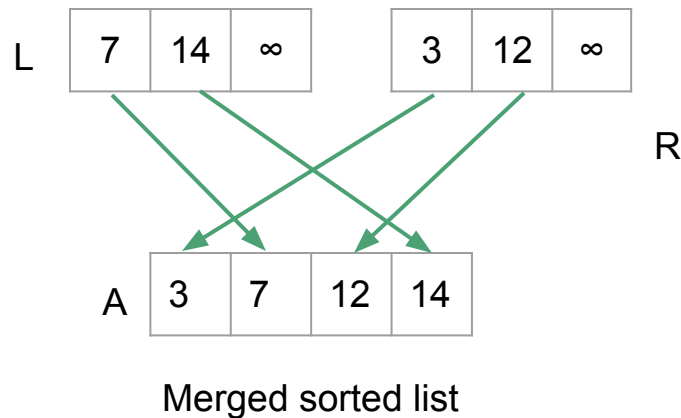
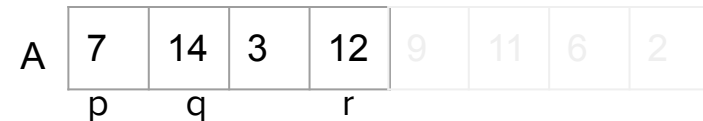
MERGE-SORT(A, p, r)

- 1 **if** $p < r$
- 2 $q = \lfloor (p + r) / 2 \rfloor$
- 3 **MERGE-SORT**(A, p, q)
- 4 **MERGE-SORT**($A, q + 1, r$)
- 5 **MERGE**(A, p, q, r)



MERGE(A, p, q, r)

```
1   $n_1 = q - p + 1$ 
2   $n_2 = r - q$ 
3  let  $L[1..n_1 + 1]$  and  $R[1..n_2 + 1]$  be new arrays
4  for  $i = 1$  to  $n_1$ 
5       $L[i] = A[p + i - 1]$ 
6  for  $j = 1$  to  $n_2$ 
7       $R[j] = A[q + j]$ 
8   $L[n_1 + 1] = \infty$ 
9   $R[n_2 + 1] = \infty$ 
10  $i = 1$ 
11  $j = 1$ 
12 for  $k = p$  to  $r$ 
13     if  $L[i] \leq R[j]$ 
14          $A[k] = L[i]$ 
15          $i = i + 1$ 
16     else  $A[k] = R[j]$ 
17          $j = j + 1$ 
```

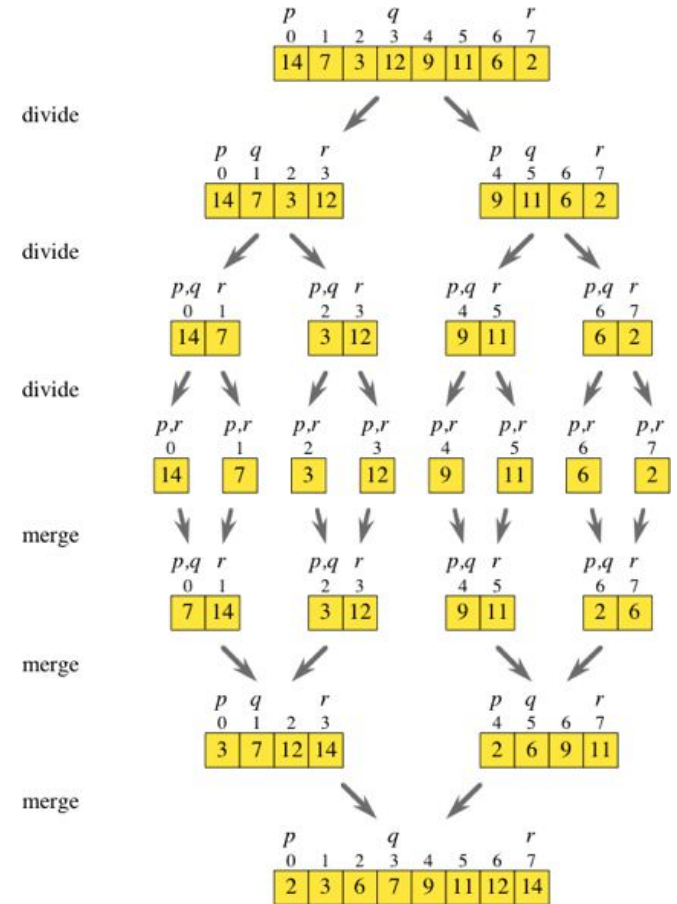


MERGE(A, p, q, r)

```

1   $n_1 = q - p + 1$ 
2   $n_2 = r - q$ 
3  let  $L[1..n_1 + 1]$  and  $R[1..n_2 + 1]$  be new arrays
4  for  $i = 1$  to  $n_1$ 
5       $L[i] = A[p + i - 1]$ 
6  for  $j = 1$  to  $n_2$ 
7       $R[j] = A[q + j]$ 
8   $L[n_1 + 1] = \infty$ 
9   $R[n_2 + 1] = \infty$ 
10  $i = 1$ 
11  $j = 1$ 
12 for  $k = p$  to  $r$ 
13     if  $L[i] \leq R[j]$ 
14          $A[k] = L[i]$ 
15          $i = i + 1$ 
16     else  $A[k] = R[j]$ 
17          $j = j + 1$ 

```



Time complexity = ?

Recurrence

When an algorithm contains a recursive call to itself, we can often describe its running time by a **recurrence equation** or **recurrence**.

A recurrence describes the overall running time on a problem of size n in terms of the running time on smaller inputs.

Recurrence

A recurrence for the running time, $T(n)$, (on a problem of size n) of a divide-and-conquer algorithm would be:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n \leq c, \\ aT(n/b) + D(n) + C(n) & \text{otherwise.} \end{cases}$$

where,

- a is the number of subproblems at the division of the problem,
- each subproblem is $1/b$ the size of the original,
- $D(n)$ is the time to divide the problem into the subproblems, and
- $C(n)$ is the time to combine the solutions to the subproblems into the solution to the original problem

Analysis of merge sort

(For simplification) Let's assume the original problem size is a power of 2.

- Each divide step yields two subsequences of size exactly $n/2$.
- Merge sort on just one element takes constant time.
- When we have $n > 1$ elements, we break down the running time as follows.
 - Divide: Takes constant time. $D(n) = \Theta(1)$
 - Conquer: We recursively solve two subproblems, each of size $n/2$. That is, $a = 2$, $b = 2$.
 - Combine: Merge procedure on an n -element subarray takes time $\Theta(n)$.



Analysis of merge sort

The recurrence for the worst-case  running time of merge sort is

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1, \\ 2T(n/2) + \Theta(n) & \text{if } n > 1. \end{cases}$$

$\Theta(n) + \Theta(1) = \Theta(n + 1) = \Theta(n)$

which can be rewritten as

$$T(n) = \begin{cases} c & \text{if } n = 1, \\ 2T(n/2) + cn & \text{if } n > 1, \end{cases}$$

where the constant c represents the time required to solve problems of size 1 as well as the time per array element of the divide and combine steps



Solving recurrences

1. Recursion-tree method
2. Substitution method
3. Master method

Recall

Minimum height of a binary tree with n nodes = ? $\text{ceil}(\log_2(n+1))$

Maximum height of a binary tree with n nodes = ? $n-1$

Minimum number of nodes in a binary tree of height h = ? $h+1$

Maximum number of nodes in a binary tree of height h = ? $2^{h+1} - 1$

Maximum number of nodes on level i of a binary tree = ? 2^i

The recurrence-tree method

This method converts the recurrence into a tree whose nodes represent the costs incurred at various levels of the recursion.

Example:

The recurrence for the worst-case running time of merge sort is

$$T(n) = \begin{cases} c & \text{if } n = 1, \\ 2T(n/2) + cn & \text{if } n > 1, \end{cases}$$

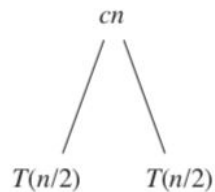
Recurrence tree

$$T(n)$$

We expand this into an equivalent tree representing the recurrence.

$$T(n) = \begin{cases} c & \text{if } n = 1, \\ 2T(n/2) + cn & \text{if } n > 1, \end{cases}$$

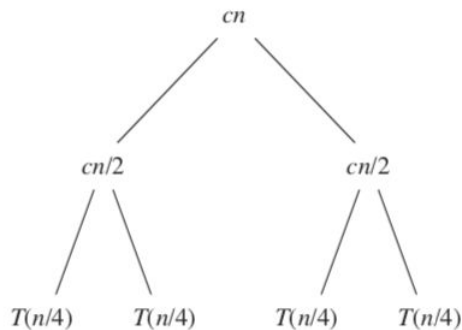
Recurrence tree



We expand this into an equivalent tree representing the recurrence.

$$T(n) = \begin{cases} c & \text{if } n = 1, \\ 2T(n/2) + cn & \text{if } n > 1, \end{cases}$$

Recurrence tree

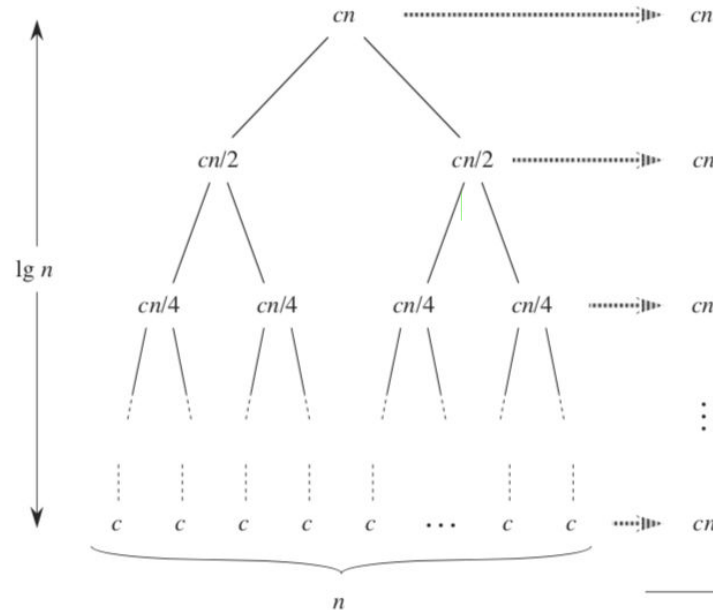


We expand this into an equivalent tree representing the recurrence.

$$T(n) = \begin{cases} c & \text{if } n = 1, \\ 2T(n/2) + cn & \text{if } n > 1, \end{cases}$$

Recurrence tree

no of level in binary tree = $\log_2 n + 1$



$T(n)$ = Total cost
 represented by the tree
 = number of levels $\times cn$
 = $(\lg n + 1) cn$
 = $cn \lg n + cn$

$T(n) = \Theta(n \lg n)$

Total: $cn \lg n + cn$

(d)

The substitution method

In this method, we guess a bound and then use mathematical induction to prove out guess correct.

- Make a guess and then prove the guess inductively (prove by induction).
- If the guess could not be proven correct, make a new guess and prove it correct inductively.

The substitution method

Let's say we have the following recurrence and we want to determine an upper bound on it

$$T(n) = \begin{cases} 7T(\frac{n}{7}) + n & \text{for } n > 1 \\ 1 & \text{for } n = 1 \end{cases}$$

Let's guess that the solution is $T(n) \leq cn$ (i.e., $T(n) = O(n)$).

We start by assuming that it holds for all positive $m < n$, in particular $m = n/7$. That is

$$T(\frac{n}{7}) \leq \frac{cn}{7}$$

The substitution method

We start by assuming that it holds for all positive $m < n$, in particular $m = n/7$. That is

$$T\left(\frac{n}{7}\right) \leq \frac{cn}{7}$$

This yields

$$T(n) \leq 7\left(\frac{cn}{7}\right) + n$$

$$\text{Or } T(n) \leq (c + 1)n$$

Unfortunately, we could not prove $T(n) \leq cn$

The substitution method

So, let's make a new guess, $T(n) \leq n \log_7(7n)$ to get the base case of $n = 1$ right.

We assume this holds true inductively for $m < n$, in particular $m = n / 7$

$$T\left(\frac{n}{7}\right) \leq \frac{n}{7} \log_7\left(7 \frac{n}{7}\right) = \frac{n}{7} \log_7 n$$

Then we get

$$\begin{aligned} T(n) &\leq 7 \left[\frac{n}{7} \log_7 n \right] + n \\ &= n \log_7 n + n \\ &= n(\log_7 n + 1) \\ &= n(\log_7 7n) \end{aligned}$$

$$T(n) \leq n(\log_7 7n)$$

That verifies our guess.



The master theorem

The master method provides a “cookbook” method for solving recurrences of the form

$$T(n) = a T\left(\frac{n}{b}\right) + f(n)$$

where $a \geq 1$ and $b > 1$ are constants, and $f(n)$ is an asymptotically positive function.

The master theorem

Theorem 4.1 (Master theorem)

Let $a \geq 1$ and $b > 1$ be constants, let $f(n)$ be a function, and let $T(n)$ be defined on the nonnegative integers by the recurrence

$$T(n) = aT(n/b) + f(n),$$

where we interpret n/b to mean either $\lfloor n/b \rfloor$ or $\lceil n/b \rceil$. Then $T(n)$ has the following asymptotic bounds:

1. If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \lg n)$.
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and if $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n , then $T(n) = \Theta(f(n))$. ■

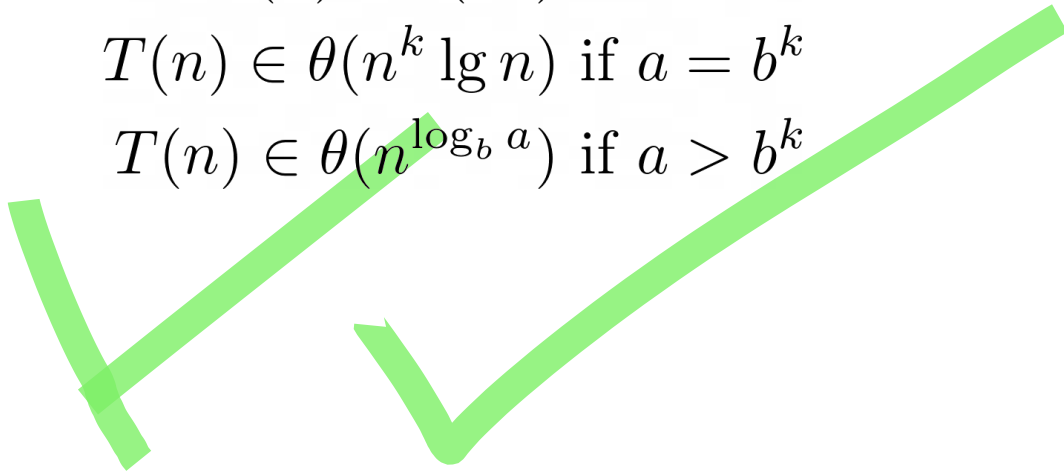
The master theorem (simpler version)

The recurrence $T(n) = a T(\frac{n}{b}) + cn^k$ where a , b , c , and k are all constants, solves to

$$T(n) \in \theta(n^k) \text{ if } a < b^k$$

$$T(n) \in \theta(n^k \lg n) \text{ if } a = b^k$$

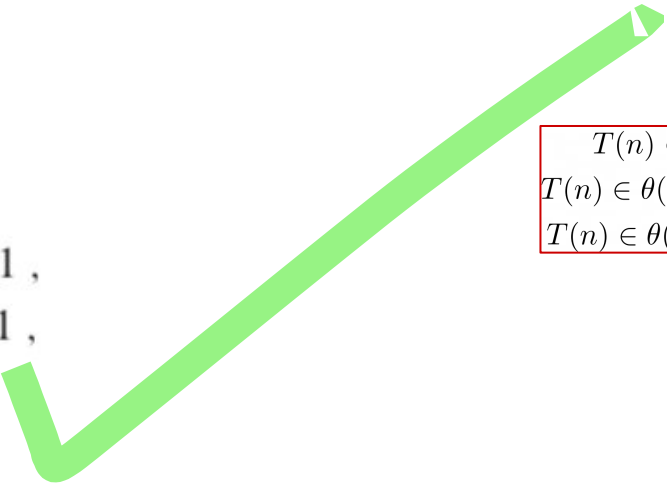
$$T(n) \in \theta(n^{\log_b a}) \text{ if } a > b^k$$



The master theorem

Example:

$$T(n) = \begin{cases} c & \text{if } n = 1, \\ 2T(n/2) + cn & \text{if } n > 1, \end{cases}$$



$T(n) \in \theta(n^k)$ if $a < b^k$
$T(n) \in \theta(n^k \lg n)$ if $a = b^k$
$T(n) \in \theta(n^{\log_b a})$ if $a > b^k$

Here, $a = 2, b = 2, k = 1$

$$b^k = 2^1 = 2 = a$$

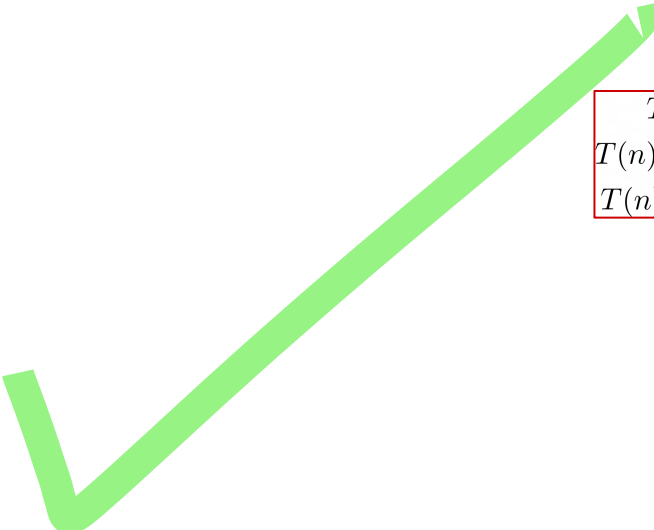
So, it follows from the case $a = b^k$ of the master theorem that

$$T(n) \in \theta(n^k \lg n) = \theta(n \lg n)$$

The master theorem

Exercise:

$$T(n) = 8T\left(\frac{n}{2}\right) + 1000n^2$$



$T(n) \in \theta(n^k)$ if $a < b^k$
$T(n) \in \theta(n^k \lg n)$ if $a = b^k$
$T(n) \in \theta(n^{\log_b a})$ if $a > b^k$

The master theorem

Example:

$$T(n) = 8T\left(\frac{n}{2}\right) + 1000n^2$$

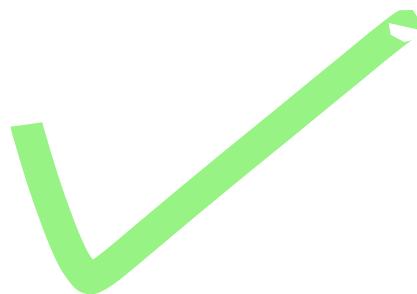
Here, $a = 8, b = 2, k = 2$

$$b^k = 2^2 = 4 < a$$

$$\begin{aligned} T(n) &\in \theta(n^k) \text{ if } a < b^k \\ T(n) &\in \theta(n^k \lg n) \text{ if } a = b^k \\ T(n) &\in \theta(n^{\log_b a}) \text{ if } a > b^k \end{aligned}$$

So, it follows from the case $a > b^k$ of the master theorem that

$$T(n) \in \theta(n^{\log_b a}) = \theta(n^{\log_2 8}) = \theta(n^3)$$

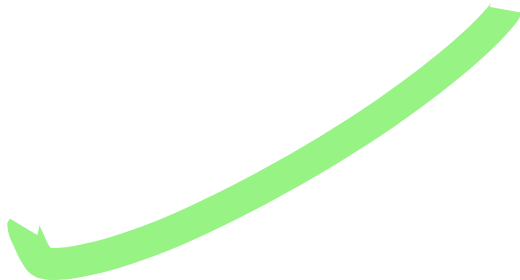


The master theorem

Exercise:

$$T(n) = 2T\left(\frac{n}{2}\right) + n^2$$

$$\begin{aligned} T(n) &\in \theta(n^k) \text{ if } a < b^k \\ T(n) &\in \theta(n^k \lg n) \text{ if } a = b^k \\ T(n) &\in \theta(n^{\log_b a}) \text{ if } a > b^k \end{aligned}$$



Chapter 1:

Introduction to Algorithms

Part III

Contents

- Algorithms (Part I)
 - Mathematical preliminaries
 - Order of growth (Part I)
 - Summation (Part II)
 - Recurrence (Part II)
 - **Proof of correctness**
 - Analysis of sorting algorithms (Part IV)
-

Proof of correctness of an algorithm

Motivation

Proof of algorithm correctness

1. Gives us more confidence in the correctness of our algorithms
2. Helps us to find subtle errors

Proving algorithm correctness

Basic techniques:

1. Handling **iteration**: using **loop invariants**
2. Handling **recursion**: using **proof by induction**

Proof of correctness using loop invariants

Loop invariant is a condition that is true immediately before and after the loop

To say an algorithm is correct, we must show 3 things about a loop invariant:

1. Initialization: It is true prior to the first iteration of the loop
2. Maintenance: Each iteration maintains the loop invariant
3. Termination: The loop terminates. When it does, the invariant gives us a useful property that helps show that the algorithm is correct.

Proof of correctness using loop invariants

Example:

```
1.  r = 0, i = 0;  
2.  while ( i < m ) {  
3.      r = r + a;  
4.      i = i + 1;  
5.  }
```

Proof of correctness using loop invariants

Example:

```
1.  r = 0, i = 0;  
2.  while ( i < m ) {  
3.      r = r + a;  
4.      i = i + 1;  
5.  }
```

Claim: This code computes $m \cdot a$

Loop invariant: $r = i \cdot a$

Proof of correctness using loop invariants

Example:

```
1.  r = 0, i = 0;
2.  while ( i < m ) {
3.      r = r + a;
4.      i = i + 1;
5.  }
```

Claim: This code computes $m.a$

Loop invariant: $r = i.a$

Initialization: Before the loop: $i = 0$,
so $r = 0.a \Rightarrow r = 0$, which is true

Maintenance:

Assume $r = i.a$ before the loop

r_{new} becomes $r_{\text{old}} + a = i.a + a = (i+1).a = i_{\text{new}}.a$

So, the loop maintains the loop invariant

Termination: Since i grows every iteration, it will be at some point be equal to m . This terminates the loop. So, when the loop is done, $i = m$. From the invariant, we derive that $r = m.a$
Q.E.D.

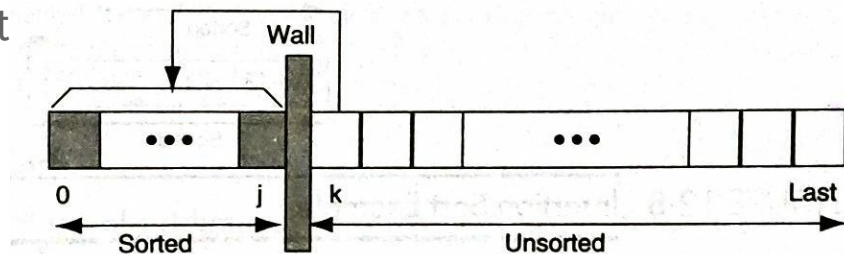
Insertion Sort

One of the most common sorting techniques used by card players. As they pick up each card, they insert it into the proper sequence in their hand.

Main idea:

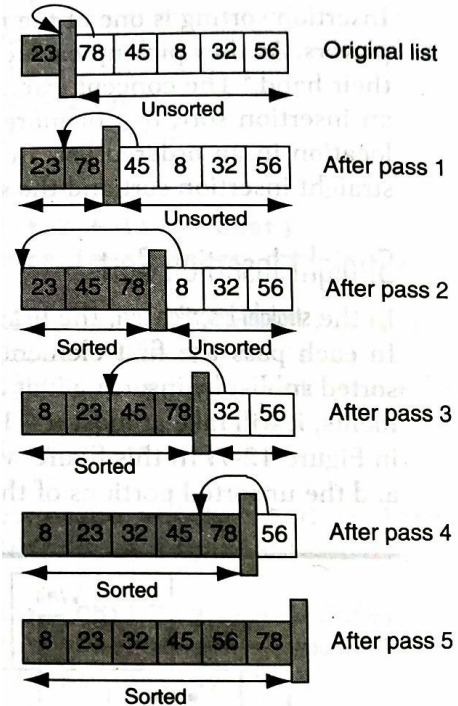
The list is divided into two parts: sorted and unsorted

In each pass of an insertion sort, the first element of the unsorted sublist is inserted into its correct location in the sorted sublist



Insertion sort

Example



Insertion sort

INSERTION-SORT(A)

```
1  for  $j = 2$  to  $A.length$ 
2       $key = A[j]$ 
3      // Insert  $A[j]$  into the sorted sequence  $A[1 \dots j - 1]$ .
4       $i = j - 1$ 
5      while  $i > 0$  and  $A[i] > key$ 
6           $A[i + 1] = A[i]$ 
7           $i = i - 1$ 
8       $A[i + 1] = key$ 
```



Assignment

1. Analyse best case and worst case time complexities of insertion sort.

Proof of correctness of insertion sort

INSERTION-SORT(A)

```
1  for  $j = 2$  to  $A.length$ 
2       $key = A[j]$ 
3      // Insert  $A[j]$  into the sorted sequence  $A[1..j-1]$ .
4       $i = j - 1$ 
5      while  $i > 0$  and  $A[i] > key$ 
6           $A[i+1] = A[i]$ 
7           $i = i - 1$ 
8       $A[i+1] = key$ 
```

Claim: Insertion sort works correctly on array of length n .

Loop invariant:

At the start of each iteration of the for loop of lines 1 - 8, the subarray $A[1..j-1]$ consists of the elements originally in $A[1..j-1]$, but in sorted order

Proof of correctness of insertion sort

Initialization: Before the iteration, $j = 2$

The subarray $A[1..j-1]$ consists of only $A[1]$, which is the original element in $A[1]$. This subarray is already sorted.

Maintenance:

The body of the for loop works by moving $A[j-1]$, $A[j-2]$, $A[j-3]$ and so on by one position to the right until it finds the proper position for $A[j]$ (lines 4 - 7), at which point it inserts the value of $A[j]$ (line 8).

The subarray $A[1..j]$ then consists of the elements originally in $A[1..j]$, but in sorted order.

Incrementing j for the next iteration then preserves the loop invariant

Proof of correctness of insertion sort

Termination:

j grows by 1 in each iteration. The loop terminates when $j = n+1$

From the loop invariant, we can derive that (by substituting $n+1$ for j) the subarray $A[1..n]$ consists of the elements originally in $A[1..n]$, but in sorted order.

Since the subarray $A[1..n]$ is the entire array, we conclude that the entire array is sorted.

QED

Proof of correctness using proof by induction

Proof by induction

Suppose that $P(n)$ is a predicate defined on $n \in \{1, 2, \dots\}$. If we can show that

1. $P(1)$ is true, and
2. $(\forall k < n [P(k)]) \Rightarrow P(n)$

Then $P(n)$ is true for all integers $n \geq 1$.

Proof by induction

In other words, to prove some assertion $P(n)$ for all positive numbers $n \geq 1$, we need to prove the induction property (in 2 parts)

1. Base case / a trivial case: $P(1)$
2. Inductive step:

Show that if the assertion holds for all smaller values, then it also holds for n , i.e. we assume that for every positive integer $n \geq 2$, $P(k)$ holds for all $k < n$ (this is called **inductive hypothesis**), and then establish that $P(n)$ holds as well.

Quick Sort

Like merge sort, quick sort also uses divide-and-conquer paradigm

Steps:

1. **Divide:** Select any element from the list. Call it the **pivot**. Then partition the list into two sublists such that all the elements in the left sublist are less than or equal to the pivot and those of the right sublist are greater than or equal to the pivot
2. **Conquer:** Recursively sort the two sublists
3. **Combine:** Since the subarrays are sorted in place, no work is needed to combine them: the entire list is now sorted.

Quick sort

QUICKSORT(A, p, r)

1 **if** $p < r$

2 $q = \text{PARTITION}(A, p, r)$

3 QUICKSORT($A, p, q - 1$)

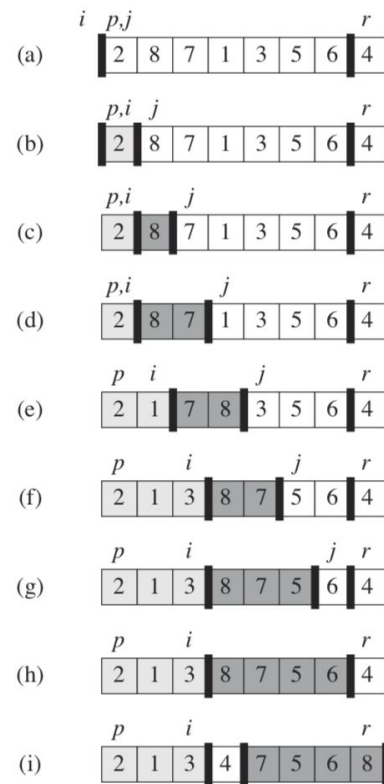
4 QUICKSORT($A, q + 1, r$)

Quick sort

PARTITION(A, p, r)

```

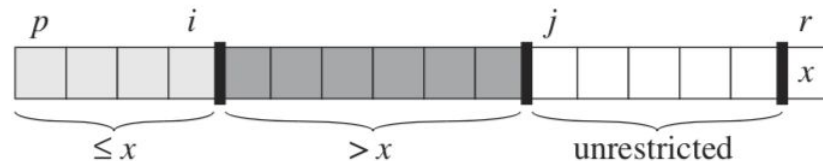
1   $x = A[r]$ 
2   $i = p - 1$ 
3  for  $j = p$  to  $r - 1$ 
4      if  $A[j] \leq x$ 
5           $i = i + 1$ 
6          exchange  $A[i]$  with  $A[j]$ 
7  exchange  $A[i + 1]$  with  $A[r]$ 
8  return  $i + 1$ 
    
```



Quick sort

PARTITION(A, p, r)

```
1   $x = A[r]$ 
2   $i = p - 1$ 
3  for  $j = p$  to  $r - 1$ 
4      if  $A[j] \leq x$ 
5           $i = i + 1$ 
6          exchange  $A[i]$  with  $A[j]$ 
7  exchange  $A[i + 1]$  with  $A[r]$ 
8  return  $i + 1$ 
```



Proof of correctness of Partition algorithm

PARTITION(A, p, r)

```
1   $x = A[r]$ 
2   $i = p - 1$ 
3  for  $j = p$  to  $r - 1$ 
4      if  $A[j] \leq x$ 
5           $i = i + 1$ 
6          exchange  $A[i]$  with  $A[j]$ 
7  exchange  $A[i + 1]$  with  $A[r]$ 
8  return  $i + 1$ 
```

Loop invariant

For any array index k ,

1. If $p \leq k \leq i$, then $A[k] \leq x$,
2. If $i + 1 \leq k \leq j - 1$, then $A[k] > x$,
3. If $k = r$, then $A[k] = x$

Proof of correctness of Partition algorithm

Initialization:

Prior to the first iteration of the loop, $i = p - 1, j = p$.
Conditions #1 and #2 are trivially satisfied because no values lie between p and i and no values lie between $i+1$ and $j-1$.

Condition #3 is satisfied because x is assigned $A[r]$ in line 1.

Loop invariant

For any array index k ,

1. If $p \leq k \leq i$, then $A[k] \leq x$,
2. If $i + 1 \leq k \leq j - 1$, then $A[k] > x$,
3. If $k = r$, then $A[k] = x$

PARTITION(A, p, r)

```
1   $x = A[r]$ 
2   $i = p - 1$ 
3  for  $j = p$  to  $r - 1$ 
4      if  $A[j] \leq x$ 
5           $i = i + 1$ 
6          exchange  $A[i]$  with  $A[j]$ 
7  exchange  $A[i + 1]$  with  $A[r]$ 
8  return  $i + 1$ 
```

Proof of correctness of Partition algorithm

Maintenance:

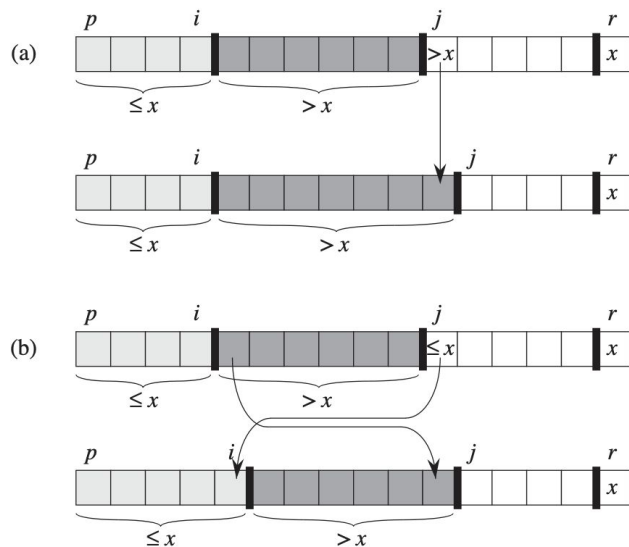


Figure 7.3 The two cases for one iteration of procedure PARTITION. (a) If $A[j] > x$, the only action is to increment j , which maintains the loop invariant. (b) If $A[j] \leq x$, index i is incremented, $A[i]$ and $A[j]$ are swapped, and then j is incremented. Again, the loop invariant is maintained.

Loop invariant

For any array index k ,

1. If $p \leq k \leq i$, then $A[k] \leq x$,
2. If $i + 1 \leq k \leq j - 1$, then $A[k] > x$,
3. If $k = r$, then $A[k] = x$

PARTITION(A, p, r)

```
1   $x = A[r]$ 
2   $i = p - 1$ 
3  for  $j = p$  to  $r - 1$ 
4      if  $A[j] \leq x$ 
5           $i = i + 1$ 
6          exchange  $A[i]$  with  $A[j]$ 
7  exchange  $A[i + 1]$  with  $A[r]$ 
8  return  $i + 1$ 
```


Proof of correctness of Partition algorithm

Termination:

The loop terminates when $j = r$.

every entry in the array is in one of the three sets described by the invariant, and we have partitioned the values in the array into three sets:

- those less than or equal to x ,
- those greater than x , and
- a singleton set containing x .

Loop invariant

For any array index k ,

1. If $p \leq k \leq i$, then $A[k] \leq x$,
2. If $i + 1 \leq k \leq j - 1$, then $A[k] > x$,
3. If $k = r$, then $A[k] = x$

PARTITION(A, p, r)

```
1   $x = A[r]$ 
2   $i = p - 1$ 
3  for  $j = p$  to  $r - 1$ 
4      if  $A[j] \leq x$ 
5           $i = i + 1$ 
6          exchange  $A[i]$  with  $A[j]$ 
7  exchange  $A[i + 1]$  with  $A[r]$ 
8  return  $i + 1$ 
```

Proof of correctness of Partition algorithm

Line 7 swaps the pivot element, $A[r]$ with the leftmost element greater than x , $A[i+1]$. If $q = i+1$, then $A[q]$ is strictly less than every element of $A[q+1..r]$. Thus the pivot is in its correct place after this line.

Line 8 returns the pivot's new index.

Loop invariant

For any array index k ,

1. If $p \leq k \leq i$, then $A[k] \leq x$,
2. If $i+1 \leq k \leq j-1$, then $A[k] > x$,
3. If $k = r$, then $A[k] = x$

PARTITION(A, p, r)

```
1   $x = A[r]$ 
2   $i = p - 1$ 
3  for  $j = p$  to  $r - 1$ 
4      if  $A[j] \leq x$ 
5           $i = i + 1$ 
6          exchange  $A[i]$  with  $A[j]$ 
7  exchange  $A[i + 1]$  with  $A[r]$ 
8  return  $i + 1$ 
```

Assignment

2. Write the recurrence equations for best case and worst case of quick sort and solve them.

Hint: In the best case, the `partition()` always picks the middle element as pivot, resulting in two subproblems, each of size no more than $n/2$. In the worst case, the list already is ordered but in reverse order (this produces one subproblem with $n - 1$ elements and one with 0 element).

Proof by induction

Example:

QUICKSORT(A, p, r)

1 **if** $p < r$

2 $q = \text{PARTITION}(A, p, r)$

3 QUICKSORT($A, p, q - 1$)

4 QUICKSORT($A, q + 1, r$)

$P(n)$ = Quicksort is always correct on inputs of length n

Base case $P(1)$ holds (input of length 1)

Inductive hypothesis: $P(k)$ holds for $k < n$

Proof by induction

Example:

QUICKSORT(A, p, r)

```
1  if  $p < r$ 
2       $q = \text{PARTITION}(A, p, r)$ 
3      QUICKSORT( $A, p, q - 1$ )
4      QUICKSORT( $A, q + 1, r$ )
```

Recall that quick sort partitions the input array into 3 parts:

1. The first subarray of size k ,
2. The pivot (which will be in its correct place),
3. The remaining subarray of size $n-k-1$

Proof by induction

Example:

Recall that quick sort partitions the input array into 3 parts:

QUICKSORT(A, p, r)

1 **if** $p < r$

2 $q = \text{PARTITION}(A, p, r)$

3 QUICKSORT($A, p, q - 1$)

4 QUICKSORT($A, q + 1, r$)

1. The first subarray of size k ,
2. The pivot (which will be in its correct place),
3. The remaining subarray of size $n-k-1$

Since $k < n$, and $n-k-1 < n$, we can apply the inductive hypothesis to imply that the two recursive calls (line 3-4) will correctly sort these two subarrays.

Thus, putting the first subarray, the pivot and the second subarray together will result in the sorted array. Thus, the quicksort algorithm is correct.

Assignment

1. Analyse best case and worst case time complexities of insertion sort.
2. Write the recurrence equations for best case and worst case of quick sort and solve them.

Hint: In the best case, the partition() always picks the middle element as pivot, resulting in two subproblems, each of size no more than $n/2$. In the worst case, the list already is ordered but in reverse order (this produces one subproblem with $n - 1$ elements and one with 0 element).

3. Prove the correctness of merge sort algorithm. You will have to prove the correctness of **merge** algorithm using loop invariants, and that of recursive **merge sort** algorithm using proof by induction.

Chapter 1:

Introduction to Algorithms

Part IV

Contents

- Algorithms (Part I)
- Mathematical preliminaries
 - Order of growth (Part I)
 - Summation (Part II)
 - Recurrence (Part II)
- Proof of correctness (Part III)
- Analysis of sorting algorithms
- Sorting in Linear time

Analysis of Sorting Algorithms

Sorting

One of the most common data-processing applications

The process of arranging a collection of items/records in a specific order

The items/records consist of one or more **fields** or **members**

One of these fields is designated as the "**sort key**" in which the records are ordered

Sort order: Data may be sorted in either ascending sequence or descending sequence

Sorting

Input

A sequence of numbers

$a_1, a_2, a_3, a_4, \dots, a_n$

Example: 2 5 6 1 12 10



Output

A permutation of the sequence of numbers

$b_1, b_2, b_3, b_4, \dots, b_n$

Example: 1 2 5 6 10 12

Sorting

Types

1. Internal sort

- All of the data are held in primary memory during the sorting process
- Examples: Insertion, selection, heap, bubble, quick, shell sort

2. External sort

- Uses primary memory for the data currently being sorted and secondary storage for any data that does not fit in primary memory
- Examples: merge sort

Sorting

Sort stability

Indicates that data with equal keys maintain their relative input order in the output

365	blue
212	green
876	white
212	yellow
119	purple
212	blue

Unsorted data

119	purple
212	green
212	yellow
212	blue
365	blue
876	white

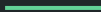
Stable sort

119	purple
212	blue
212	green
212	yellow
365	blue
876	white

Unstable sort

Sorting algorithms

- Selection Sort ✓
- Insertion Sort ✓
- Merge Sort ✓
- Quick Sort ✓
- Heap Sort



Sorting algorithms

Algorithm	Best case	Worst case	Average case
Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$
Quick sort	$O(n \log n)$	$O(n^2)$	$O(n \log n)$
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Heap sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

Quick sort vs merge sort

In merge sort, the divide step does hardly anything, and all the real work happens in the combine step whereas in quick sort, the real work happens in the divide step.

Quicksort works in place.

In practice, quicksort outperforms merge sort, and it significantly outperforms selection sort and insertion sort.

Heap sort

- Is among the fastest sorting algorithms
- Is used with very large arrays
- Uses heap data structure for sorting

Heap

- A *nearly complete binary tree* in which the root contains the largest (or smallest) element in the tree.
- Is completely filled on all levels except possibly the lowest, which is filled from the left up to a point.

Heap property

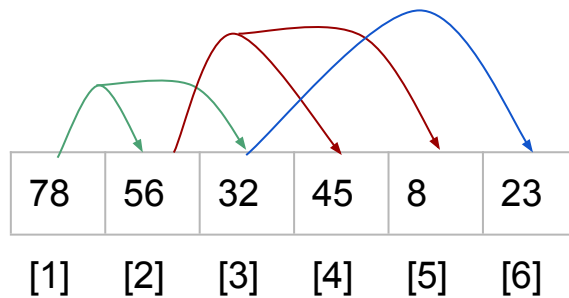
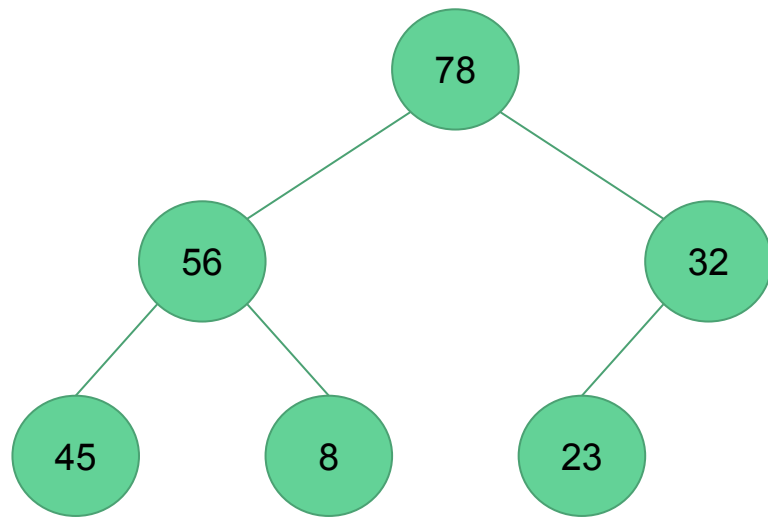
There are two kinds of binary heaps: **max-heaps** and **min-heaps**.

In both kinds, the values in the nodes satisfy a **heap property**.

In a max-heap, the **max-heap property** is that for every node other than the root, **the value of a node is at most the value of its parent**. Thus, the largest element in a max-heap is stored at the root.

Similarly, in a min-heap, the smallest element is at the root.

Heap



Heap in its array form

PARENT(i)
1 **return** $\lfloor i/2 \rfloor$

LEFT(i)
1 **return** $2i$

RIGHT(i)
1 **return** $2i + 1$

Heap sort

To implement the heap sort using a max-heap, we need two basic algorithms:

1. **Max-heapify**

Maintains the max-heap property by pushing the root down the tree until it is in its correct position in the heap.

2. **Build-max-heap**

Produces a max-heap from an unordered input array.

Heap sort

MAX-HEAPIFY(A, i)

```
1   $l = \text{LEFT}(i)$ 
2   $r = \text{RIGHT}(i)$ 
3  if  $l \leq A.\text{heap-size}$  and  $A[l] > A[i]$ 
4       $\text{largest} = l$ 
5  else  $\text{largest} = i$ 
6  if  $r \leq A.\text{heap-size}$  and  $A[r] > A[\text{largest}]$ 
7       $\text{largest} = r$ 
8  if  $\text{largest} \neq i$ 
9      exchange  $A[i]$  with  $A[\text{largest}]$ 
10     MAX-HEAPIFY( $A, \text{largest}$ )
```

BUILD-MAX-HEAP(A)

```
1   $A.\text{heap-size} = A.\text{length}$ 
2  for  $i = \lfloor A.\text{length}/2 \rfloor$  downto 1
3      MAX-HEAPIFY( $A, i$ )
```

HEAPSORT(A)

```
1  BUILD-MAX-HEAP( $A$ )
2  for  $i = A.\text{length}$  downto 2
3      exchange  $A[1]$  with  $A[i]$ 
4       $A.\text{heap-size} = A.\text{heap-size} - 1$ 
5      MAX-HEAPIFY( $A, 1$ )
```

Heap sort

Steps:

1. Convert the array into a max heap
2. Find the largest element of the list (i.e., the root of the heap) and then place it at the end of the list. Decrement the heap size by 1 and readjust the heap
3. Repeat Step 2 until the unsorted list is empty

HEAPSORT(A)

```
1  BUILD-MAX-HEAP( $A$ )
2  for  $i = A.length$  downto 2
3      exchange  $A[1]$  with  $A[i]$ 
4       $A.heap-size = A.heap-size - 1$ 
5      MAX-HEAPIFY( $A, 1$ )
```

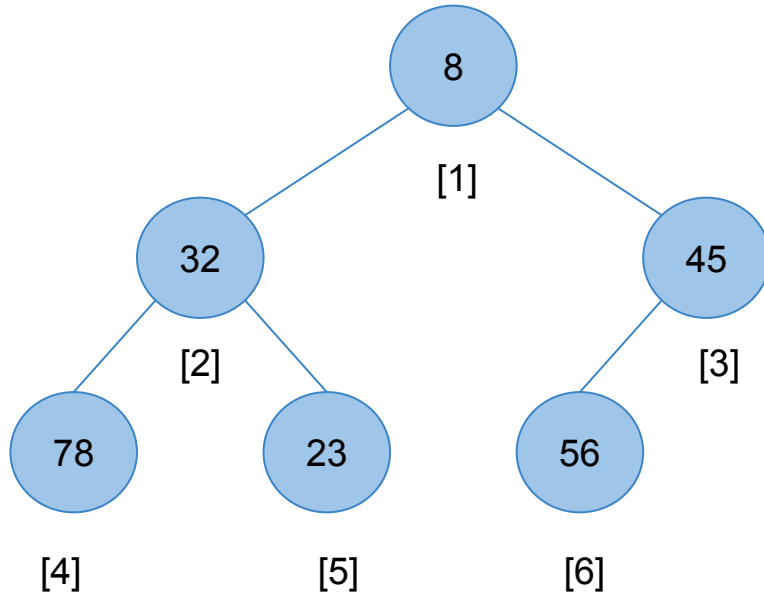

Heap sort

Example:

Sort the following data using heap sort:

8	32	45	78	23	56
---	----	----	----	----	----

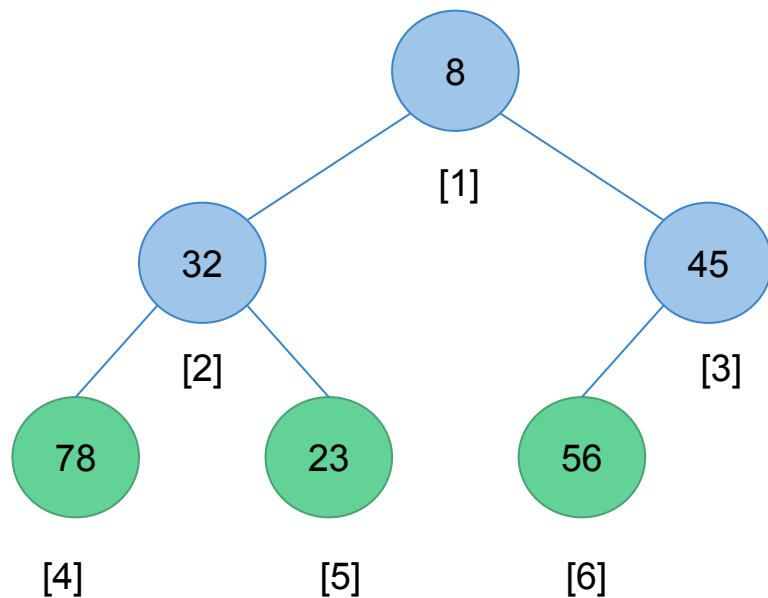
Heap sort



Convert the array into a max heap

8	32	45	78	23	56
---	----	----	----	----	----

Heap sort

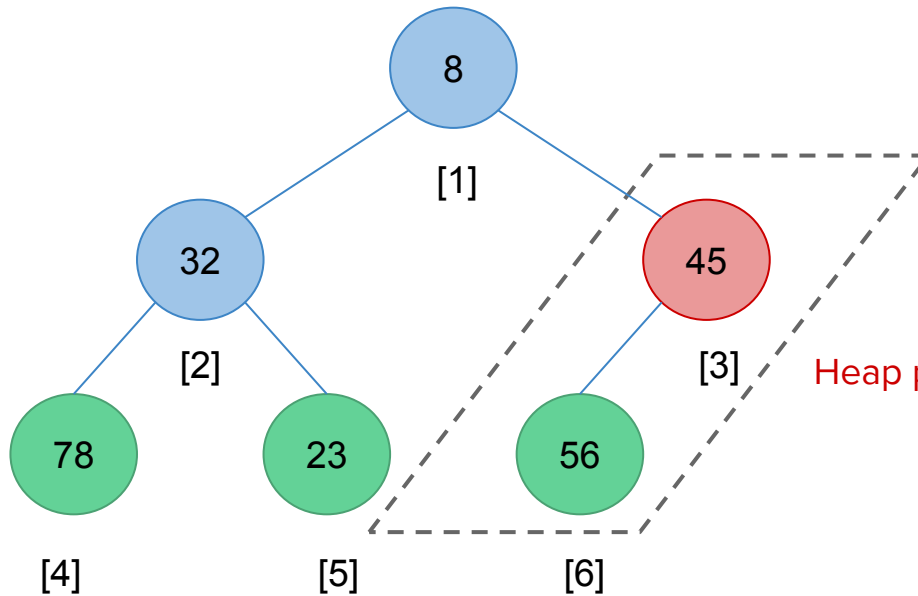


Convert the array into a max heap

8	32	45	78	23	56
---	----	----	----	----	----

Starting from the index, i , of the node just above the leaf level, check if the tree starting at i is a max-heap. If not, fix it (by reheap down)

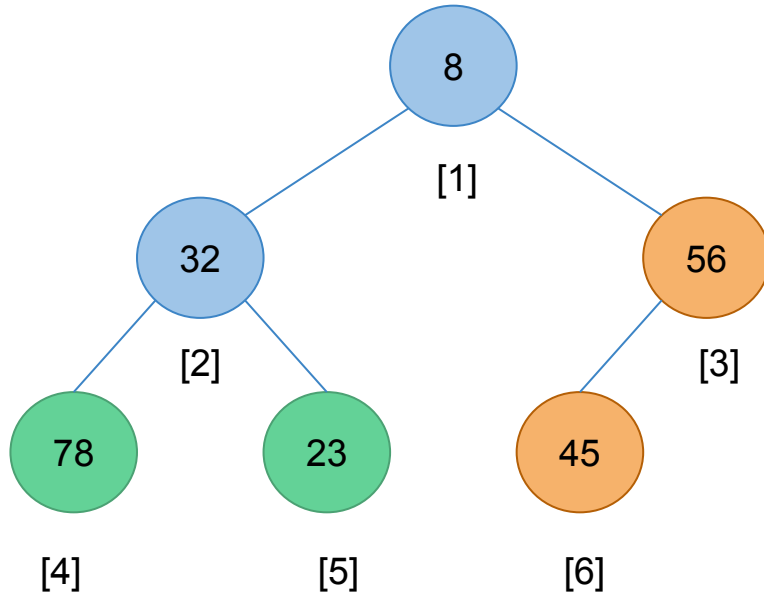
Heap sort



Convert the array into a max heap

8	32	45	78	23	56
---	----	----	----	----	----

Heap sort



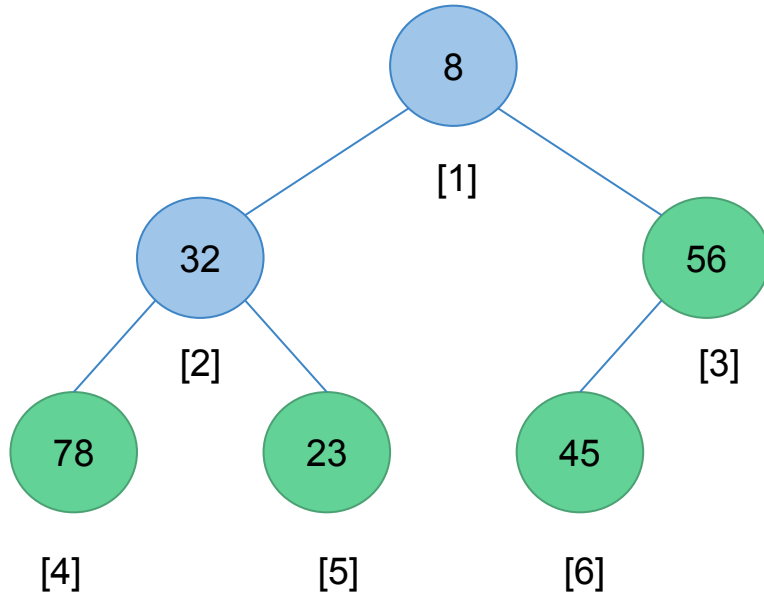
Convert the array into a max heap

8	32	56	78	23	45
---	----	----	----	----	----



Heap property is not satisfied. Therefore, swap them

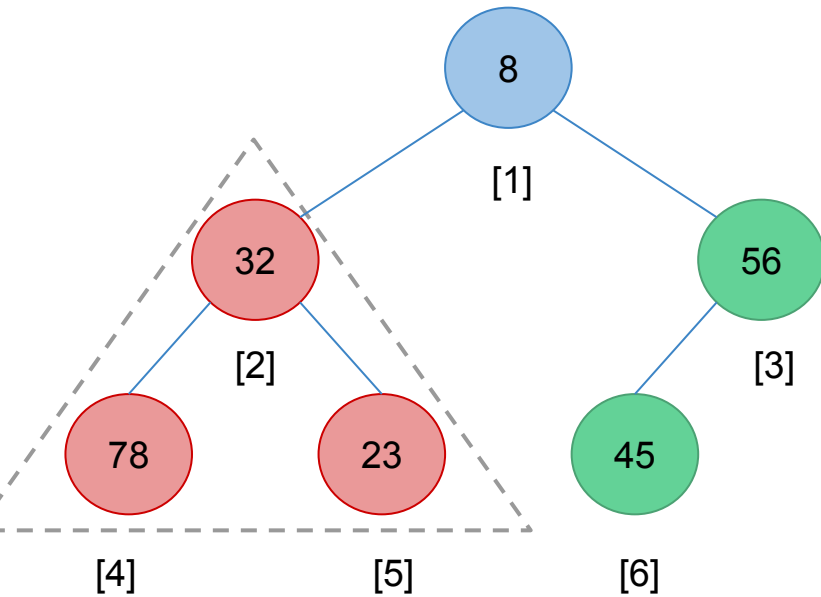
Heap sort



Convert the array into a max heap

8	32	56	78	23	45
---	----	----	----	----	----

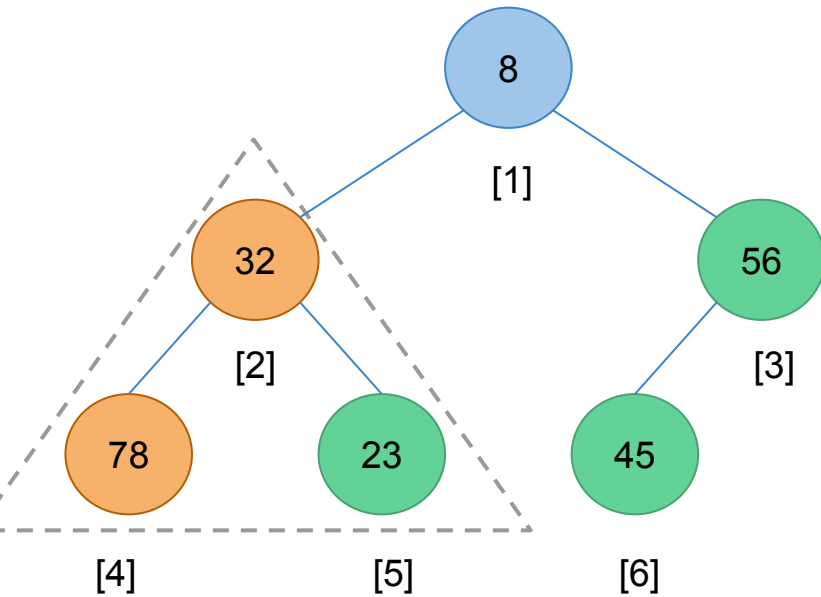
Heap sort



Convert the array into a max heap

8	32	56	78	23	45
---	----	----	----	----	----

Heap sort

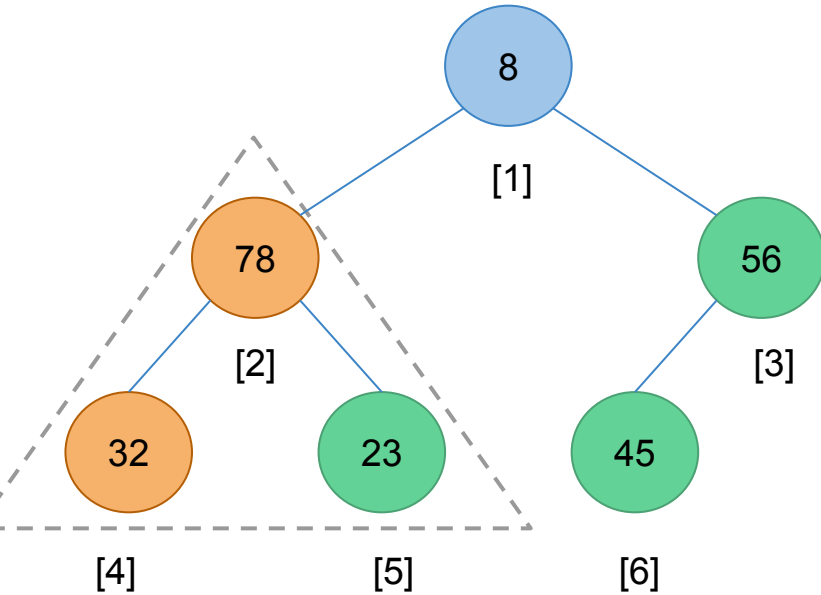


Convert the array into a max heap

8	32	56	78	23	45
---	----	----	----	----	----

Swap the root with the largest child

Heap sort

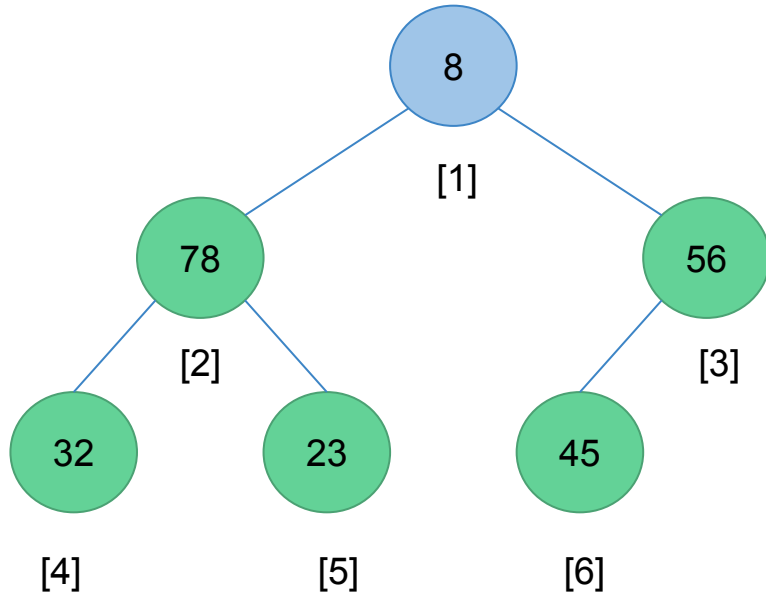


Convert the array into a max heap

8	78	56	32	23	45
---	----	----	----	----	----

Swap the root with the largest child

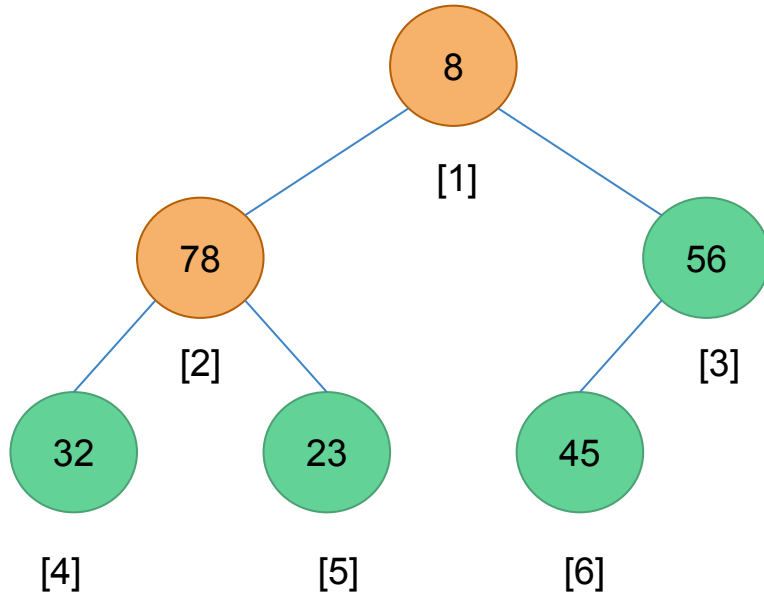
Heap sort



Convert the array into a max heap

8	78	56	32	23	45
---	----	----	----	----	----

Heap sort

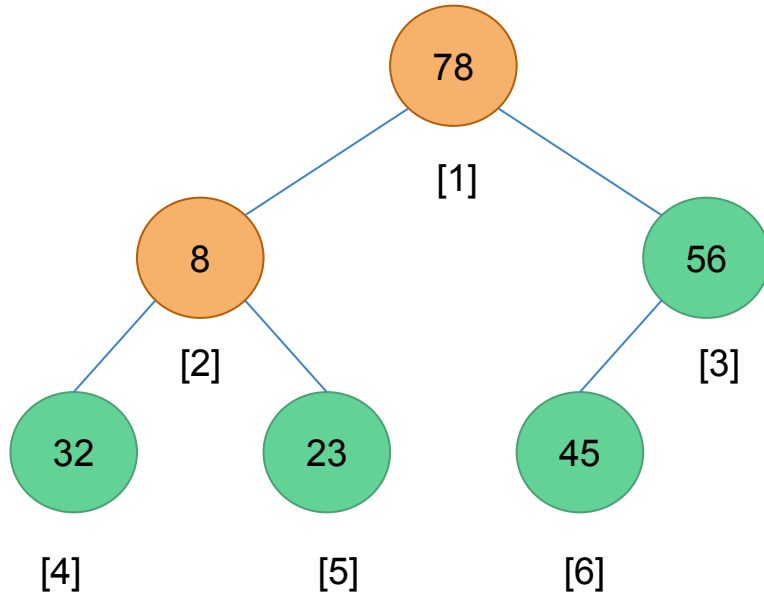


Convert the array into a max heap

8	78	56	32	23	45
---	----	----	----	----	----

Swap the root with the largest child

Heap sort

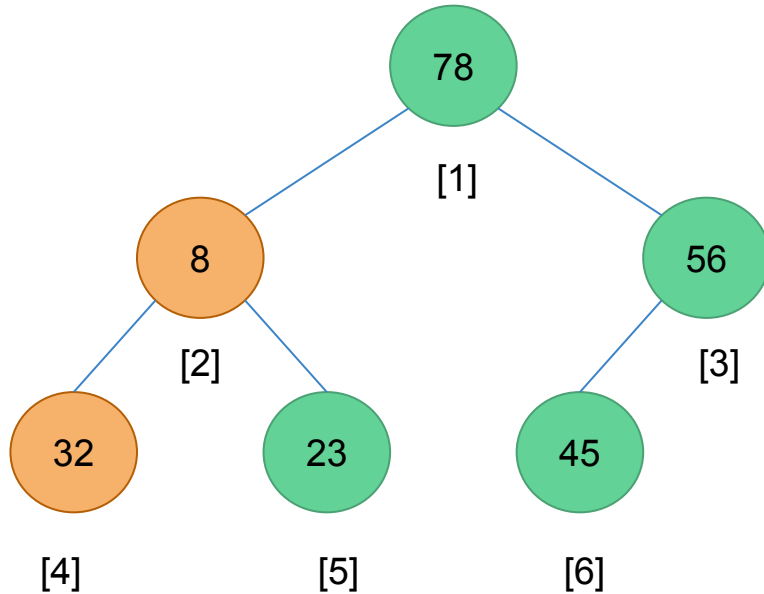


Convert the array into a max heap

78	8	56	32	23	45
----	---	----	----	----	----

Swap the root with the largest child

Heap sort

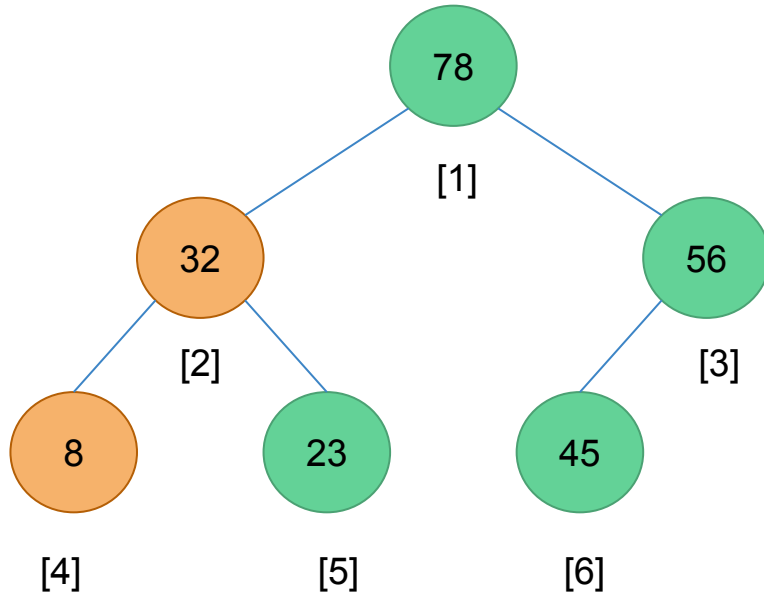


Convert the array into a max heap

78	8	56	32	23	45
----	---	----	----	----	----

Swap the root with the largest child

Heap sort

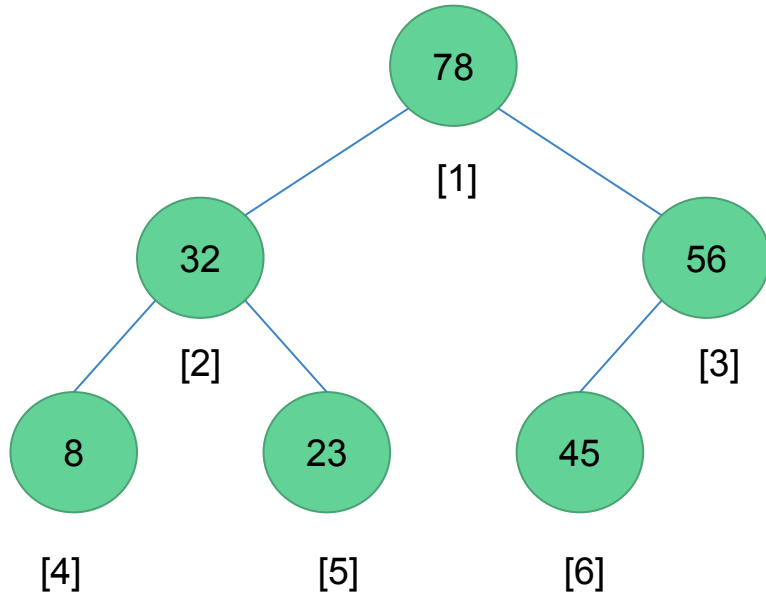


Convert the array into a max heap

78	32	56	8	23	45
----	----	----	---	----	----

Swap the root with the largest child

Heap sort

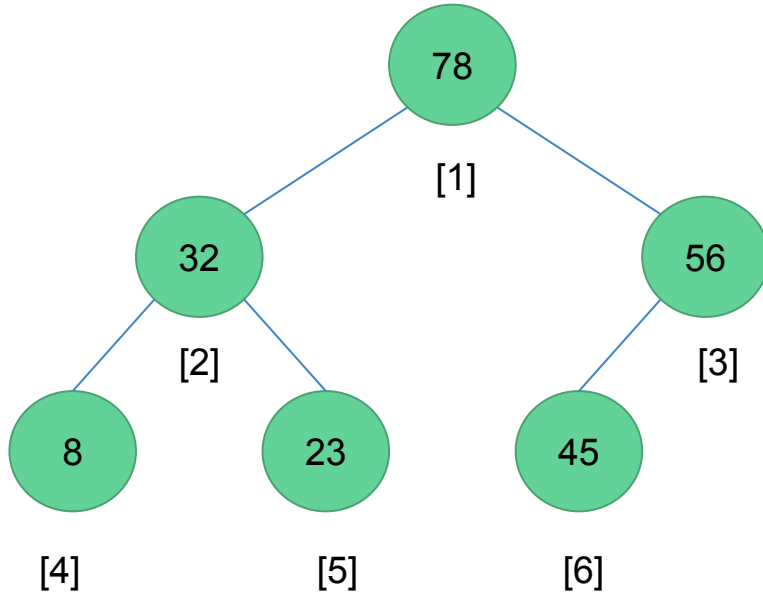


Convert the array into a max heap

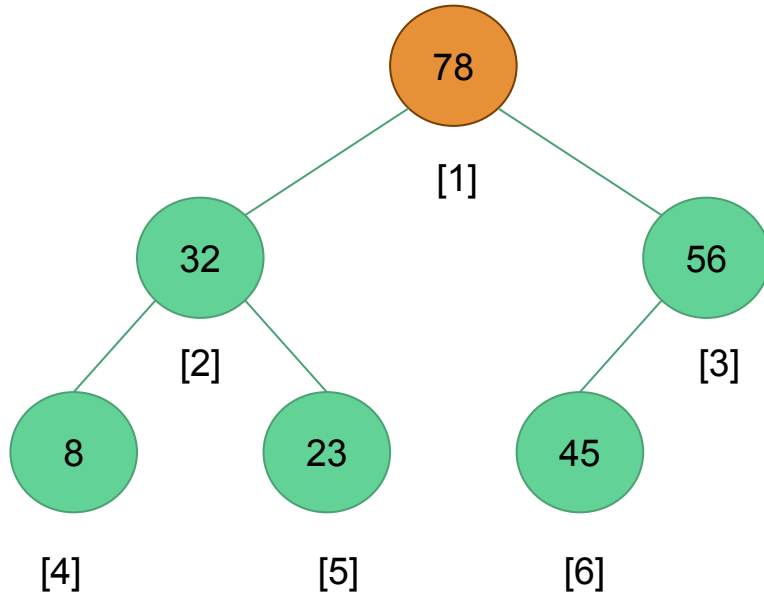
78	32	56	8	23	45
----	----	----	---	----	----

Heap sort

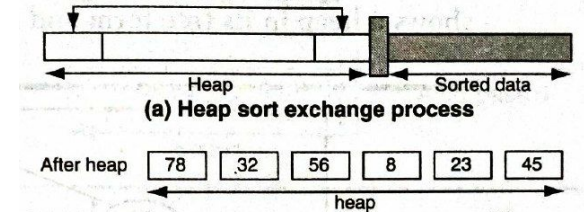
Swap the root of the heap with the element at the end of the list. Decrement the heap size by 1 and readjust the heap.



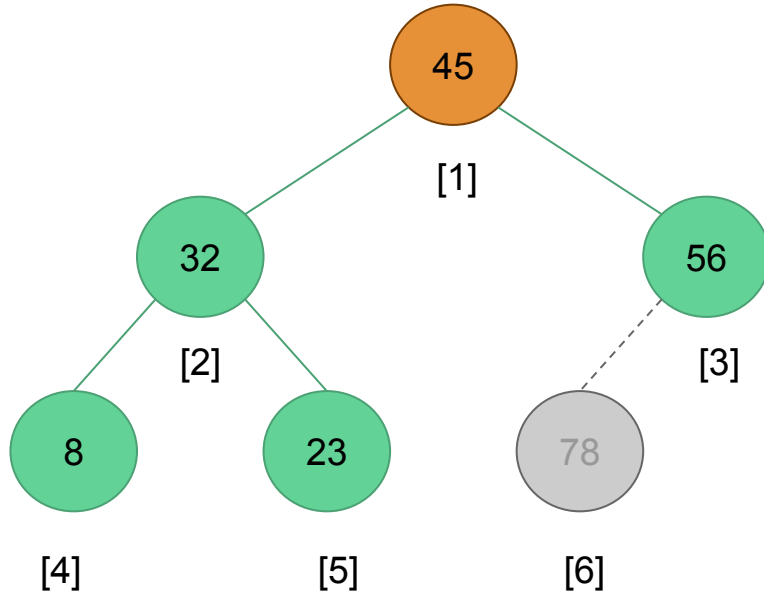
Heap sort



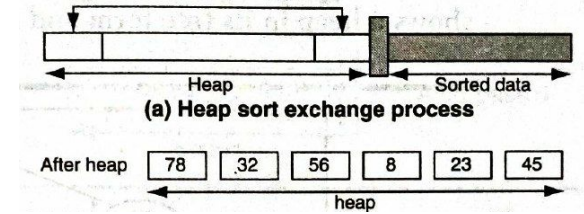
Swap the root of the heap with the element at the end of the list. Decrement the heap size by 1 and readjust the heap.



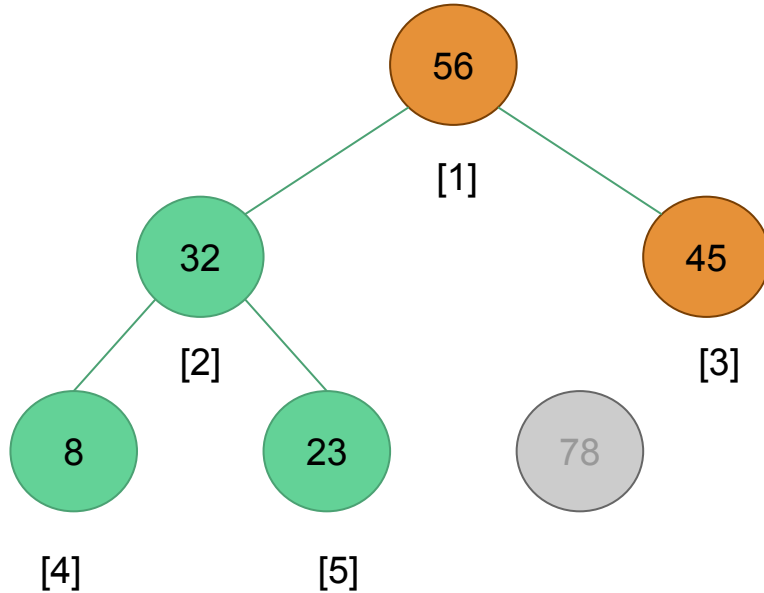
Heap sort



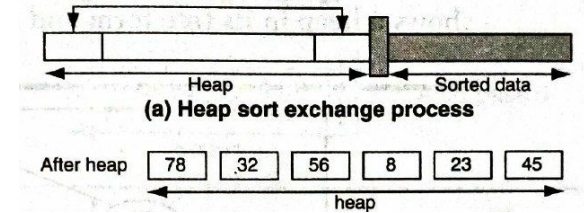
Swap the root of the heap with the element at the end of the list. Decrement the heap size by 1 and readjust the heap.



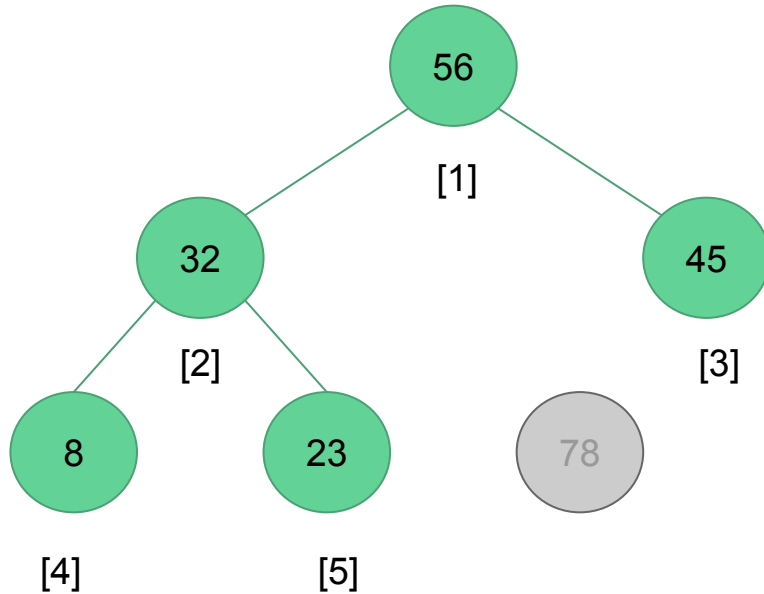
Heap sort



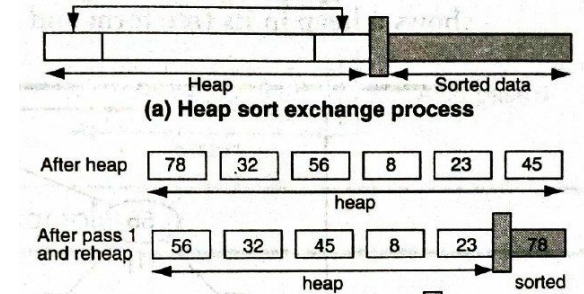
Swap the root of the heap with the element at the end of the list. Decrement the heap size by 1 and readjust the heap.



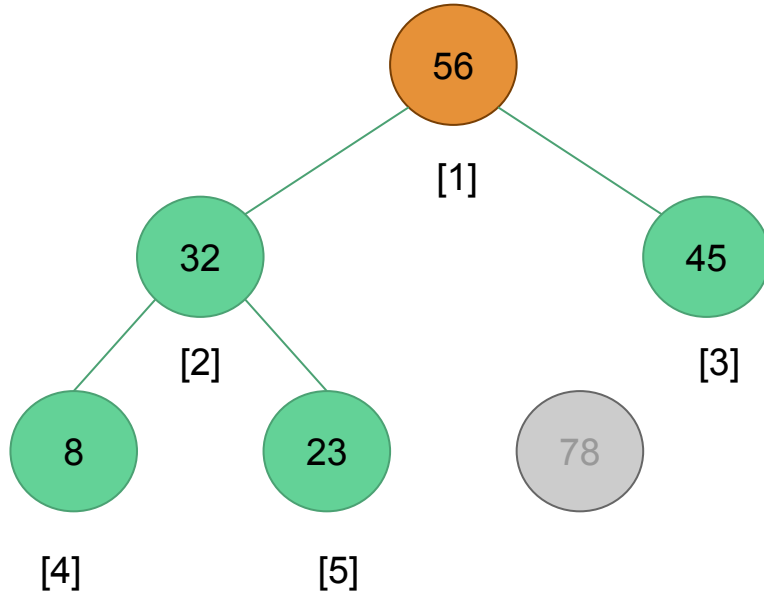
Heap sort



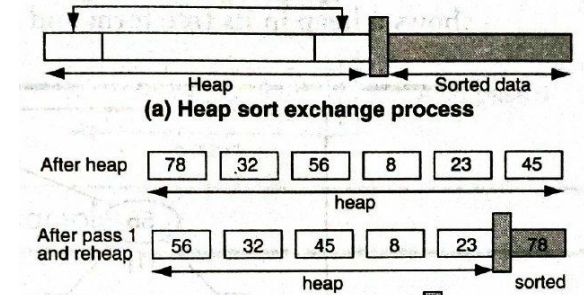
Swap the root of the heap with the element at the end of the list. Decrement the heap size by 1 and readjust the heap.



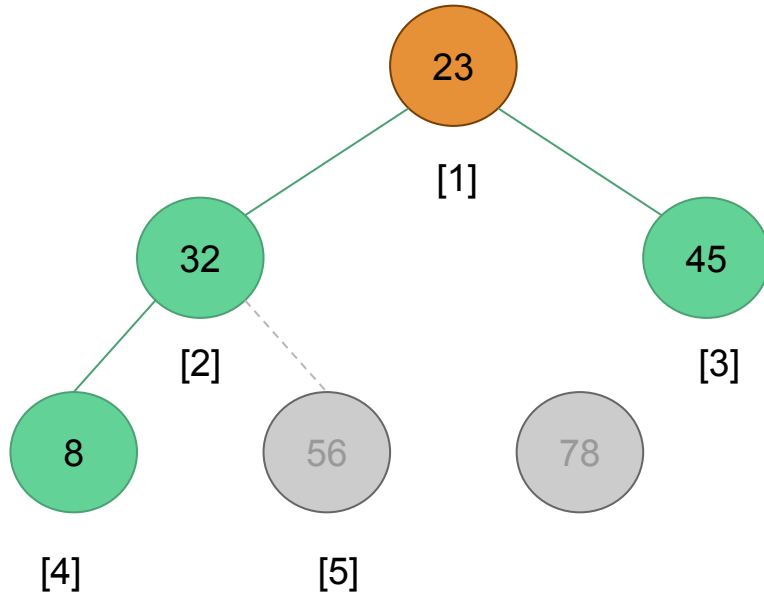
Heap sort



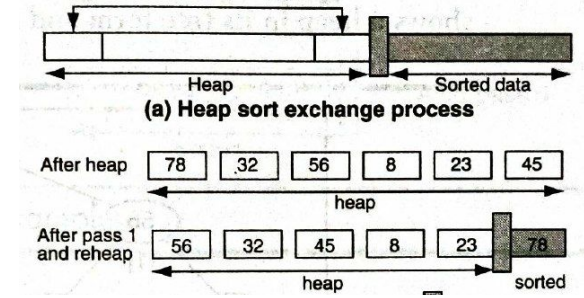
Swap the root of the heap with the element at the end of the list. Decrement the heap size by 1 and readjust the heap.



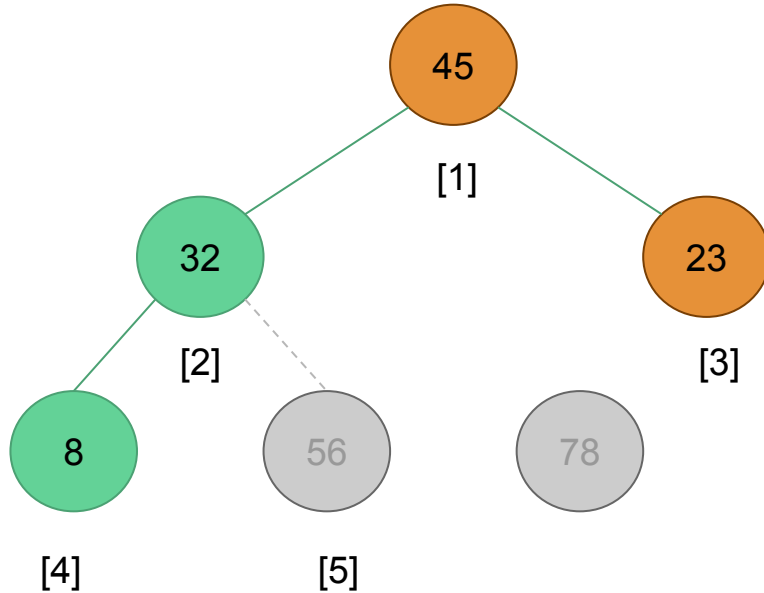
Heap sort



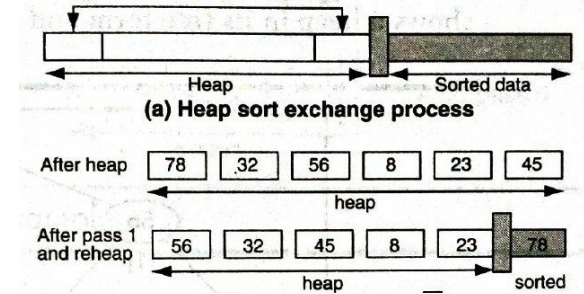
Swap the root of the heap with the element at the end of the list. Decrement the heap size by 1 and readjust the heap.



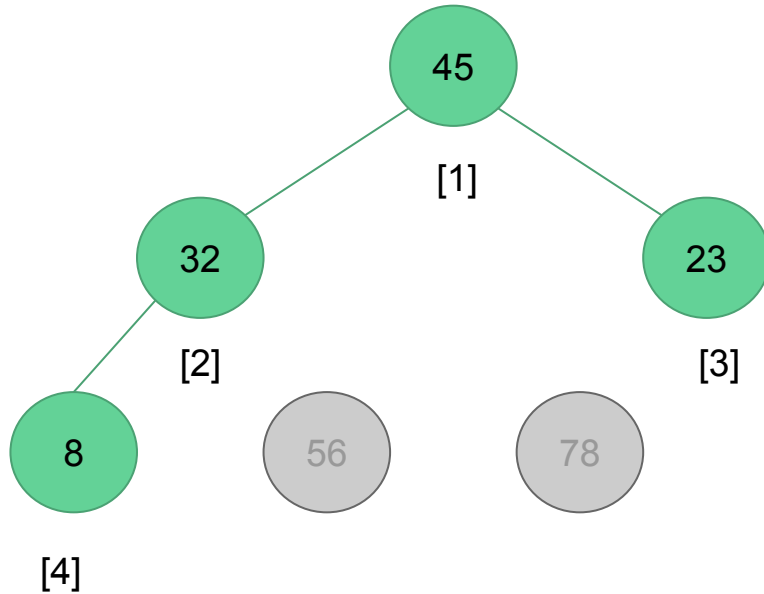
Heap sort



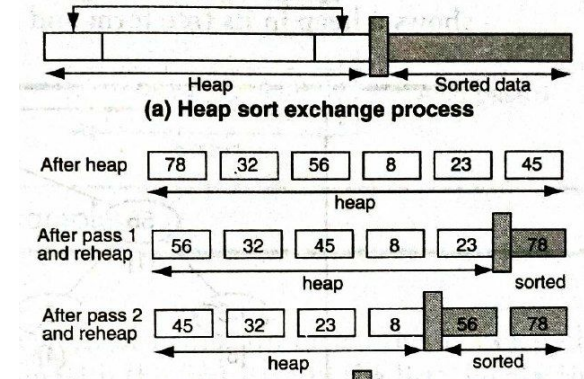
Swap the root of the heap with the element at the end of the list. Decrement the heap size by 1 and readjust the heap.



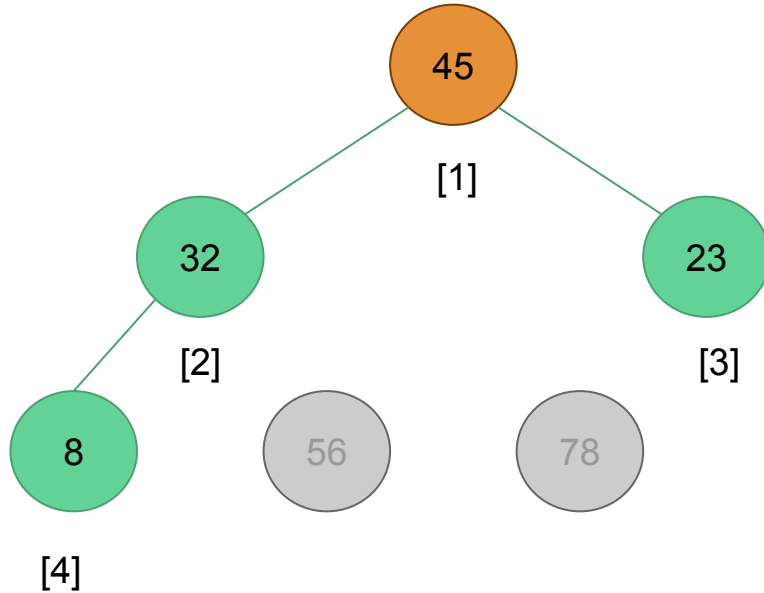
Heap sort



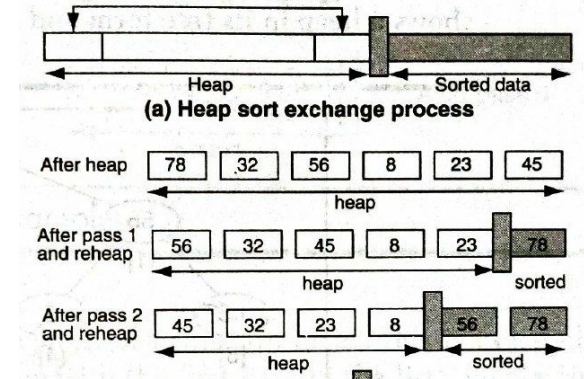
Swap the root of the heap with the element at the end of the list. Decrement the heap size by 1 and readjust the heap.



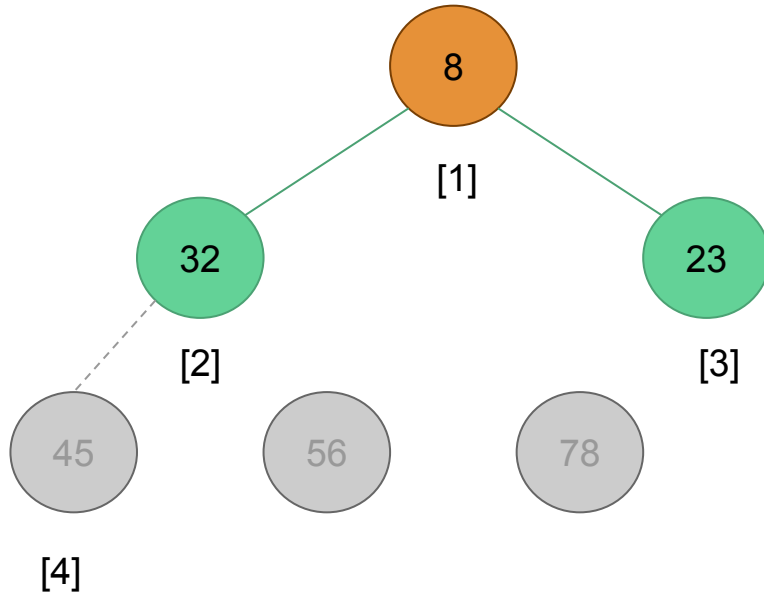
Heap sort



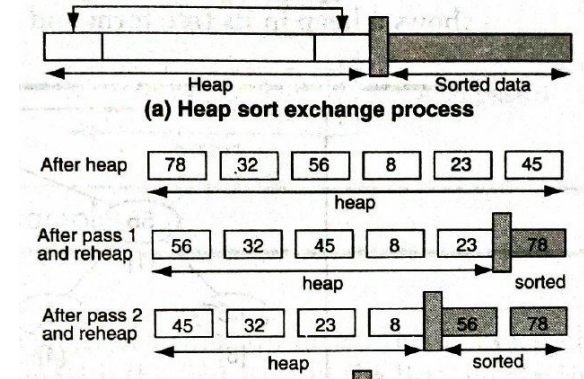
Swap the root of the heap with the element at the end of the list. Decrement the heap size by 1 and readjust the heap.



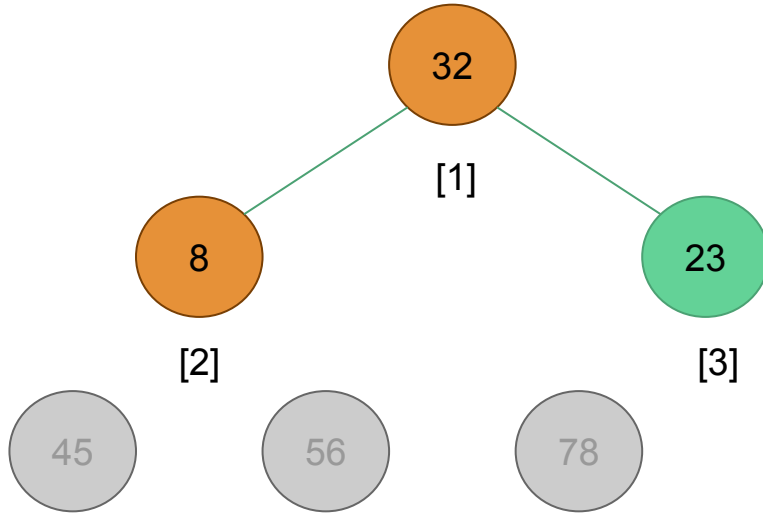
Heap sort



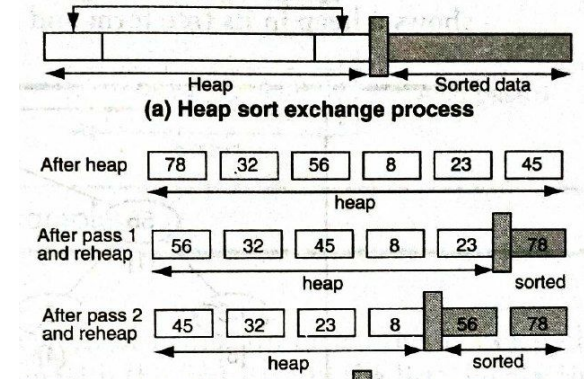
Swap the root of the heap with the element at the end of the list. Decrement the heap size by 1 and readjust the heap.



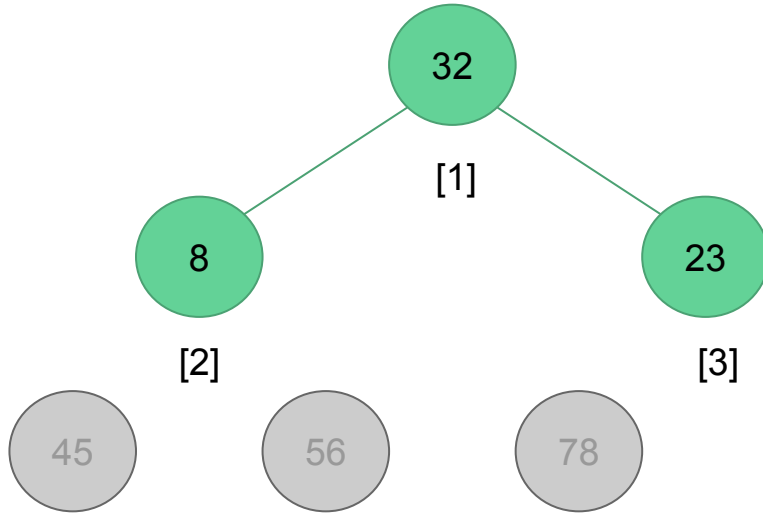
Heap sort



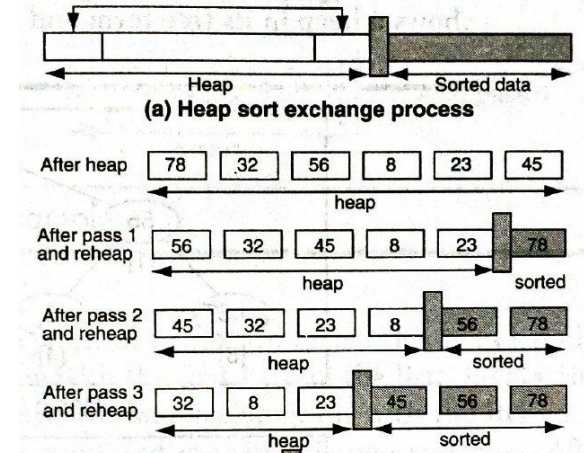
Swap the root of the heap with the element at the end of the list. Decrement the heap size by 1 and readjust the heap.



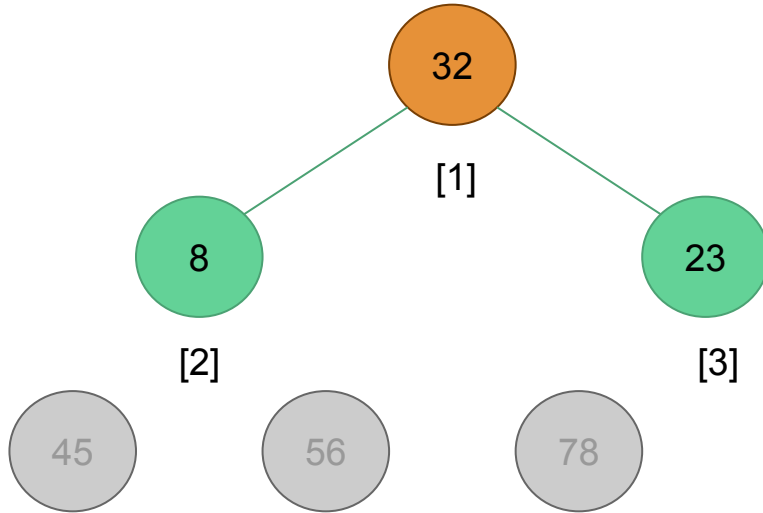
Heap sort



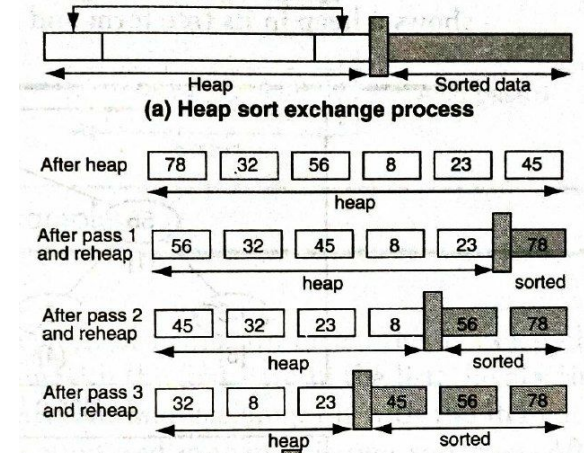
Swap the root of the heap with the element at the end of the list. Decrement the heap size by 1 and readjust the heap.



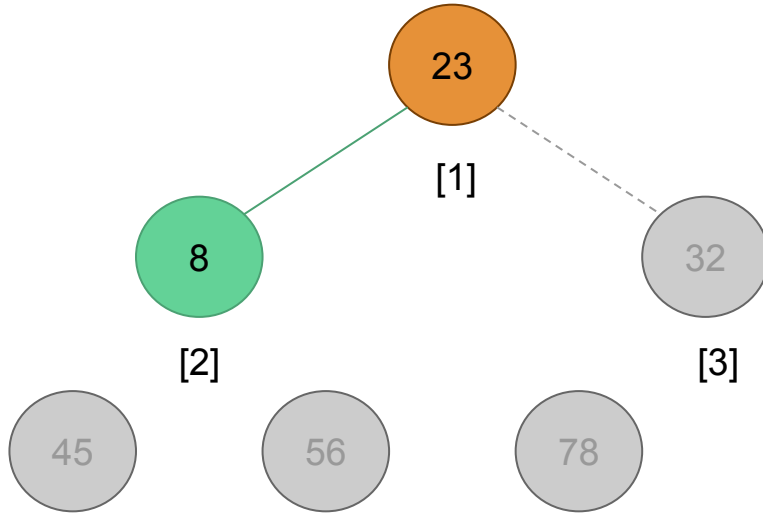
Heap sort



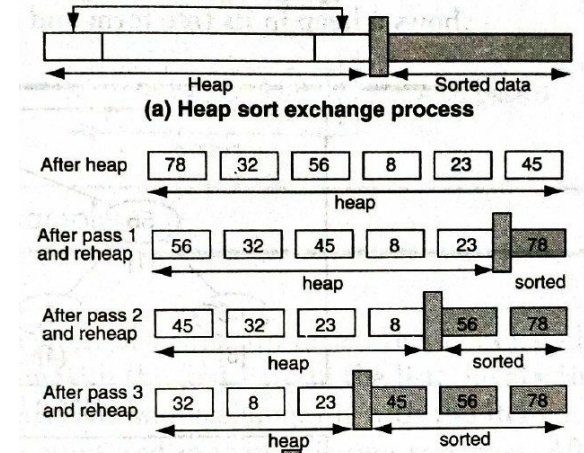
Swap the root of the heap with the element at the end of the list. Decrement the heap size by 1 and readjust the heap.



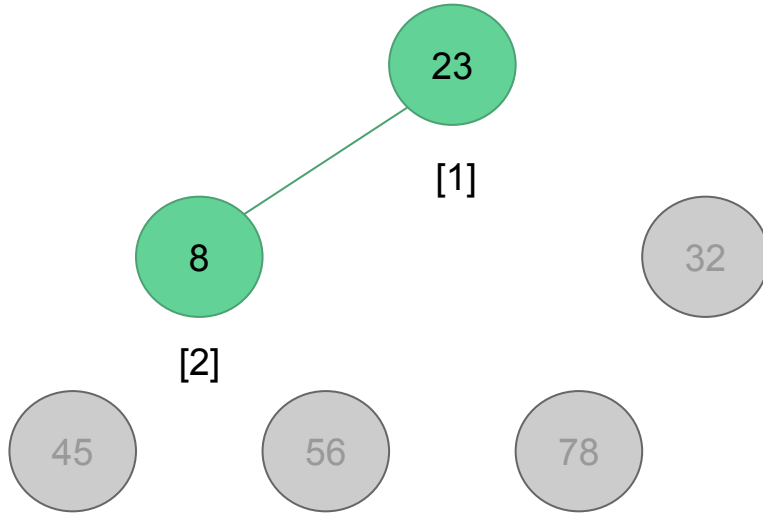
Heap sort



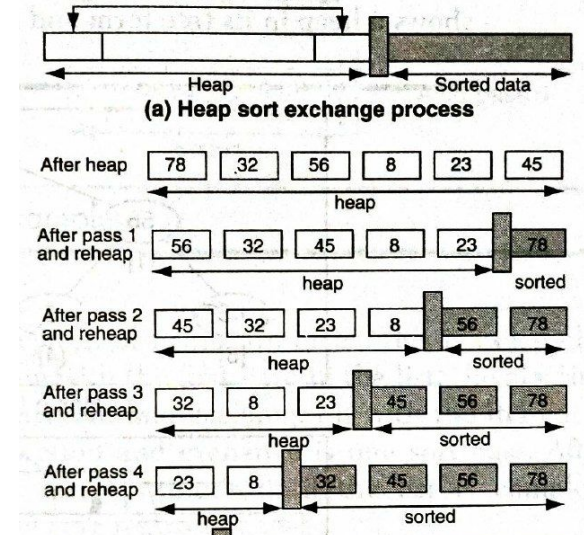
Swap the root of the heap with the element at the end of the list. Decrement the heap size by 1 and readjust the heap.



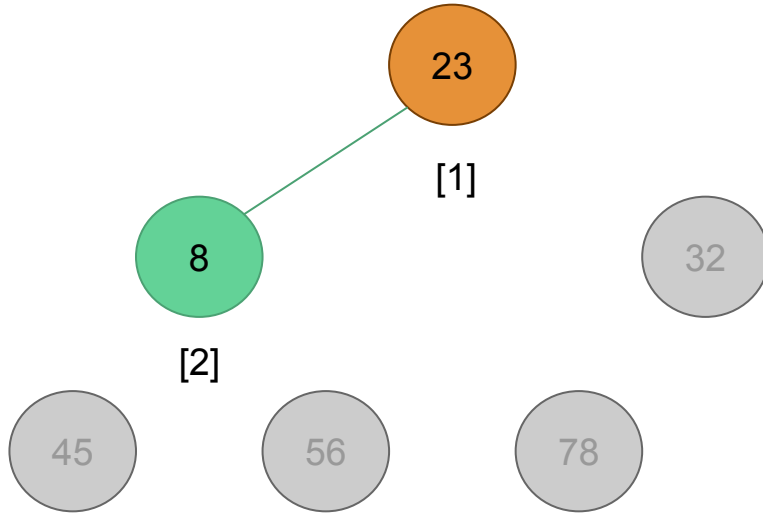
Heap sort



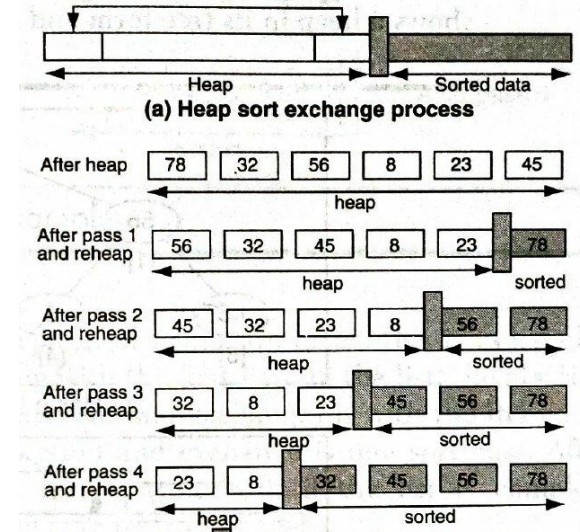
Swap the root of the heap with the element at the end of the list. Decrement the heap size by 1 and readjust the heap.



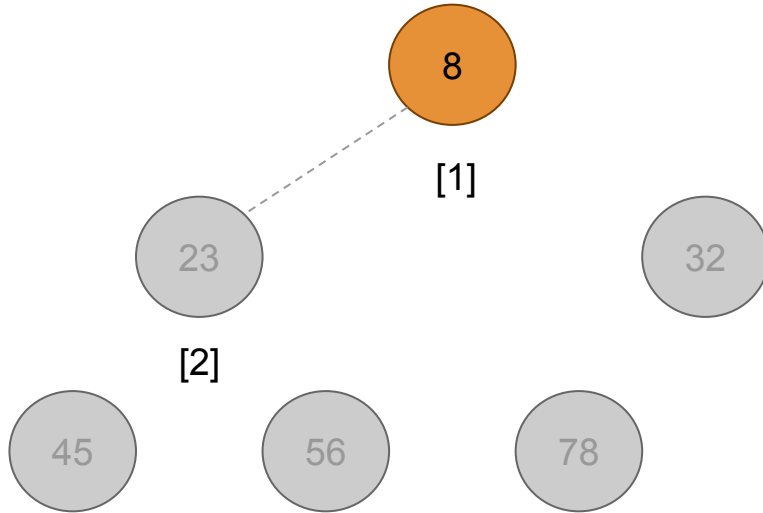
Heap sort



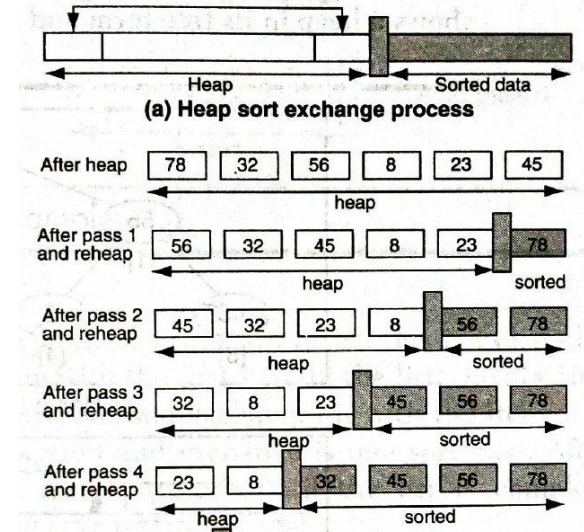
Swap the root of the heap with the element at the end of the list. Decrement the heap size by 1 and readjust the heap.



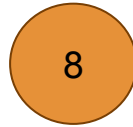
Heap sort



Swap the root of the heap with the element at the end of the list. Decrement the heap size by 1 and readjust the heap.



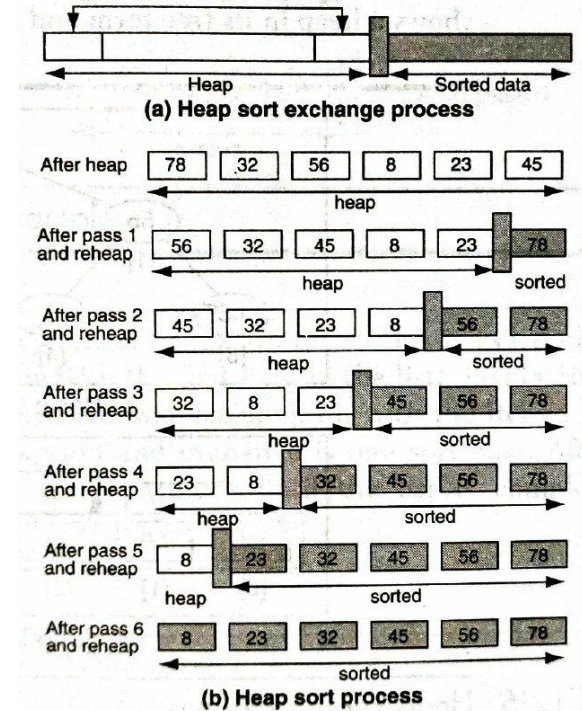
Heap sort



[1]



Swap the root of the heap with the element at the end of the list. Decrement the heap size by 1 and readjust the heap.



Analysis of Max-Heapify

MAX-HEAPIFY(A, i)

```
1   $l = \text{LEFT}(i)$ 
2   $r = \text{RIGHT}(i)$ 
3  if  $l \leq A.\text{heap-size}$  and  $A[l] > A[i]$ 
4       $\text{largest} = l$ 
5  else  $\text{largest} = i$ 
6  if  $r \leq A.\text{heap-size}$  and  $A[r] > A[\text{largest}]$ 
7       $\text{largest} = r$ 
8  if  $\text{largest} \neq i$ 
9      exchange  $A[i]$  with  $A[\text{largest}]$ 
10     MAX-HEAPIFY( $A, \text{largest}$ )
```

Analysis of Max-Heapify

MAX-HEAPIFY(A, i)

```
1   $l = \text{LEFT}(i)$ 
2   $r = \text{RIGHT}(i)$ 
3  if  $l \leq A.\text{heap-size}$  and  $A[l] > A[i]$ 
4       $largest = l$ 
5  else  $largest = i$ 
6  if  $r \leq A.\text{heap-size}$  and  $A[r] > A[largest]$ 
7       $largest = r$ 
8  if  $largest \neq i$ 
9      exchange  $A[i]$  with  $A[largest]$ 
10     MAX-HEAPIFY( $A, largest$ )
```

Running time of lines 1 - 9 = $\theta(1)$

Running time of line 10 = ?

Analysis of Max-Heapify

MAX-HEAPIFY(A, i)

```
1   $l = \text{LEFT}(i)$ 
2   $r = \text{RIGHT}(i)$ 
3  if  $l \leq A.\text{heap-size}$  and  $A[l] > A[i]$ 
4       $\text{largest} = l$ 
5  else  $\text{largest} = i$ 
6  if  $r \leq A.\text{heap-size}$  and  $A[r] > A[\text{largest}]$ 
7       $\text{largest} = r$ 
8  if  $\text{largest} \neq i$ 
9      exchange  $A[i]$  with  $A[\text{largest}]$ 
10     MAX-HEAPIFY( $A, \text{largest}$ )
```

Running time of lines 1 - 9 = $\theta(1)$

Running time of line 10 = ?

The running time $T(n)$ of Max-Heapify on a subtree of size n rooted at a given node i is

$\theta(1)$ + Time to run Max-Heapify on a subtree rooted at one of the children of node i

Analysis of Max-Heapify

MAX-HEAPIFY(A, i)

```
1   $l = \text{LEFT}(i)$ 
2   $r = \text{RIGHT}(i)$ 
3  if  $l \leq A.\text{heap-size}$  and  $A[l] > A[i]$ 
4       $largest = l$ 
5  else  $largest = i$ 
6  if  $r \leq A.\text{heap-size}$  and  $A[r] > A[largest]$ 
7       $largest = r$ 
8  if  $largest \neq i$ 
9      exchange  $A[i]$  with  $A[largest]$ 
10     MAX-HEAPIFY( $A, largest$ )
```

Max-Heapify may need to be called all the way down to the bottom level.

Recall

What is the maximum number of nodes on level i of a binary tree?

What is the maximum number of nodes in a binary tree of height h ?

$$\sum_{k=0}^n x^k = \frac{x^{n+1} - 1}{x - 1}$$

Analysis of Max-Heapify

MAX-HEAPIFY(A, i)

```
1   $l = \text{LEFT}(i)$ 
2   $r = \text{RIGHT}(i)$ 
3  if  $l \leq A.\text{heap-size}$  and  $A[l] > A[i]$ 
4       $\text{largest} = l$ 
5  else  $\text{largest} = i$ 
6  if  $r \leq A.\text{heap-size}$  and  $A[r] > A[\text{largest}]$ 
7       $\text{largest} = r$ 
8  if  $\text{largest} \neq i$ 
9      exchange  $A[i]$  with  $A[\text{largest}]$ 
10     MAX-HEAPIFY( $A, \text{largest}$ )
```

Max-Heapify may need to be called all the way down to the bottom level.

In such case, children's subtrees each will have size of $n/2 - 1$ if the heap is a complete binary tree, and at most $2n/3$ if the bottom level of the tree is exactly half full. (The latter is the worst case.)

$$T(n) \leq T(2n/3) + \theta(1)$$

From the master theorem, this recurrence solves to $T(n) = O(\log_2 n)$

Analysis of Max-Heapify

MAX-HEAPIFY(A, i)

```
1   $l = \text{LEFT}(i)$ 
2   $r = \text{RIGHT}(i)$ 
3  if  $l \leq A.\text{heap-size}$  and  $A[l] > A[i]$ 
4       $\text{largest} = l$ 
5  else  $\text{largest} = i$ 
6  if  $r \leq A.\text{heap-size}$  and  $A[r] > A[\text{largest}]$ 
7       $\text{largest} = r$ 
8  if  $\text{largest} \neq i$ 
9      exchange  $A[i]$  with  $A[\text{largest}]$ 
10     MAX-HEAPIFY( $A, \text{largest}$ )
```

Alternatively, we can characterize the running time, $T(n)$ of Max-Heapify on a node of height h as $O(h)$.

The height of a heap with n nodes is $\log_2 n$

$$T(n) = O(\log_2 n)$$

Analysis of Build-Max-Heap

BUILD-MAX-HEAP(A)

```
1   $A.heap-size = A.length$   
2  for  $i = \lfloor A.length/2 \rfloor$  downto 1  
3      MAX-HEAPIFY( $A, i$ )
```

Trivial Analysis: Each call to Max-Heapify requires $\log n$ time, we make $n/2$ such calls $\Rightarrow O(n \log n)$.

Analysis of Build-Max-Heap

BUILD-MAX-HEAP(A)

```
1   $A.heap-size = A.length$   
2  for  $i = \lfloor A.length/2 \rfloor$  downto 1  
3      MAX-HEAPIFY( $A, i$ )
```

Tighter Bound: When `Max-Heapify` is called, the running time depends on how far an element might shift down before the process terminates.

In the worst case, the element might shift down all the way to the leaf level.

To simplify the analysis, let's assume that the heap is a complete binary tree, i.e.

$$n = 2^{h+1} - 1$$

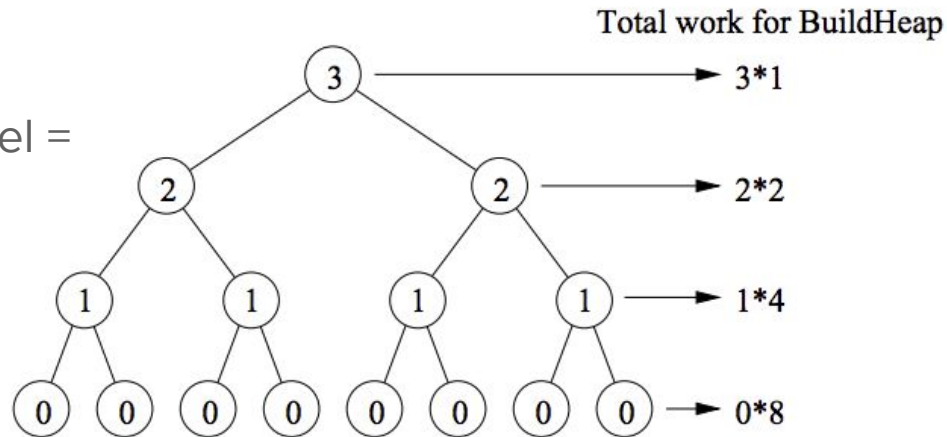
Analysis of build_heap

Tighter bound (contd.)

Number of nodes at the bottommost level = 2^h but we do not call heapify on any of these.

Number of nodes at the next to bottommost level = 2^{h-1} , each might shift down 1 level.

And so on.

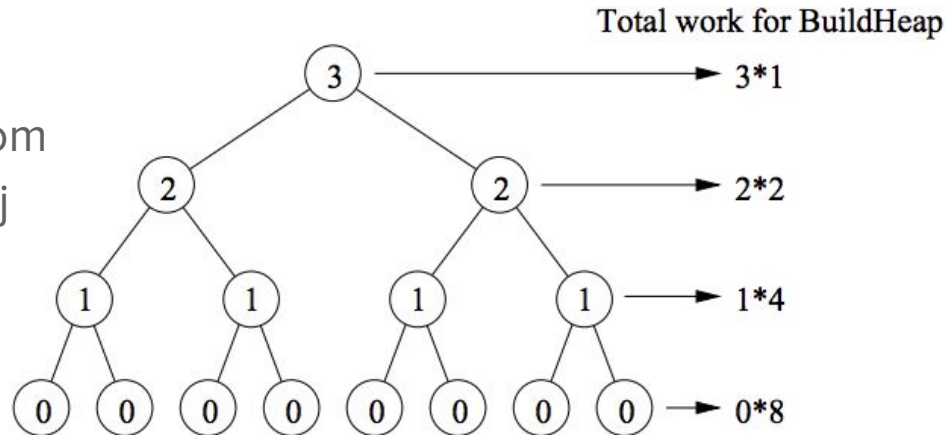


Analysis of build_heap

Tighter bound (contd.)

In general, number of nodes at level j from the bottom = 2^{h-j} , each might shift down j level

So, the total time is proportional to



$$T(n) = \sum_{j=0}^h j 2^{h-j} = \sum_{j=0}^h j \frac{2^h}{2^j} = 2^h \sum_{j=0}^h \frac{j}{2^j}.$$

Analysis of build_heap

Recall: The infinite geometric series, for any constant $x < 1$

$$\sum_{j=0}^{\infty} x^j = \frac{1}{1-x}.$$

Taking the derivative of both sides w.r.t. x gives

$$\sum_{j=0}^{\infty} jx^{j-1} = \frac{1}{(1-x)^2} \qquad \sum_{j=0}^{\infty} jx^j = \frac{x}{(1-x)^2},$$

Analysis of build_heap

When $x = 1/2$, we get

$$\sum_{j=0}^{\infty} \frac{j}{2^j} = \frac{1/2}{(1 - (1/2))^2} = \frac{1/2}{1/4} = 2.$$

Using this as an approximation, we get

$$T(n) = 2^h \sum_{j=0}^h \frac{j}{2^j} \leq 2^h \sum_{j=0}^{\infty} \frac{j}{2^j} \leq 2^h \cdot 2 = 2^{h+1}.$$

Since $n = 2^{h+1} - 1$, so we have $T(n) \leq n + 1$, which implies $T(n)$ is $O(n)$.

Also, $T(n)$ is $\Omega(n)$ since every element of the array must be accessed at least once.

Therefore, $T(n)$ is $\Theta(n)$.

Analysis of heap sort

HEAPSORT(A)

```
1  BUILD-MAX-HEAP( $A$ )
2  for  $i = A.length$  downto 2
3      exchange  $A[1]$  with  $A[i]$ 
4       $A.heap-size = A.heap-size - 1$ 
5      MAX-HEAPIFY( $A, 1$ )
```


Analysis of heap sort

HEAPSORT(*A*)

1 BUILD-MAX-HEAP(*A*)

2 **for** *i* = *A.length* **downto** 2

3 exchange *A*[1] with *A*[*i*]

4 *A.heap-size* = *A.heap-size* - 1

5 MAX-HEAPIFY(*A*, 1)

Running time of line 1 = $O(n)$

Running time of line 3-4 = $O(1)$

Running time of line 5 = $O(\log n)$

Lines 3-5 are executed $n-1$ times.

Therefore, the running time of Heap-Sort is

$$T(n) = O(n) + O(1) + (n - 1) O(\log n) = O(n \log n)$$

Stability

Algorithm	Stable ?
Selection sort	No
Insertion sort	Yes
Heap sort	No
Merge sort	Yes
Quick sort	No

Sorting in linear time

Sorting algorithms that run in linear time

- Counting sort
- Radix sort
- Bucket sort



Readings

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Chapter 8: Sorting in Linear Time. *In Introduction to algorithms*. MIT press.

Andersson, A., Hagerup, T., Nilsson, S., & Raman, R. (1998). Sorting in linear time?. *Journal of Computer and System Sciences*, 57(1), 74-93.

Comparison sorts

Sorting algorithms studied so far are **comparison sorts**, i.e. they are based on comparisons of input elements.

Sorting algorithm	Time complexity
Insertion sort	$O(n^2)$
Selection sort	$O(n^2)$
Merge sort	$O(n \lg n)$
Heap sort	$O(n \lg n)$
Quick sort	$O(n^2)$

Any comparison sort algorithm requires $\Omega(n \lg n)$ comparisons in the worst case.

Comparison sorts

To get information about an input sequence $\langle a_1, a_2, \dots, a_n \rangle$, comparison sort algorithms only use comparisons of two elements a_i and a_j

- $a_i < a_j$,
- $a_i \leq a_j$,
- $a_i = a_j$,
- $a_i \geq a_j$,
- $a_i > a_j$

If we assume all input elements are distinct, then comparisons of the form $a_i = a_j$ are useless. Also, $a_i < a_j$, $a_i \leq a_j$, $a_i \geq a_j$, and $a_i > a_j$ are equivalent as they yield identical information about the relative order of a_i and a_j .

Comparison sorts can be viewed abstractly in terms of **decision trees**.

The decision tree model

- A **decision tree** is a full binary tree that represents the comparisons between elements that are performed by a particular sorting algorithm operating on an input of a given size.
- It does not consider control, data movement, and all other aspects of the algorithm but only comparisons.
- In a decision tree, for an input of size n ,
 - Each internal node is annotated by $i : j$ for some i and j in the range $1 \leq i, j \leq n$, indicating a comparison between a_i and a_j
 - Each leaf is annotated by a permutation $\langle \pi(1), \pi(2), \dots, \pi(n) \rangle$, indicating the ordering $a_{\pi(1)} \leq a_{\pi(2)} \leq a_{\pi(3)} \dots \leq a_{\pi(n)}$

The decision tree model

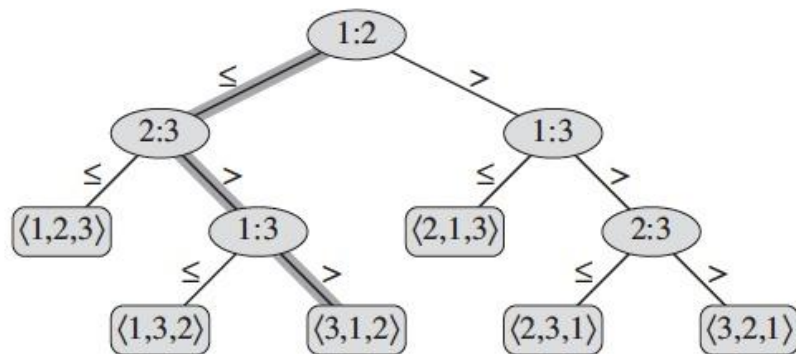
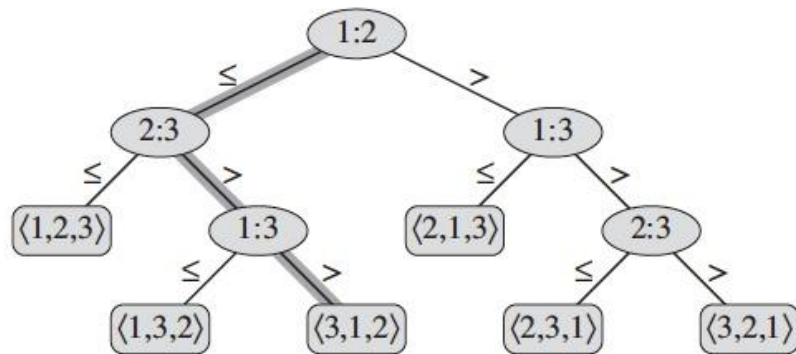


Figure 8.1 The decision tree for insertion sort operating on three elements. An internal node annotated by $i:j$ indicates a comparison between a_i and a_j . A leaf annotated by the permutation $\langle \pi(1), \pi(2), \dots, \pi(n) \rangle$ indicates the ordering $a_{\pi(1)} \leq a_{\pi(2)} \leq \dots \leq a_{\pi(n)}$. The shaded path indicates the decisions made when sorting the input sequence $\langle a_1 = 6, a_2 = 8, a_3 = 5 \rangle$; the permutation $\langle 3, 1, 2 \rangle$ at the leaf indicates that the sorted ordering is $a_3 = 5 \leq a_1 = 6 \leq a_2 = 8$. There are $3! = 6$ possible permutations of the input elements, and so the decision tree must have at least 6 leaves.

The decision tree model



Because any correct sorting algorithm must be able to produce each permutation of its input, **each of the $n!$ permutations on n elements must appear as one of the leaves of the decision tree.**

Figure 8.1 The decision tree for insertion sort operating on three elements. An internal node annotated by $i:j$ indicates a comparison between a_i and a_j . A leaf annotated by the permutation $\langle \pi(1), \pi(2), \dots, \pi(n) \rangle$ indicates the ordering $a_{\pi(1)} \leq a_{\pi(2)} \leq \dots \leq a_{\pi(n)}$. The shaded path indicates the decisions made when sorting the input sequence $\langle a_1 = 6, a_2 = 8, a_3 = 5 \rangle$; the permutation $\langle 3, 1, 2 \rangle$ at the leaf indicates that the sorted ordering is $a_3 = 5 \leq a_1 = 6 \leq a_2 = 8$. There are $3! = 6$ possible permutations of the input elements, and so the decision tree must have at least 6 leaves.

Lower bound of comparison sort algorithms

The length of the longest simple path from the root of a decision tree to any of its reachable leaves

= The worst-case number of comparisons that the corresponding sorting algorithm performs

= The height of its decision tree

Lower bound of comparison sort algorithms

Consider a decision tree of height h with l reachable leaves.

Since each of the $n!$ permutations of the input appears as some leaf, we have

$$n! \leq l$$

Since a binary tree of height h has no more than 2^h leaves, we have

$$n! \leq l \leq 2^h$$

By taking logarithms, we get $h \geq \lg(n!) = \Omega(n \lg n)$

∴ Any comparison sort algorithm requires $\Omega(n \lg n)$ comparisons in the worst case.

Counting sort

- Assumes that each of the n input elements is an integer in the range from 0 to k , for some integer k
- When $k = O(n)$, the sort runs in $\Theta(n)$ time
- Determines, for each input element x , the number of elements less than x .
 - This is how it positions x in its place in the output array
 - If there are 17 elements smaller than x , then x will be assigned position 18
- To sort an array $A[1..n]$, we need two additional arrays
 - $B[1..n]$ holds the sorted output
 - $C[1..k]$ stores the number of repetitions of number in A

Counting sort

	1	2	3	4	5	6	7	8
<i>A</i>	2	5	3	0	2	3	0	3

	0	1	2	3	4	5
<i>C</i>	2	0	2	3	0	1

(a)

Counting sort

	1	2	3	4	5	6	7	8
<i>A</i>	2	5	3	0	2	3	0	3
	0	1	2	3	4	5		
<i>C</i>	2	0	2	3	0	1		

(a)

	0	1	2	3	4	5
<i>C</i>	2	2	4	7	7	8

(b)

Counting sort

	1	2	3	4	5	6	7	8
<i>A</i>	2	5	3	0	2	3	0	3
	0	1	2	3	4	5		
<i>C</i>	2	0	2	3	0	1		

(a)

	0	1	2	3	4	5
<i>C</i>	2	2	4	7	7	8

(b)

	1	2	3	4	5	6	7	8
<i>B</i>							3	
	0	1	2	3	4	5		
<i>C</i>	2	2	4	6	7	8		

(c)

Counting sort

	1	2	3	4	5	6	7	8
<i>A</i>	2	5	3	0	2	3	0	3
	0	1	2	3	4	5		
<i>C</i>	2	0	2	3	0	1		

(a)

	0	1	2	3	4	5
<i>C</i>	2	2	4	7	7	8

(b)

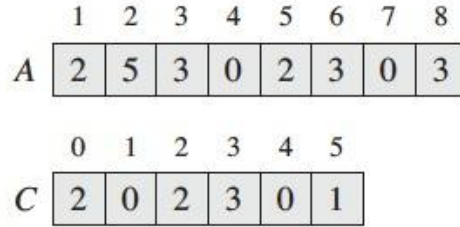
	1	2	3	4	5	6	7	8
<i>B</i>							3	
	0	1	2	3	4	5		
<i>C</i>	2	2	4	6	7	8		

(c)

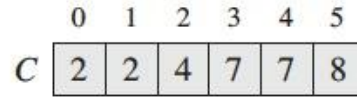
	1	2	3	4	5	6	7	8
<i>B</i>		0					3	
	0	1	2	3	4	5		
<i>C</i>	1	2	4	6	7	8		

(d)

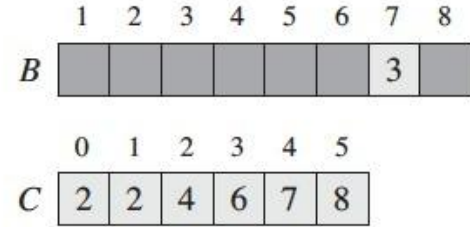
Counting sort



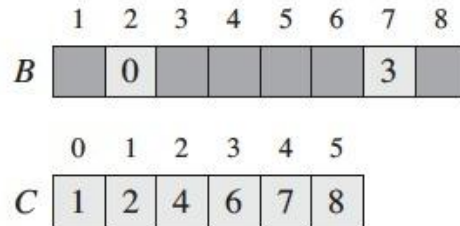
(a)



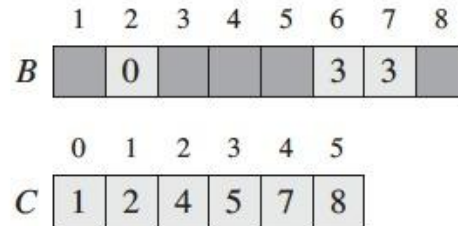
(b)



(c)

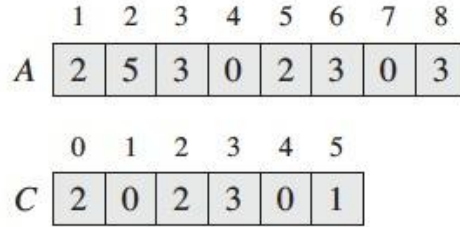


(d)

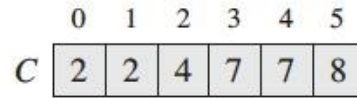


(e)

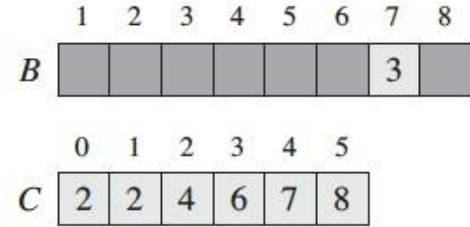
Counting sort



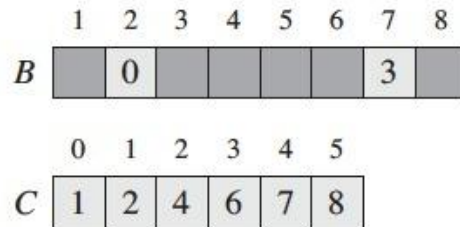
(a)



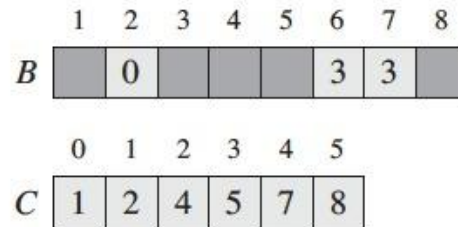
(b)



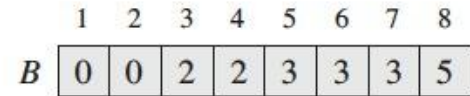
(c)



(d)



(e)



(f)

Counting sort

COUNTING-SORT(A, B, k)

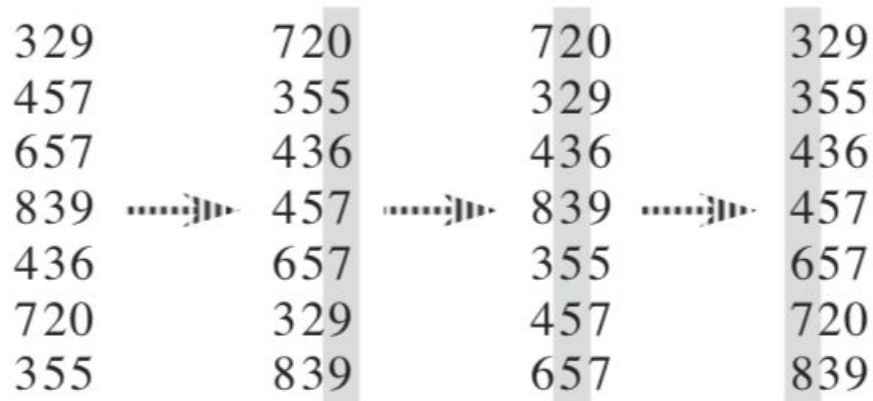
```
1  let  $C[0..k]$  be a new array
2  for  $i = 0$  to  $k$                                      Line 2 - 3  $\rightarrow \Theta(k)$ 
3       $C[i] = 0$ 
4  for  $j = 1$  to  $A.length$                                Line 4 - 5  $\rightarrow \Theta(n)$ 
5       $C[A[j]] = C[A[j]] + 1$ 
6  //  $C[i]$  now contains the number of elements equal to  $i$ .
7  for  $i = 1$  to  $k$                                      Line 7 - 8  $\rightarrow \Theta(k)$ 
8       $C[i] = C[i] + C[i - 1]$ 
9  //  $C[i]$  now contains the number of elements less than or equal to  $i$ .
10 for  $j = A.length$  downto 1
11      $B[C[A[j]]] = A[j]$ 
12      $C[A[j]] = C[A[j]] - 1$                            Line 10 - 12  $\rightarrow \Theta(n)$ 
```

Counting sort

- Is **not** a comparison sort
- Beats the lower bound of $\Omega(n \lg n)$
- Is **stable**
- Is often used as a subroutine in radix sort

Radix sort

- The idea of Radix Sort is to do digit by digit sort starting from least significant digit to most significant digit.
- In order for radix sort to work correctly, the digit sorts must be stable.
- Radix sort uses counting sort as a subroutine to sort.



Radix sort

RADIX-SORT(A, d)

1 **for** $i = 1$ **to** d

2 use a stable sort to sort array A on digit i

Each element in the n -element array A has d digits, where digit 1 is the lowest-order digit and digit d is the highest-order digit.

Radix sort

RADIX-SORT(A, d)

- 1 **for** $i = 1$ **to** d
- 2 use a stable sort to sort array A on digit i

When each digit is in the range 0 to $k - 1$ (so that it can take on k possible values), and k is not too large, counting sort is the obvious choice.

Each step for a digit takes $\Theta(n+k)$.

For d digits $\Theta(dn+dk) \Rightarrow$ The total time for radix sort is $\Theta(d(n+k))$

When d is constant and $k = O(n)$, we can make radix sort run in linear time.

Bucket sort

- Assumes that the input is drawn from a uniform distribution over the interval $[0, 1)$
- Divides the interval $[0, 1)$ into n equal-sized subintervals, or **buckets**, and then distributes the n input numbers into the buckets
- To produce the output, we sort numbers in each bucket, then go through buckets in order

Bucket sort

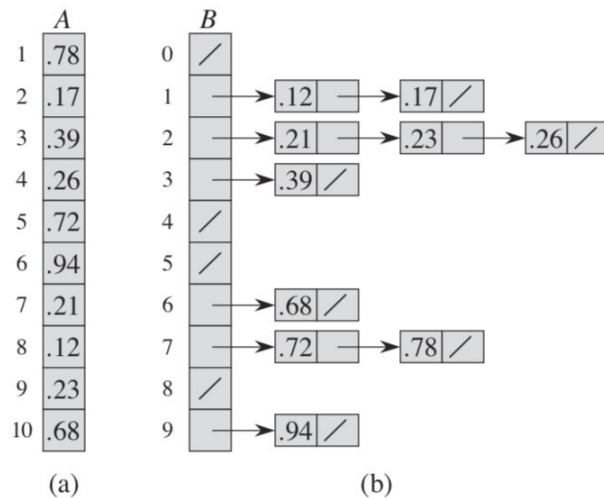


Figure 8.4 The operation of BUCKET-SORT for $n = 10$. (a) The input array $A[1..10]$. (b) The array $B[0..9]$ of sorted lists (buckets) after line 8 of the algorithm. Bucket i holds values in the half-open interval $[i/10, (i + 1)/10)$. The sorted output consists of a concatenation in order of the lists $B[0], B[1], \dots, B[9]$.

Bucket sort

BUCKET-SORT(A)

```
1  let  $B[0 \dots n - 1]$  be a new array
2   $n = A.length$ 
3  for  $i = 0$  to  $n - 1$ 
4      make  $B[i]$  an empty list
5  for  $i = 1$  to  $n$ 
6      insert  $A[i]$  into list  $B[\lfloor n A[i] \rfloor]$ 
7  for  $i = 0$  to  $n - 1$ 
8      sort list  $B[i]$  with insertion sort
9  concatenate the lists  $B[0], B[1], \dots, B[n - 1]$  together in order
```

Chapter 3:

Algorithm Strategies

Contents

- Brute-force
- The Greedy method
- Divide-and-conquer
- Backtracking
- Branch-and-bound
- Heuristics



Brute-force

Brute-force

- A straightforward approach to solving a problem, usually directly based on the problem statement and definition
- Systematically enumerates all possible candidates for the solution and checks whether each candidate satisfies the problem statement

Brute-force algorithms

Selection sort

Based on sequentially finding the smallest elements

Bubble Sort

Based on consecutive swapping adjacent pairs. This causes a slow migration of the smallest elements to the left of the array.

Brute-force algorithms

Bubble Sort

Based on consecutive swapping adjacent pairs. This causes a slow migration of the smallest elements to the left of the array.

```
1: procedure BUBBLESORT( $A[0 \dots n - 1]$ )
2:   for  $i \leftarrow 0$  to  $n - 2$  do
3:     for  $j \leftarrow 0$  to  $n - 2 - i$  do
4:       if  $A[j + 1] < A[j]$  then
5:         swap  $A[j + 1]$  and  $A[j]$ 
6:       end if
7:     end for
8:   end for
9: end procedure
```

Brute-force algorithms

Sequential Searching

```
1: procedure SEQUENTIALSEARCH( $A[0 \dots n-1]$ ,  $K$ )     $\triangleright K$  is the search key
2:    $i \leftarrow 0$ 
3:   while  $i < n$  and  $A[i] \neq K$  do
4:      $i \leftarrow i + 1$ 
5:   end while
6:   if  $i < n$  then
7:     return  $i$ 
8:   else
9:     return -1
10:  end if
11: end procedure
```

Brute-force algorithms

Brute-Force String Matching: Searching for a pattern, $P[0 \dots m-1]$, in text, $T[0 \dots n-1]$

```
1: procedure BRUTEFORCESTRINGMATCH( $T[0 \dots n-1], P[0 \dots m-1]$ )
2:   for  $i \leftarrow 0$  to  $n - m$  do
3:      $j \leftarrow 0$ 
4:     while  $j < m$  and  $P[j] == T[i + j]$  do
5:        $j \leftarrow j + 1$ 
6:     end while
7:     if  $j == m$  then
8:       return  $i$ 
9:     end if
10:  end for
11:  return -1
12: end procedure
```

The Greedy method

The greedy method

- Builds up a solution piece by piece, always choosing the next piece that looks best at the moment
- The main idea is to make locally optimal choice in the hope that this choice will lead to a globally optimal solution
- Greedy algorithms do not always yield optimal solutions, but for many problems they do

Huffman coding

Coding: Assigning binary codewords to (blocks of) source symbols

Huffman coding is a lossless data compression algorithm.

Idea:

- Assign variable-length codes to input characters, based on the frequencies of corresponding characters.
- Instead of using ASCII codes, store the more frequently occurring characters using fewer bits and less frequently occurring characters using more bits.

Huffman coding

There are mainly two major parts in Huffman Coding

1. Build a **Huffman Tree** from input characters.
2. Traverse the Huffman Tree and assign codes to characters.

Huffman coding

Building a Huffman tree:

1. Organize the entire character set into a row, ordered according to frequency from highest to lowest (or vice versa). Each character is now a node at the leaf level of a tree
2. Find two nodes with the smallest combined frequency weights and join them to form a third node, resulting in a simple two-level tree. The weight of the new node is the combined weights of the original two nodes.
3. Repeat step 2 until all of the nodes, on every level, are combined into a single tree.

Huffman coding example

Input character string: "**datastructures**"

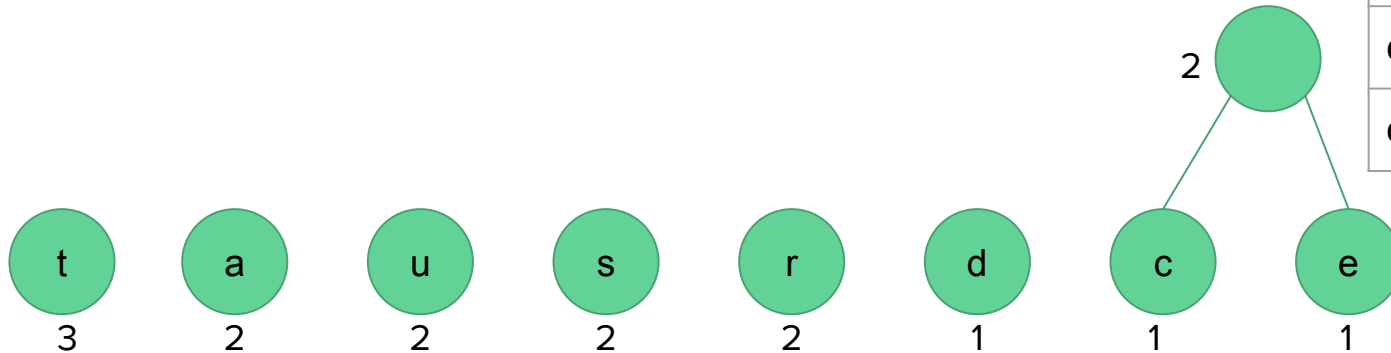
We first build the frequency table

character	frequency
t	3
a	2
u	2
s	2
r	2
d	1
c	1
e	1

Huffman coding example

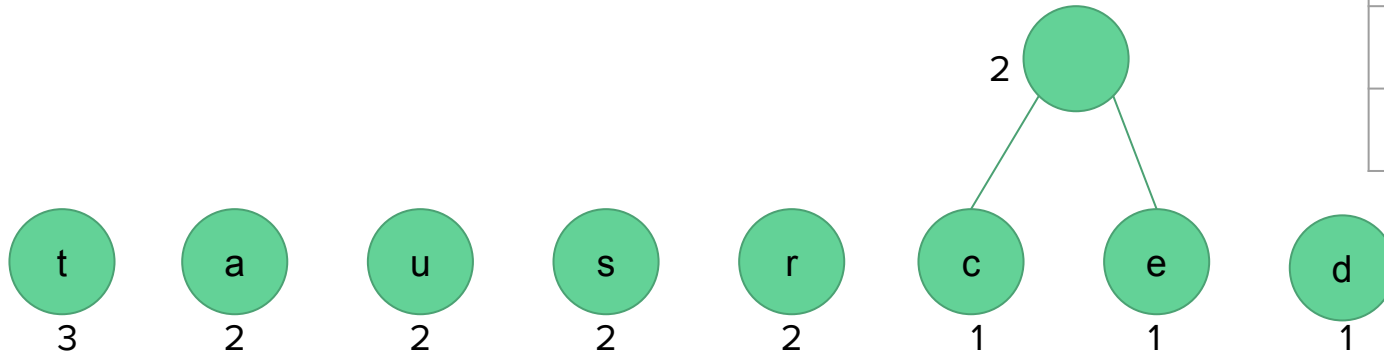
Build a Huffman tree:

character	frequency
t	3
a	2
u	2
s	2
r	2
d	1
c	1
e	1



Huffman coding example

Build a Huffman tree:

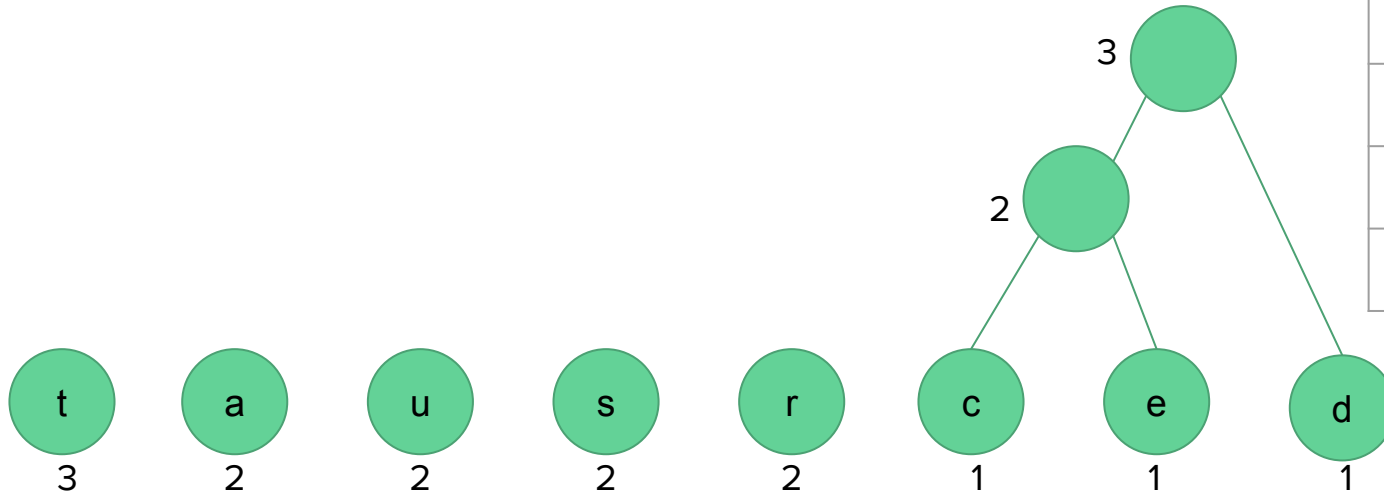


character	frequency
t	3
a	2
u	2
s	2
r	2
d	1
c	1
e	1

Huffman coding example

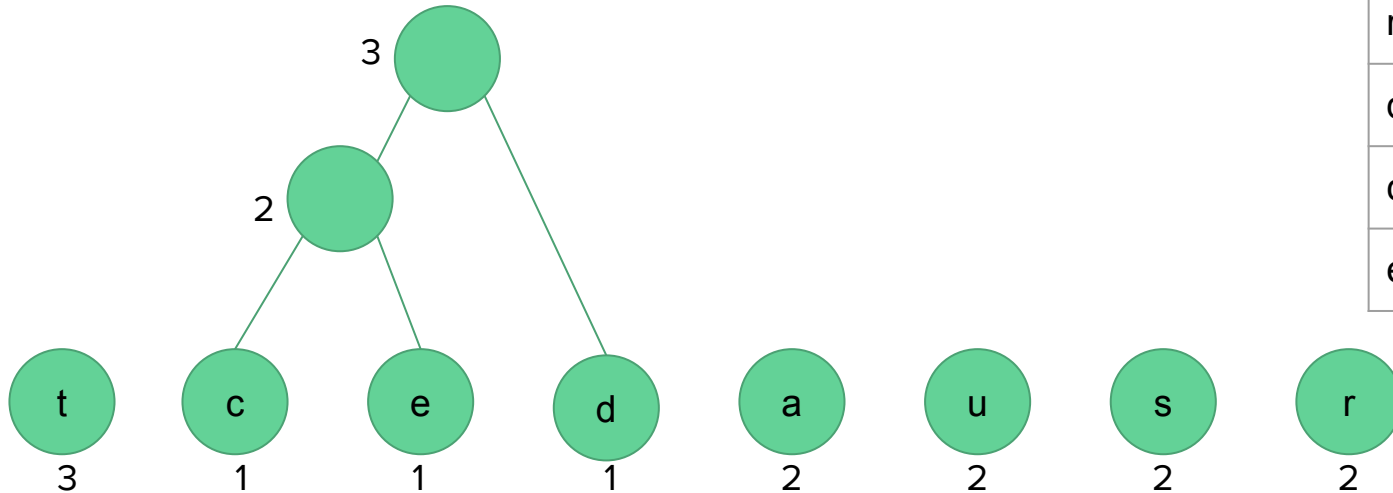
Build a Huffman tree:

character	frequency
t	3
a	2
u	2
s	2
r	2
d	1
c	1
e	1



Huffman coding example

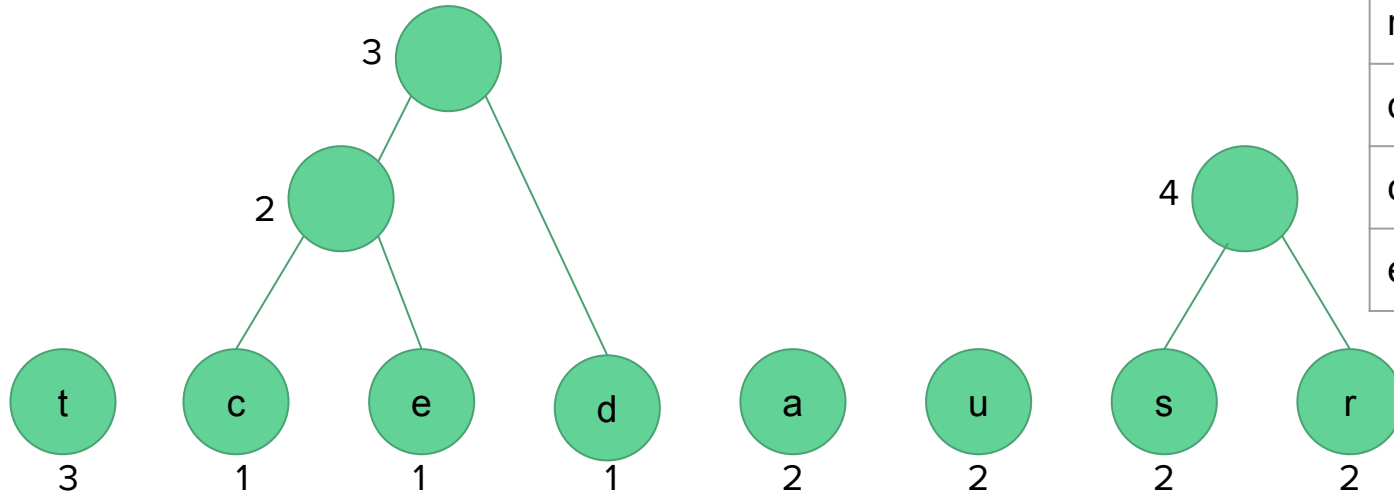
Build a Huffman tree:



character	frequency
t	3
a	2
u	2
s	2
r	2
d	1
c	1
e	1

Huffman coding example

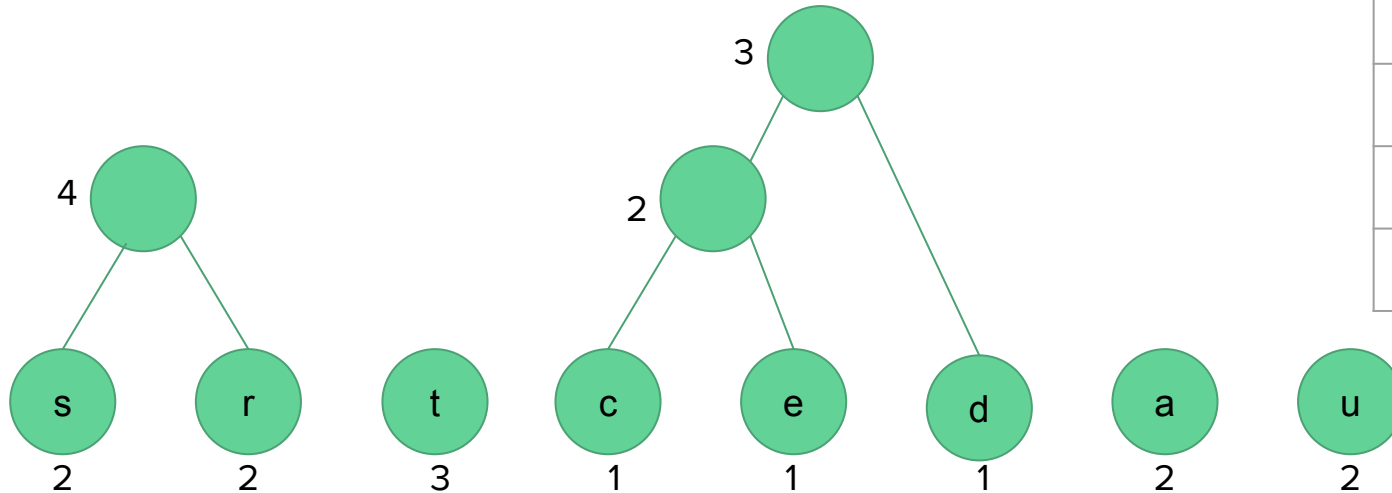
Build a Huffman tree:



character	frequency
t	3
a	2
u	2
s	2
r	2
d	1
c	1
e	1

Huffman coding example

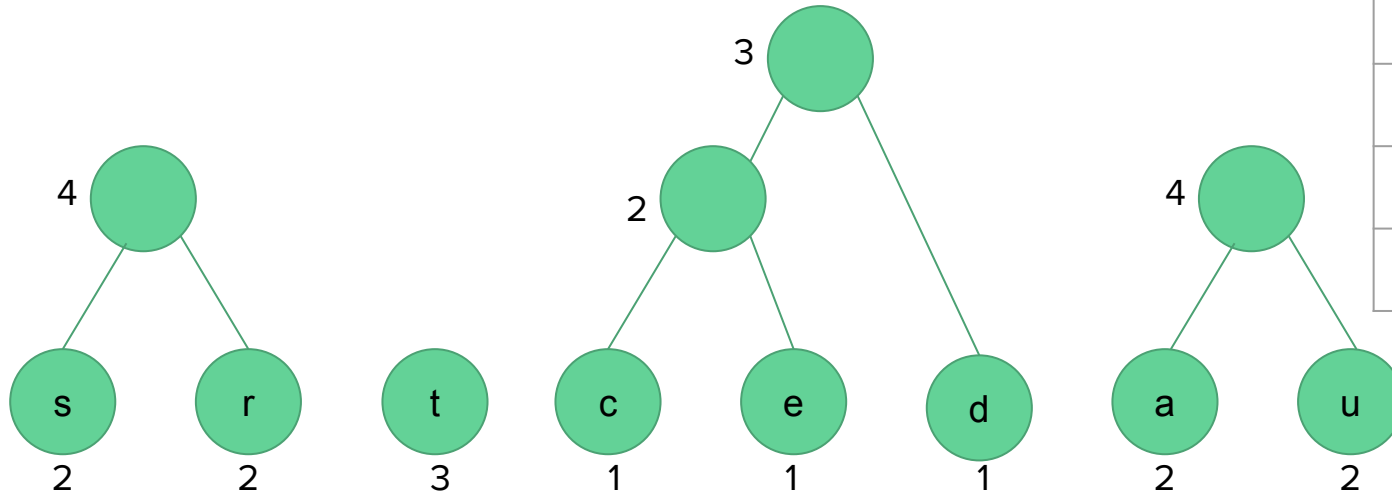
Build a Huffman tree:



character	frequency
t	3
a	2
u	2
s	2
r	2
d	1
c	1
e	1

Huffman coding example

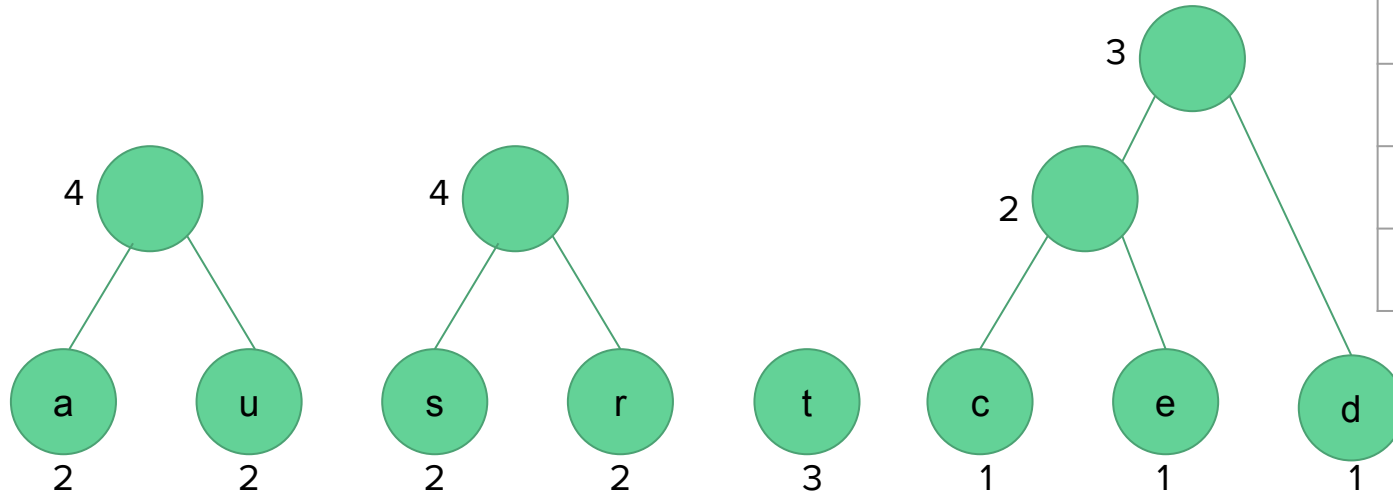
Build a Huffman tree:



character	frequency
t	3
a	2
u	2
s	2
r	2
d	1
c	1
e	1

Huffman coding example

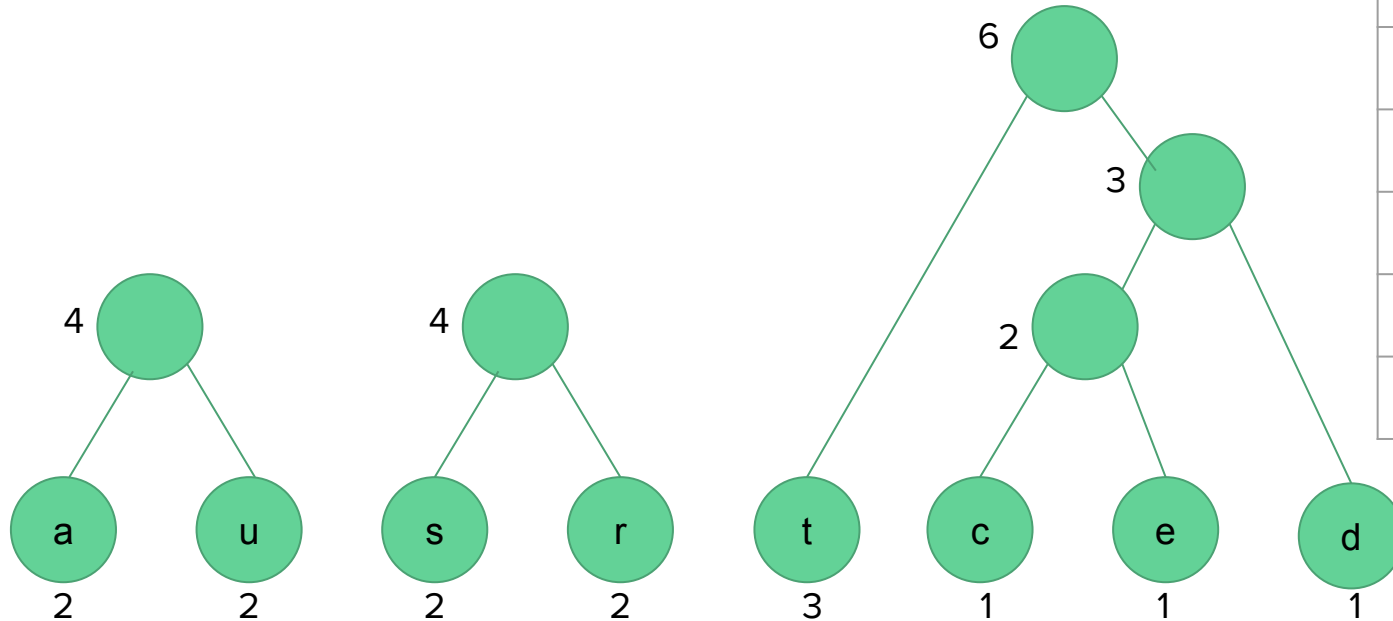
Build a Huffman tree:



character	frequency
t	3
a	2
u	2
s	2
r	2
d	1
c	1
e	1

Huffman coding example

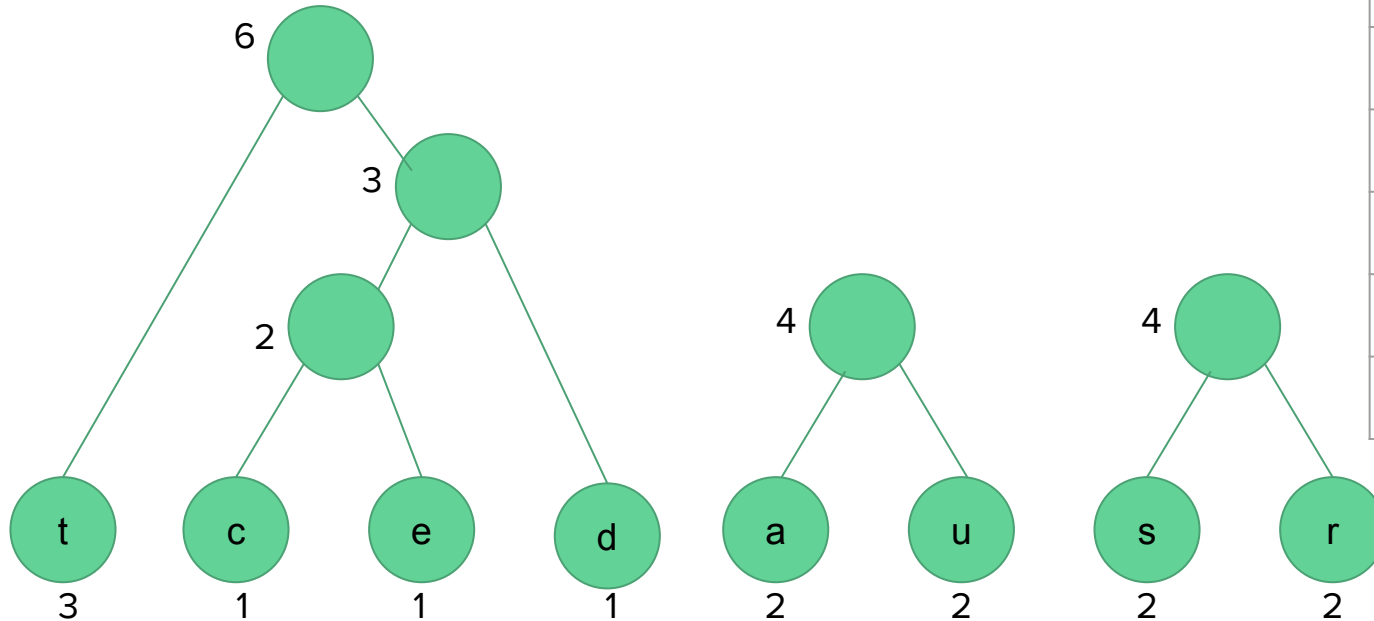
Build a Huffman tree:



character	frequency
t	3
a	2
u	2
s	2
r	2
d	1
c	1
e	1

Huffman coding example

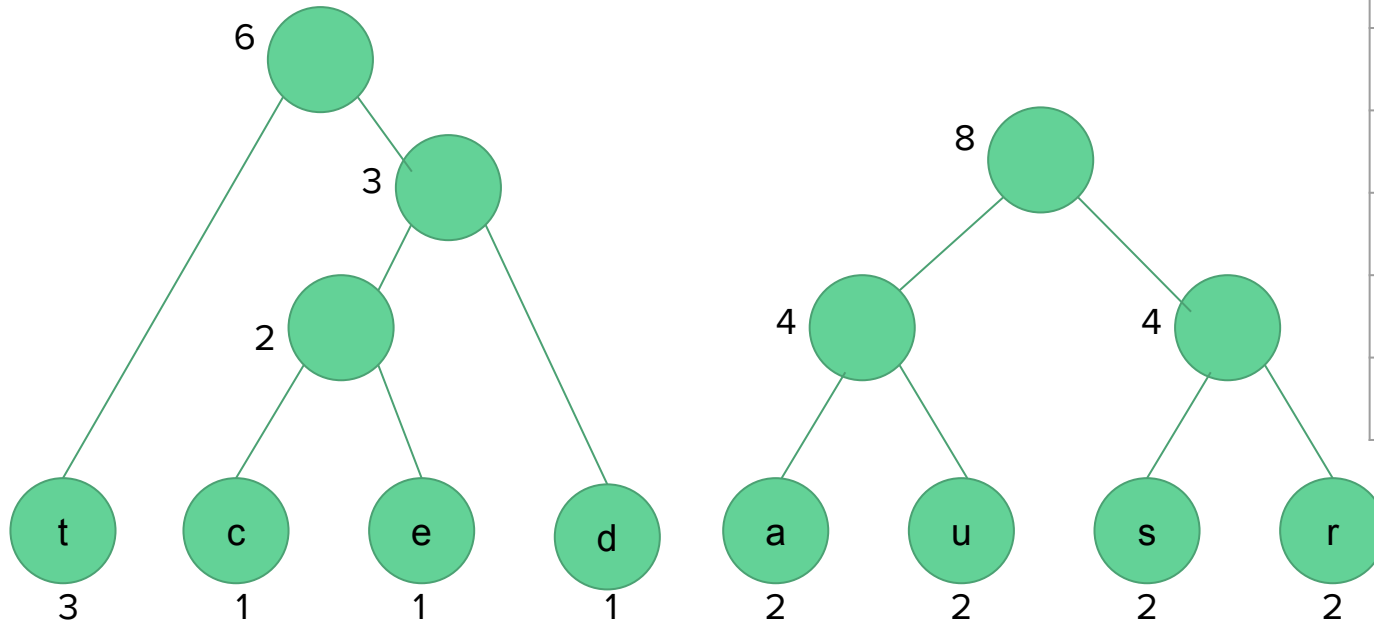
Build a Huffman tree:



character	frequency
t	3
a	2
u	2
s	2
r	2
d	1
c	1
e	1

Huffman coding example

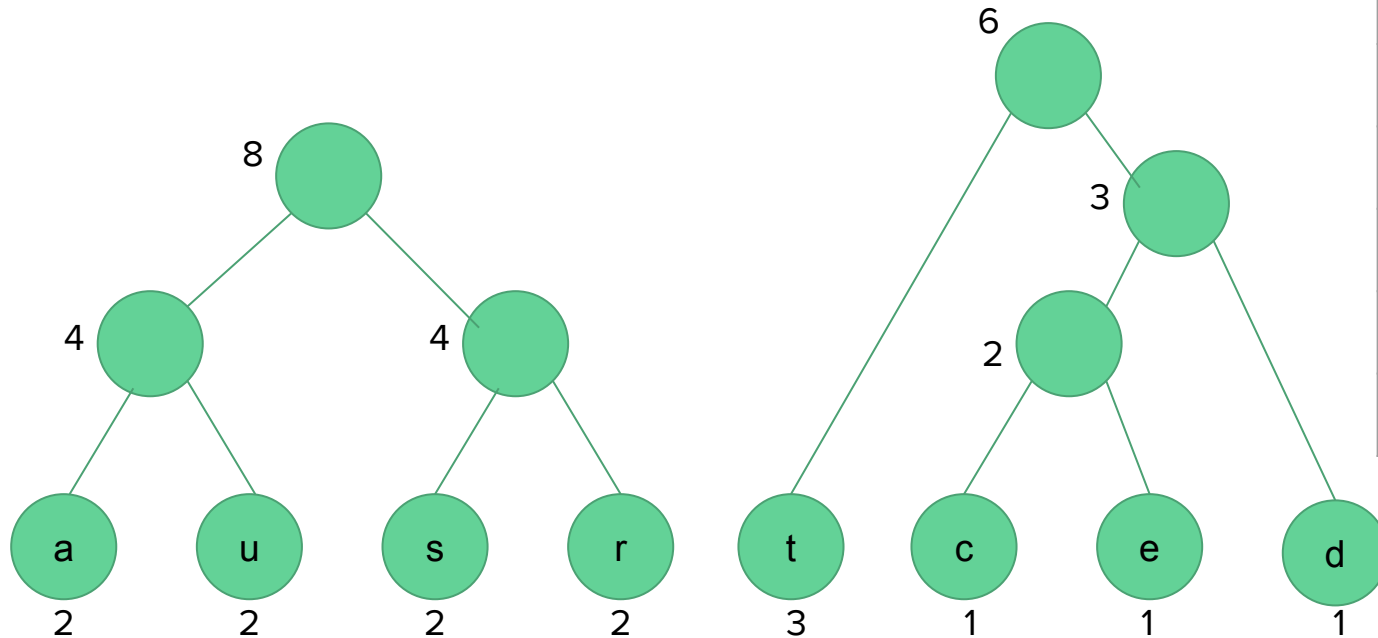
Build a Huffman tree:



character	frequency
t	3
a	2
u	2
s	2
r	2
d	1
c	1
e	1

Huffman coding example

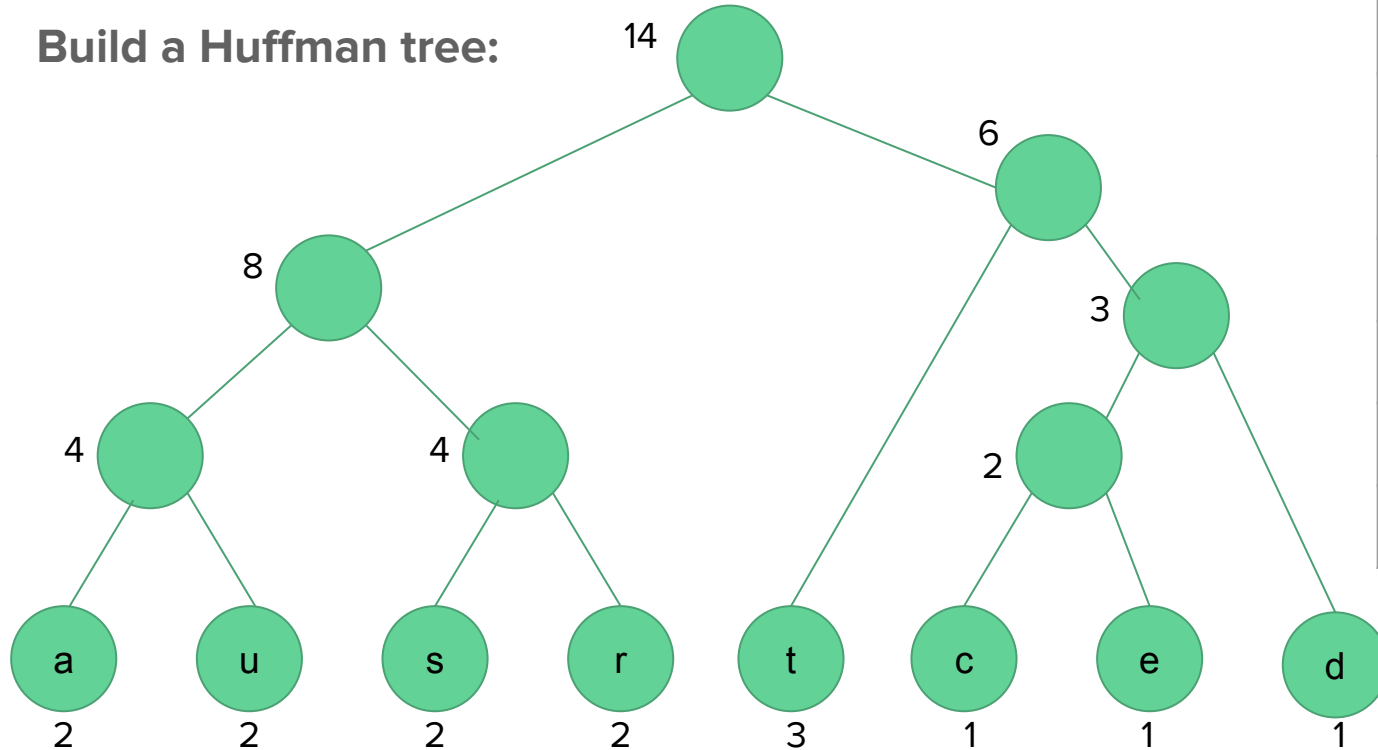
Build a Huffman tree:



character	frequency
t	3
a	2
u	2
s	2
r	2
d	1
c	1
e	1

Huffman coding example

Build a Huffman tree:



character	frequency
t	3
a	2
u	2
s	2
r	2
d	1
c	1
e	1

Huffman coding example

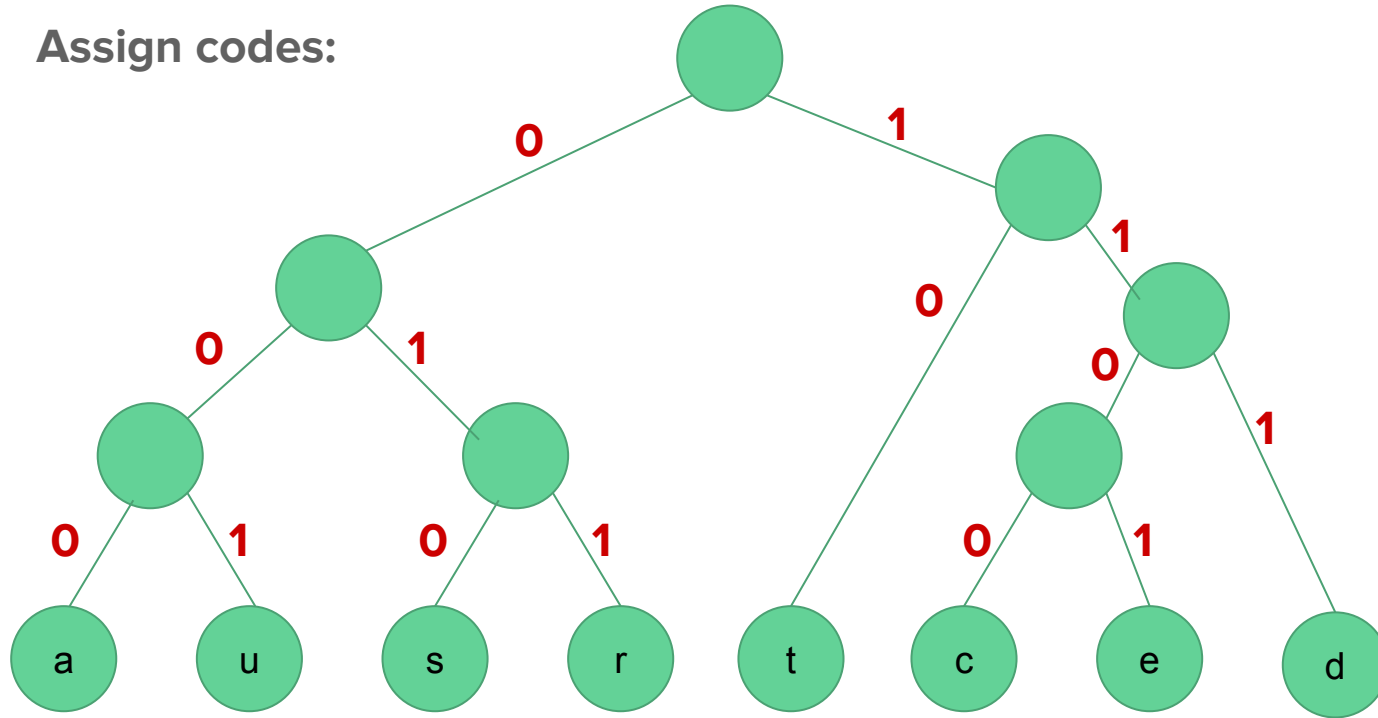
Now we **assign codes** to the tree by **placing a 0 on every left branch and a 1 on every right branch**

A traversal of the tree from root to leaf give the Huffman code for that particular leaf character

These codes are then used to encode the string

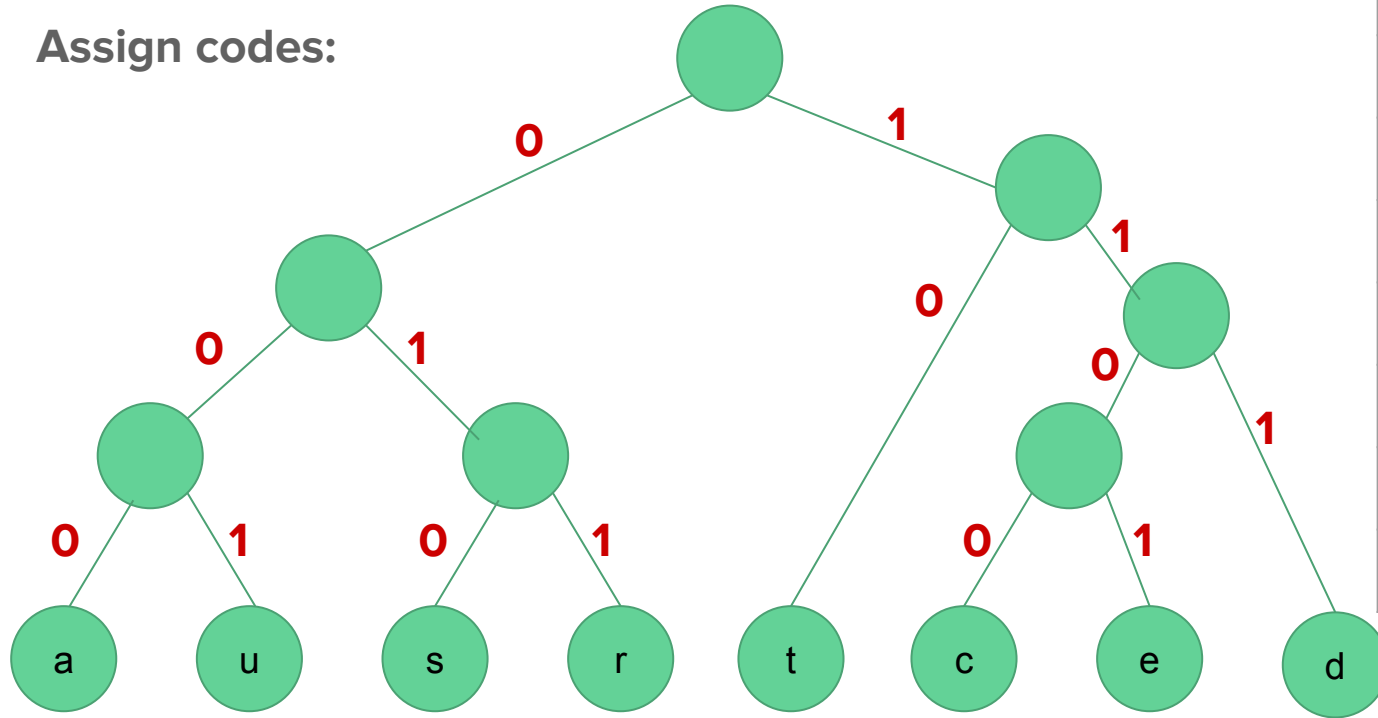
Huffman coding example

Assign codes:



Huffman coding example

Assign codes:



character	Huffman code
t	10
a	000
u	001
s	010
r	011
d	111
c	1100
e	1101

Huffman coding example

Thus “datastructures” turns into

11100010000010100110011100100010111101010

If 8-bit ASCII code had been used instead of Huffman coding, “datastructures” would have been

0110010001100001011101000110000101110011

0111010001110010011101010110001101110100

01110101011100100110010101110011

character	Huffman code	ASCII code
t	10	01110100
a	000	01100001
u	001	01110101
s	010	01110011
r	011	01110010
d	111	01100100
c	1100	01100011
e	1101	01100101

Huffman coding

Uncompression:

Read the file bit by bit

1. Start at the root of the tree
2. If a 0 is read, head left
3. If a 1 is read, head right
4. When a leaf is reached, decode that character and start over again at the root of the tree

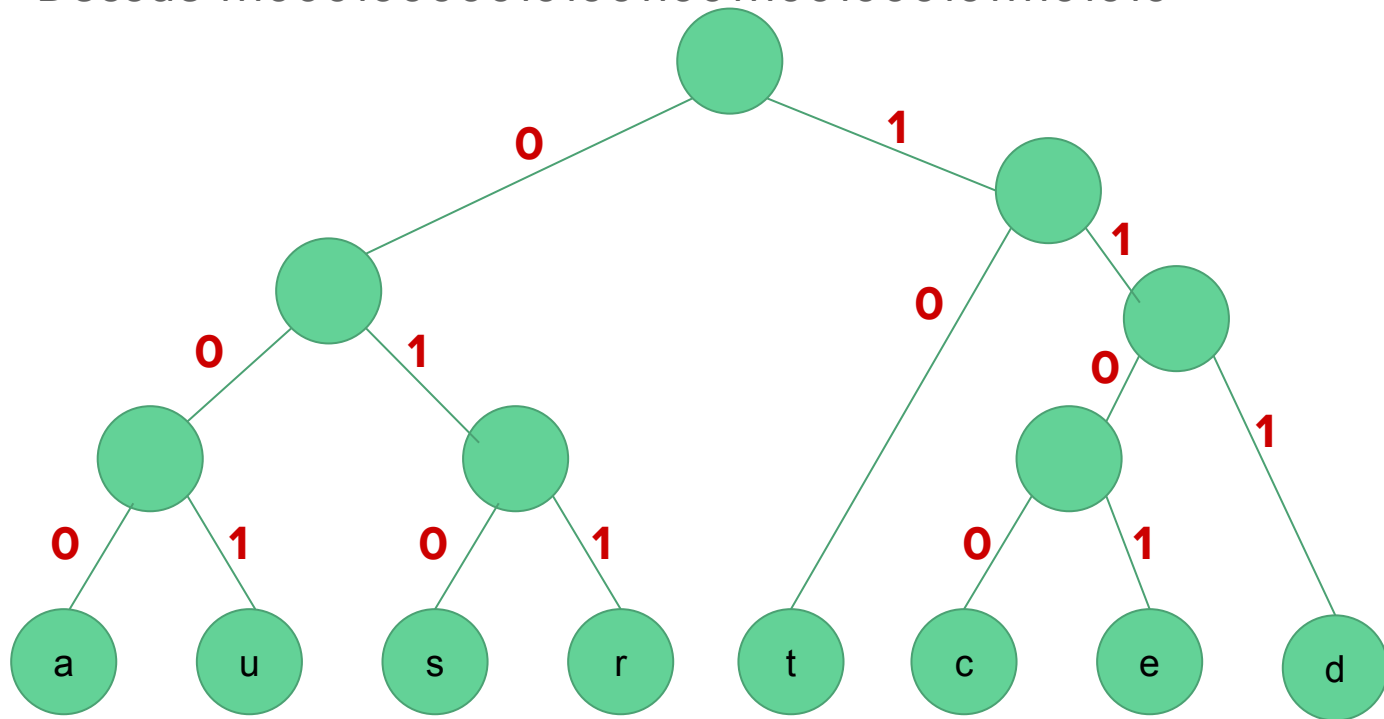
Huffman coding

Uncompression example:

Decode 11100010000010100110011100100010111101010 using the previous Huffman tree

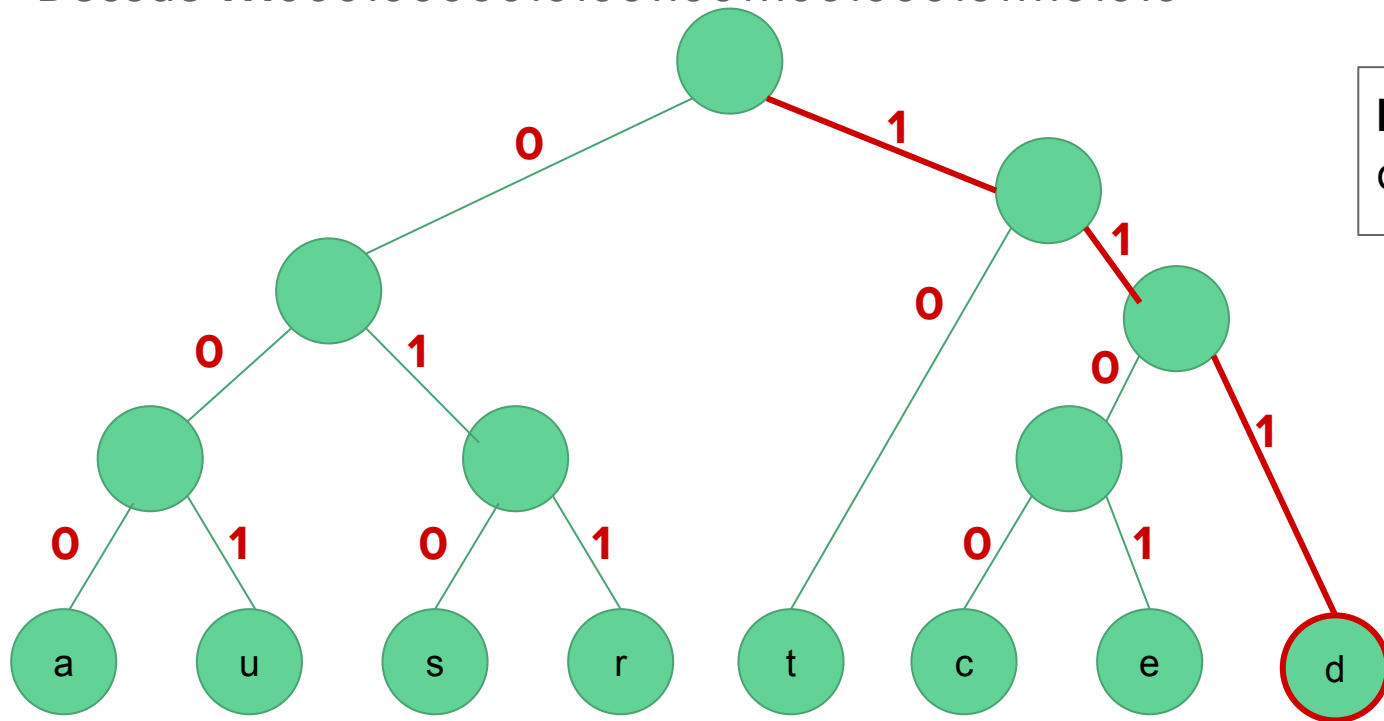
Uncompression example

Decode 111000100000010100110011100100010111101010



Uncompression example

Decode **1**1100010000010100110011100100010111101010

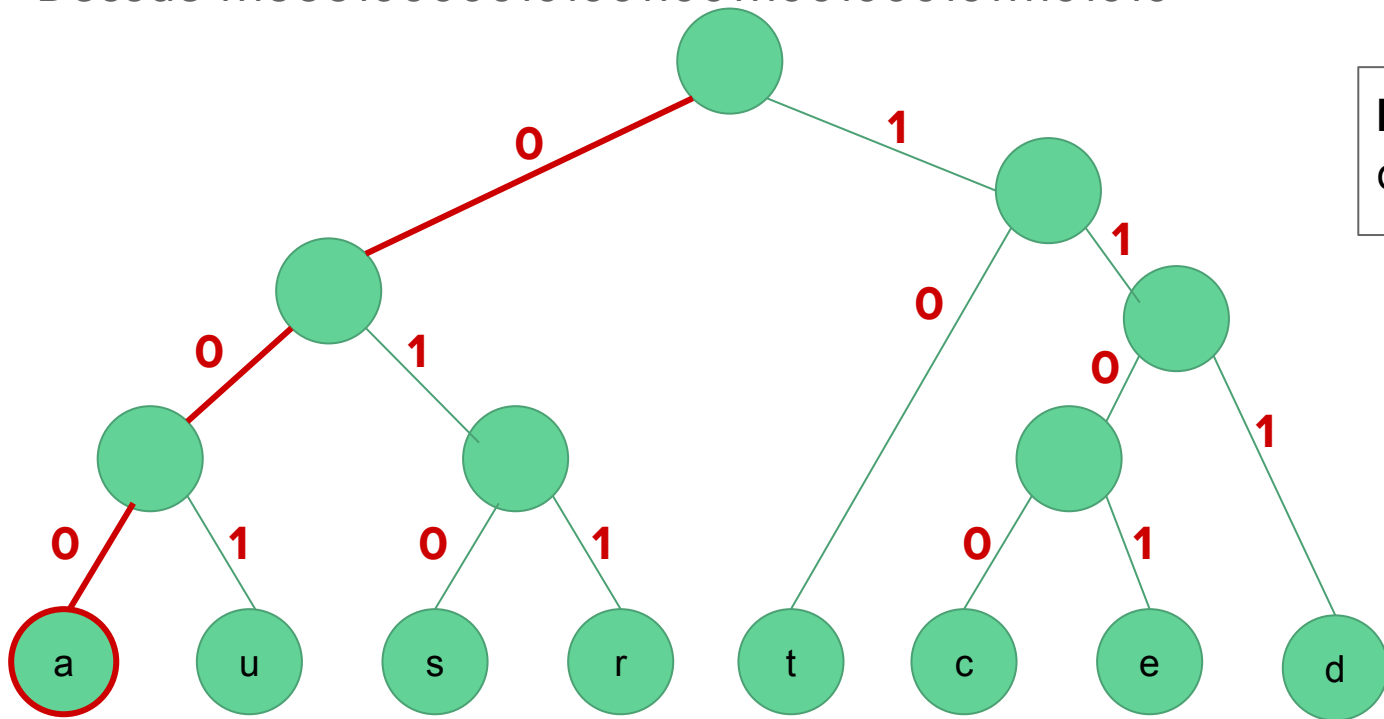


Decoded string:

d

Uncompression example

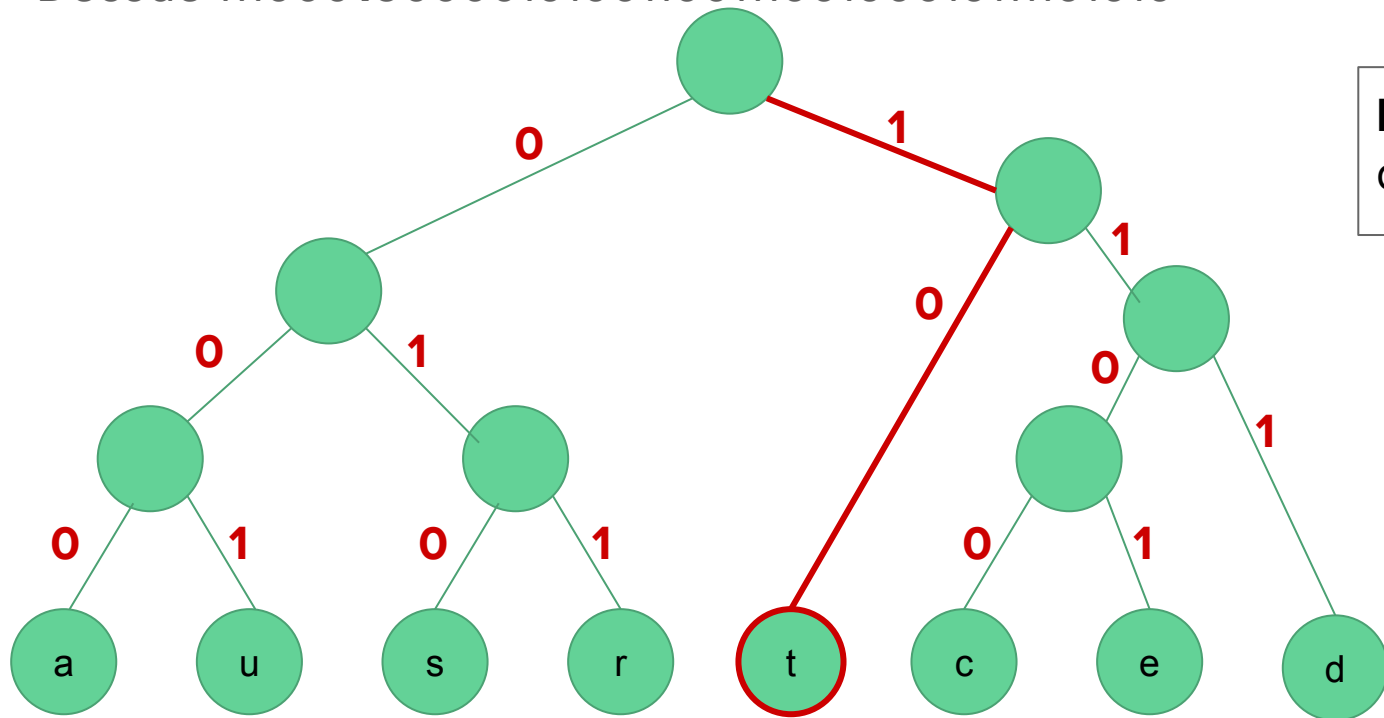
Decode 111**000**1000001010011001100100010111101010



Decoded string:
da

Uncompression example

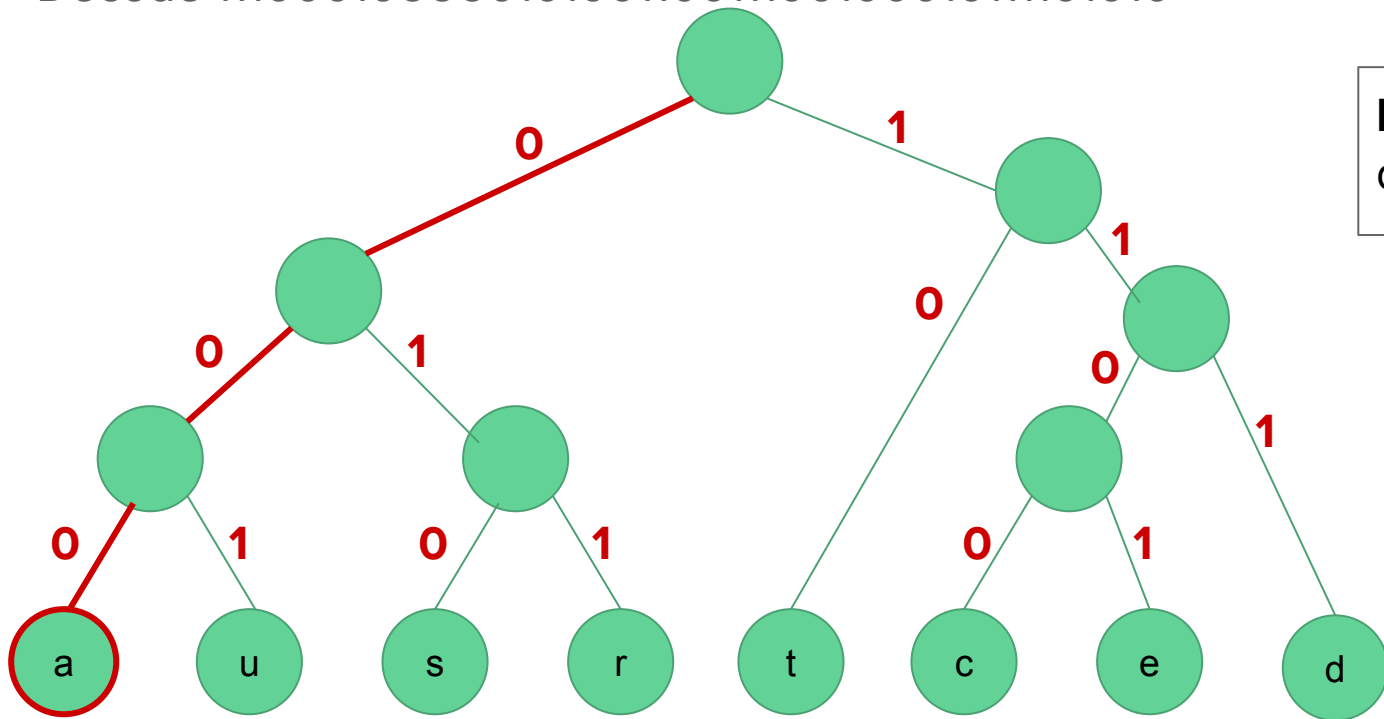
Decode 111000**1**0000010100110011100100010111101010



Decoded string:
dat

Uncompression example

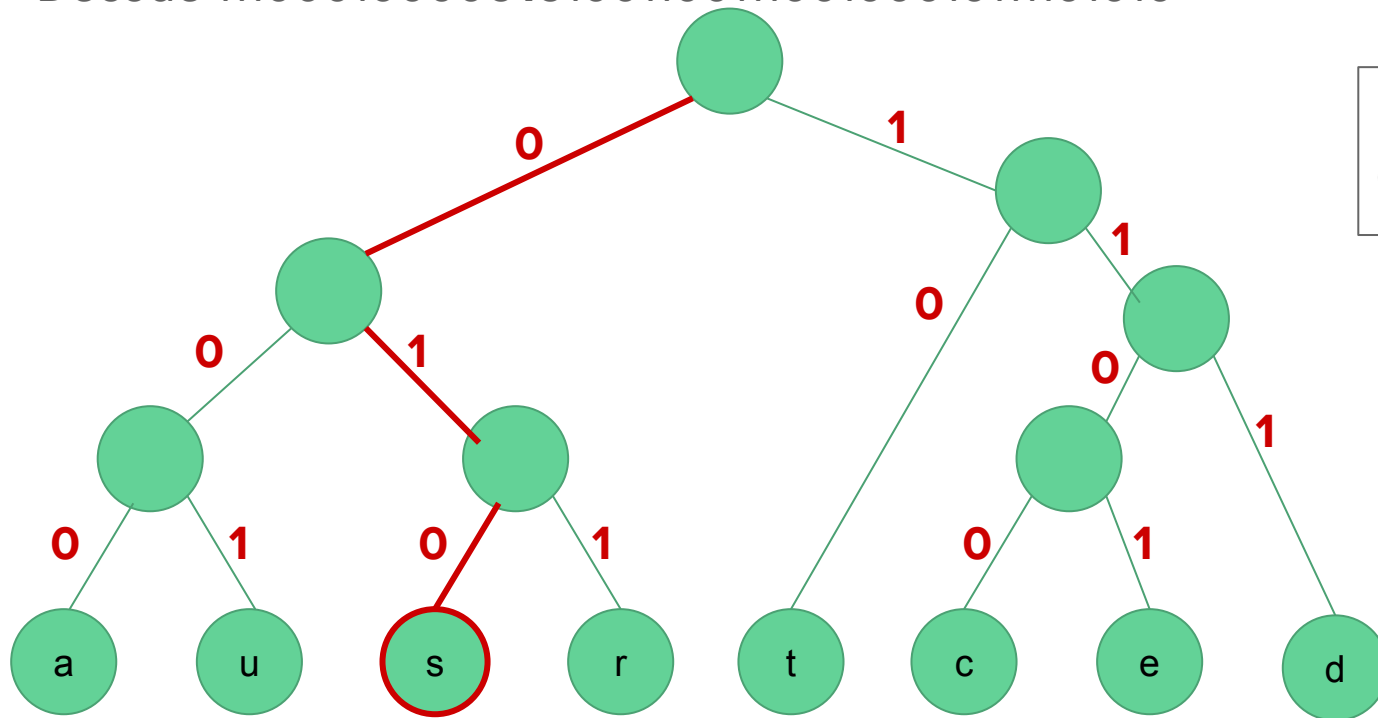
Decode 11100010**000**010100110011100100010111101010



Decoded string:
data

Uncompression example

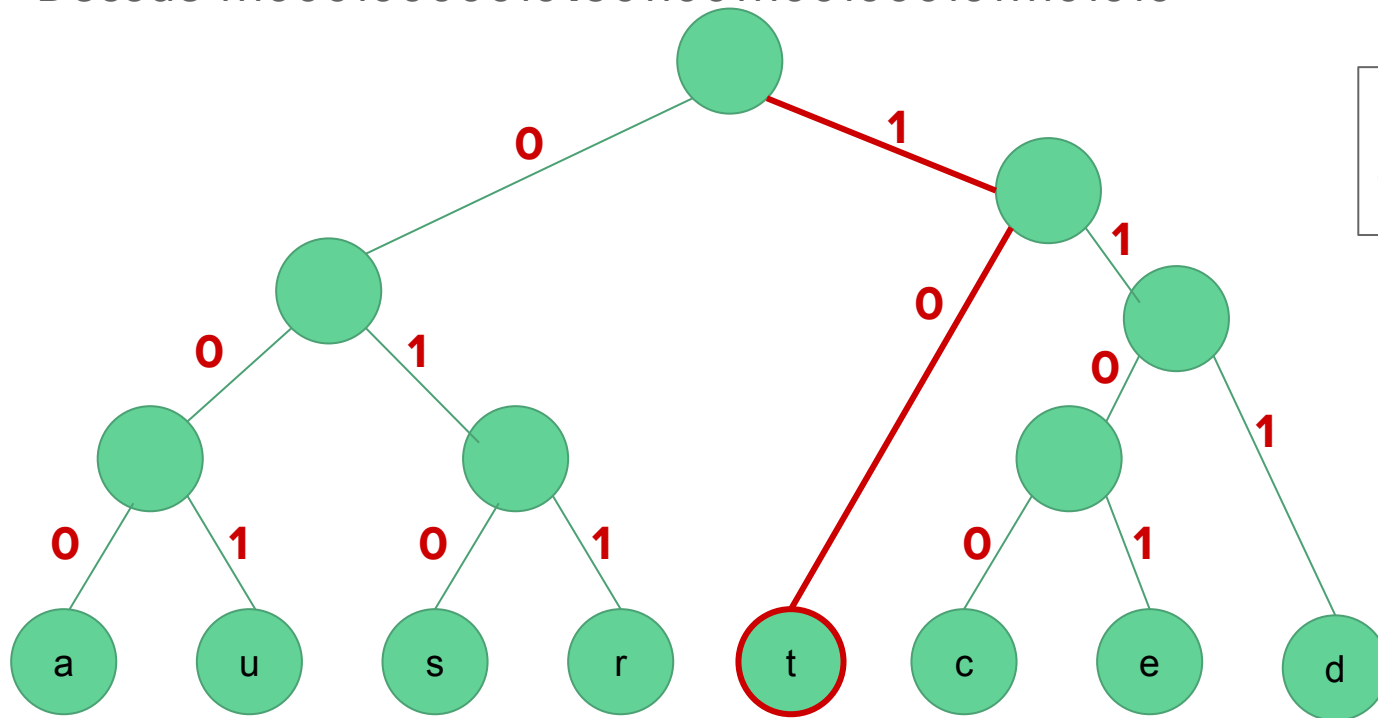
Decode 111000100000**010**10011001100100010111101010



Decoded string:
datas

Uncompression example

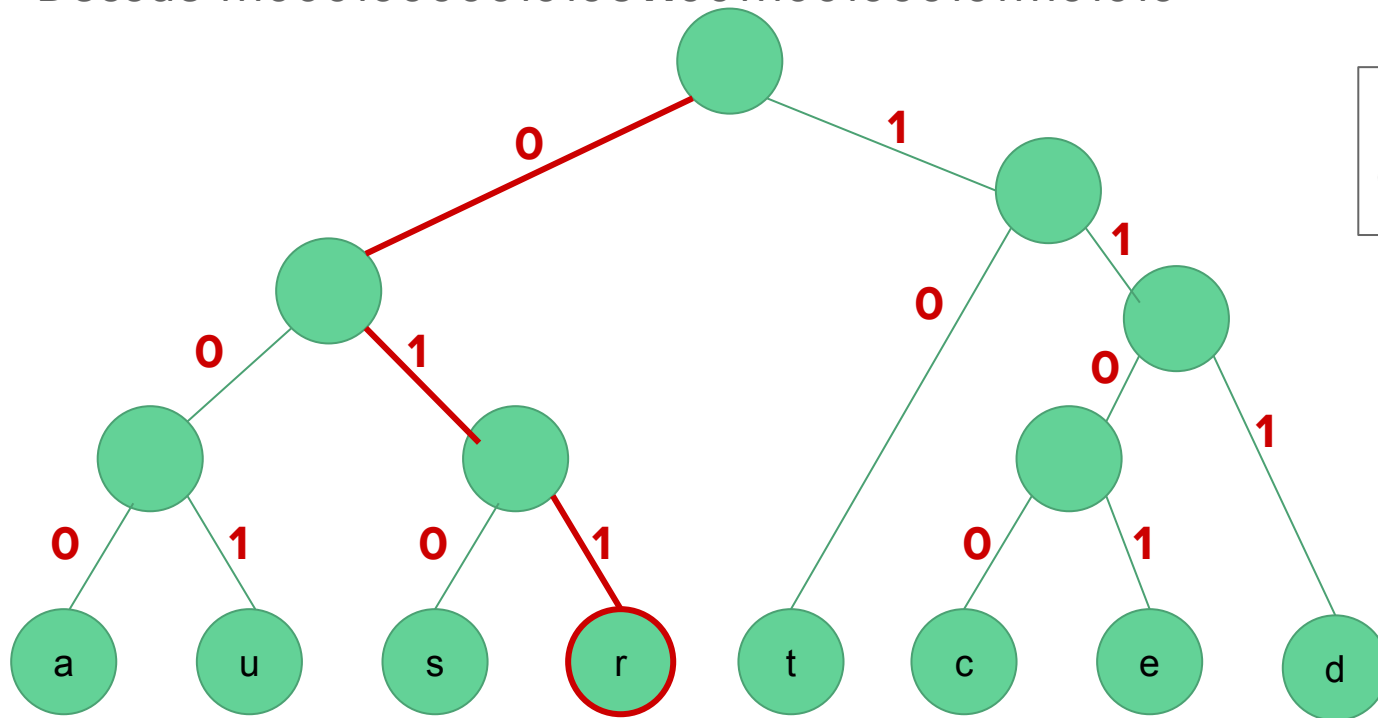
Decode 11100010000010**1**00110011100100010111101010



Decoded string:
datast

Uncompression example

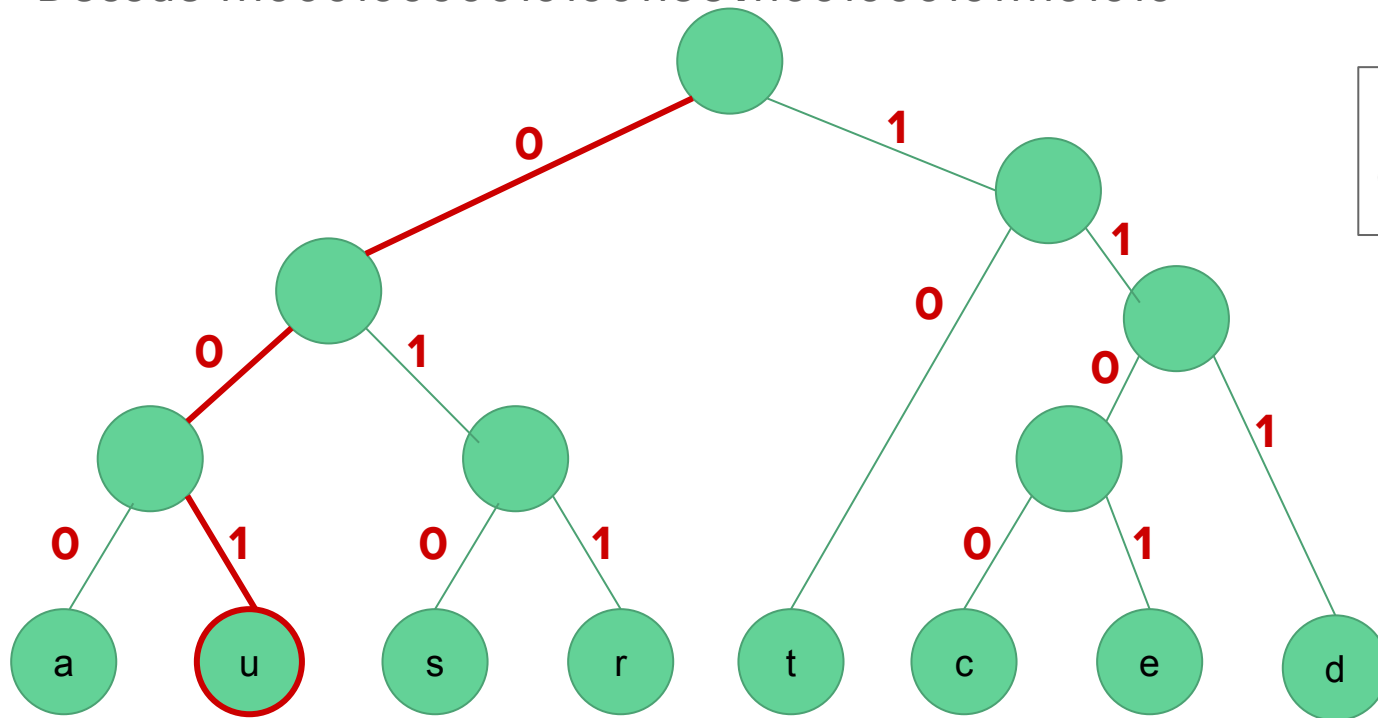
Decode 1110001000001010**0**1110011100100010111101010



Decoded string:
datastr

Uncompression example

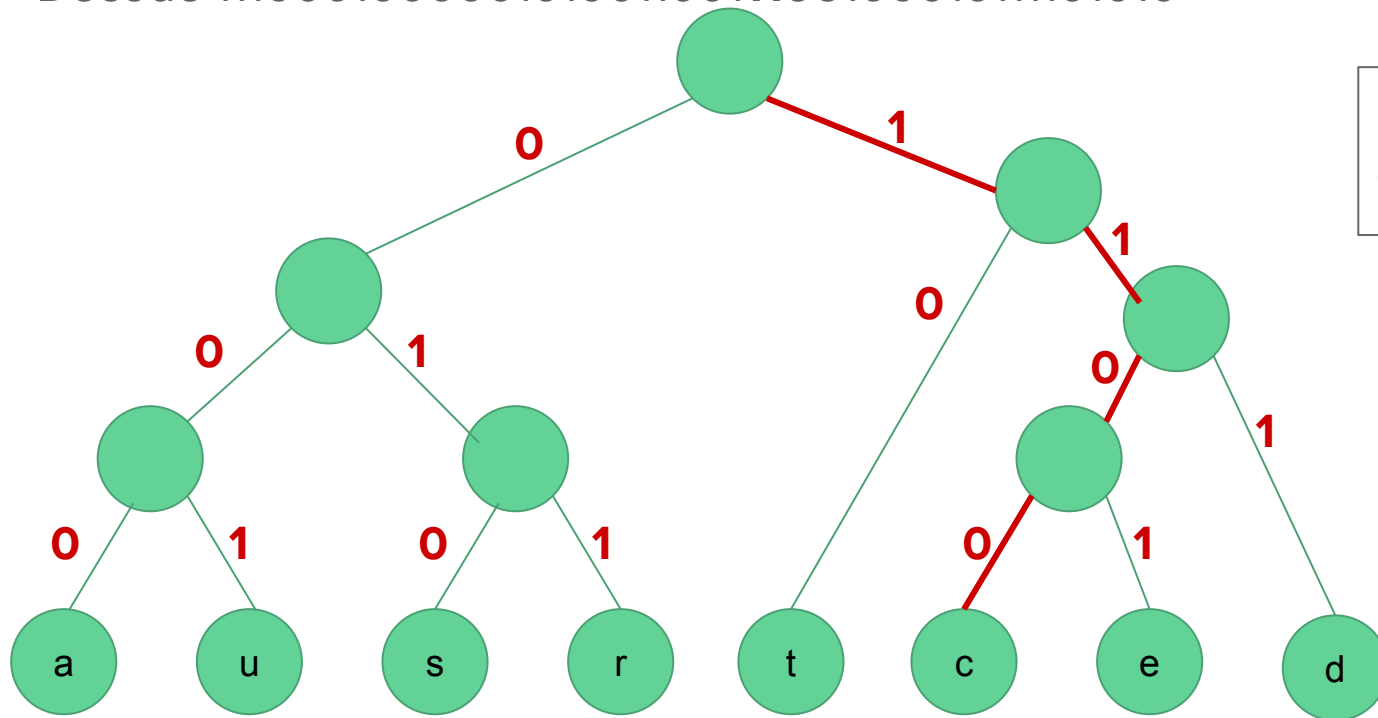
Decode 111000100000010100110011100100010111101010



Decoded string:
datastru

Uncompression example

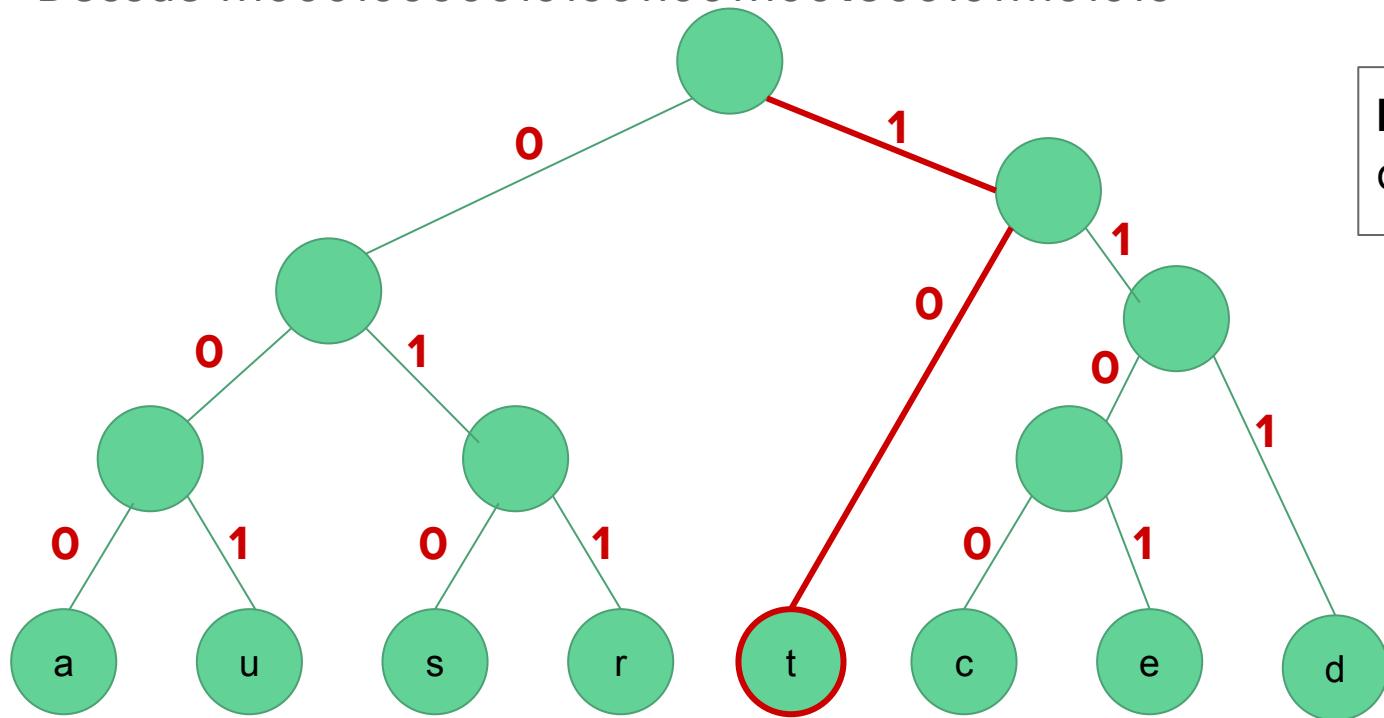
Decode 1110001000001010011001**1100**100010111101010



Decoded string:
datastruc

Uncompression example

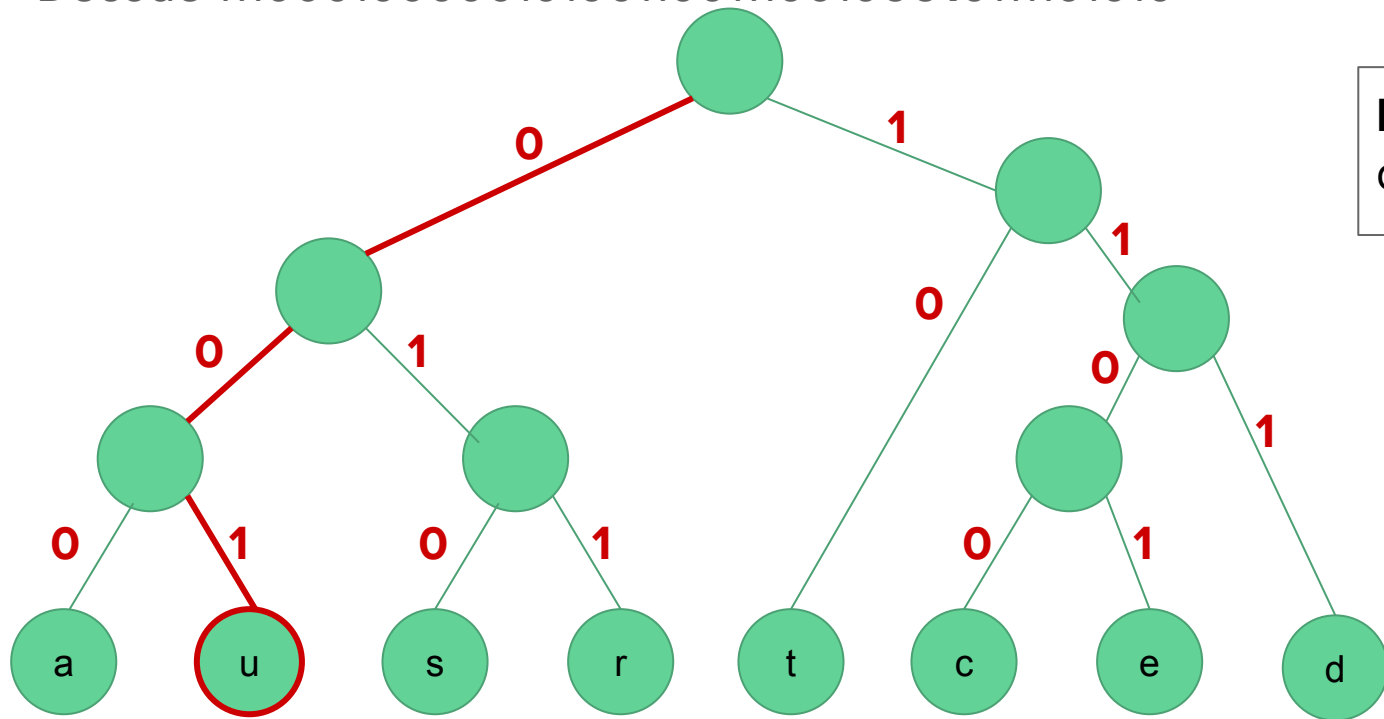
Decode 111000100000101001100111001000010111101010



Decoded string:
datastruct

Uncompression example

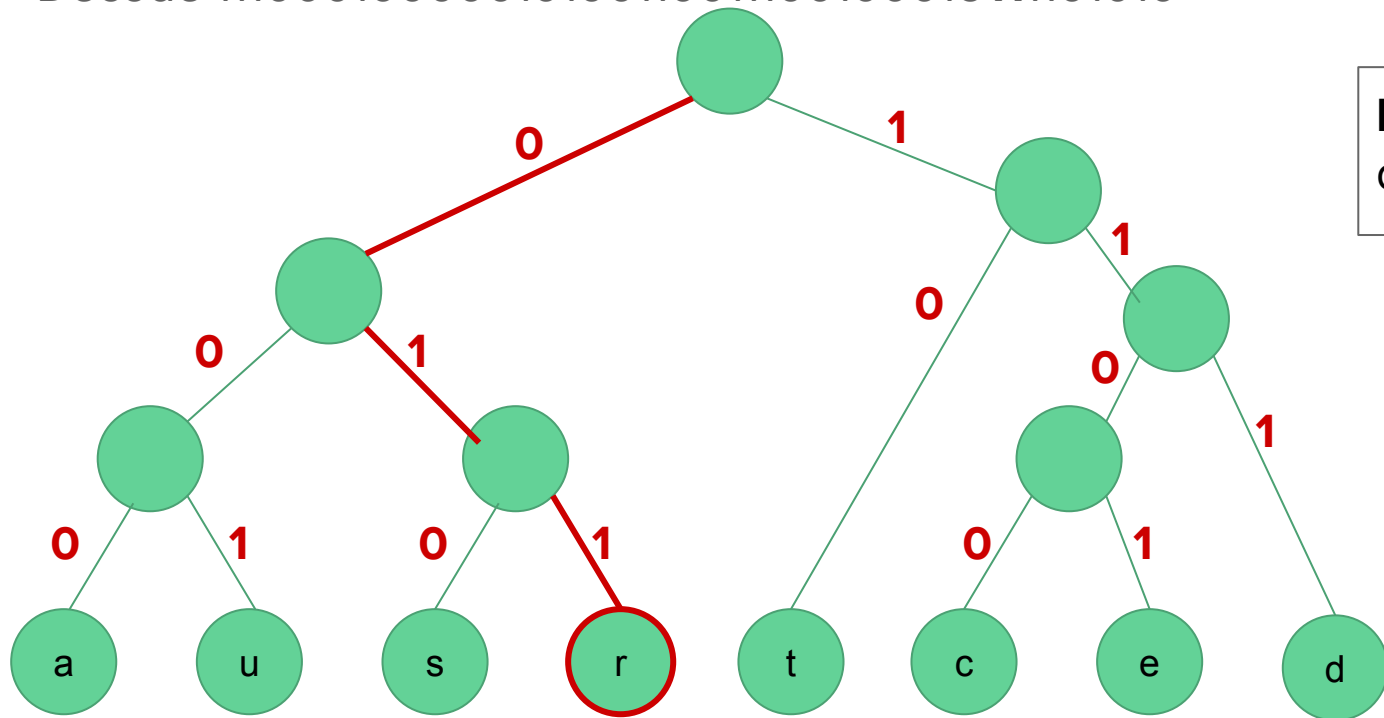
Decode 111000100000010100110011100100**00**10111101010



Decoded string:
datastructu

Uncompression example

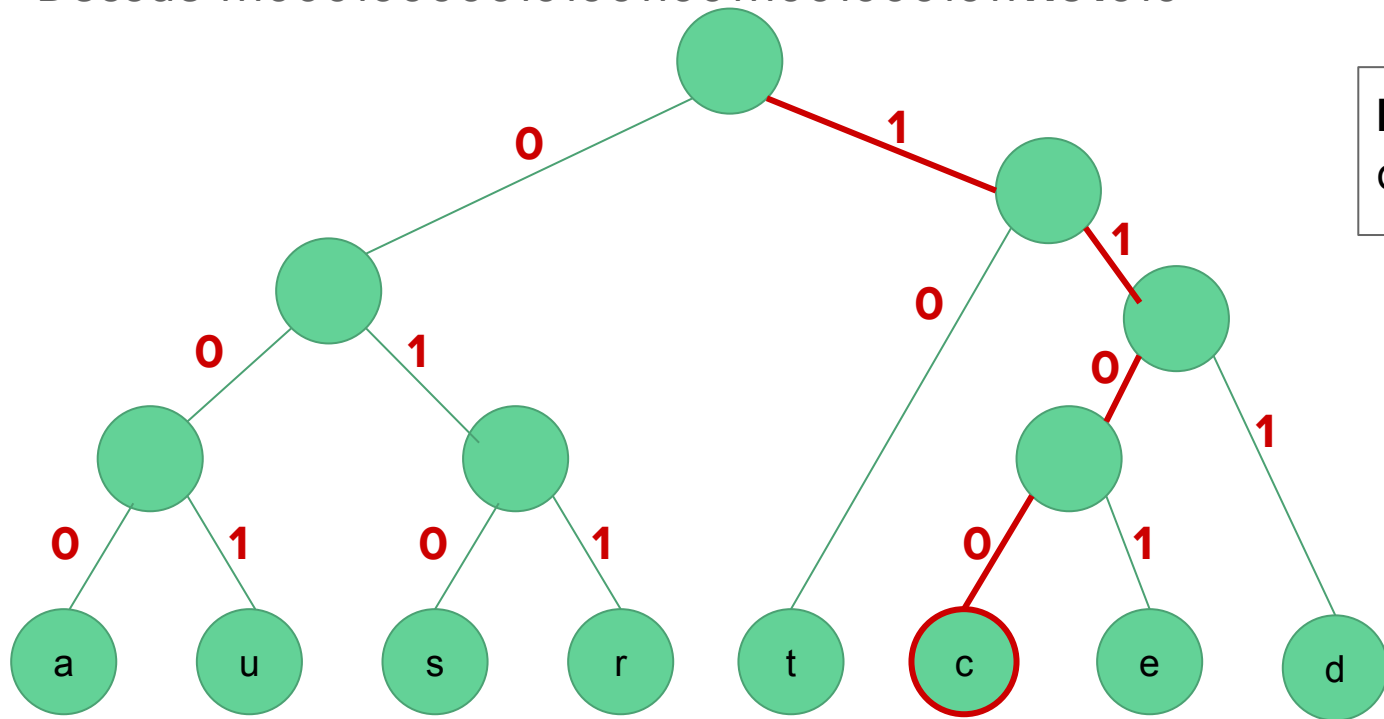
Decode 111000100000010100110011100100010**1**11101010



Decoded string:
datastructur

Uncompression example

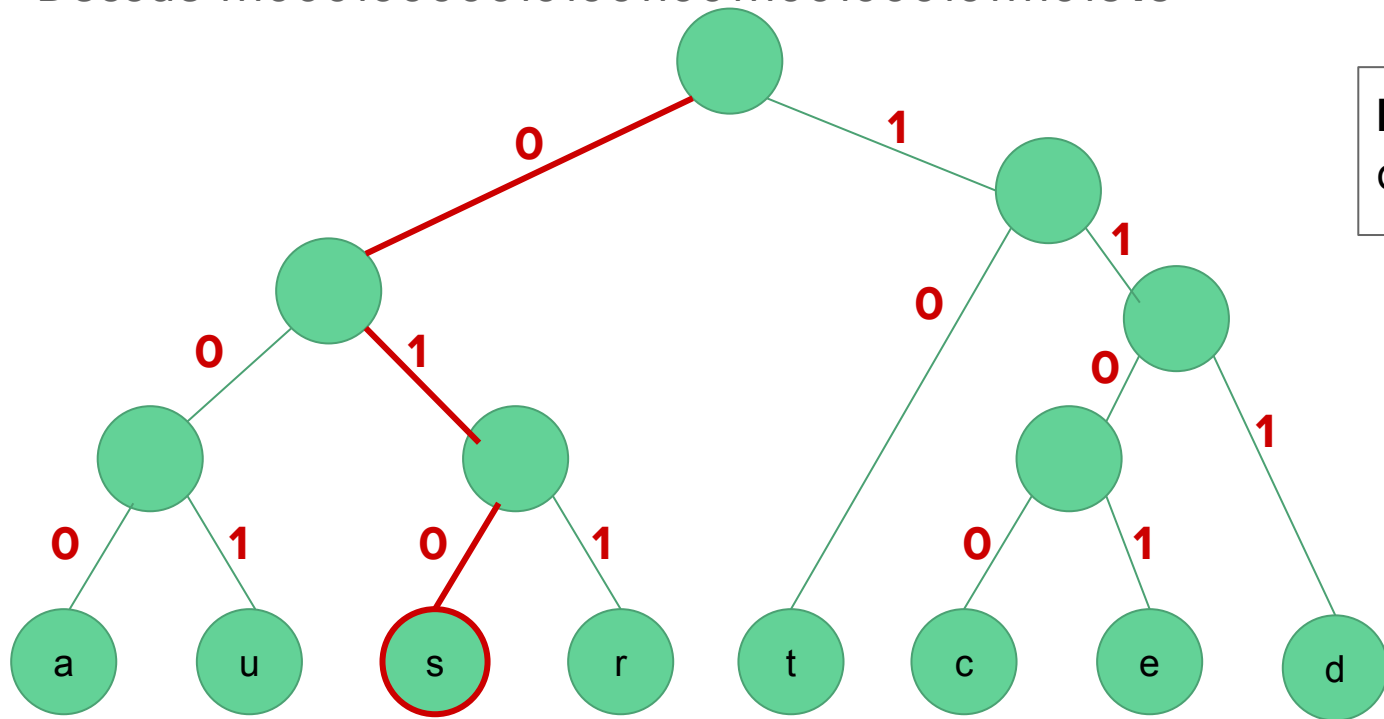
Decode 1110001000001010011001110010001011**110**1010



Decoded string:
datastructure

Uncompression example

Decode 111000100000010100110011100100010111101**010**



Decoded string:
datastructures

Activity-Selection Problem

The problem of scheduling several competing activities that require exclusive use of a common resource, with a goal of selecting a maximum-size set of mutually compatible activities.

Activity-Selection Problem

Input: A set of activities that we wish to use a resource (such as classroom) which can serve only one activity at a time. Each activity a_i in the set $S = \{a_1, a_2, \dots, a_n\}$ has a start time s_i and a finish time f_i , where $0 \leq s_i < f_i < \infty$.

If selected, activity a_i takes place during the time interval $[s_i, f_i)$

Output: A maximum-size subset of **mutually compatible** activities.

Two activities are compatible if and only if their intervals do not overlap.

Activity-Selection

Example

Consider the following set S of activities

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

Activity-Selection

Greedy approach

- Choose an activity that leaves the resource available for as many other activities, i.e.

Choose an activity with the earliest finish time

Activity-Selection

Greedy algorithm:

We assume that n input activities are already ordered by monotonically increasing finish time:

$$f_1 \leq f_2, \leq f_3 \leq \dots \leq f_{n-1} \leq f_n$$

1. Select the activity with the earliest finish time
2. Eliminate the activities that could not be scheduled / incompatible activities
3. Repeat

Activity-Selection: Recursive greedy algorithm

Input: start times s , finish times f , the index k that defines the subproblem S_k it is to solve, and the size n of the original problem

Activity-Selection: Recursive greedy algorithm

Input: start times s , finish times f , the index k that defines the subproblem S_k it is to solve, and the size n of the original problem

```
RECURSIVE-ACTIVITY-SELECTOR( $s, f, k, n$ )  
1   $m = k + 1$   
2  while  $m \leq n$  and  $s[m] < f[k]$       // find the first activity in  $S_k$  to finish  
3       $m = m + 1$   
4  if  $m \leq n$   
5      return  $\{a_m\} \cup \text{RECURSIVE-ACTIVITY-SELECTOR}(s, f, m, n)$   
6  else return  $\emptyset$ 
```

We start with $k = 0$ and a fictitious activity a_0 with $f_0 = 0$

Activity-Selection: Recursive greedy algorithm

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

RECURSIVE-ACTIVITY-SELECTOR(s, f, k, n)

```

1   $m = k + 1$ 
2  while  $m \leq n$  and  $s[m] < f[k]$            // find the first activity in  $S_k$  to finish
3       $m = m + 1$ 
4  if  $m \leq n$ 
5      return  $\{a_m\} \cup \text{RECURSIVE-ACTIVITY-SELECTOR}(s, f, m, n)$ 
6  else return  $\emptyset$ 

```

[illegible]

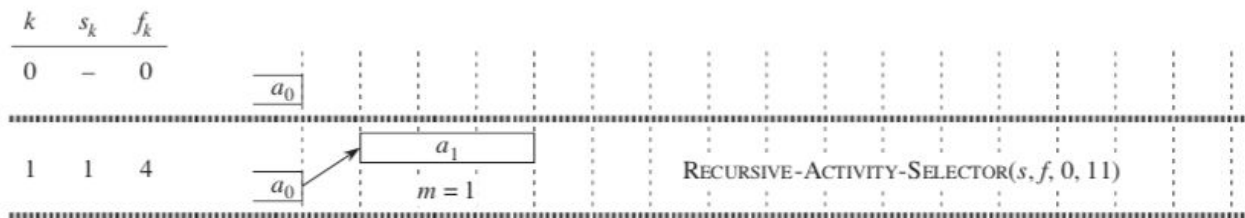
Activity-Selection: Recursive greedy algorithm

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

RECURSIVE-ACTIVITY-SELECTOR(s, f, k, n)

```

1   $m = k + 1$ 
2  while  $m \leq n$  and  $s[m] < f[k]$       // find the first activity in  $S_k$  to finish
3       $m = m + 1$ 
4  if  $m \leq n$ 
5      return  $\{a_m\} \cup \text{RECURSIVE-ACTIVITY-SELECTOR}(s, f, m, n)$ 
6  else return  $\emptyset$ 
```



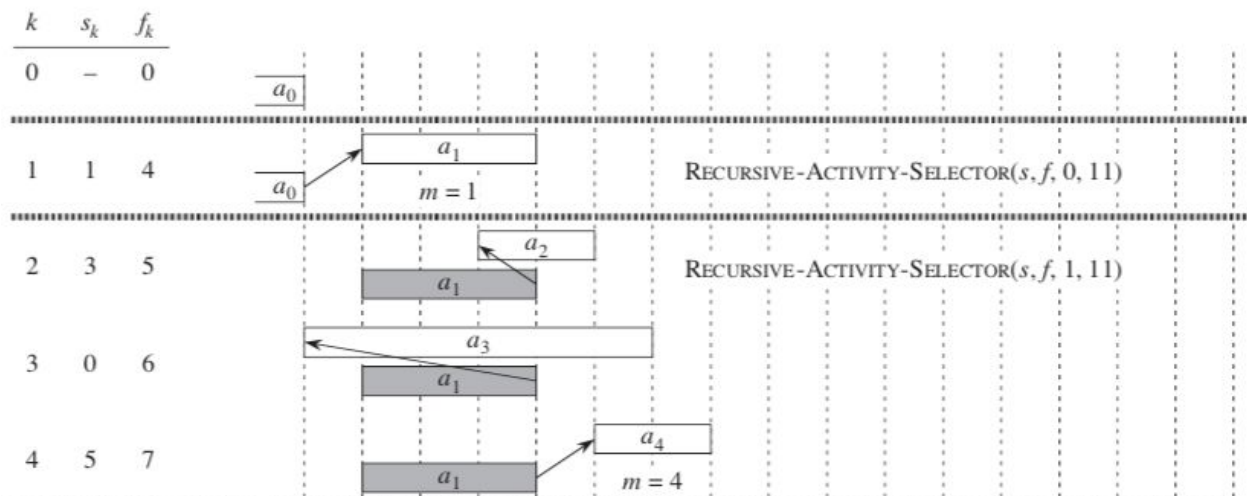
Activity-Selection: Recursive greedy algorithm

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

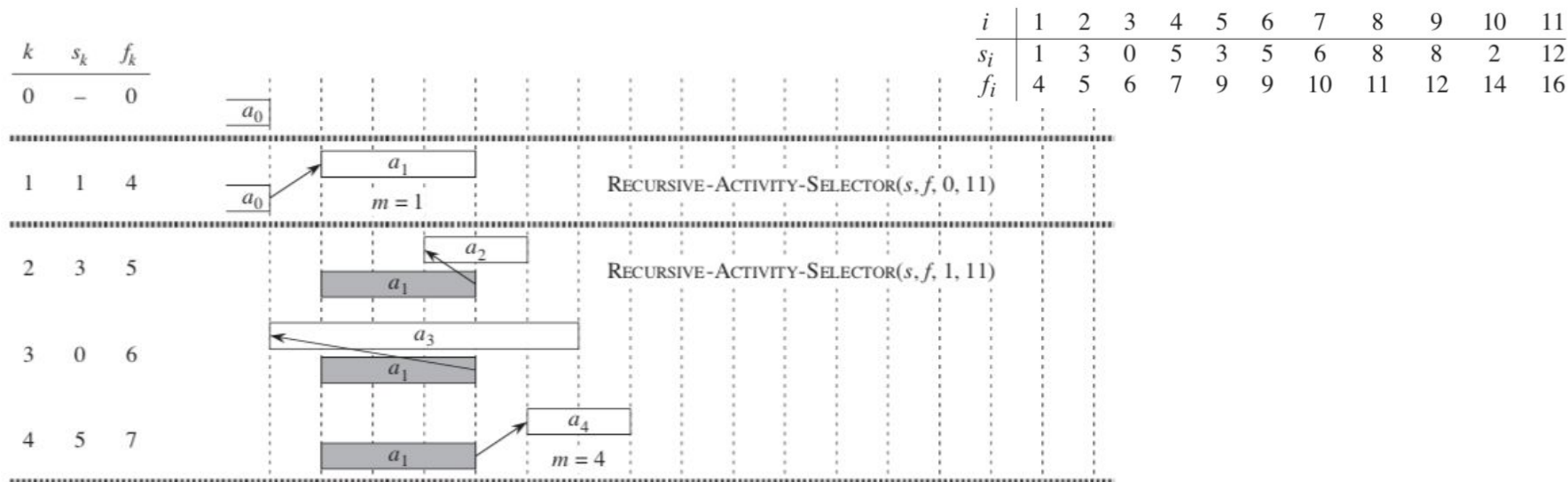
RECURSIVE-ACTIVITY-SELECTOR(s, f, k, n)

```

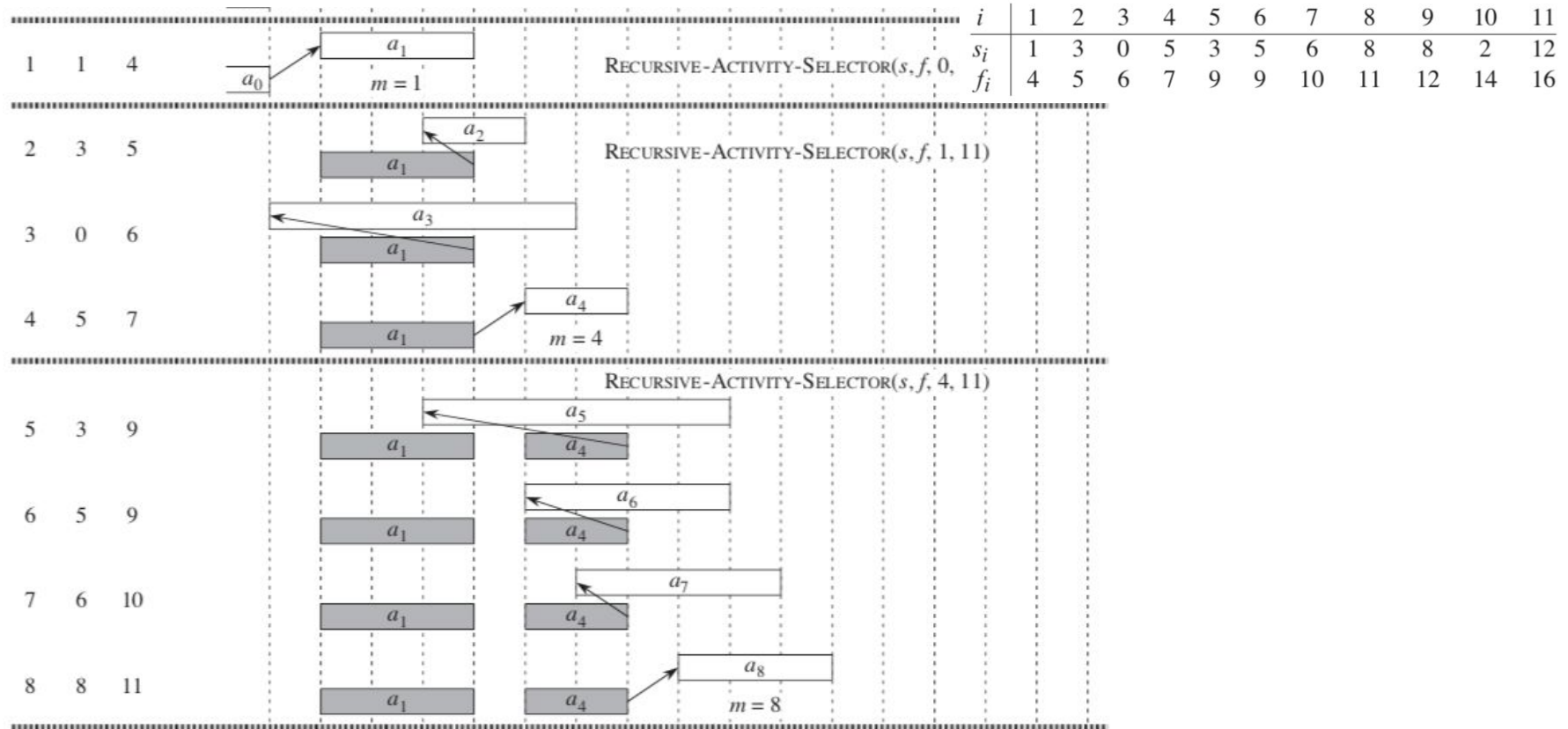
1   $m = k + 1$ 
2  while  $m \leq n$  and  $s[m] < f[k]$       // find the first activity in  $S_k$  to finish
3       $m = m + 1$ 
4  if  $m \leq n$ 
5      return  $\{a_m\} \cup \text{RECURSIVE-ACTIVITY-SELECTOR}(s, f, m, n)$ 
6  else return  $\emptyset$ 
```



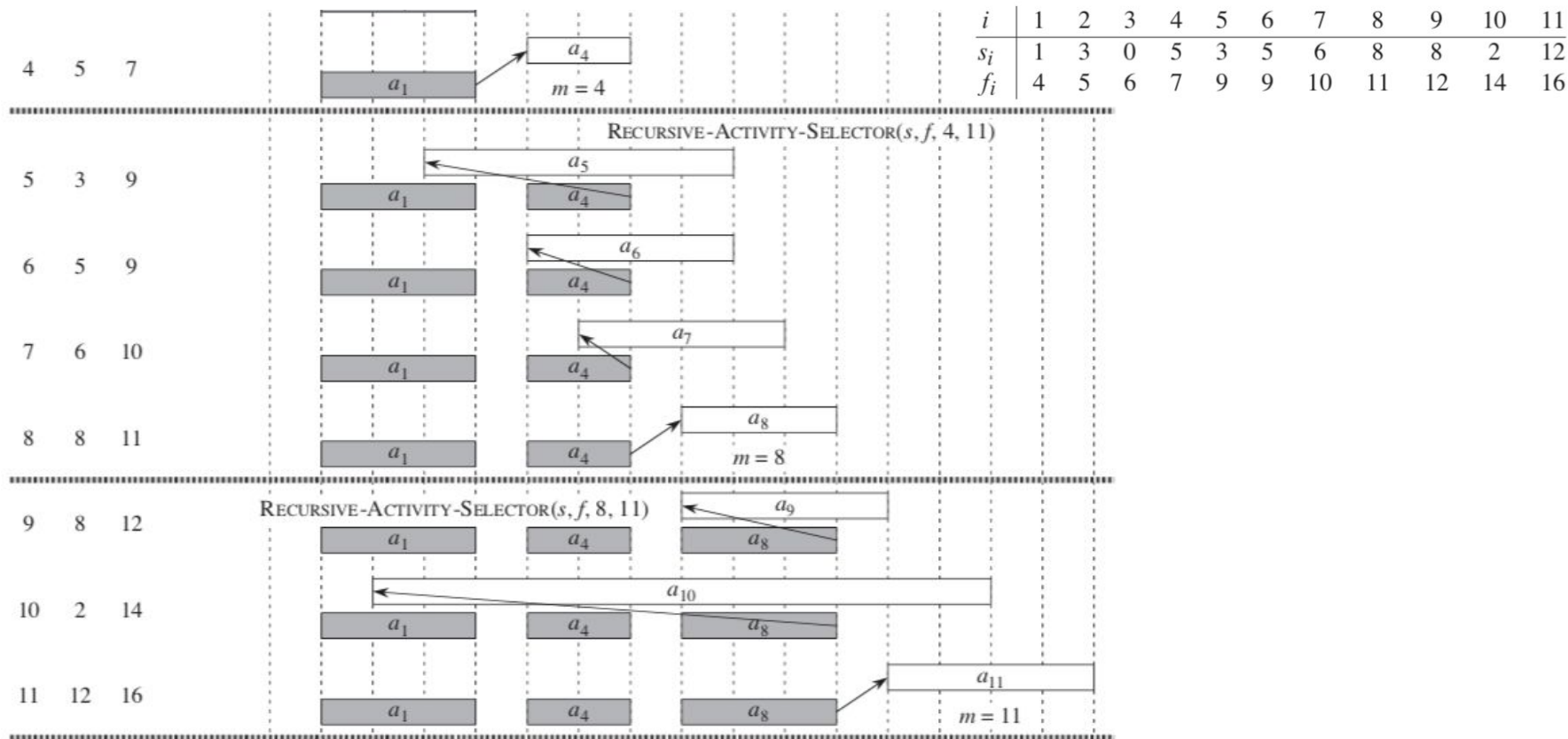
Activity-Selection: Recursive greedy algorithm



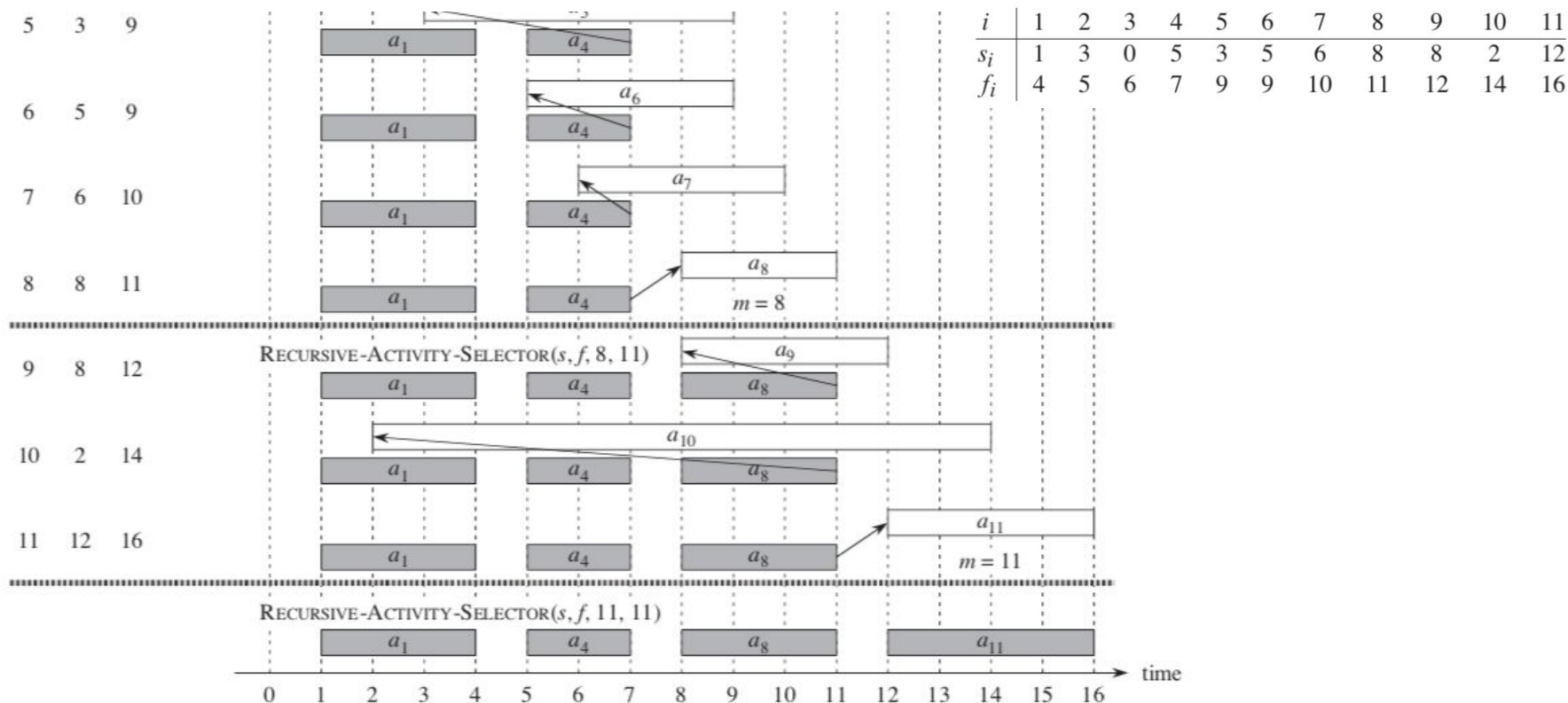
Activity-Selection: Recursive greedy algorithm



Activity-Selection: Recursive greedy algorithm



Activity-Selection: Recursive greedy algorithm



Activity-Selection: Iterative greedy algorithm

GREEDY-ACTIVITY-SELECTOR(s, f)

```
1   $n = s.length$ 
2   $A = \{a_1\}$ 
3   $k = 1$ 
4  for  $m = 2$  to  $n$ 
5      if  $s[m] \geq f[k]$ 
6           $A = A \cup \{a_m\}$ 
7           $k = m$ 
8  return  $A$ 
```

Divide-and-conquer

Divide-and-conquer

The divide-and-conquer strategy solves a problem by:

1. Breaking it into subproblems that are themselves smaller instances of the same type of problem
2. Recursively solving these subproblems
3. Appropriately combining their answers

Divide-and-conquer examples

- Merge sort (already studied)
- Quick sort (already studied)
- Power algorithm:

Compute b^n as $b^{n/2} * b^{n/2}$

$b^n = 1$ if $n = 0$

$b^n = b^{n/2} * b^{n/2}$ if $n > 0$ and n is even

$b^n = b * b^{n/2} * b^{n/2}$ if $n > 0$ and n is odd

Power algorithm using divide-and-conquer strategy

```
power(b, n)
{
    if (n == 0)
        return 1;
    else {
        int p = power(b, n/2);
        if (n % 2 == 0)
            return p * p;
        else
            return b * p * p;
    }
}
```

Optimal substructure

A problem is said to have optimal substructure if an optimal solution can be constructed from optimal solutions of its subproblems.

Assignment

1. Improve the power algorithm to work with negative powers, i.e. it should be able to calculate a^{-n} .

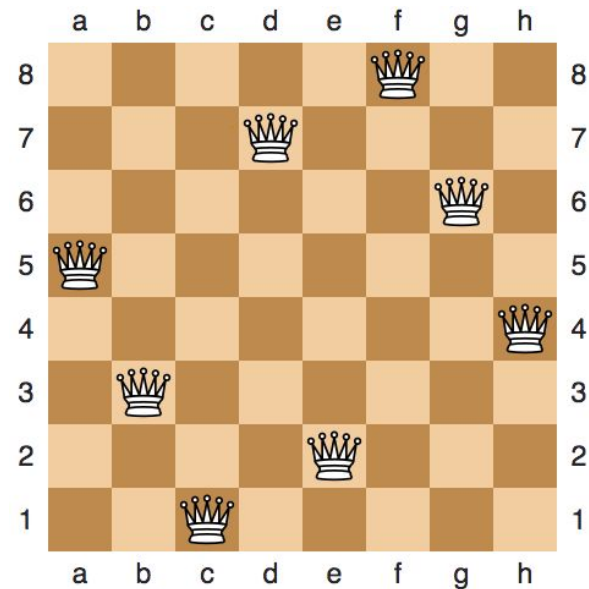
Backtracking

Backtracking

Examples of problems which can be solved using backtracking:

1. N-queens problem

The problem of placing N chess queens on an NxN chessboard so that no two queens attack each other.



Source: https://en.wikipedia.org/wiki/Eight_queens_puzzle

Backtracking

Examples of problems which can be solved using backtracking (Contd.):

2. **Sudoku**

The problem of filling a 9x9 grid with digits so that each column, each row, and each of the nine 3x3 subgrids that compose the grid contains all of the digits from 1 to 9.

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

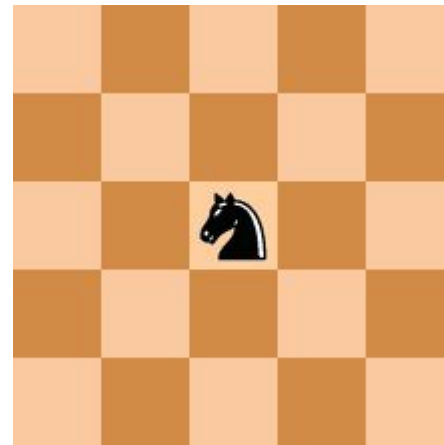
Source: <https://en.wikipedia.org/wiki/Sudoku>

Backtracking

Examples of problems which can be solved using backtracking (Contd.):

3. **The Knight's tour problem**

Moving a knight on a chessboard such that the knight visits every square only once.



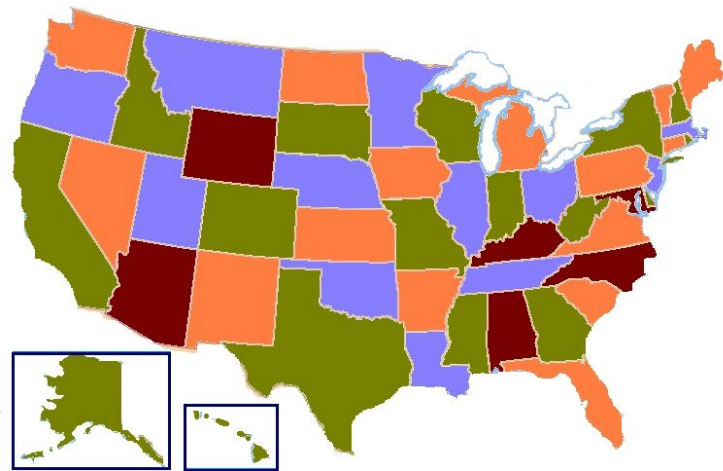
Source: https://en.wikipedia.org/wiki/Knight%27s_tour

Backtracking

Examples of problems which can be solved using backtracking (Contd.):

4. **Map coloring**

Coloring each country/state/entity in a map with a color from the given set of colors, such that no two adjacent countries/states/entities have the same color.



Source: https://en.wikipedia.org/wiki/Four_color_theorem

Backtracking

Examples of problems which can be solved using backtracking (Contd.):

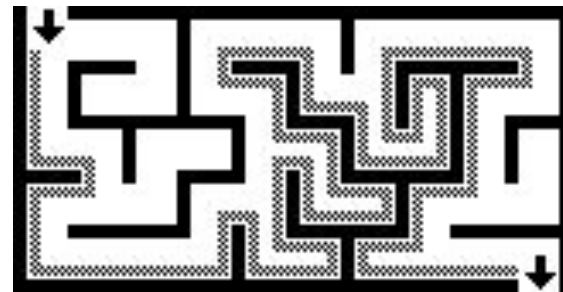
5. Maze solving

Given a maze, find a path from start to finish.

At each intersection, one has to decide between

Three or fewer choices:

- Go straight
- Go left
- Go right



Source: https://en.wikipedia.org/wiki/Maze_solving_algorithm

Backtracking

An approach to solving problems which requires that a series of decision, among various choices, be made where

- We don't have enough information to know what to choose
- Each decision leads to a new set of choices
- Some sequence of choices may be a solution to our problem

Backtracking

An approach to solving constraint-satisfaction problems without trying all possibilities.

Constraint-satisfaction problems require that all the solutions satisfy a complex set of constraints.

Constraints may be explicit or implicit.

Backtracking

The desired solution is expressible as an n -tuple $(x_1, \dots, x_n)$ where the x_i are chosen from some finite set S_i .

Often the problem to be solved calls for finding one vector that maximizes (minimizes/satisfies) a criterion function $P(x_1, \dots, x_n)$.

Example:

All solutions to the N -queens problem can be represented as n -tuples $(x_1, \dots, x_n)$, where x_i is the column on which each queen i is placed.

Backtracking

All solutions to the N-queens problem can be represented as n-tuples $(x_1, \dots, x_n)$, where x_i is the column on which each queen i is placed.

	1	2	3	4
1			q_1	
2	q_2			
3				q_3
4		q_4		

One solution to the 4-queens problem is (3, 1, 4, 2).

Backtracking

Explicit constraints are rules that restrict each x_i to take on values only from a given set.

Example:

In the N-queens problem, explicit constraints are:

$$S_i = \{1, 2, \dots, n\}$$

$$1 \leq x_i \leq n$$

Backtracking

Implicit constraints are rules that determine which of the tuples in the solution space of I satisfy the criterion function.

They describe the way in which the x_i must relate to each other.

Example:

In the N-queens problem, the implicit constraints are

1. No two x_i 's can be the same.
2. No two queens can be on the same diagonal.

N-Queens problem

- N queens are to be placed on an $N \times N$ chessboard without attacking each other.
- All solutions to the N-queens problem can be represented as n-tuples $(x_1, \dots, x_n)$, where x_i is the column on which each queen i is placed.
- Explicit constraints:
 - $S_i = \{1, 2, \dots, n\}$
 - $1 \leq x_i \leq n$
- Implicit constraints:
 - No two x_i 's can be the same.
 - No two queens can be on the same diagonal

N-Queens problem

Possible solutions / Solution space:

(1, 2, 3, 4)

(1, 3, 2, 4)

(1, 3, 4, 2)

and so on.

That is, the solution space consists of all $n!$ Permutations of the n -tuple $(1, 2, \dots, n)$.

Permutation tree

Searching the solution space is facilitated by using a tree organization.

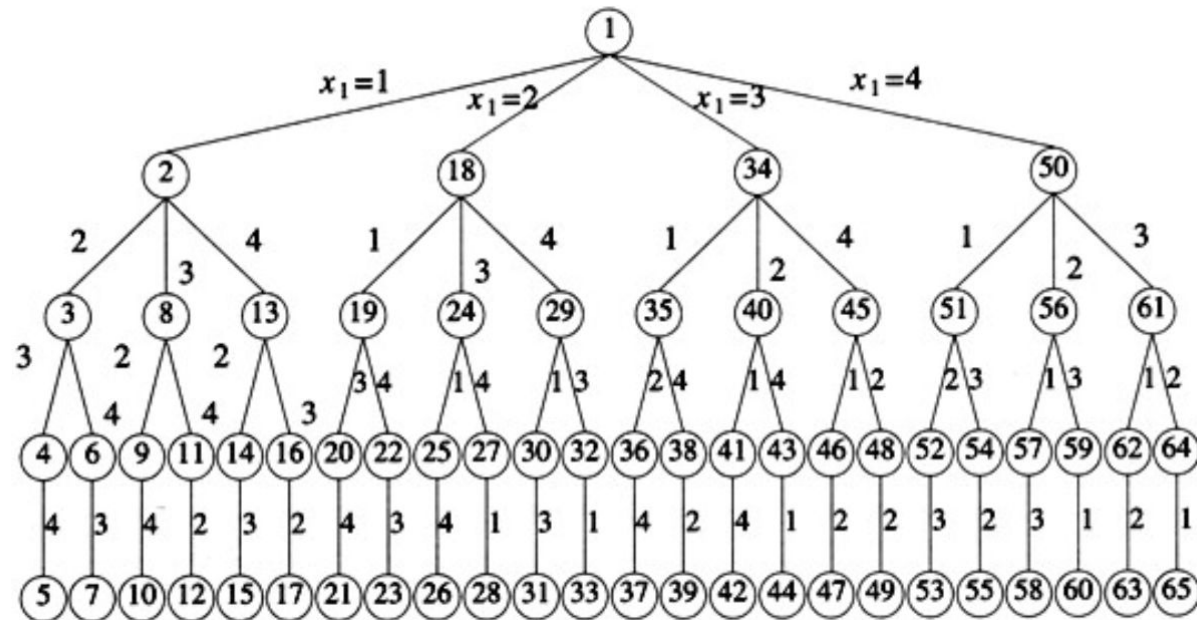
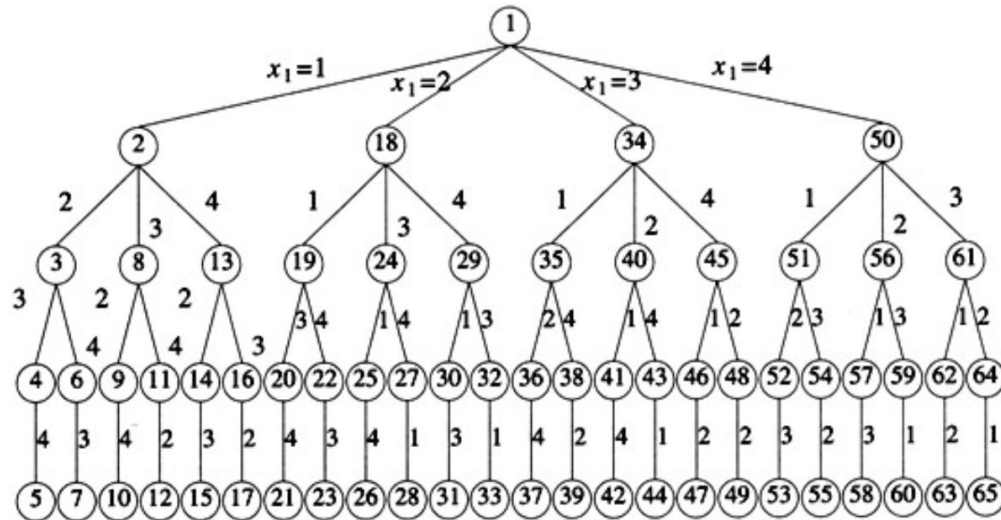


Fig: State space tree of 4-queens problem. Nodes are numbered as in depth first search.

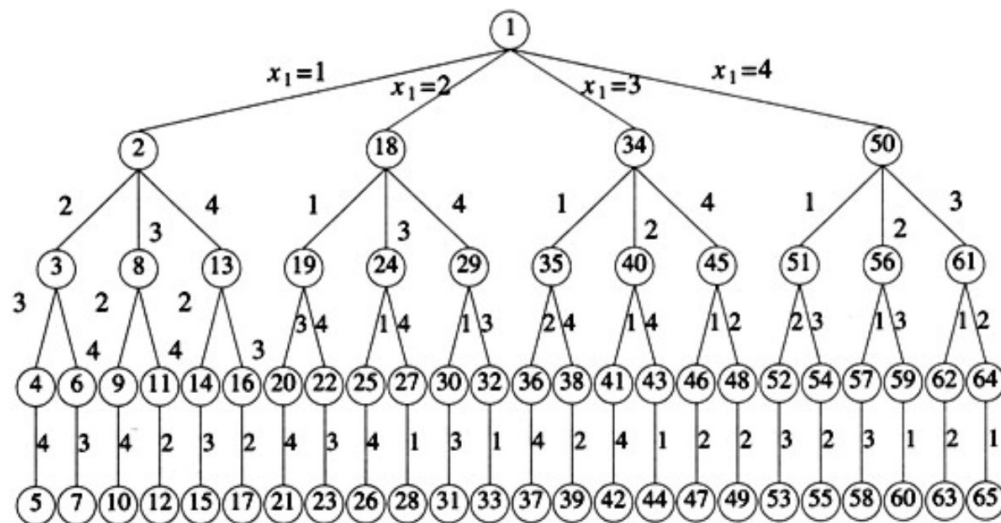
Permutation tree

A brute-force algorithm searches the whole tree, but with backtracking, we got to throw away massive parts of the tree (prune the tree) when we discover a partial solution cannot be extended to a complete solution.



Permutation tree

- **Edges** from level i to $i+1$: possible values of x_i
- Each **node** is a **partial solution**
- **Solution space** is defined by all paths from the root node to a leaf node.



N-queens problem: Backtracking algorithm

1. Place a queen on the first available square in row 1.
2. Move onto the next row, placing a queen on the first available square there (that doesn't conflict with the previously placed queens).
3. Continue in this fashion until either:
 - a. you have solved the problem, or
 - b. you get stuck.

When you get stuck, remove the queens that got you there, until you get to a row where there is another valid square to try.

q_1			
		q_2	

N-queens problem: Backtracking algorithm

The problem can be solved by systematically generating nodes of the state space tree, determining which of the nodes are the solutions.

While exploring the tree, the tree is pruned if the partial solution does not satisfy the constraints.

Backtracking begins with the root and generate other nodes in depth-first manner.

N-queens problem: Backtracking algorithm

Terminologies

- **Live node** : a node which has been generated and all of whose children have not yet been generated.
- **E-node** : A live node whose children are currently being generated
- **Dead node** : A generated node which is not to be expanded further or all of whose children have been generated.

N-queens problem: Backtracking algorithm

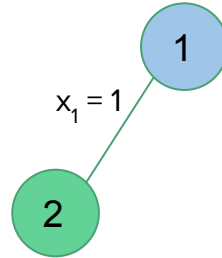
We start with the root node as the only live node. This becomes the E-node and the path is ().

1

N-queens problem: Backtracking algorithm

We start with the root node as the only live node. This becomes the E-node and the path is ().

We generate one child (Node 2) and the path is (1). This corresponds to placing a queen 1 on column 1.

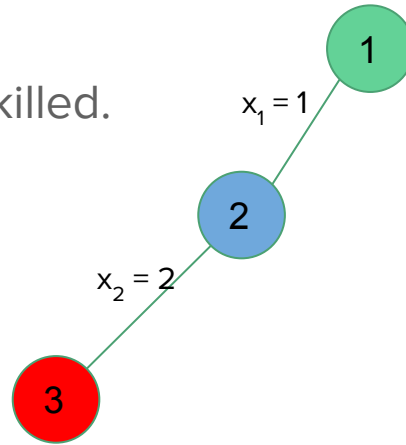


q ₁			

N-queens problem: Backtracking algorithm

Node 2 becomes the E-node.

Node 3 is generated and immediately killed.

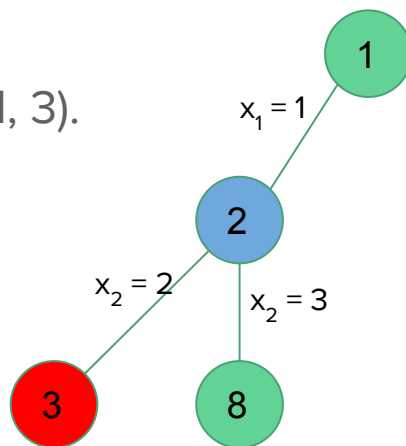


q_1			

N-queens problem: Backtracking algorithm

Node 2 is still the E-node.

Node 3 is generated and the path is (1, 3).

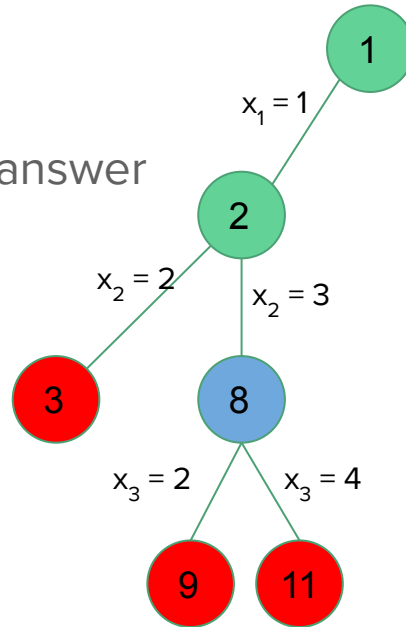


q_1			
		q_2	

N-queens problem: Backtracking algorithm

Node 8 becomes the E-node.

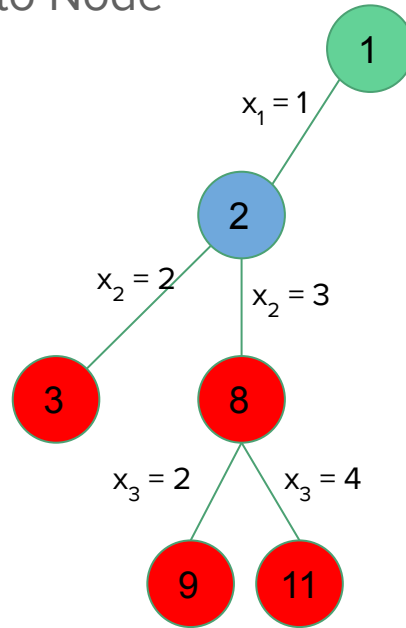
All of its children represent board configurations that cannot lead to an answer node.



q_1			
		q_2	

N-queens problem: Backtracking algorithm

Node 8 gets killed and we backtrack to Node 2.

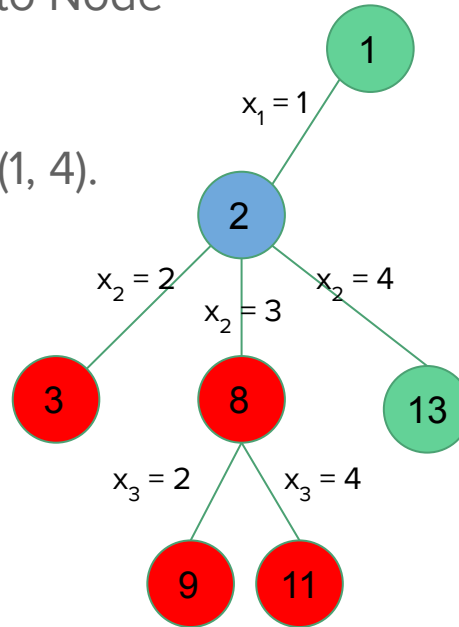


q ₁			

N-queens problem: Backtracking algorithm

Node 8 gets killed and we backtrack to Node 2, which then becomes the E-node.

Node 13 is generated and the path is (1, 4).

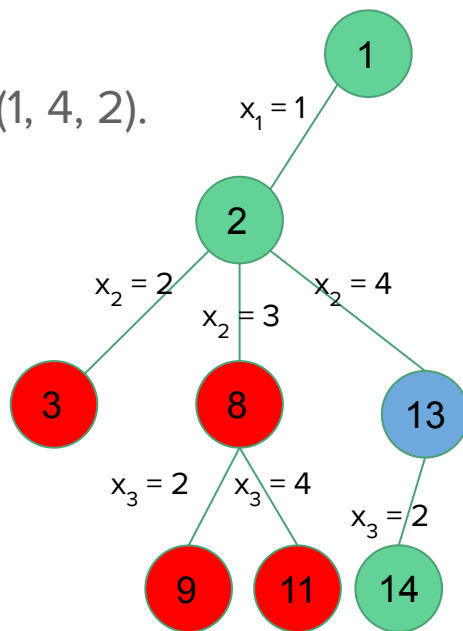


q ₁			
			q ₂

N-queens problem: Backtracking algorithm

Node 13 becomes the E-node.

Node 14 is generated and the path is (1, 4, 2).

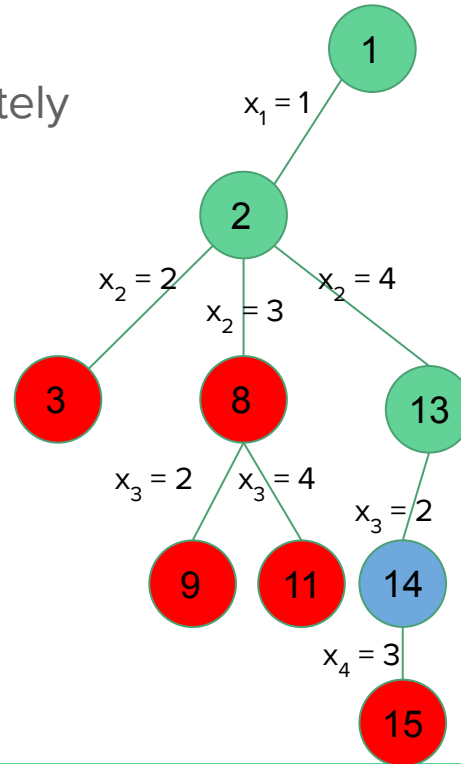


q_1			
			q_2
	q_3		

N-queens problem: Backtracking algorithm

Node 14 becomes the E-node.

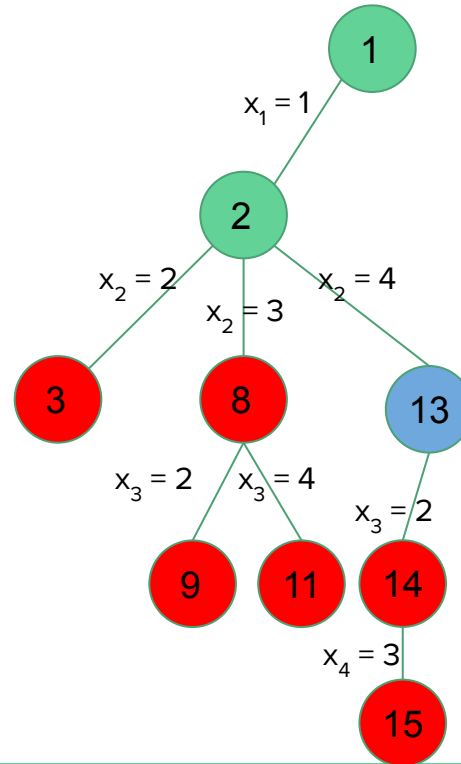
Node 15 is generated and is immediately killed.



q_1			
			q_2
	q_3		

N-queens problem: Backtracking algorithm

We backtrack to Node 13.

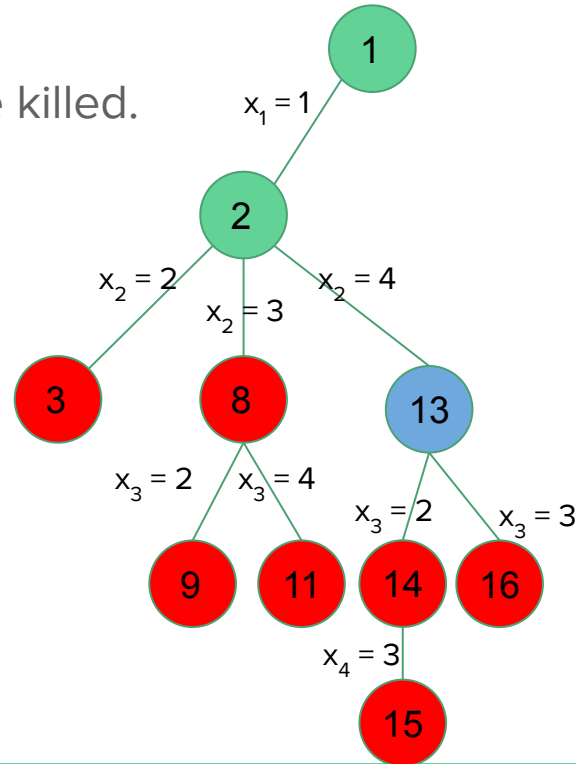


q_1			
			q_2

N-queens problem: Backtracking algorithm

Now the E-node is Node 13.

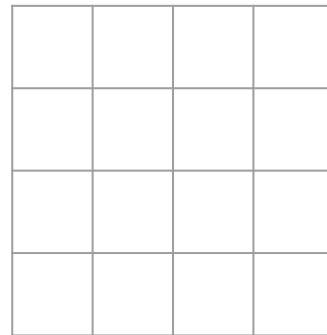
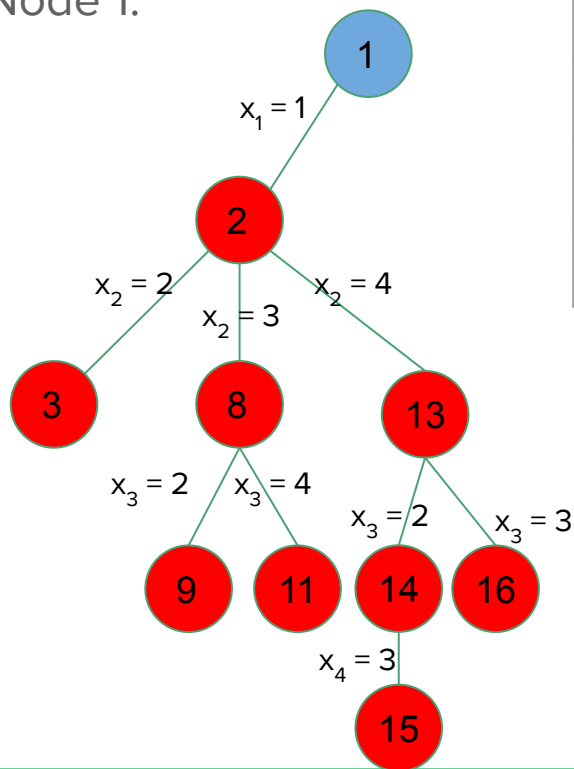
Its another child, Node 16, will also be killed.



q_1			
			q_2

N-queens problem: Backtracking algorithm

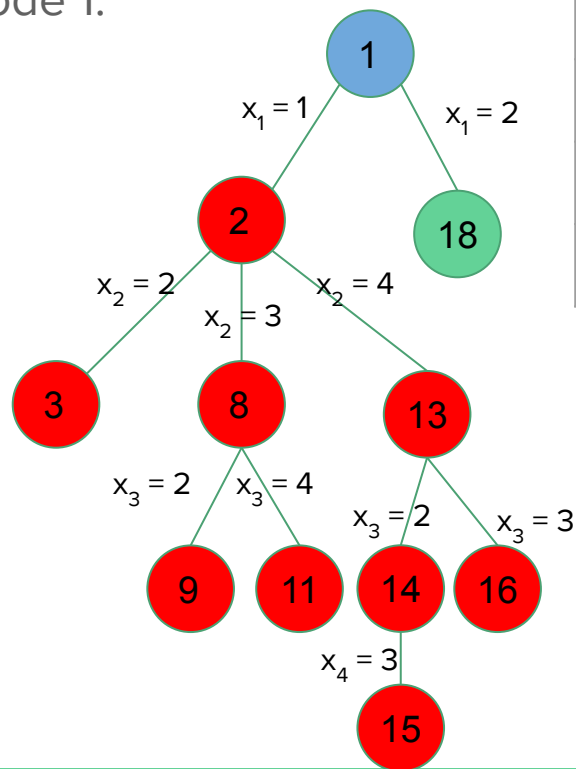
We backtrack to Node 2 and then to Node 1.



N-queens problem: Backtracking algorithm

Now, we generate another child of Node 1.

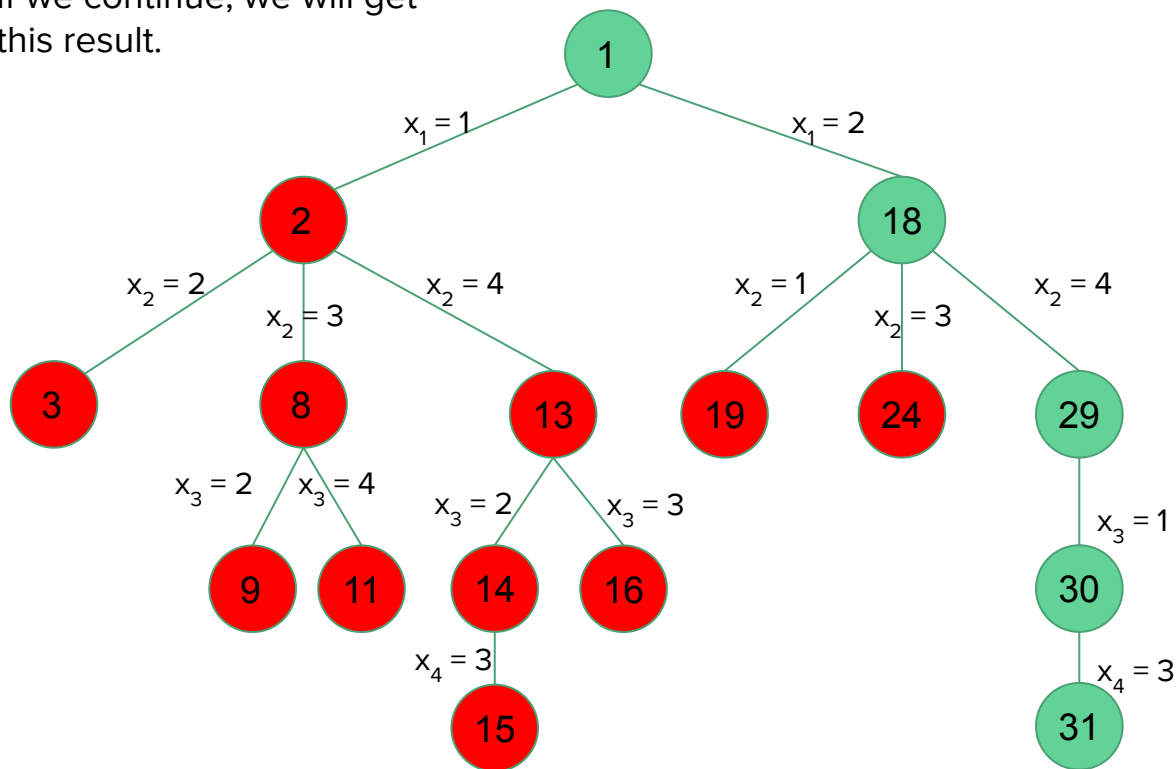
And the path will be (2).



	q_1		

N-queens problem: Backtracking algorithm

If we continue, we will get this result.



	q_1		
			q_2
q_3			
		q_4	

The solution is
(2, 4, 1, 3).

N-queens problem: Backtracking algorithm

```
1  Algorithm NQueens( $k, n$ )
2  // Using backtracking, this procedure prints all
3  // possible placements of  $n$  queens on an  $n \times n$ 
4  // chessboard so that they are nonattacking.
5  {
6      for  $i := 1$  to  $n$  do
7      {
8          if Place( $k, i$ ) then
9          {
10              $x[k] := i$ ;
11             if ( $k = n$ ) then write ( $x[1 : n]$ );
12             else NQueens( $k + 1, n$ );
13         }
14     }
15 }
```

```
1  Algorithm Place( $k, i$ )
2  // Returns true if a queen can be placed in  $k$ th row and
3  //  $i$ th column. Otherwise it returns false.  $x[ ]$  is a
4  // global array whose first  $(k - 1)$  values have been set.
5  // Abs( $r$ ) returns the absolute value of  $r$ .
6  {
7      for  $j := 1$  to  $k - 1$  do
8          if (( $x[j] = i$ ) // Two in the same column
9              or ( $\text{Abs}(x[j] - i) = \text{Abs}(j - k)$ ))
10             // or in the same diagonal
11             then return false;
12     return true;
13 }
```

Branch-and-bound

Recall

A **state space tree** consists of

- a set of nodes representing each state of the problem,
- arcs between nodes representing the legal moves from one state to another,
- an initial state and
- a goal state

In state space tree methods of problem solving, we first represent the problem as a state space tree and then search the tree to find the solution to the problem by systematically generating nodes of the tree.

Recall

Terminologies

- A node which has been generated and all of whose children have not yet been generated is called a **live node**
- The live node whose children are being generated is called the **E-node**
- A **dead node** is a generated node which is not to be expanded further or all of whose children have generated

Graph search strategies

- Depth-first search
- Breadth-first search

Branch and bound

In **backtracking**, nodes are generated in depth-first manner, i.e. as soon as a new child of the current E-node is generated, this child will become the new E-node

In **branch-and-bound**, nodes are generated in breadth-first manner, i.e. E-node remains the E-node until it is dead.

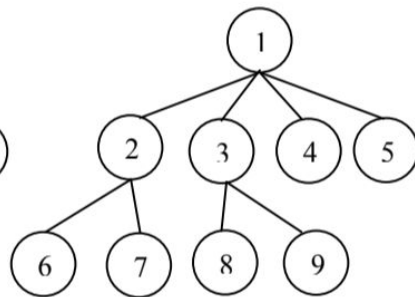
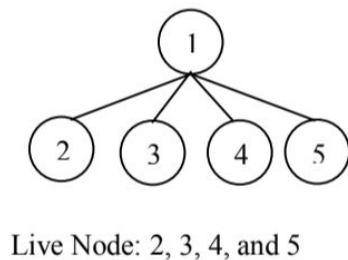
Branch-and-bound

- A branch-and-bound method searches a state space tree using any search mechanism in which all the children of the E-node are generated before another node becomes the E-node
- Common search techniques
 - FIFO search
 - LIFO search
 - Least-cost search

State space tree search

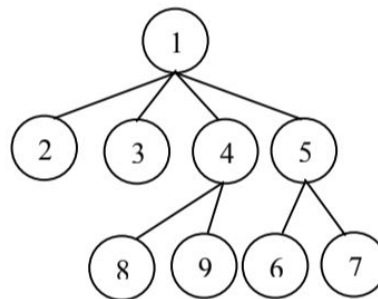
FIFO search

The list of live nodes is a FIFO list



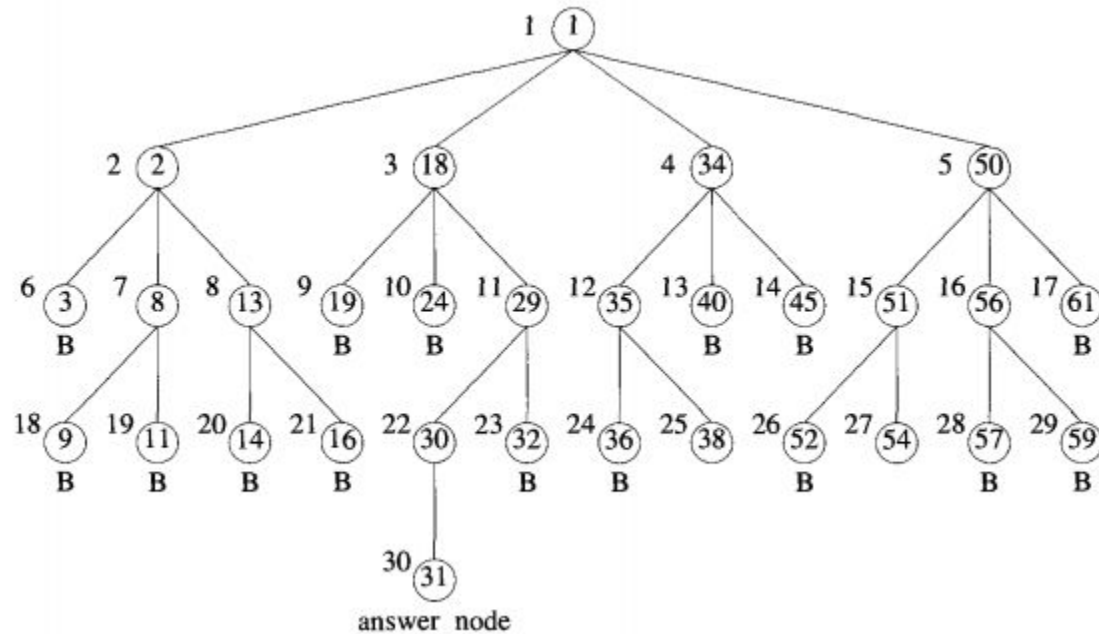
LIFO search

The list of live nodes is a LIFO list



FIFO search for 4-queens problem

FIFO search for 4-queens problem



State space tree search

Least cost (LC) search

- In FIFO and LIFO branch and bound, the selection rule for the next E-node does not give any preference to a node that has a very good chance of getting the search to an answer quickly
- The search for an answer node can often be speeded by using an “intelligent” ranking function, also called an **approximate cost function**, $\hat{c}$, for live nodes
- In **LC search**, we expand the node with the best cost

Branch-and-bound

- A branch-and-bound method searches a state space tree using any search mechanism in which all the children of the E-node are generated before another node becomes the E-node
- We assume that each answer node x has a cost $c(x)$ associated with it
- The goal is to find the minimum-cost answer node

Branch-and-bound

- A branch-and-bound method searches a state space tree using any search mechanism in which all the children of the E-node are generated before another node becomes the E-node
- Requirements
 - **Branching:** A set of solutions, which is represented by a node, can be partitioned into mutually exclusive sets. Each subset in the partition is represented by a child of the original node.
 - **Lower bounding:** An algorithm is available for calculating a lower bound on the cost of any solution in a given subset.

0/1 Knapsack problem

We are given n objects and a knapsack or bag. Object i has a weight w_i and value p_i . The knapsack has a capacity m . If an object i is placed into the knapsack, then a profit of $p_i x_i$ is earned. The objective is to obtain a filling of the knapsack that maximizes the total profit earned.

0/1 Knapsack problem

In **0/1 Knapsack problem**, each object is either included or excluded in the knapsack, i.e. the object cannot be divided.

In **fractional knapsack problem**, fractions of objects can be included in the knapsack.

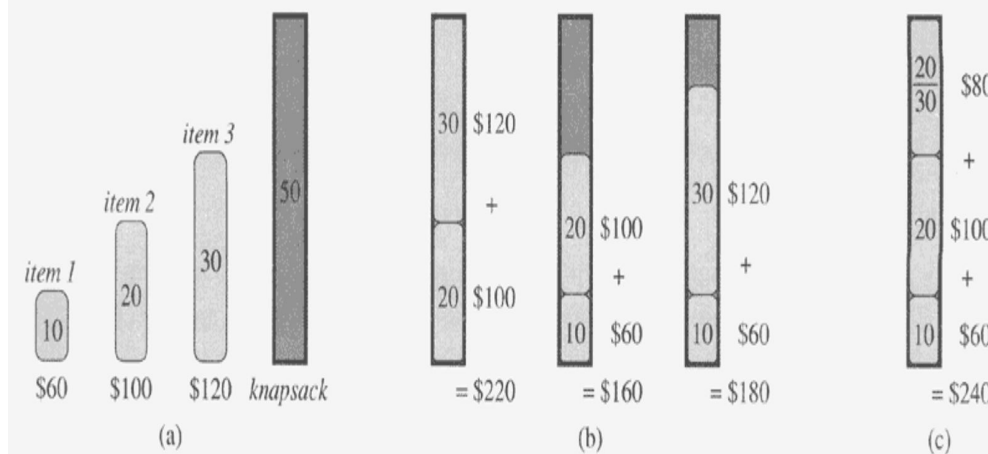


Fig:
(a) A knapsack problem
(b) Solutions for 0/1 knapsack problem
(c) A solution for fractional knapsack problem

0/1 Knapsack problem

We are given n objects and a knapsack or bag. Object i has a weight w_i and value p_i . The knapsack has a capacity m . If an object i is placed into the knapsack, then a profit of $p_i x_i$ is earned. The objective is to obtain a filling of the knapsack that maximizes the total profit earned.

Since LC BB deals with minimization problems, we transform this maximization problem into a minimization problem as follows:

$$\text{minimize } - \sum_{i=1}^n p_i x_i$$

$$\text{subject to } \sum_{i=1}^n w_i x_i \leq m$$

$$x_i = 0 \text{ or } 1, \quad 1 \leq i \leq n$$

0/1 Knapsack problem

Consider the following knapsack problem:

4 objects are to be put in a knapsack of capacity 16. The objects have the following values and weights

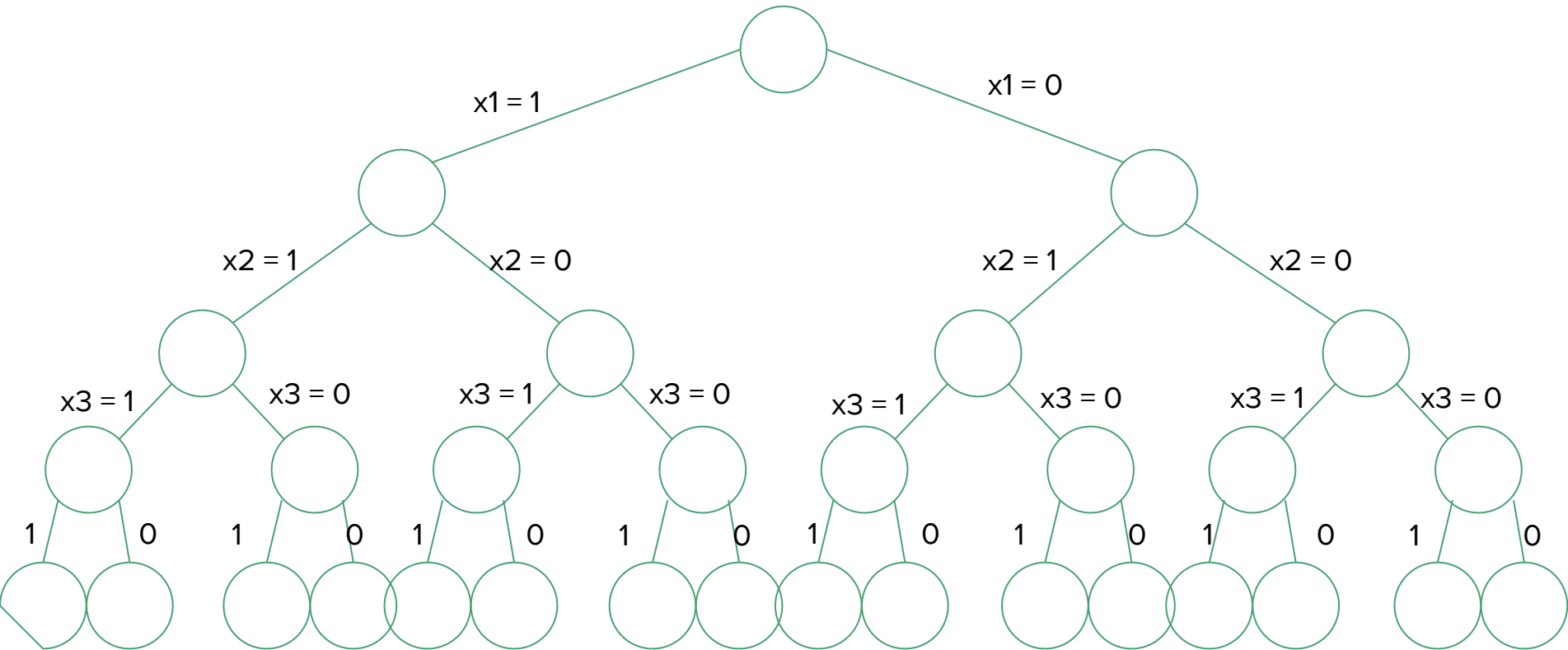
p	45	30	45	10
w	3	5	9	5

Let the solution of this problem be an n-tuple $(x_1, x_2, \dots, x_n)$ where $x_i \in \{0, 1\}$

$x_i = 0$ if object i is not taken

$x_i = 1$ if object i is taken

State space tree



LC branch-and-bound solution

Every leaf node in the state space tree representing an assignment for which

$$\sum_{i=1}^n w_i x_i \leq m \text{ is an **answer node**}$$

For a minimum-cost answer node to correspond to any optimal solution, we need

to define $c(x) = - \sum_{i=1}^n p_i x_i$ for every answer node

LC branch-and-bound

We need two functions, $\hat{c}(\cdot)$ and $u(\cdot)$, (for lower and upper bounds) such that

$$\hat{c}(x) \leq c(x) \leq u(x)$$

For 0/1 knapsack problem,

$$u(x) = - \sum_{i=1}^n p_i x_i$$

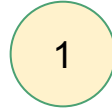
$$\hat{c}(x) = - \sum_{i=1}^n p_i x_i \quad (\text{with fraction})$$

If `upper` is an upper bound on the cost of a minimum-cost solution, all live nodes with $\hat{c}(x) > \text{upper}$ may be killed

The starting value of `upper` can be obtained by some heuristics or can be set to ∞

LC branch-and-bound solution

p	45	30	45	10
w	3	5	9	5



$$u = -75$$
$$\hat{c} = -115$$

LC BB starts with upper = ∞

$$u(1) = -(45 + 30) = -75$$

$$\text{Remaining capacity} = 16 - 3 - 5 = 8$$

$$\hat{c}(1) = -(45 + 30 + 45/9 \times 8) = -115$$

$$\text{upper} = -75$$

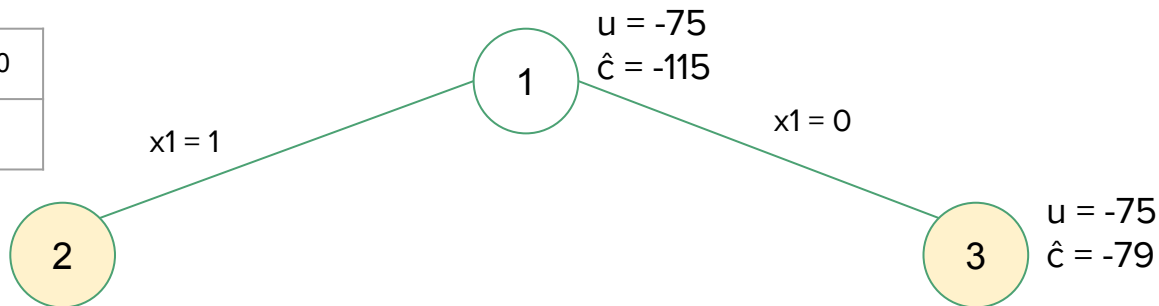
Live nodes: {1}

LC branch-and-bound solution

p	45	30	45	10
w	3	5	9	5

$$u = -75$$

$$\hat{c} = -115$$



upper = -75

Live nodes: {2, 3}

$\hat{c}(2)$ is the lowest

$$u(2) = -(45 + 30) = -75$$

$$\text{Remaining capacity} = 16 - 3 - 5 = 8$$

$$\hat{c}(2) = -(45 + 30 + 45/9 \times 8) = -115$$

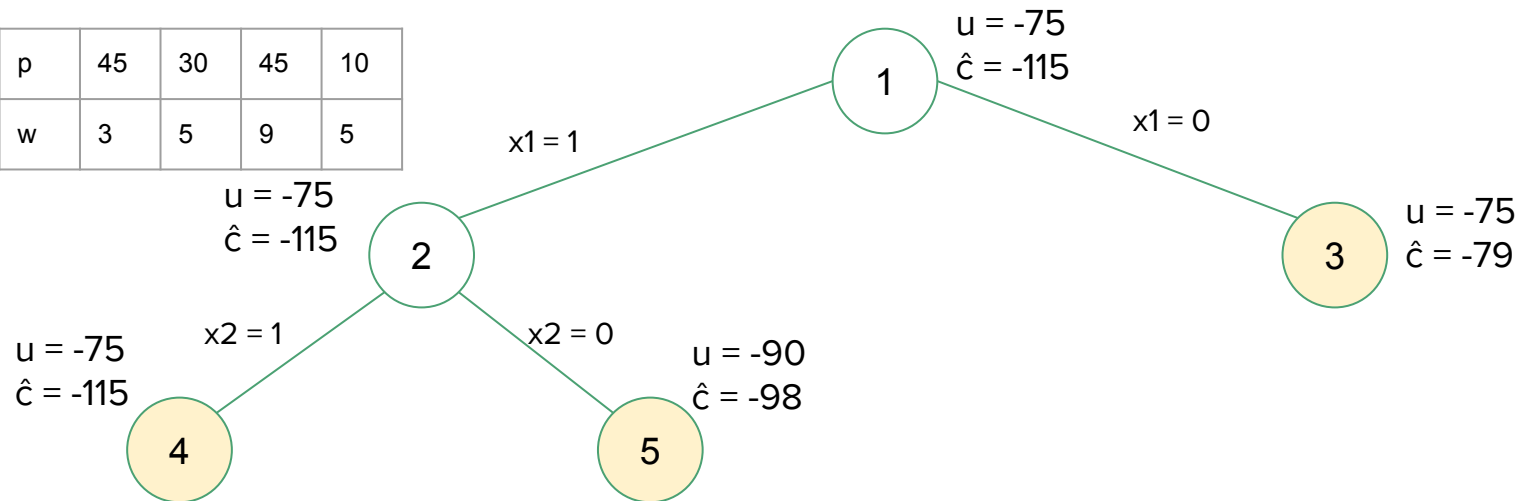
$$u(3) = -(30 + 45) = -75$$

$$\text{Remaining capacity} = 16 - 5 - 9 = 2$$

$$\hat{c}(3) = -(30 + 45 + 10/5 \times 2) = -79$$

LC branch-and-bound solution

p	45	30	45	10
w	3	5	9	5



upper = -75

Live nodes: {3, 4, 5}
 $\hat{c}(4)$ is the lowest

$$u(4) = -(45 + 30) = -75$$

$$\text{Remaining capacity} = 16 - 3 - 5 = 8$$

$$\hat{c}(4) = -(45 + 30 + 45/9 \times 8) = -115$$

$$u(5) = -(45 + 45) = -90$$

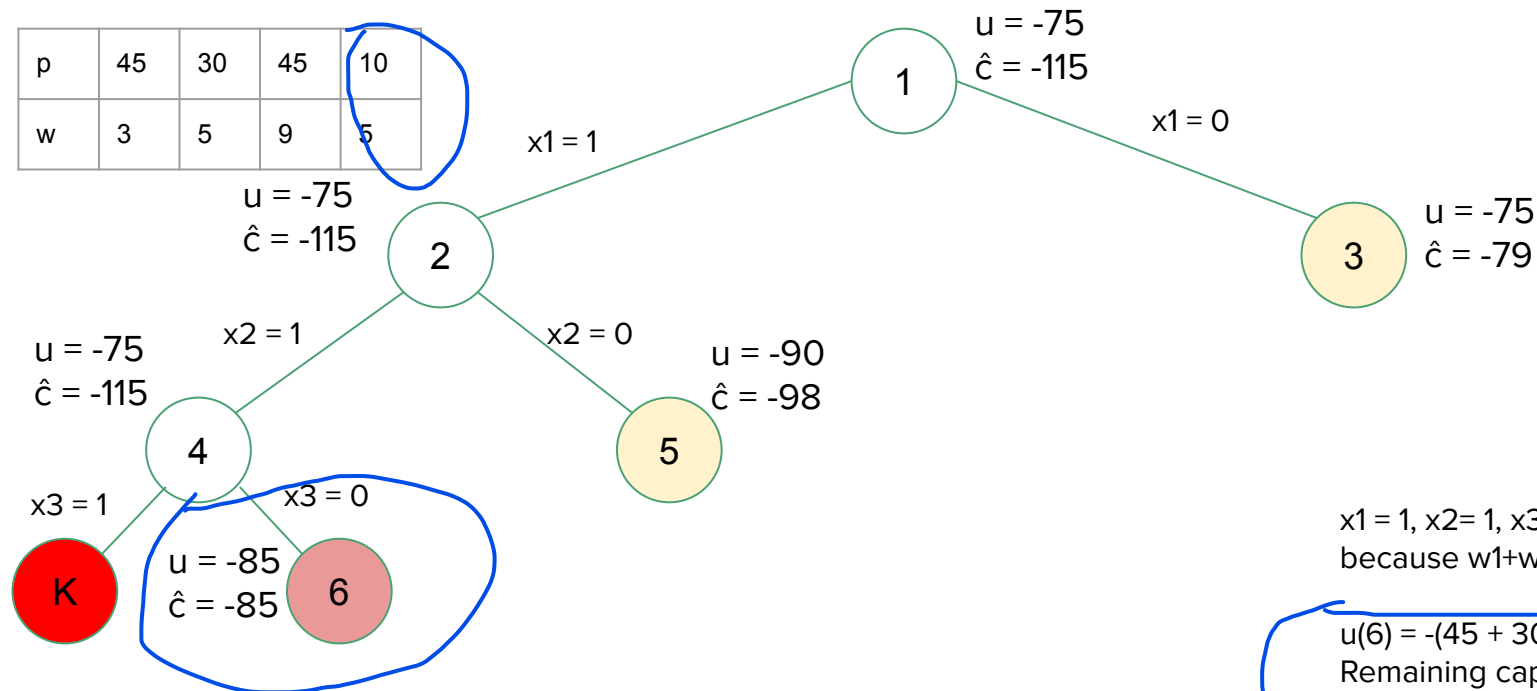
$$\text{Remaining capacity} = 16 - 3 - 9 = 4$$

$$\hat{c}(5) = -(45 + 45 + 10/5 \times 4) = -98$$

upper = -90

LC branch-and-bound solution

p	45	30	45	10
w	3	5	9	5



upper = -90

Live nodes: {3, 5}
 $\hat{c}(5)$ is the lowest

$x_1 = 1, x_2 = 1, x_3 = 1$ is not possible
 because $w_1 + w_2 + w_3$ exceeds 16.

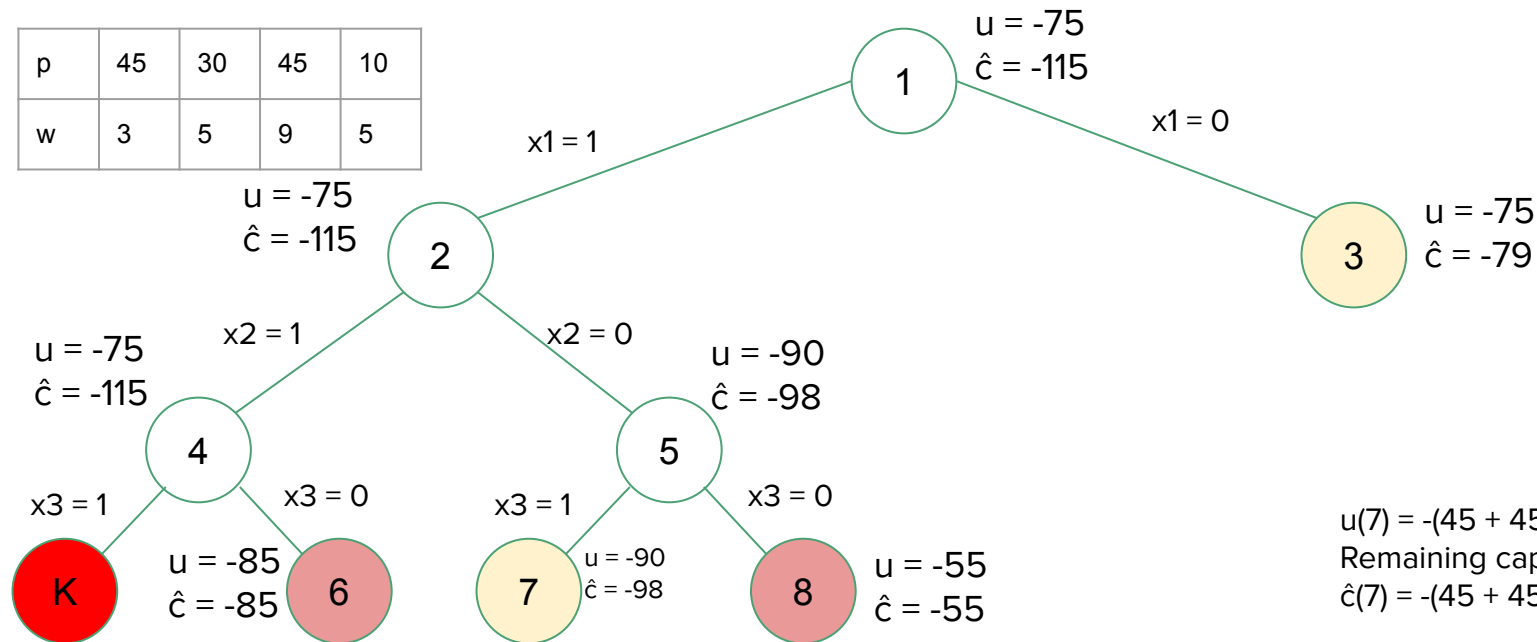
$u(6) = -(45 + 30 + 10) = -85$
 Remaining capacity = $16 - 3 - 5 = 8$
 $\hat{c}(6) = -(45 + 30 + 10) = -85$

$\hat{c}(6) > \text{upper}$. So Node 6 is killed.

LC branch-and-bound solution

p	45	30	45	10
w	3	5	9	5

upper = -90



Live nodes: {3, 7}
 $\hat{c}(7)$ is the lowest

$$u(7) = -(45 + 45) = -90$$

$$\text{Remaining capacity} = 16 - 3 - 9 = 4$$

$$\hat{c}(7) = -(45 + 45 + 10/5 \times 4) = -98$$

$$u(8) = -(45 + 10) = -55$$

$$\text{Remaining capacity} = 16 - 3 - 5 = 8$$

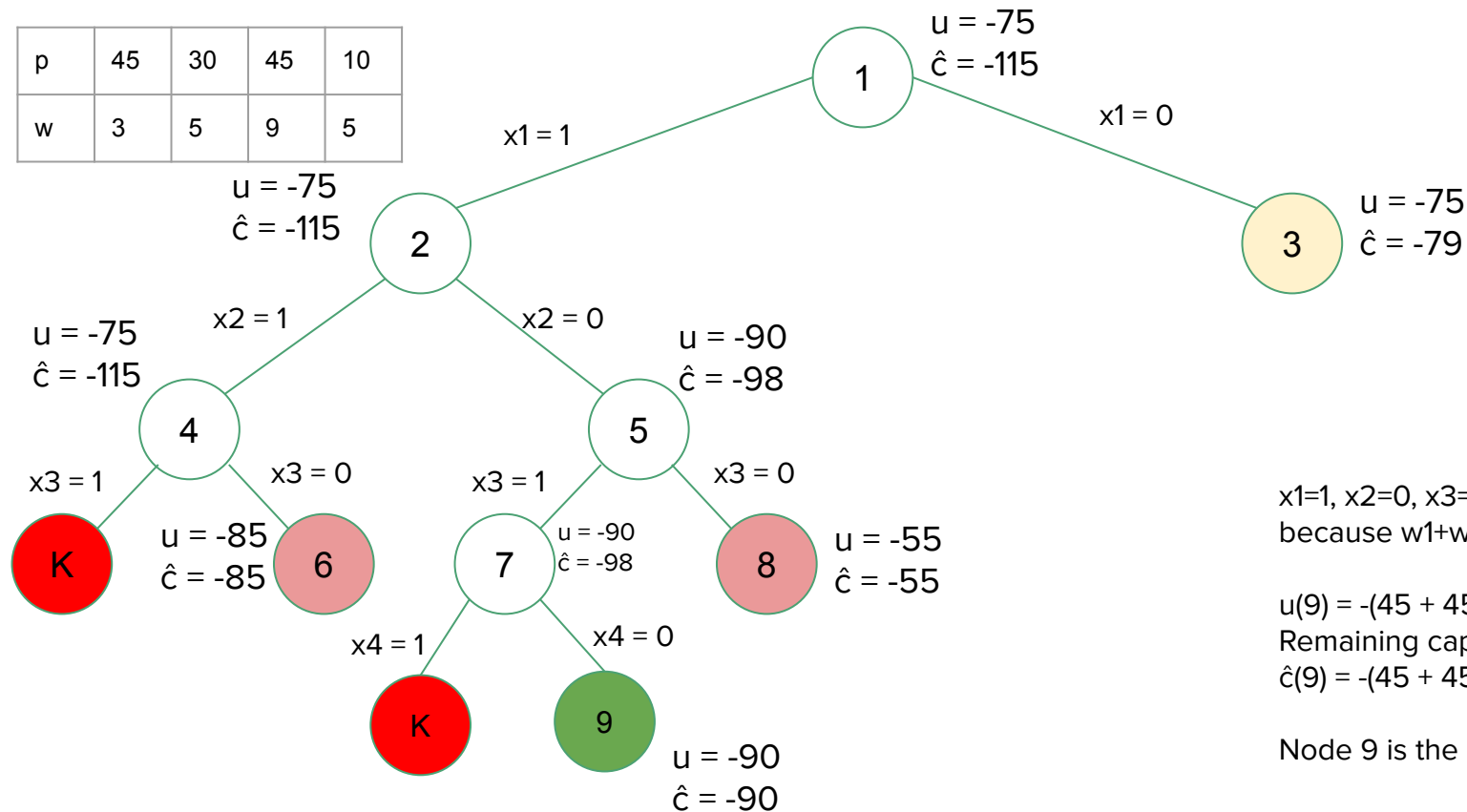
$$\hat{c}(8) = -(45 + 10) = -55$$

$\hat{c}(8) > \text{upper}$. So Node 8 is killed.

LC branch-and-bound solution

p	45	30	45	10
w	3	5	9	5

upper = -90

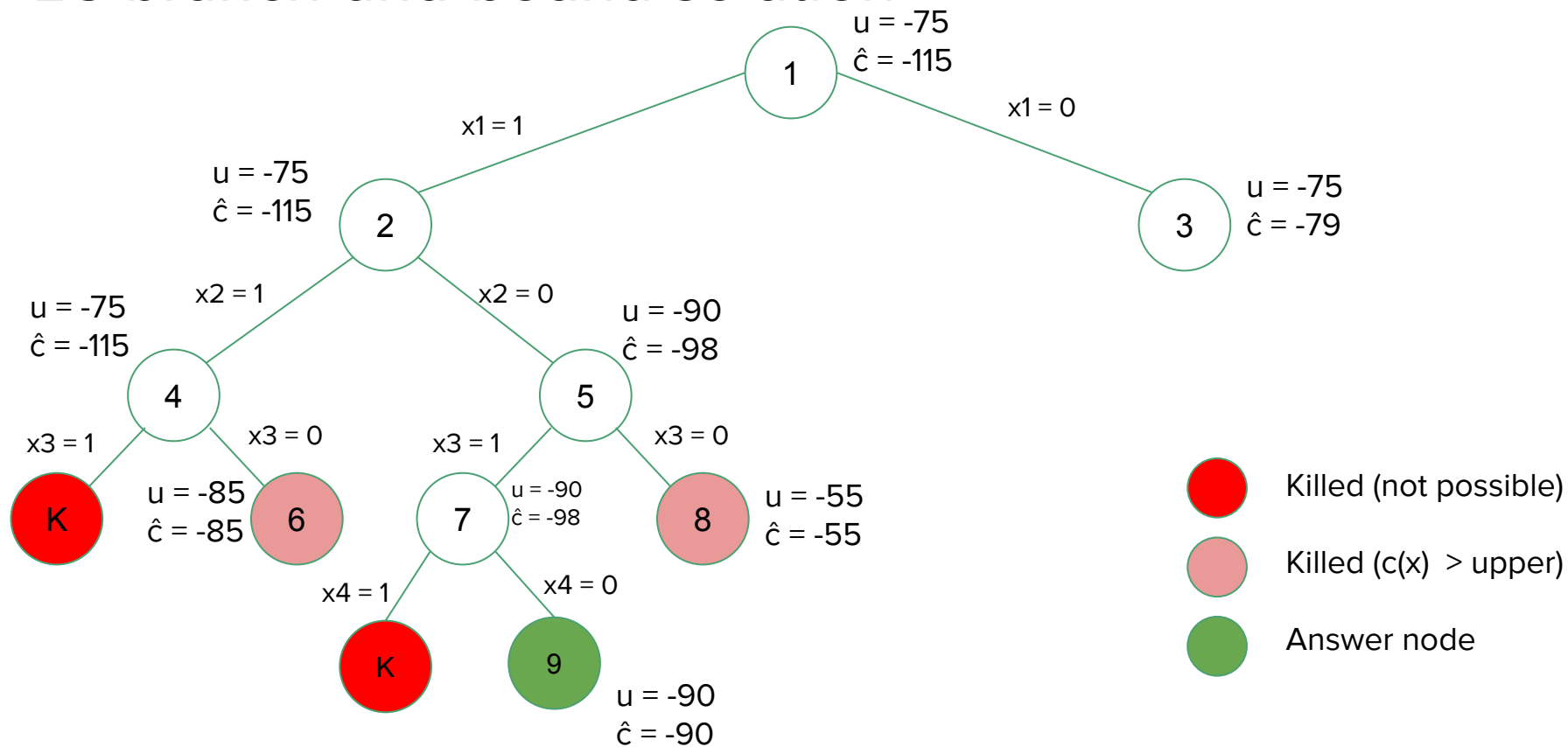


$x_1=1, x_2=0, x_3=1, x_4=1$ is not possible because $w_1+w_3+w_4 = 3 + 9 + 5 > 16$

$u(9) = -(45 + 45) = -90$
 Remaining capacity = $16 - 3 - 9 = 4$
 $\hat{c}(9) = -(45 + 45) = -90$

Node 9 is the solution.

LC branch-and-bound solution



Heuristics

Heuristics

- A **heuristic** is a technique designed or *solving a problem more quickly when classic methods are too slow*, or for finding *an approximate solution* when classic methods fail to find any exact solution
- It guides an algorithm to find good choices without exhaustively searching all possible solutions
- A **heuristic function** is a function that ranks alternatives in search algorithms at each branching step based on available information to decide which branch to follow.

Examples of heuristics

Travelling Salesman Problem (TSP):

- "Given a list of cities and the distances between each pair of cities, what is the shortest possible route that visits each city and returns to the origin city?"
- **The nearest-neighbor heuristic:** pick the **nearest** unvisited city as the next city on the path.

Knapsack problem:

- **Heuristics:** Sort the items by value/weight ratio, then select the next item with the highest ratio.

Examples of heuristics

Antivirus:

- **Heuristic scanning** looks for code and/or behavioral patterns common to a class or family of viruses, with different sets of rules for different viruses

A* search (pronounced A star)

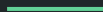
- A search algorithm that finds the shortest path between some nodes in a graph
- (will be covered in Chapter Graph Algorithms)

NP-Completeness

Chapter 6

Outline

- NP-completeness and the classes P and NP
- Polynomial time verification
- NP-completeness and reducibility
- NP-complete problems



Efficient algorithm

Classical definition:

An algorithm is **efficient** iff it runs in polynomial time: $O(n^c)$ for some constant c , where n is the size of the input to the problem

Runtimes of “efficient” algorithms: $O(n)$ $O(n \log n)$ $O(n^3 \log_2 n)$

Runtimes of “inefficient” algorithms: $O(2^n)$ $O(n!)$

Tractability and Intractability

A problem is called **tractable** iff there is an efficient algorithm that solves it.

A problem is called **intractable** iff there is no efficient algorithm that solves it.

Deterministic and non-deterministic algorithms

Given a particular input, a **deterministic algorithm** will always produce the same output.

Even for the same input, a **nondeterministic algorithm** can exhibit different behaviors on different runs.

Such algorithms have multiple choices in some points during its control flow. The choice is not determined by the input value and the state; instead an arbitrary choice among the possibilities can be made in a given run of the program, e.g. due to a *race condition*, or a random number generator etc.

The class P

- A **decision problem** is a problem with a yes/no answer.
- **P** is the class of decision problems that can be solved by a polynomial-time algorithm.
- In other words, given an instance of the problem of class P, the answer yes or no can be decided in polynomial time.
- **Example:** Is there a path from node x to node y in a graph G ? This problem can be solved in linear time using depth-first search.
- *Note: To be in P, a problem only needs an algorithm that runs in time less than some polynomial. Hence, problems that can be solved in $n \log n$ time are in P.*

The class NP

The **class NP** (**N**on-deterministic **P**olynomial time) consists of those problems that are “verifiable” in polynomial time.

What do we mean by a **problem being verifiable**?

- A. if someone gives us an instance of the problem and a certificate to the answer being yes, we can check that it is correct in polynomial time.
In other words, if we were given the solution, we can verify a Yes answer quickly.

Example: Hamiltonian cycle problem

The class NP

Hamiltonian cycle problem:

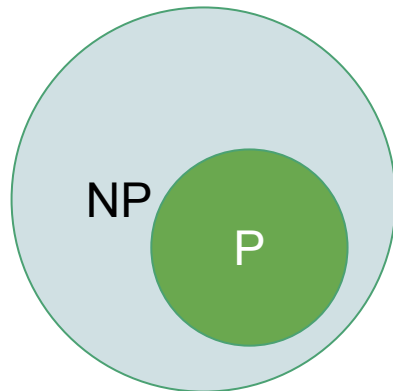
A **hamiltonian cycle** of a directed graph $G = (V, E)$ is a simple cycle that contains each vertex in V .

There is no polynomial-time algorithm for determining whether a directed graph has a hamiltonian cycle.

However, if we are given a certificate, which would be a sequence $\langle v_1, v_2, \dots, v_{|V|} \rangle$ of $|V|$ vertices, we could easily check in polynomial time that $(v_i, v_{i+1}) \in E$ for $i = 1, 2, 3, \dots, |V| - 1$ and that $(v_{|V|}, v_1) \in E$ as well.

The classes P and NP

If a problem is in P, then it is also in NP.



However, **the open question is whether or not P is a proper subset of NP** (i.e. if $P = NP$, P would not be a proper subset of NP).

If the solution to a problem is easy to check for correctness, must the problem be easy to solve?

This is one of the seven Millenium Prize problems, selected by the Clay Mathematics Institute

P = NP ?

An answer to the $P = NP$ question would determine whether problems that can be verified in polynomial time can also be solved in polynomial time.

If it turned out that $P \neq NP$, it would mean that there are problems in NP that are harder to compute than to verify.

NP-completeness

A problem M is said to be **NP-complete** if it is in NP, and for every problem L in NP, there is a **polytime reduction** from L to M.

Reduction:

A problem Q can be **reduced to** another problem Q' if any instance of Q can be “**easily rephrased**” as an instance of Q', the solution to which provides a solution to the instance of Q.

If a problem Q reduces to another problem P in polynomial time (i.e. *polytime reduction*), we write $Q \leq_p P$

NP-completeness

Example: SAT: The satisfiability problem

Given a boolean formula, for example, $(x_1 \vee x_2 \vee x_3 \vee x_4) \wedge (x_1 \vee \neg x_3 \vee x_5) \wedge x_6$, is it possible to assign the inputs $x_1, x_2, \dots, x_n$ such that the formula evaluates to true?

Cook's theorem states that this problem is NP-complete (proof not in the scope of this course)

NP-completeness

Example: 3-SAT. (aka 3-CNF-SAT), 3 CNF satisfiability problem

A boolean formula is in **conjunctive normal form (CNF)** if it is a conjunction (AND) of several clauses, each of which is the disjunction (OR) of several literals, each of which is either a variable or its negation. Example: $(A \vee B \vee C) \wedge (A \vee \neg C \vee D)$

The problem: Given a 3CNF formula, is there an assignment to the variables that makes the formula evaluate to true?

Karp showed the problem 3SAT to be NP-complete by showing how to reduce (in polynomial time) any instance of SAT to an equivalent instance of 3SAT.

Example: Reduction

3SAT $\leq$ 0/1 Knapsack

NP-completeness

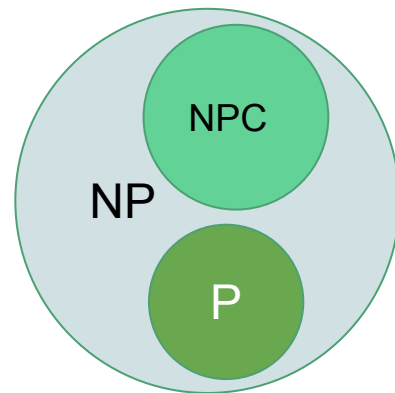
A problem M is said to be **NP-complete** if

1. $M \in \text{NP}$, and
2. $L \leq_p M$ for every $L \in \text{NP}$

Fundamental property:

If M has a polytime algorithm, then so does L . That means, if anyone finds a polynomial algorithm for even one NP-complete problem, then that would imply a polynomial algorithm for every NP-complete problem, i.e., if M is in P , then every L in NP is also in P , or “ $P = \text{NP}$.”

As of now, there are no known polynomial-time algorithms for any NP-complete problem.



NP-hard

Recall: A problem M is said to be **NP-complete** if

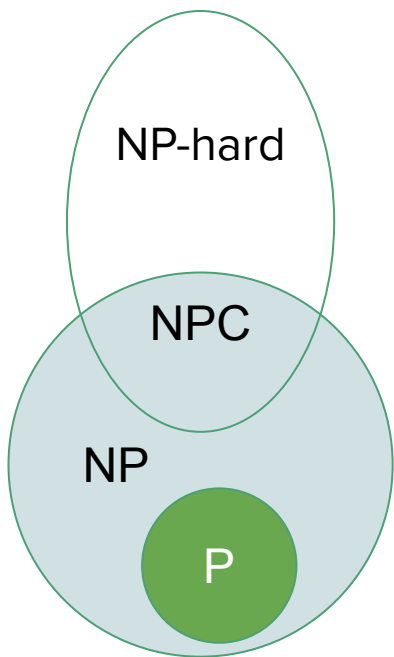
1. $M \in \text{NP}$, and
2. $L \leq_p M$ for every $L \in \text{NP}$

If a problem M' satisfies property 2, but not necessarily property 1, we say that M' is NP-hard.

In other words, NP-hard problems are those problems that are at least as hard as the NP-complete problems.

Note that NP-hard problems do not have to be in NP, and they do not have to be decision problems.

P, NP, NP-Complete, and NP-hard



Note that these classes characterize a problem, not an algorithm

NP-complete problems

The clique problem:

- A clique is a complete subgraph of G .
- The size of a clique is the number of vertices it contains.
- The clique problem is the optimization problem of finding a clique of maximum size in a graph
- As a decision problem, we ask simply whether a clique of a given size k exists in the graph.

$\text{CLIQUE} = \{(G, k): G \text{ is a graph containing a clique of size } k\}$

- To prove that the clique problem is NP-complete, we show that $\text{CLIQUE} \in \text{NP}$, and a known NP-complete problem can be reduced to CLIQUE, e.g.

$3\text{-CNF-SAT} \leq_p \text{CLIQUE}$

NP-complete problems

The vertex-cover problem:

- Given an undirected graph $G(V, E)$, $S \subseteq V$ is a vertex cover if every vertex is incident on a vertex in S .
- The vertex-cover problem is to find a vertex cover of minimum size in a given graph.

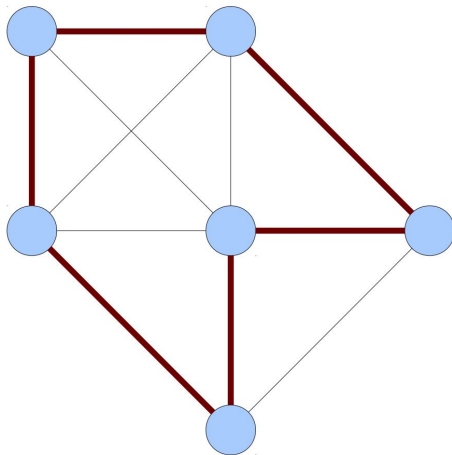
$\text{VERTEX-COVER} = \{(G, k): \text{graph } G \text{ has a vertex cover of size } k\}$

- To prove that this problem is NP-complete, we show that $\text{VERTEX-COVER} \in \text{NP}$ and $\text{CLIQUE} \leq_p \text{VERTEX-COVER}$

NP-complete problems

The hamiltonian-cycle problem:

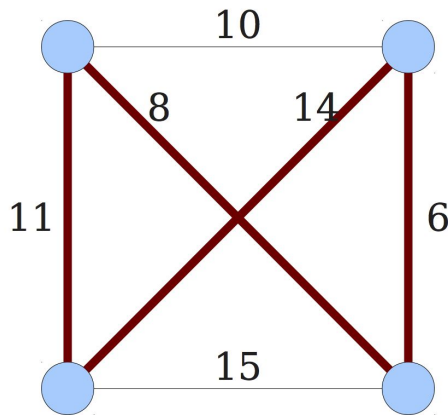
- The problem is to determine whether a directed graph has a hamiltonian cycle
 $\text{HAM-CYCLE} = \{G: G \text{ is a hamiltonian graph}\}$



NP-complete problems

The traveling-salesman problem:

- We are given a complete undirected graph $G = (V, E)$ that has a nonnegative integer cost $c(u, v)$ associated with each edge $(u, v) \in E$, and we must find a hamiltonian cycle (a tour) of G with minimum cost.



Chapter 5: Probabilistic and Parallel Algorithms

—

Contents

- Probabilistic algorithms
 - Monte Carlo algorithm - Primality testing
 - Las Vegas algorithm - N-queen problem
 - Parallel algorithms
 - Finding connected components in a graph
 - Parallel sorting
-

Probabilistic Algorithms

—

Different types of algorithms

Suppose an algorithm produces X as the maximum value, when the true maximum is Y .

- An **exact** or **deterministic** algorithm guarantees $X = Y$.
- An **approximation** algorithm guarantees X 's rank is “close to” Y 's rank
- A **probabilistic** algorithm gives up its guarantee of getting $X = Y$ and may say “ X is usually Y ” in exchange for an improved running time. They sacrifice reliability for speed.

Disadvantages of deterministic algorithms

Some problems may be too complex to solve. Deterministic algorithms to solve such problems may have high resource (time and memory) consumption.

If there is more than one correct answer, deterministic algorithms always come up with the same answer.

Some applications require nondeterminism (unpredictability), e.g., in card games to avoid cheating.

Probabilistic/Randomized Algorithms

Probabilistic or randomised algorithms employ some form of random element in an attempt to obtain improved performance.

- Sometimes, improvement can be dramatic, from intractable to tractable
- Some loss in reliability of results can occur
- Different runs may produce different results for the same input, reliability may be improved by running several times
- Two main categories
 - **Monte Carlo algorithms** have good running time, their result is not guaranteed
 - **Las Vegas algorithms** have a guaranteed result, but do not guarantee fast running time

Monte Carlo Algorithms

These are randomized algorithms which may produce incorrect results with some small probability, but whose execution time is deterministic.

If such an algorithm is run multiple times with independent random choices each time, the failure probability can be made arbitrarily small at the cost of a slight increase in computing time .

Primality Test

Prime number is a natural number greater than 1 whose only factors are 1 and itself.

Primality test is an algorithm for determining whether an input number is prime.

No known algorithm can solve this problem with certainty in a reasonable time when the number to be tested has more than a few hundred decimal digits.

The Simplest Primality Test

Given an input number, n , check whether it is divisible by all numbers from 2 to $n-1$. If we find any number that divides, n is composite, otherwise it is prime.

Time complexity: $O(n)$

Optimized way: Instead of checking till $n-1$, we can check till $\sqrt{n}$ because a larger factor of n must be a multiple of a smaller factor that has been already checked, i.e. for any divisor $p > \sqrt{n}$, there must be another divisor $n/p < \sqrt{n}$.

Time Complexity: $O(\sqrt{n})$

Probabilistic Primality Test

- Many popular probabilistic primality tests use, apart from the input number n , some other numbers a which are chosen at random from some sample space.
- They never report a prime number as composite, but it is possible for a composite number to be reported as prime.
- The probability of error can be reduced by repeating the test with several independently chosen values of a .
- Some popular probabilistic primality tests:
 - Fermat Primality Test
 - Miller-Rabin Primality Test

Fermat Primality Test

Fermat primality test is based on the **Fermat's Little Theorem**, which states that

If n is a prime number, then for any integer a , $1 < a < n-1$, $a^{n-1} \equiv 1 \pmod{n}$
read it as a^{n-1} and 1 are congruent modulo n [i.e., $a^{n-1} \% n = 1$]

Examples:

- Since 5 is prime,
 $2^4 \equiv 1 \pmod{5}$ [or $2^4 \% 5 = 1$], $3^4 \equiv 1 \pmod{5}$ and $4^4 \equiv 1 \pmod{5}$
- Since 7 is prime,
 $2^6 \equiv 1 \pmod{7}$, $3^6 \equiv 1 \pmod{7}$, $4^6 \equiv 1 \pmod{7}$, $5^6 \equiv 1 \pmod{7}$ and $6^6 \equiv 1 \pmod{7}$

Fermat Primality Test

Given an integer n , choose some integer a coprime to n and calculate $a^{n-1} \bmod n$. If the result is different from 1, then n is composite. If it is 1, then n may be prime.

If n is prime, then this method always returns true. If n is composite (or non-prime), then it may return true or false.

The probability of producing incorrect results for composite is low and can be reduced by doing more iterations.

Fermat Primality Test

Inputs: n : a value to test for primality, $n > 3$; k : a parameter that determines the number of times to test for primality

Output: composite if n is composite, otherwise probably prime

Steps:

1. Repeat k times:
 - a. Pick a randomly in the range $[2, n - 2]$
 - b. If $a^{n-1} \not\equiv 1 \pmod{n}$, then return composite
2. Return probably prime

Fermat Primality Test

Example:

$$n = 17$$

Step 1: Pick a randomly in the range $[2, 15]$. Let's say $a = 9$.

Step 2: Check $a^{n-1} \not\equiv 1 \pmod{n}$

$$9^{16} \% 17 = 1$$

Repeat. Let's say $a = 12$. $12^{16} \% 17 = 1$

Fermat Primality Test

Example:

$$n = 239$$

Step 1: Pick a randomly in the range $[2, 237]$. Let's say $a = 210$.

Step 2: Check $a^{n-1} \not\equiv 1 \pmod{n}$

$$210^{238} \% 239 = 1$$

Repeat. Let's say $a = 25$. $25^{238} \% 239 = 1$

Fermat Primality Test

Example:

$$n = 250$$

Step 1: Pick a randomly in the range $[2, 250]$. Let's say $a = 30$.

Step 2: Check $a^{n-1} \not\equiv 1 \pmod{n}$

$$30^{249} \% 250 = 0$$

Thus 250 is composite.

Fermat Primality Test

Time complexity: $O(k \log n)$. Note that the modular exponentiation of step 1.b takes $O(\log n)$ time.

Note that the Fermat Primality Test may fail even if we increase the number of iterations (higher k). There exist some composite numbers (e.g., the number 561) with the property that for every $a < n$ and coprime to n , we have $a^{n-1} \equiv 1 \pmod{n}$. Such numbers are called **Carmichael numbers**.

Fermat's primality test is often used if a rapid method is needed for filtering, for example in the key generation phase of the RSA public key cryptographic algorithm.

Miller-Rabin Primality Test

- It is also based on the Fermat's Little Theorem
- The result of this test will be whether the given number is a composite number or a probable prime number
- The probable prime numbers in the Miller-Rabin Primality Test are called strong probable numbers, as they have a higher chance of being a prime number than in the Fermat's Primality Test.
- A number n is said to be strong probable prime number to base a if one of these congruence relations hold:
 - $a^d \equiv \pm 1 \pmod{n}$
 - $a^{2^r d} \equiv -1 \pmod{n}$ for all $0 \leq r \leq s - 1$

where $n-1 = 2^s d$

Miller-Rabin Primality Test

Steps to check whether n is prime or not:

1. Find $n-1=2^s \cdot d$, such that d is an odd number
2. Choose a such that $1 < a < n-1$
3. Compute $b_0 = a^d \pmod{n}$. If b_0 is ± 1 , n is probably prime else

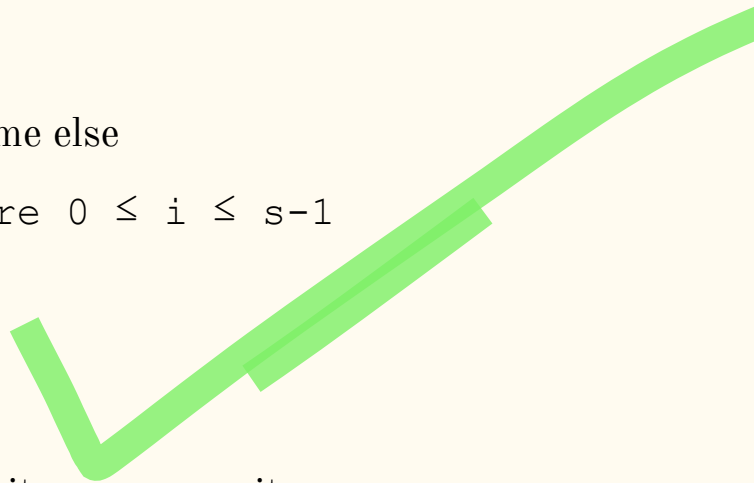
Compute $b_1 = b_0^2 \pmod{n}$... $b_i = b_{i-1}^2 \pmod{n}$ where $0 \leq i \leq s-1$
until b_i is ± 1 or -1

If b_i is $\neq -1$ the number is composite

If b_i is -1 the number is probably prime

Simulate for **561**. Fermat's returns it as a Prime, MR returns it as a composite

Check for **23** and **27**.



Miller-Rabin Primality Test

Input #1: $n > 2$, an odd integer to be tested for primality

Input #2: k , the number of rounds of testing to perform

Output: "*composite*" if n is found to be composite, "*probably prime*" otherwise

let $s > 0$ and d odd > 0 such that $n - 1 = 2^s d$

repeat k **times:**

$a \leftarrow \text{random}(2, n - 2)$

$x \leftarrow a^d \bmod n$

repeat s **times:**

$y \leftarrow x^2 \bmod n$

if $y = 1$ and $x \neq 1$ and $x \neq n - 1$ **then**

return "*composite*"

$x \leftarrow y$

if $y \neq 1$ **then**

return "*composite*"

return "*probably prime*"

Las Vegas algorithms - Characteristics

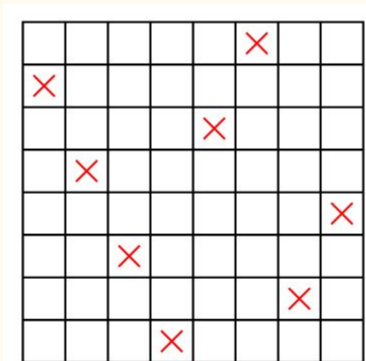
- Las Vegas algorithms make probabilistic choices to help guide them more quickly to a correct solution.
- Unlike Monte Carlo algorithms, they never return a wrong answer.
- Two main categories
 - Those which guarantee a correct solution though they may take longer if unfortunate choices are made
 - Those that may fail to produce a result, but if they return a result, it is always correct

Eight queens problem

Place 8 queens on a chess board so that no one attacks another.

Remember: a queen attacks other pieces on the same row, same column and same diagonals.

A possible solution:



Backtracking algorithm

Place first queen in top-left corner.

Assume now the situation with some non-attacking queens in place.

If < 8 queens on board, place a queen in the next row — if it attacks a queen, move along the row one square at a time. If no position is possible, move the queen in the immediately preceding row on one square. If no position is suitable, move the queen in the next row back, etc..

This algorithm actually returns the first solution after examining 114 of the 2057 possible states.



A Las Vegas approach – non-backtracking

- Place queens randomly on successive rows
- No attempt is made to relocate previous queens when there is no possibility left for the next queen
- The algorithm either ends successfully or fails if there is no square in which the next queen can be placed
- The process can be repeated if a failure is detected, and will consider a probably different placement

A Las Vegas approach – non-backtracking

The expected number of states explored can be calculated to be around 56, compared with 114 for the deterministic algorithm.

Further improvements can be made by combining random placement with some backtracking, first fixing some queens at random and then completing the arrangement via backtracking.

When the first two rows are occupied at random, then the expected number of states explored to find a solution becomes just 29.

References

- Brassard, G. & Bratley, P. Fundamentals of Algorithmics.
- Barringer H. Randomized algorithms - a brief introduction
- <https://www.geeksforgeeks.org/fermat-method-of-primality-test/>
- <https://www.geeksforgeeks.org/primality-test-set-3-miller-rabin/>
- <https://www.baeldung.com/cs/fermat-primality-test>

Chapter 5: Probabilistic and Parallel Algorithms

—

Contents

- Probabilistic algorithms
 - Monte Carlo algorithm - Primality testing
 - Las Vegas algorithm - N-queen problem
 - Parallel algorithms
 - Finding connected components in a graph
 - Parallel sorting
-

Parallel Computing

- **Parallelism** or **parallel computing** refers to the ability to carry out multiple calculations or the execution of processes simultaneously
- Parallel computing is commonly used to solve computationally large and data-intensive tasks
- Today parallelism is available in all computer systems
 - Multicore chips are used in essentially all computing devices
- At the larger scale, many computers can be connected by a network and used together to solve large problems.
- *Parallel computing requires design of efficient parallel algorithms*

Why Parallel Computing?

- It is not possible to increase processor and memory frequencies indefinitely
- Power consumption rises with processor frequency while the energy efficiency decreases.
- Parallelism has become a part of any computer

Parallel Execution of Algorithms

- We first need to identify the **subtasks** that can execute in parallel (not a trivial task)
- Assuming a task T can be divided into n parallel subtasks $t_1, \dots, t_n$, these subtasks need to be **mapped/assigned** to processors of the parallel system. This process is denoted as a function from the task set T to the processor set P as $M : T \rightarrow P$
- Subtasks may have dependencies (i.e., t_i may not start before t_j)
- If we know all of the task dependencies and also characteristics such as the execution time, we can distribute tasks evenly to the processors before running them (**static scheduling**).
- In many cases, we do not have such information beforehand and thus **dynamic load balancing** is used to provide each processor with a fair share of workload at runtime.

Concepts and Terminology

Embarrassingly parallel

A computational problem is called embarrassingly parallel if it requires little or no effort to divide that problem into subproblems that can be computed independently on separate computing resources.

Parallel Algorithm Design Methods

- Data Parallelism
- Task Parallelism

Data Parallelism

- Focuses on distribution of data sets across the multiple computation programs.
- Same operations are performed on different parallel computing processors on the distributed data subset.
- Some of Big Data frameworks that utilize data parallelism are Apache Spark, Apache MapReduce and Apache YARN (it supports more of hybrid parallelism providing both task and data parallelism).

Data Parallelism Example

- Multiplying two matrices A and B of size $n \times n$ to get a matrix C can be parallelized by partitioning these matrices as $n/2 \times n/2$ sub-matrices

$$\begin{pmatrix} C_1 & C_2 \\ C_3 & C_4 \end{pmatrix} = \begin{pmatrix} A_1 & A_2 \\ A_3 & A_4 \end{pmatrix} \times \begin{pmatrix} B_1 & B_2 \\ B_3 & B_4 \end{pmatrix}$$

$$C_1 = (A_1 \times B_1) + (A_2 \times B_3) \rightarrow p_1$$

$$C_2 = (A_1 \times B_2) + (A_2 \times B_4) \rightarrow p_2$$

$$C_3 = (A_3 \times B_1) + (A_4 \times B_3) \rightarrow p_3$$

$$C_4 = (A_3 \times B_2) + (A_4 \times B_4) \rightarrow p_4$$

Task Parallelism

- Covers the execution of computer programs across multiple processors on same or multiple machines on the same or different data sets.
- Focuses on executing different operations in parallel to fully utilize the available computing resources in form of processors and memory.
- The divide and conquer algorithmic method can be efficiently implemented using this model
- Some of Big Data frameworks that utilize task parallelism are Apache Storm and Apache YARN (it supports more of hybrid parallelism providing both task and data parallelism).

Task Parallelism Example

```
1: function SUM(A[1..n])
2:   if  $n = 1$  then
3:     return A[1]
4:   else
5:      $x \leftarrow \text{Sum}(A[1..n/2])$ 
6:      $y \leftarrow \text{Sum}(A[n/2 + 1..n])$ 
7:     return  $x + y$ 
8:   end if
9: end function
```

In order to have a parallel version of this algorithm, we note the recursive calls are independent as they operate on different data partitions; hence, we can perform these calls in parallel simply by performing the operations within the else statement between lines 4 and 8 of this algorithm in parallel.

Parallel Computing Architectures

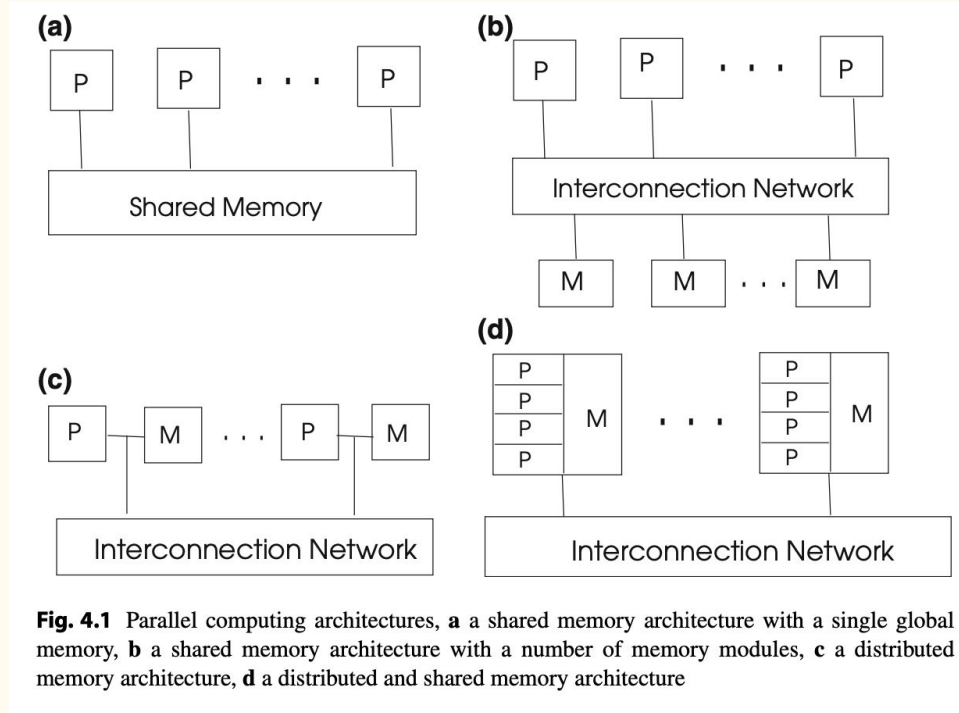
1. Shared memory architectures

- a. Each processor has some local memory
- b. Interprocess communication and synchronization are performed using a shared memory that provides access to all processors
- c. Data is read from and written to the shared memory locations; however, we need to provide some form of control on access to this memory to prevent race conditions

2. Distributed memory architectures

- a. There is no shared memory
- b. Interprocess communication and synchronization are performed by sending and receiving of messages (message passing)

Parallel Computing Architectures



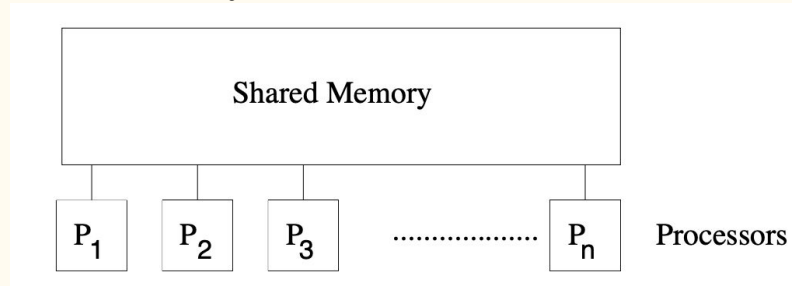
Models of Parallel Computing

Two basic models for parallel computing

1. Parallel Random Access Model (PRAM) Model
2. Message Passing Model

PRAM Model

- The PRAM model extends the basic Random Access Model (RAM) model to parallel computing
- The PRAM model has p processing units that are all connected to a common unbounded shared memory
- In a PRAM model, a processor can access any word of memory in a single step, and these accesses can occur in parallel, i.e., in a single step, every processor can access the shared memory.



PRAM Model

PRAM instructions execute in 3-phase cycles

1. Read (if any) from a shared memory cell
2. Local computation (if any)
3. Write (if any) to a shared memory cell

Processors execute these 3-phase PRAM instructions synchronously

Simultaneous access to the same location may end in unpredictable data in the accessing processing units as well as in the accessed location.

Variants of PRAM

- **Exclusive Read Exclusive Write (EREW) PRAM**

No two processors are allowed to read or write the same shared memory cell simultaneously

- **Concurrent Read Exclusive Write (CREW)**

Simultaneous read allowed, but only one processor can write

- **Concurrent Read Concurrent Write (CRCW)**

Simultaneous reads and writes are allowed; different additional restrictions are imposed on simultaneous writes to avoid unpredictable effects

- Priority CRCW: processors assigned fixed distinct priorities, highest priority wins
- Arbitrary CRCW: one randomly chosen write wins
- Common/Consistent CRCW: all processors are allowed to complete write if and only if all the values to be written are the same

Message Passing Model

- There is no shared memory
- The parallel tasks that run on different processors communicate and synchronize using two basic primitives: send and receive
- Particularly useful in a distributed environment where the communicating processes may reside on different, network connected, systems

PRAM	Message Passing
Threads OpenMP	MPI, PVM
locks, semaphores	messages (send, receive)
Shared Memory	Distributed Memory

ALGORITHM MODEL

PROGRAMMING

OPERATING SYSTEM

HARDWARE

Analysis of Parallel Algorithms

The **running time** of an algorithm, the number of processors it uses, and its cost are used to determine the efficiency of a parallel algorithm.

The running time of a parallel algorithm T_p is

$$T_p = t_{\text{fin}} - t_{\text{st}}$$

where t_{st} is the start time of the algorithm on the first (earliest) processor and t_{fin} is the finishing time of the algorithm in the last (latest) processor.

Analysis of Parallel Algorithms (Contd.)

If T_p is the worst-case running time of a particular algorithm A for a problem Q using p identical processors, and T_s is the worst-case running time of the fastest known sequential algorithm to solve Q, the **speedup** S_p is defined as

$$S_p = T_s / T_p$$

We need the speedup to be as large as possible.

Increasing the number of processors will decrease the speedup due to the increased interprocess communication.

Placing parallel tasks in fewer processors to reduce network traffic decreases parallelism.

Analysis of Parallel Algorithms (Contd.)

Efficiency of a parallel algorithm is the speedup per processor, i.e.

$$E_p = S_p / p$$

A parallel algorithm is said to be **scalable** if its efficiency remains almost constant when both the number of processors and the size of the problem are increased.

Analysis of Parallel Algorithms (Contd.)

The **work** of an algorithm corresponds to the total number of primitive operations performed by an algorithm.

Ignoring communication overhead from synchronizing the processors, this is equal to the time used to run the computation on a single processor

Analysis of Parallel Algorithms (Contd.)

The **span** or **depth** of an algorithm basically corresponds to the longest sequence of dependences in the computation.

It can be thought of as the time an algorithm would take if we had an unlimited number of processors on an ideal machine.

It enables analyzing to what extent the work of an algorithm can be divided among processors.

Minimizing the depth/span is important in designing parallel algorithms.

Parallel Computing with a Binary Tree

Computing sum of n integers

```
for  $i$  from  $\lg n - 1$  downto 0 do  
    for  $2^i \leq j \leq 2^{i+1} - 1$  in parallel do  
         $A[j] \leftarrow A[2j] + A[2j+1]$   
return  $A[1]$ 
```


Parallel Computing with a Binary Tree

Computing Prefix Sum

Given: array $A[0 \dots n-1]$

Goal: Compute all prefix sums $A[0] + \dots + A[i]$ for $i = 0, \dots, n-1$

Sequential solution:

```
for i from 0 to n-1 do
```

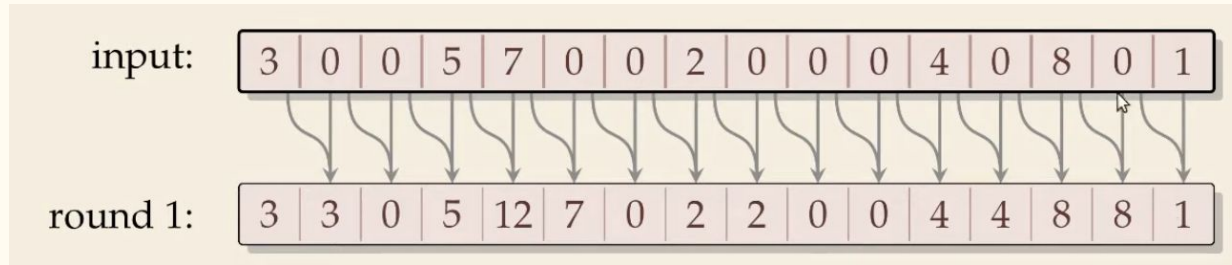
```
     $A[i] = A[i-1] + A[i]$ 
```

Time complexity: $O(n)$

Cannot parallelize due to data

Parallel Computing with a Binary Tree (Contd.)

Parallel Prefix Sum

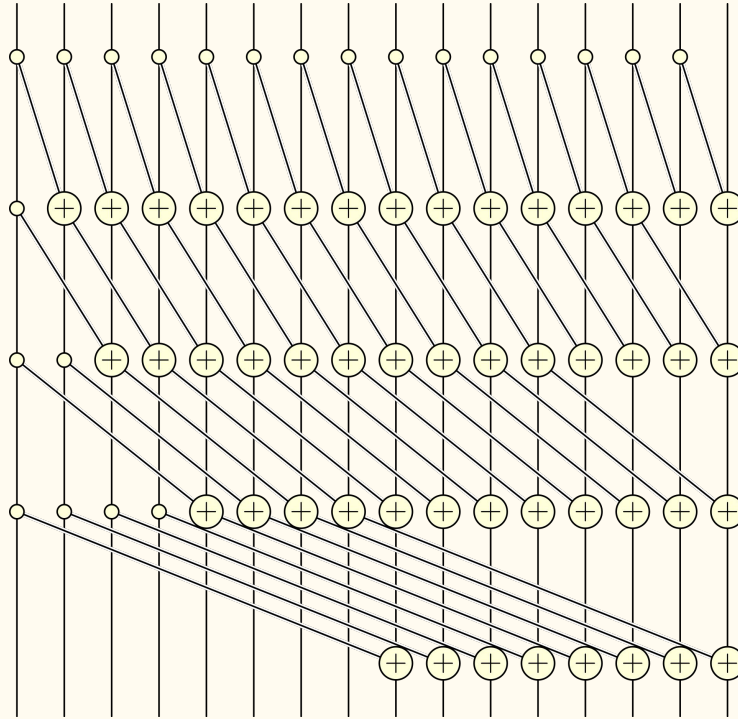


Parallel Computing with a Binary Tree (Contd.)

Parallel Prefix Sum



Parallel Computing with a Binary Tree (Contd.)



Parallel Computing with a Binary Tree (Contd.)

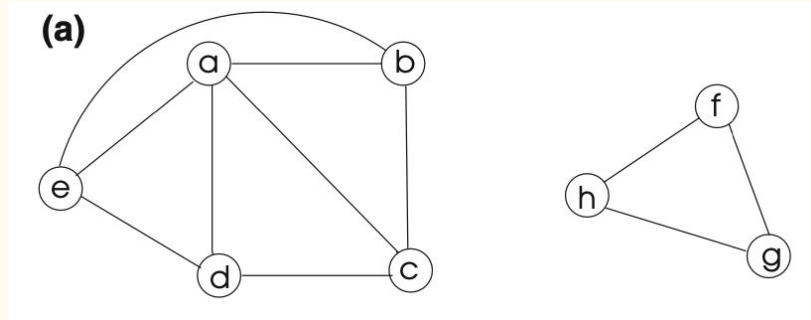
Parallel Prefix Sum

```
1 procedure parallelPrefixSums( $A[0..n - 1]$ )  
2   for  $r := 1, \dots \lceil \lg n \rceil$  do  
3      $step := 2^{r-1}$   
4     for  $i := step, \dots n - 1$  do in parallel  
5        $x := A[i] + A[i - step]$   
6        $A[i] := x$   
7     end parallel for  
8   end for
```

Connected Components

A connected component or simply component of an undirected graph is a subgraph in which each pair of nodes is connected with each other via a path.

In connected components, all the nodes are always reachable from each other.



A graph containing two connected components

Finding Connected Components

The problem is to label all the vertices in a graph G such that two vertices u and v have the same label if and only if there is a path between the two vertices.

Sequentially the connected components of a graph can easily be labeled using either depth-first or breadth-first search.

Two approaches for parallelizing the algorithm for finding connected components:

1. Partitioning the adjacency matrix
2. Contracting the graph

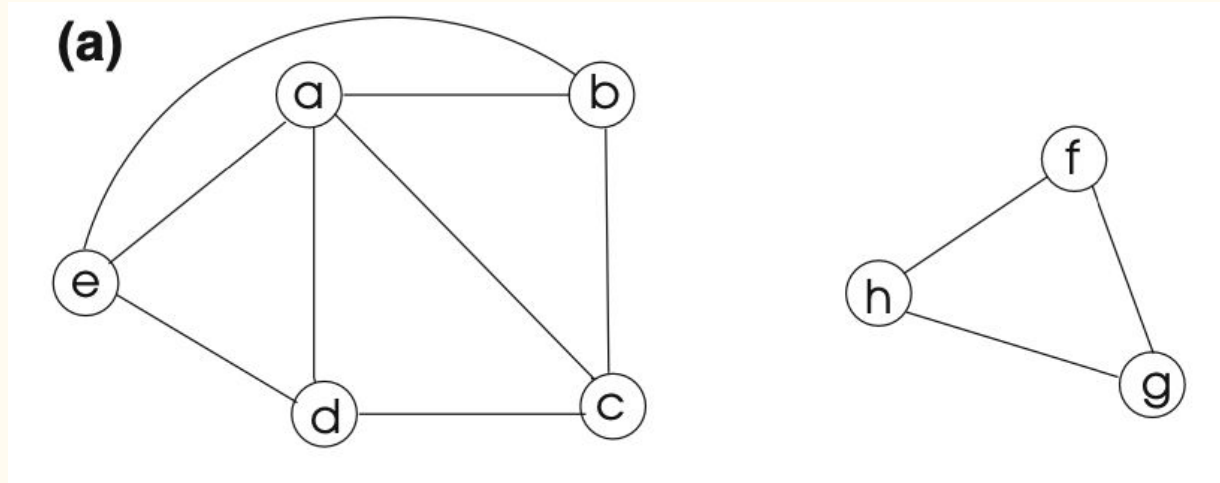
Finding Connected Components Using Adjacency Matrix

Steps

1. Row-wise partitioning of the adjacency matrix
2. Apply DFS on each partition in parallel
3. Merge the forests

Finding Connected Components Using Adjacency Matrix

Given graph



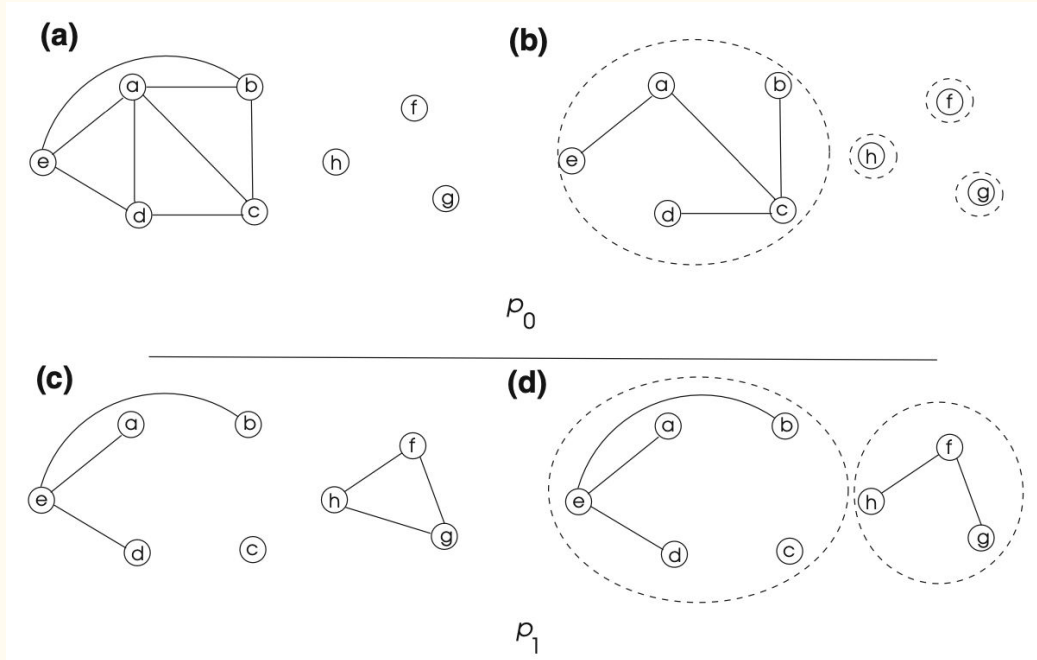
Finding Connected Components Using Adjacency Matrix

Step 1: Row-wise partitioning of the adjacency matrix

(b)		a	b	c	d	e	f	g	h	
A =	a	0	1	1	1	1	0	0	0	p_0
	b	1	0	1	0	1	0	0	0	
	c	1	1	0	1	0	0	0	0	
	d	1	0	1	0	1	0	0	0	
	e	1	1	0	1	0	0	0	0	p_1
	f	0	0	0	0	0	0	1	1	
	g	0	0	0	0	0	1	0	1	
	h	0	0	0	0	0	1	1	0	

Finding Connected Components Using Adjacency Matrix

Step 2: Apply DFS on each partition in parallel



Finding Connected Components Using Adjacency Matrix

Step 3: Merge the forests

Merging can be efficiently done using the find and union operations of disjoint set data structure.

For given two forests A and B,

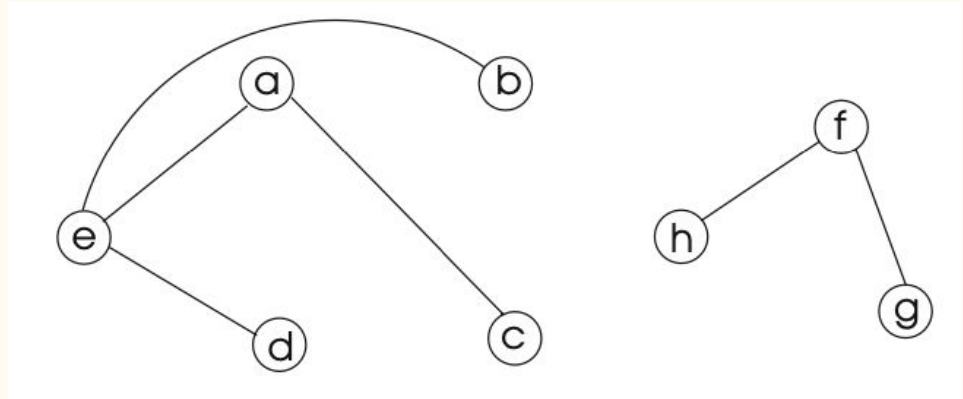
For each edge $(u, v) \in A$:

$x = \text{find}(u)$ in B

$y = \text{find}(v)$ in B

if $x \neq y$

merge (u, v) in B



Finding Connected Components Using Adjacency Matrix

Step 3: Merge the forests

Merging can be efficiently done using the find and union operations of disjoint set data structure.

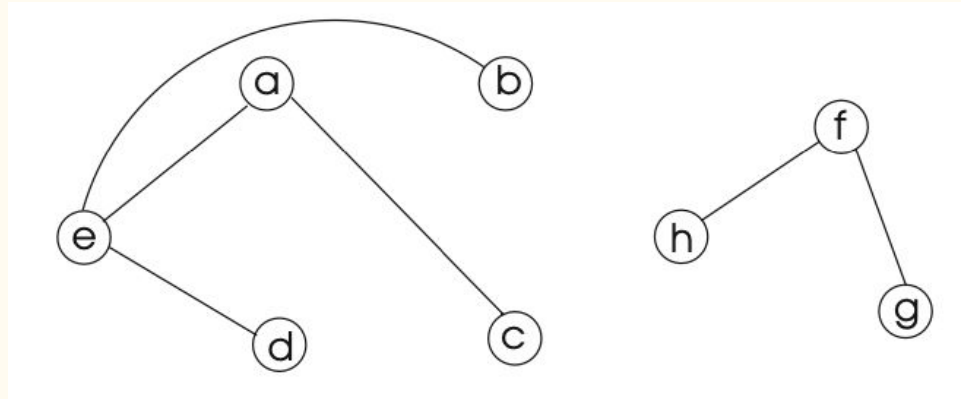
For example,

p0 returns $\{\{a, b, c, d, e\}, \{f\}, \{g\}, \{h\}\}$

p1 returns $\{\{a, b, d, e\}, \{c\}, \{f, g, h\}\}$

Merging will result into

$\{\{a, b, c, d, e\}, \{f, g, h\}\}$



Finding Connected Components Using Graph Contraction

Graph contraction is a technique for computing properties of graph in parallel.

The key idea behind graph contraction is to “shrink” the input graph, ideally by a constant factor in size so that we can solve the problem on a smaller graph, and then use the solution to the smaller graph to construct the solution for the input graph.

Parallel Sorting

Quick sort (sequential)

Parallel Quick Sort

- Recursive calls can run in parallel
- Partition algorithm needs to parallelize
 - Split partitioning into rounds. In each round
 - Compare all elements with pivot (in parallel), store bitvector
 - Compute prefix sums of bit vectors (in parallel)
 - Compact subsequences of small and large elements (in parallel)

Parallel Quick Sort

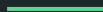
```
1 procedure parQuicksort( $A[l..r]$ )
2    $b := \text{choosePivot}(A[l..r])$ 
3    $j := \text{parallelPartition}(A[l..r], b)$ 
4   in parallel { parQuicksort( $A[l..j - 1]$ ), parQuicksort( $A[j + 1..r]$ ) }
5
6 procedure parallelPartition( $A[l..r], b$ )
7   swap( $A[n - 1], A[b]$ );  $p := A[n - 1]$ 
8   for  $i = 0, \dots, n - 2$  do in parallel
9      $S[i] := \lfloor A[i] \leq p \rfloor$  //  $S[i]$  is 1 or 0
10     $L[i] := 1 - S[i]$ 
11  end parallel for
12  in parallel { parallelPrefixSum( $S[0..n - 2]$ ); parallelPrefixSum( $L[0..n - 2]$ ) }
13   $j := S[n - 2] + 1$ 
14  for  $i = 0, \dots, n - 2$  do in parallel
15     $x := A[i]$ 
16    if  $x \leq p$  then  $A[S[i] - 1] := x$ 
17    else  $A[j + L[i]] := x$ 
18  end parallel for
19   $A[j] := p$ 
20  return  $j$ 
```

Chapter 4:

Dynamic Programming

Outline

- Introduction
- Matrix chain multiplication method
- Longest common subsequence



Dynamic Programming

- Dynamic programming, like the divide-and-conquer method, solves problems by combining the solutions to subproblems
- However, it applies when the subproblems overlap, i.e. when subproblems share subsubproblems
- It solves each subsubproblem just once and then saves its answer in a table, thereby avoiding the work of recomputing the answer every time it solves each subsubproblem

Dynamic programming

Two ways to implement dynamic programming:

1. Memoization - Top down approach
 - a. Maintain a map of already solved sub problems
2. Tabulation - Bottom up approach
 - a. Solve all related sub-problems first
 - b. Based on the results in the table, compute the solution to the "top" / original problem

Fibonacci numbers

Recall definition of Fibonacci numbers:

$$F(n) = F(n-1) + F(n-2)$$

$$F(0) = 0$$

$$F(1) = 1$$

The beginning of the sequence is thus:

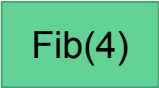
0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ...

Fibonacci numbers: Recursion

```
def Fib(n):  
    if n==0:  
        result = 0  
    elif n==1:  
        result = 1  
    else:  
        result = Fib(n-1) + Fib(n-2)  
    return result
```

Fibonacci numbers: Recursion

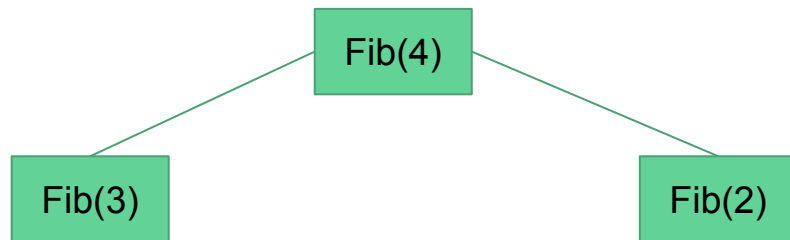
```
def Fib(n):  
    if n==0:  
        result = 0  
    elif n==1:  
        result = 1  
    else:  
        result = Fib(n-1) + Fib(n-2)  
    return result
```



Fib(4)

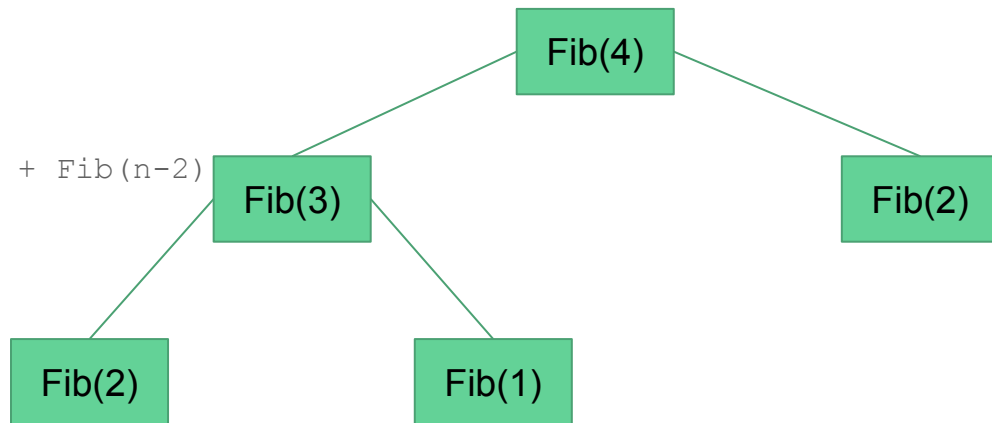
Fibonacci numbers: Recursion

```
def Fib(n):  
    if n==0:  
        result = 0  
    elif n==1:  
        result = 1  
    else:  
        result = Fib(n-1) + Fib(n-2)  
    return result
```



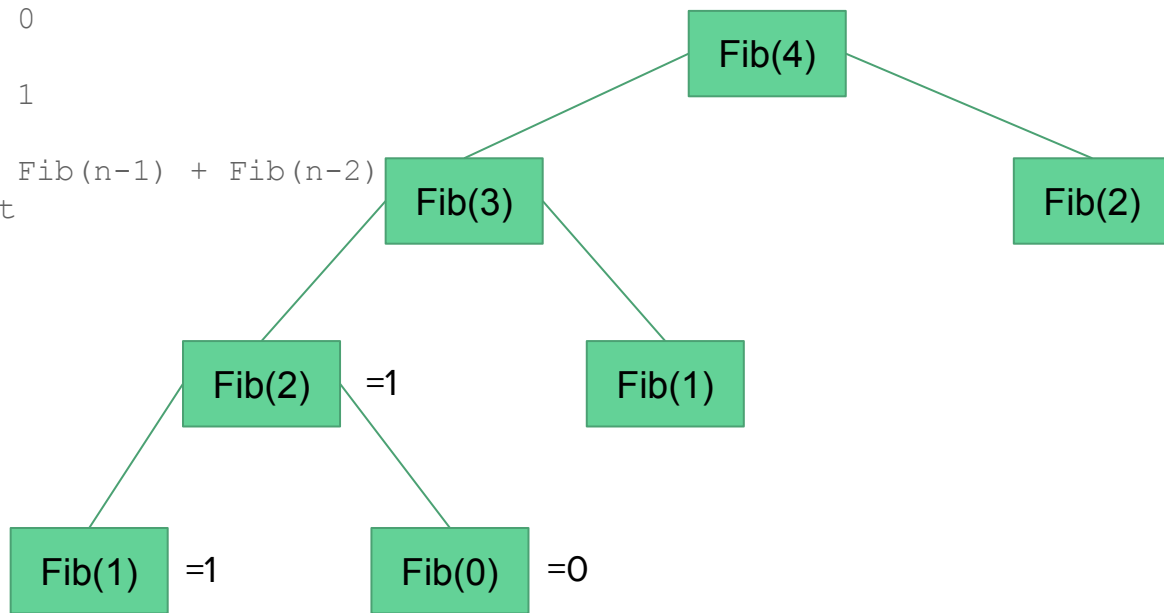
Fibonacci numbers: Recursion

```
def Fib(n):  
    if n==0:  
        result = 0  
    elif n==1:  
        result = 1  
    else:  
        result = Fib(n-1) + Fib(n-2)  
    return result
```



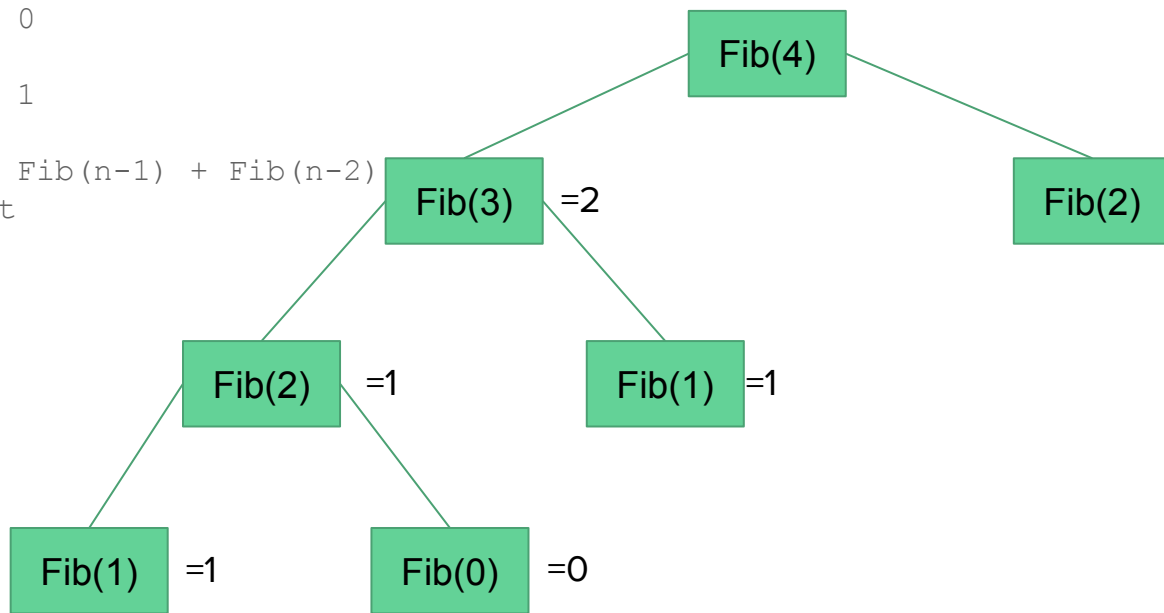
Fibonacci numbers: Recursion

```
def Fib(n):  
    if n==0:  
        result = 0  
    elif n==1:  
        result = 1  
    else:  
        result = Fib(n-1) + Fib(n-2)  
    return result
```



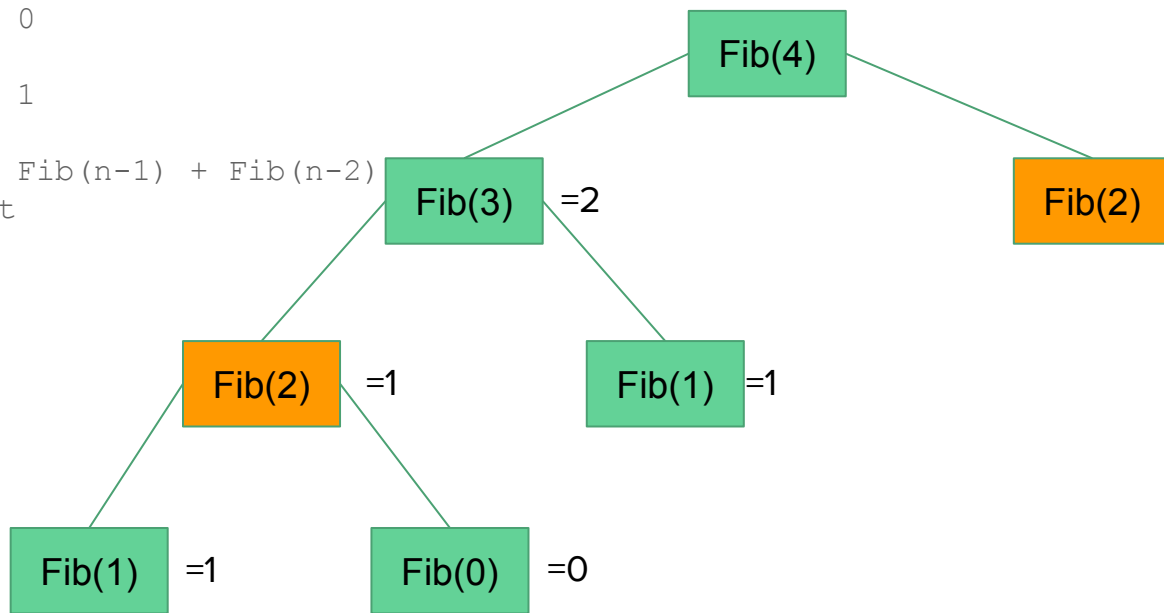
Fibonacci numbers: Recursion

```
def Fib(n):  
    if n==0:  
        result = 0  
    elif n==1:  
        result = 1  
    else:  
        result = Fib(n-1) + Fib(n-2)  
    return result
```



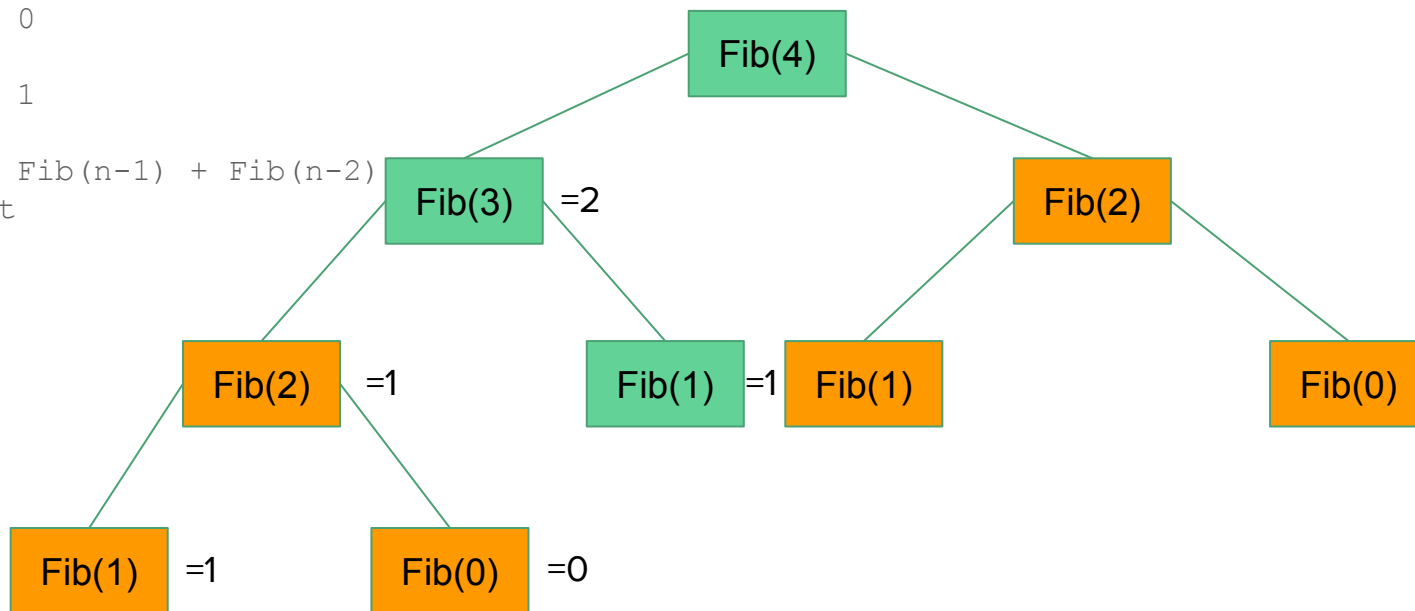
Fibonacci numbers: Recursion

```
def Fib(n):  
    if n==0:  
        result = 0  
    elif n==1:  
        result = 1  
    else:  
        result = Fib(n-1) + Fib(n-2)  
    return result
```



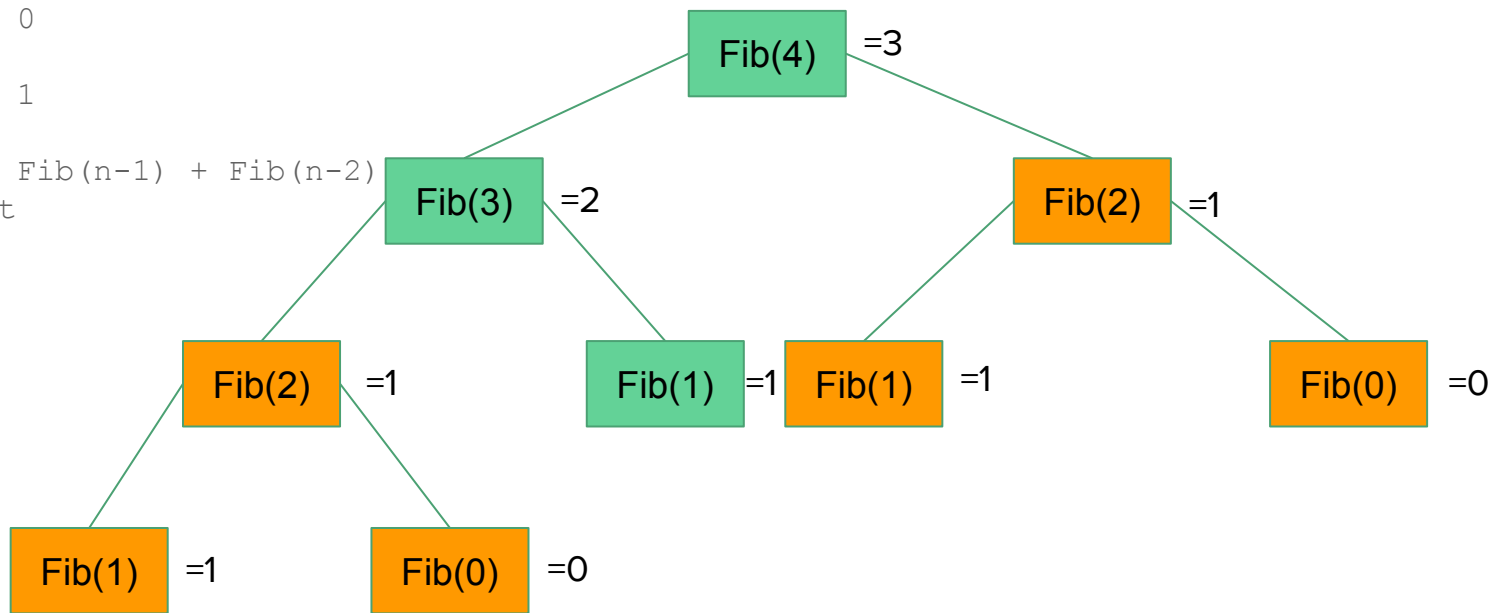
Fibonacci numbers: Recursion

```
def Fib(n):  
    if n==0:  
        result = 0  
    elif n==1:  
        result = 1  
    else:  
        result = Fib(n-1) + Fib(n-2)  
    return result
```



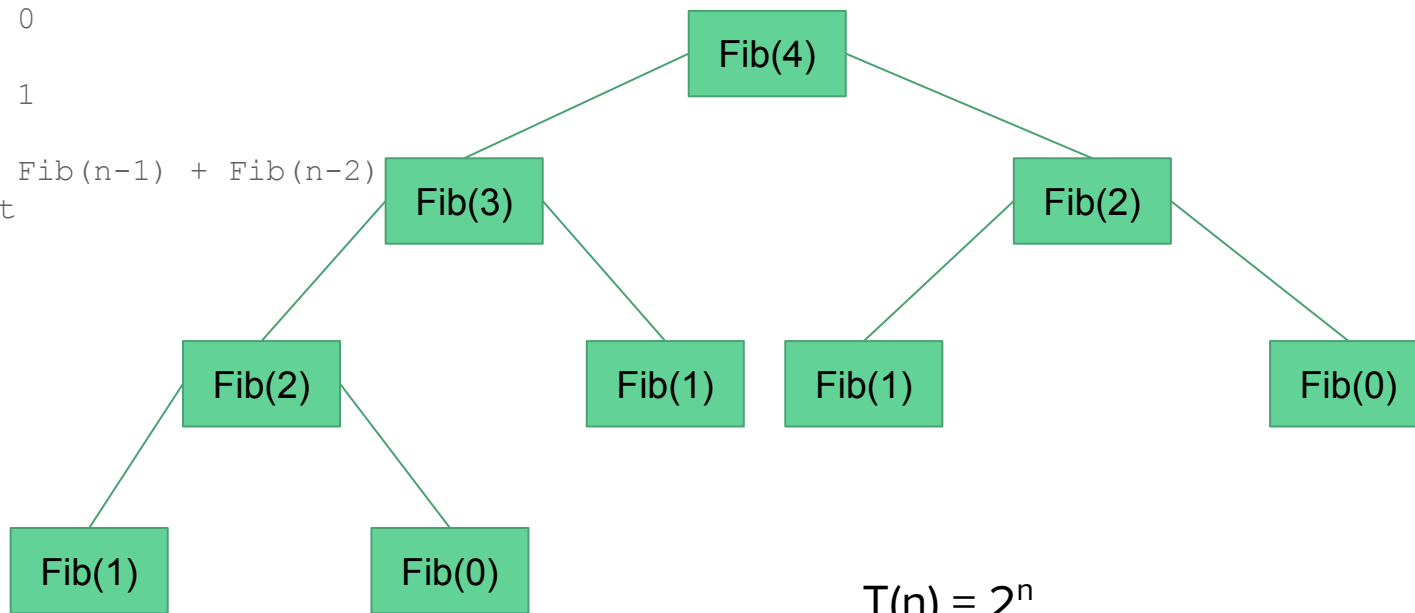
Fibonacci numbers: Recursion

```
def Fib(n):  
    if n==0:  
        result = 0  
    elif n==1:  
        result = 1  
    else:  
        result = Fib(n-1) + Fib(n-2)  
    return result
```



Fibonacci numbers: Recursion

```
def Fib(n):  
    if n==0:  
        result = 0  
    elif n==1:  
        result = 1  
    else:  
        result = Fib(n-1) + Fib(n-2)  
    return result
```



$$T(n) = 2^n$$

Fibonacci numbers: Memoization

Storing the results of
subsubproblems so as to avoid
recomputing them

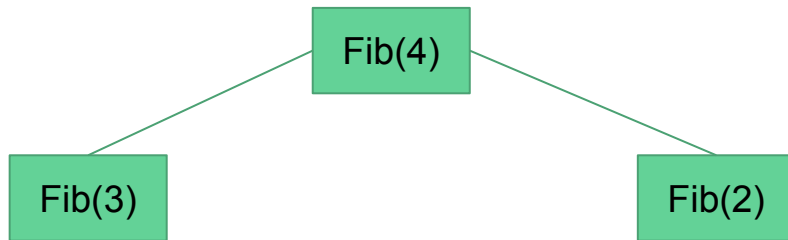
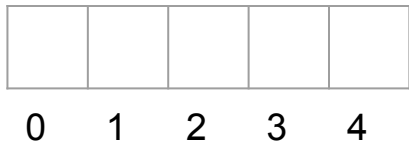
Fib(4)

--	--	--	--	--

0 1 2 3 4

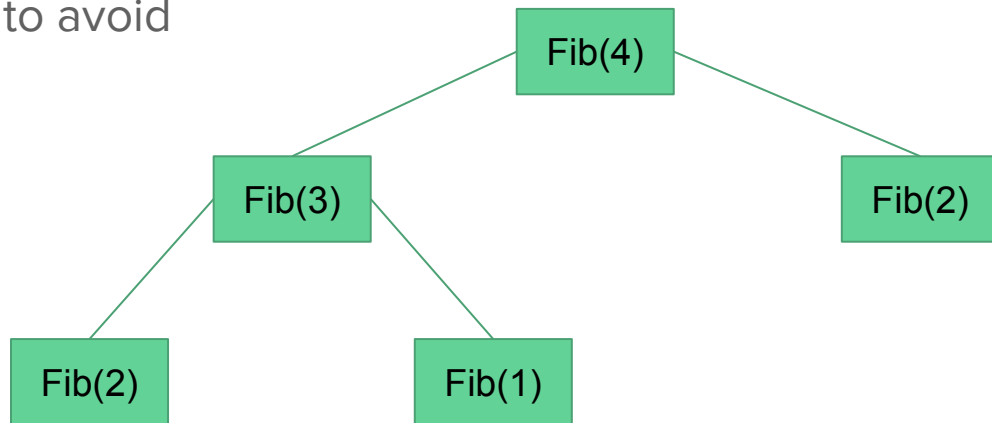
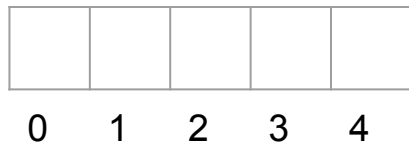
Fibonacci numbers: Memoization

Storing the results of subsubproblems so as to avoid recomputing them



Fibonacci numbers: Memoization

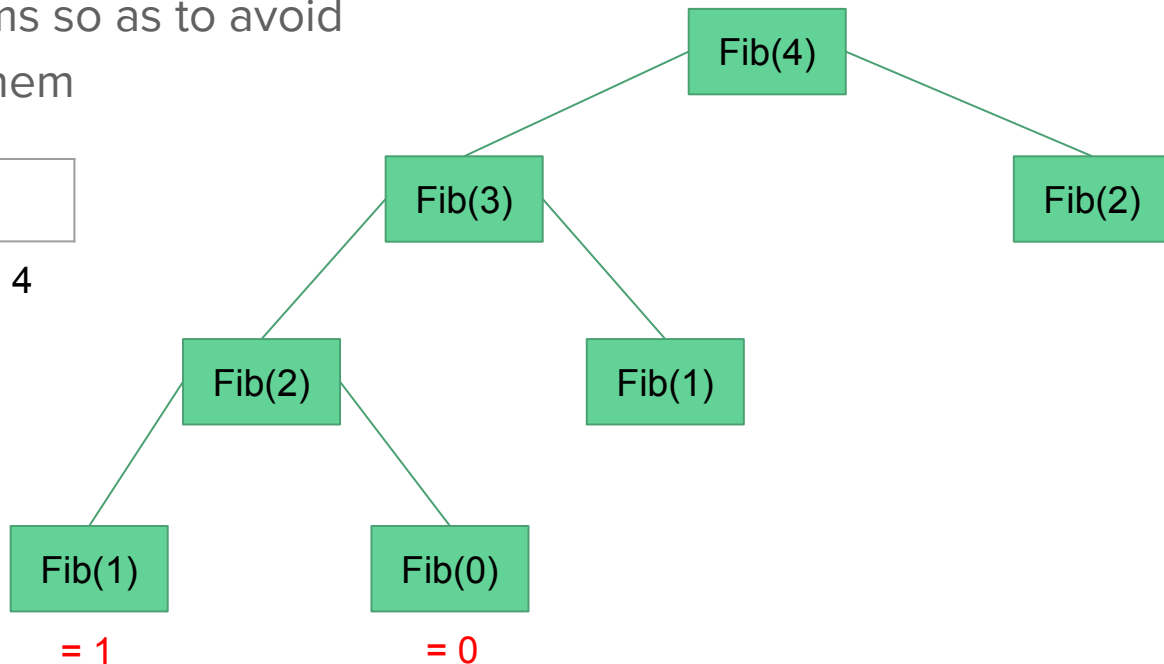
Storing the results of
subsubproblems so as to avoid
recomputing them



Fibonacci numbers: Memoization

Storing the results of
subsubproblems so as to avoid
recomputing them

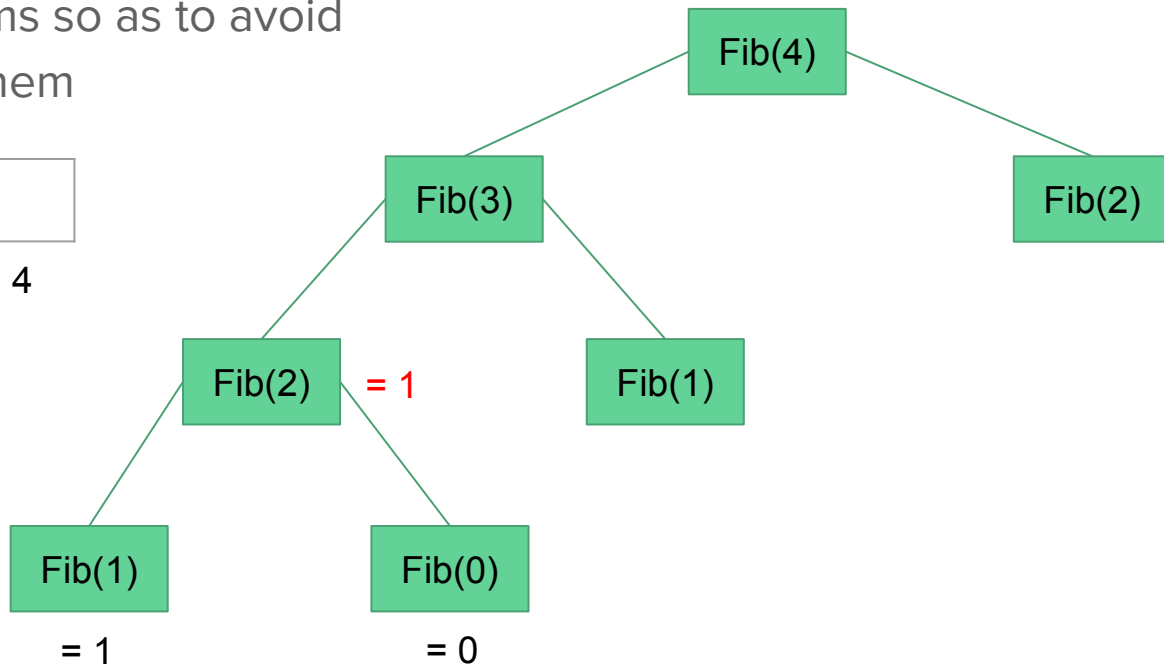
0	1			
0	1	2	3	4



Fibonacci numbers: Memoization

Storing the results of
subsubproblems so as to avoid
recomputing them

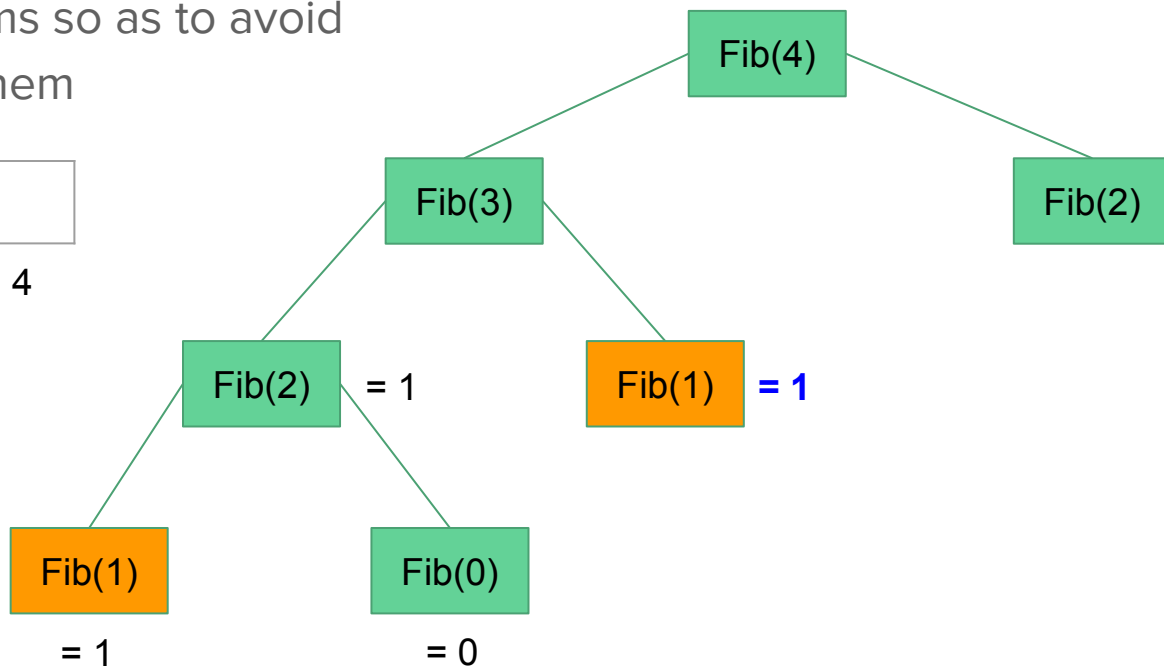
0	1	1		
0	1	2	3	4



Fibonacci numbers: Memoization

Storing the results of
subsubproblems so as to avoid
recomputing them

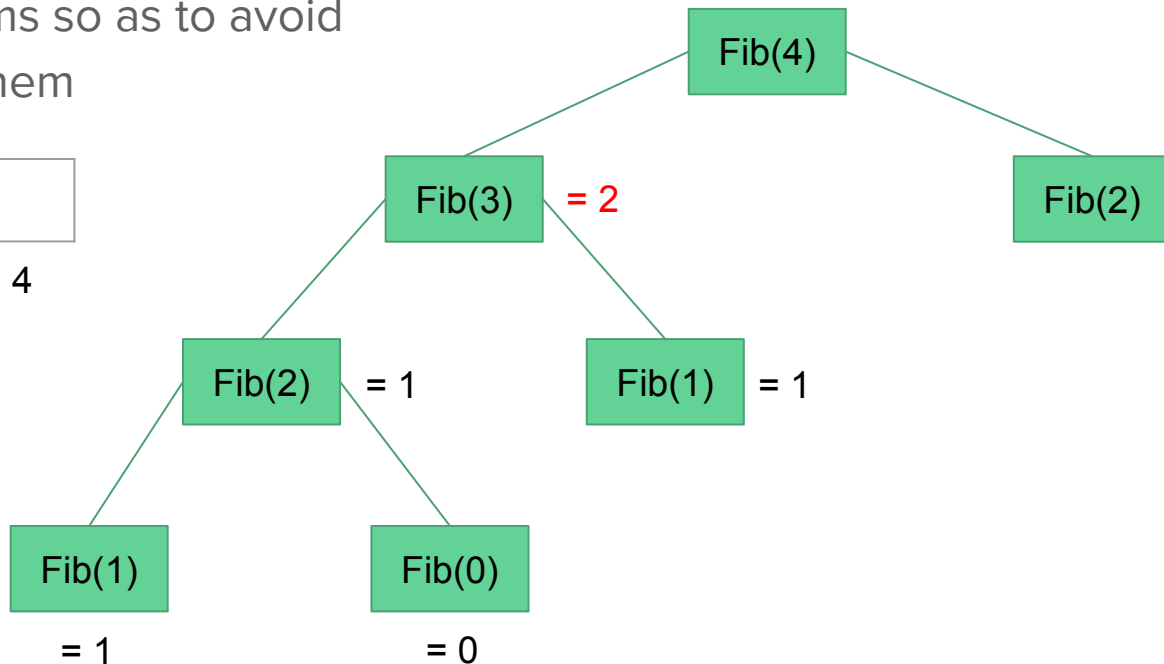
0	1	1		
0	1	2	3	4



Fibonacci numbers: Memoization

Storing the results of
subsubproblems so as to avoid
recomputing them

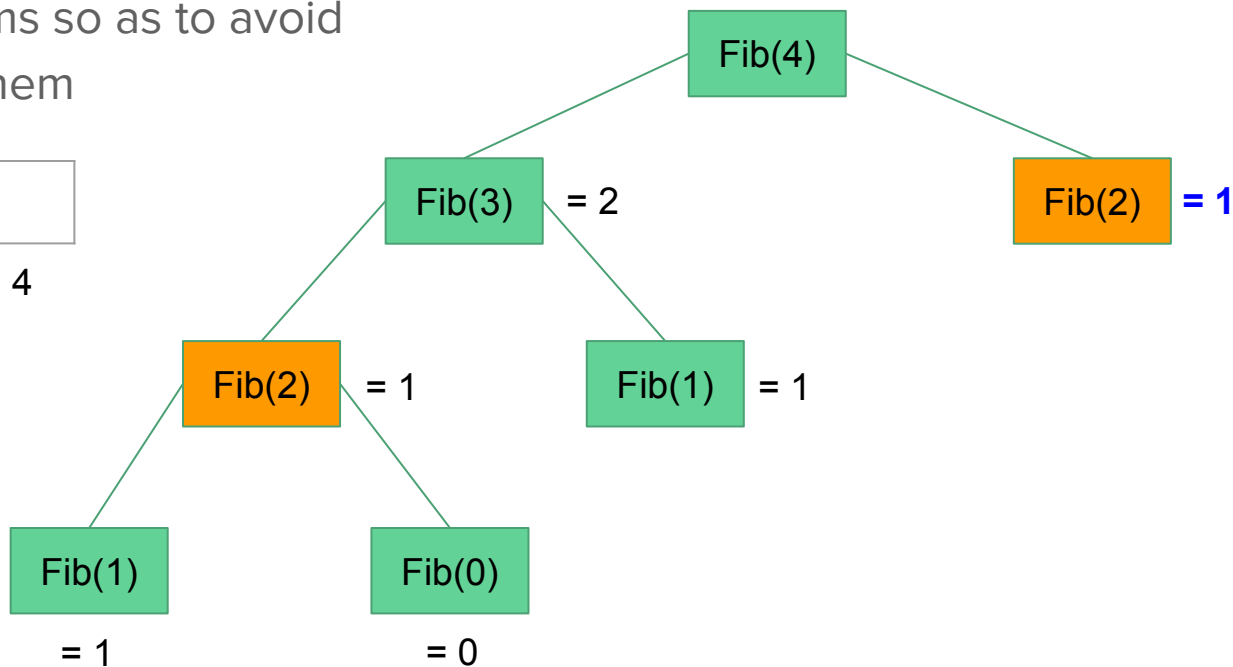
0	1	1	2	
0	1	2	3	4



Fibonacci numbers: Memoization

Storing the results of
subsubproblems so as to avoid
recomputing them

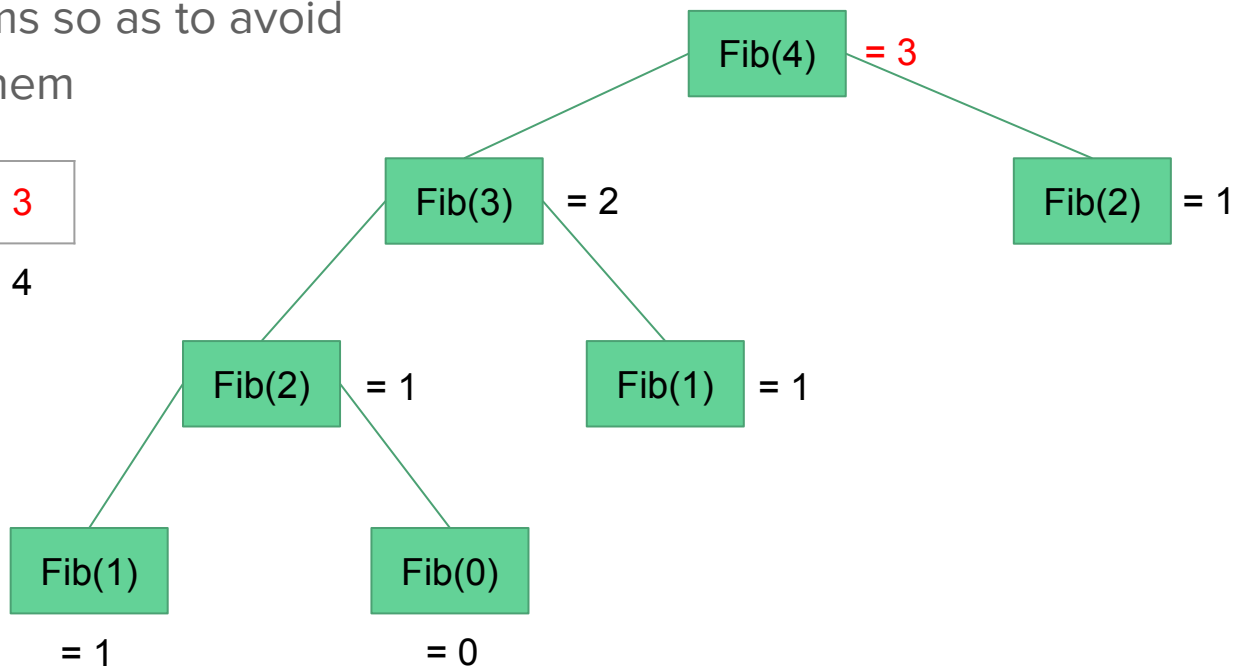
0	1	1	2	
0	1	2	3	4



Fibonacci numbers: Memoization

Storing the results of
subsubproblems so as to avoid
recomputing them

0	1	1	2	3
0	1	2	3	4



Fibonacci numbers: Memoization

```
1. def Fib(n, memo):
2.     if memo[n] != null:
3.         return memo[n]
4.     elif n==0:
5.         result = 0
6.     elif n==1:
7.         result = 1
8.     else:
9.         result = Fib(n-1, memo) + Fib(n-2, memo)
10.    memo[n] = result
11.    return result
```

$T(n) = O(n)$

Fibonacci numbers: bottom-up approach

- Solve all related sub-problems first
- Based on the results in the table, compute the solution to the "top" / original problem

For example, to find $F(4)$, first find out $F(0)$, $F(1)$, and so on, i.e. if the array from left to right

$F(0) = 0$, $F(1) = 1$

0	1			
0	1	2	3	4

Fibonacci numbers: bottom-up approach

- Solve all related sub-problems first
- Based on the results in the table, compute the solution to the "top" / original problem

For example, to find $F(4)$, first find out $F(0)$, $F(1)$, and so on, i.e. if the array from left to right

$$F(0) = 0, F(1) = 1$$

0	1			
---	---	--	--	--

0 1 2 3 4

$$F(2) = F(1) + F(0)$$

0	1	1		
----------	----------	----------	--	--

0 1 2 3 4

Fibonacci numbers: bottom-up approach

- Solve all related sub-problems first
- Based on the results in the table, compute the solution to the "top" / original problem

For example, to find $F(4)$, first find out $F(0)$, $F(1)$, and so on, i.e. if the array from left to right

$$F(0) = 0, F(1) = 1$$

0	1			
---	---	--	--	--

0 1 2 3 4

$$F(2) = F(1) + F(0)$$

0	1	1		
----------	----------	----------	--	--

0 1 2 3 4

$$F(3) = F(2) + F(1)$$

0	1	1	2	
---	---	---	---	--

0 1 2 3 4

Fibonacci numbers: bottom-up approach

- Solve all related sub-problems first
- Based on the results in the table, compute the solution to the "top" / original problem

For example, to find $F(4)$, first find out $F(0)$, $F(1)$, and so on, i.e. if the array from left to right

$$F(0) = 0, F(1) = 1$$

0	1			
---	---	--	--	--

0 1 2 3 4

$$F(2) = F(1) + F(0)$$

0	1	1		
----------	----------	----------	--	--

0 1 2 3 4

$$F(3) = F(2) + F(1)$$

0	1	1	2	
---	----------	----------	----------	--

0 1 2 3 4

$$F(4) = F(3) + F(2)$$

0	1	1	2	3
---	---	----------	----------	----------

0 1 2 3 4

Fibonacci numbers: bottom-up approach

```
def fib_bottom_up(n):  
    if n <= 1:  
        return n  
    else:  
        arr = [0 for i in range(n+1)]  
        arr[1] = 1  
        for i in range(2, n+1):  
            arr[i] = arr[i-1] + arr[i-2]  
        return arr[n]
```

$T(n) = O(n)$
No overhead for recursion

Longest Common Subsequence

A subsequence of a sequence/string S is obtained by deleting zero or more symbols from S .

Examples: subsequences of “algorithms” are alg, lits, oms, agihs etc.

The letters of a subsequence of S appear in order in S , but they are not required to be consecutive

Longest Common Subsequence

The longest common subsequence problem is to find a maximum length common subsequence between two sequences

The LCS of the sequences “president” and “providence” is “priden”

One of the LCSs of the sequences “algorithm” and “alignment” is “algm”

LCS: recursion

```
# Find LCS of two subsequences X and Y of length m and n
# respectively
def lcs(X, Y, m, n):
    if m == len(X) or n == len(Y):
        return 0
    elif X[m] == Y[n]:
        return 1 + lcs(X, Y, m+1, n+1)
    else:
        return max(lcs(X, Y, m, n+1), lcs(X, Y, m+1, n))
```

X[0], Y[0]
A, T

LCS: recursion

```
if m == len(X) or n == len(Y):  
    return 0  
elif X[m] == Y[n]:  
    return 1 + lcs(X, Y, m+1, n+1)  
else:  
    return max(lcs(X, Y, m, n+1), lcs(X, Y, m+1, n))
```

X = "AC"

Y = "TAGC"

LCS: recursion

```
if m == len(X) or n == len(Y):
```

```
    return 0
```

```
elif X[m] == Y[n]:
```

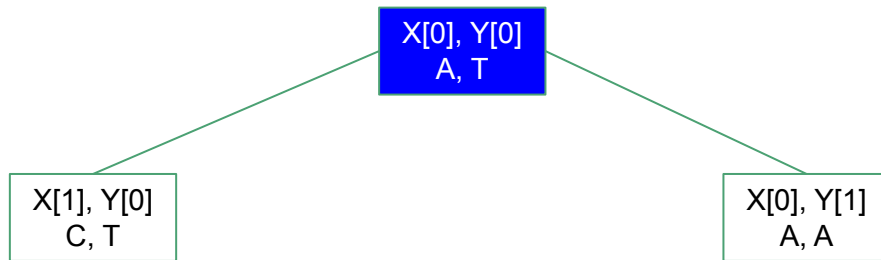
```
    return 1 + lcs(X, Y, m+1, n+1)
```

```
else:
```

```
    return max(lcs(X, Y, m, n+1), lcs(X, Y, m+1, n))
```

X = "AC"

Y = "TAGC"



LCS: recursion

```
if m == len(X) or n == len(Y):
```

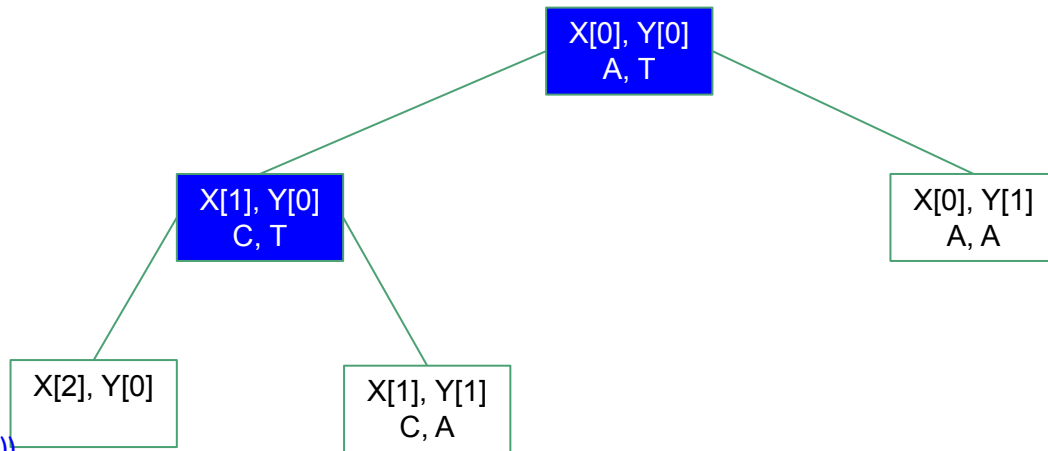
```
    return 0
```

```
elif X[m] == Y[n]:
```

```
    return 1 + lcs(X, Y, m+1, n+1)
```

```
else:
```

```
    return max(lcs(X, Y, m, n+1), lcs(X, Y, m+1, n))
```



$X = \text{"AC"}$

$Y = \text{"TAGC"}$

LCS: recursion

```
if m == len(X) or n == len(Y):
```

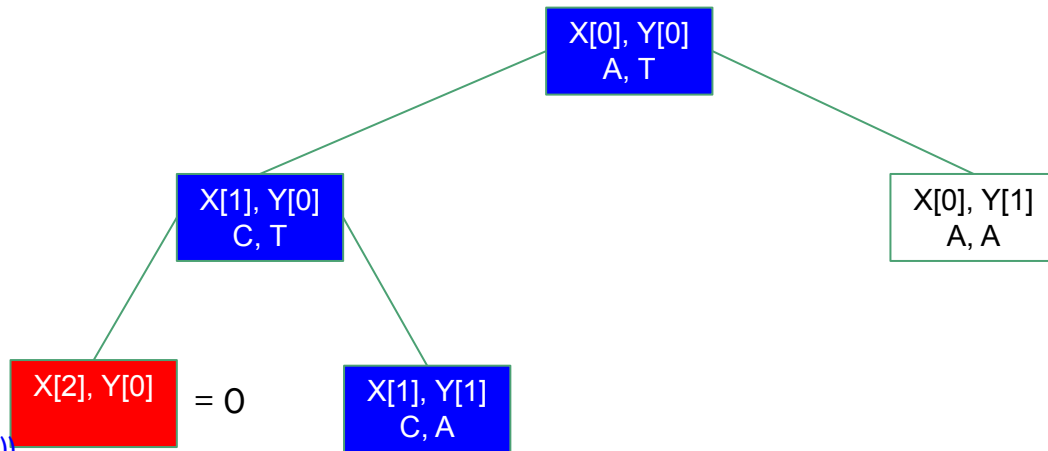
```
    return 0
```

```
elif X[m] == Y[n]:
```

```
    return 1 + lcs(X, Y, m+1, n+1)
```

```
else:
```

```
    return max(lcs(X, Y, m, n+1), lcs(X, Y, m+1, n))
```



X = "AC"

Y = "TAGC"

LCS: recursion

if $m == \text{len}(X)$ or $n == \text{len}(Y)$:

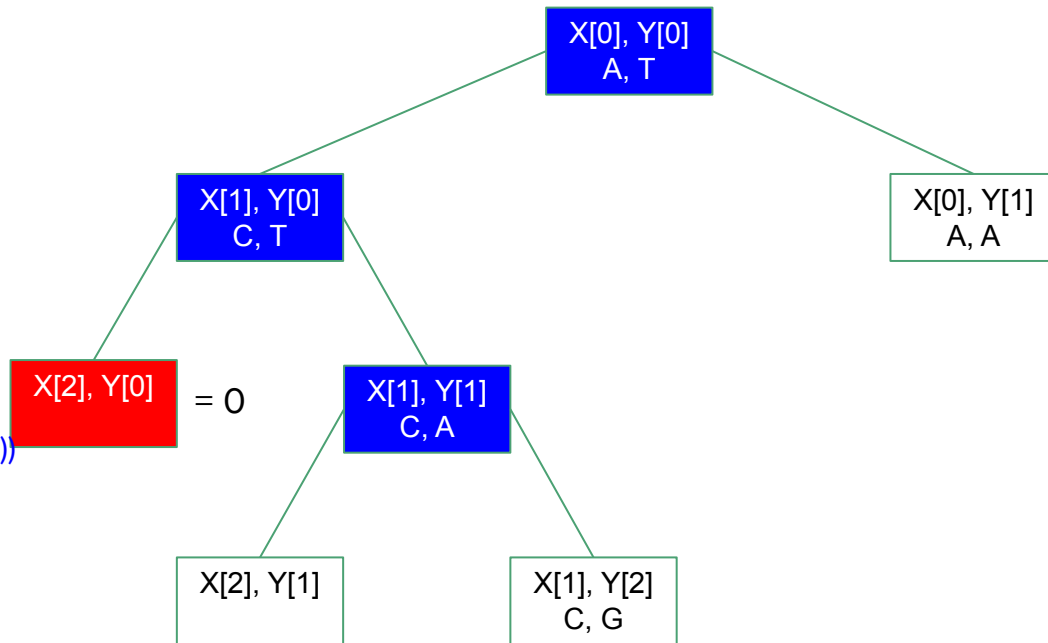
return 0

elif $X[m] == Y[n]$:

return $1 + \text{lcs}(X, Y, m+1, n+1)$

else:

return $\max(\text{lcs}(X, Y, m, n+1), \text{lcs}(X, Y, m+1, n))$



$X = \text{"AC"}$

$Y = \text{"TAGC"}$

LCS: recursion

if $m == \text{len}(X)$ or $n == \text{len}(Y)$:

```
return 0
```

```
elif X[m] == Y[n]:
```

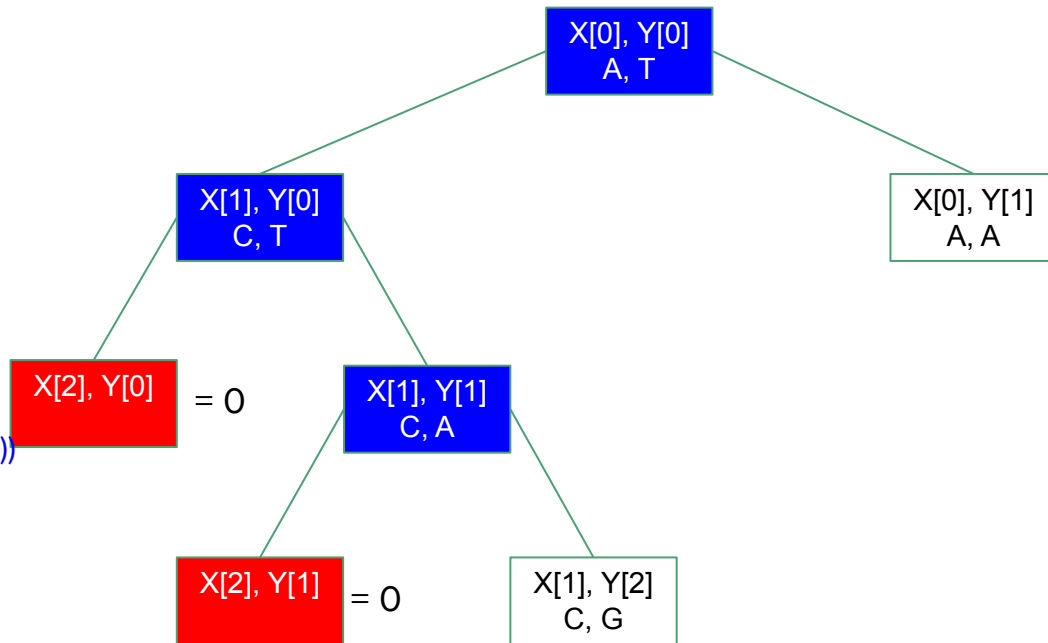
```
return 1 + lcs(X, Y, m+1, n+1)
```

```
else:
```

```
return max(lcs(X, Y, m, n+1), lcs(X, Y, m+1, n))
```

X = "AC"

Y = "TAGC"



LCS: recursion

if $m == \text{len}(X)$ or $n == \text{len}(Y)$:

return 0

elif $X[m] == Y[n]$:

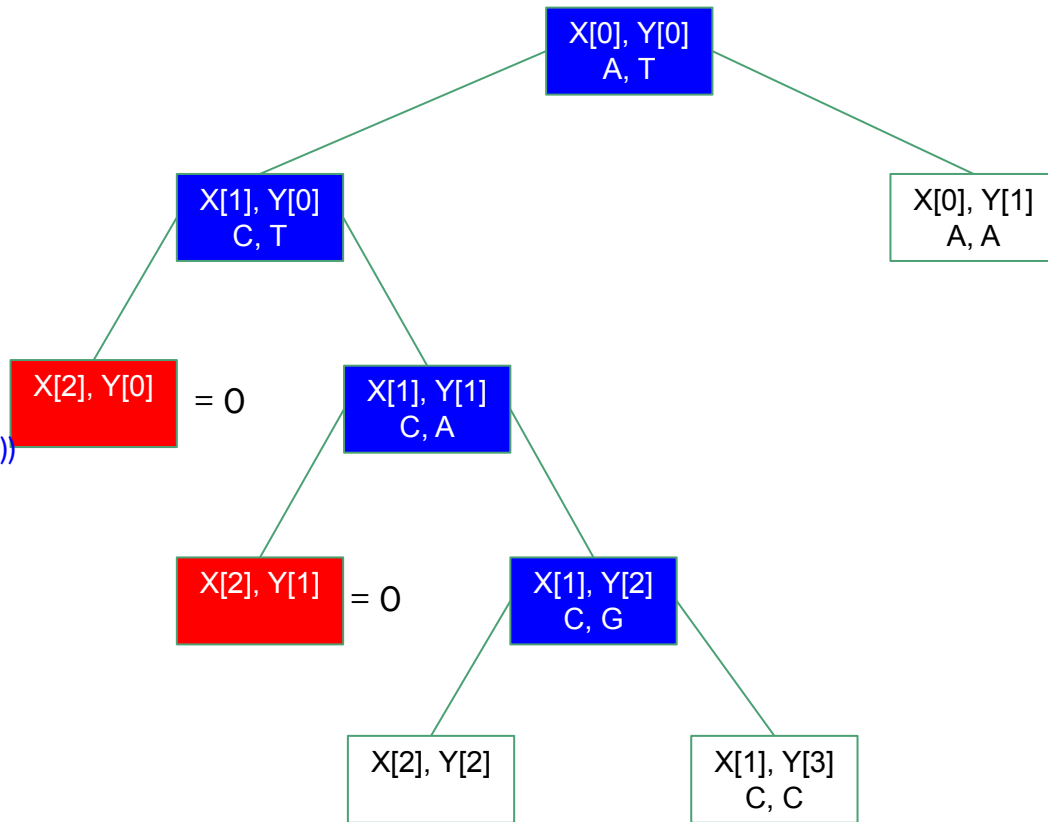
return $1 + \text{lcs}(X, Y, m+1, n+1)$

else:

return $\max(\text{lcs}(X, Y, m, n+1), \text{lcs}(X, Y, m+1, n))$

$X = \text{"AC"}$

$Y = \text{"TAGC"}$



LCS: recursion

if $m == \text{len}(X)$ or $n == \text{len}(Y)$:

return 0

elif $X[m] == Y[n]$:

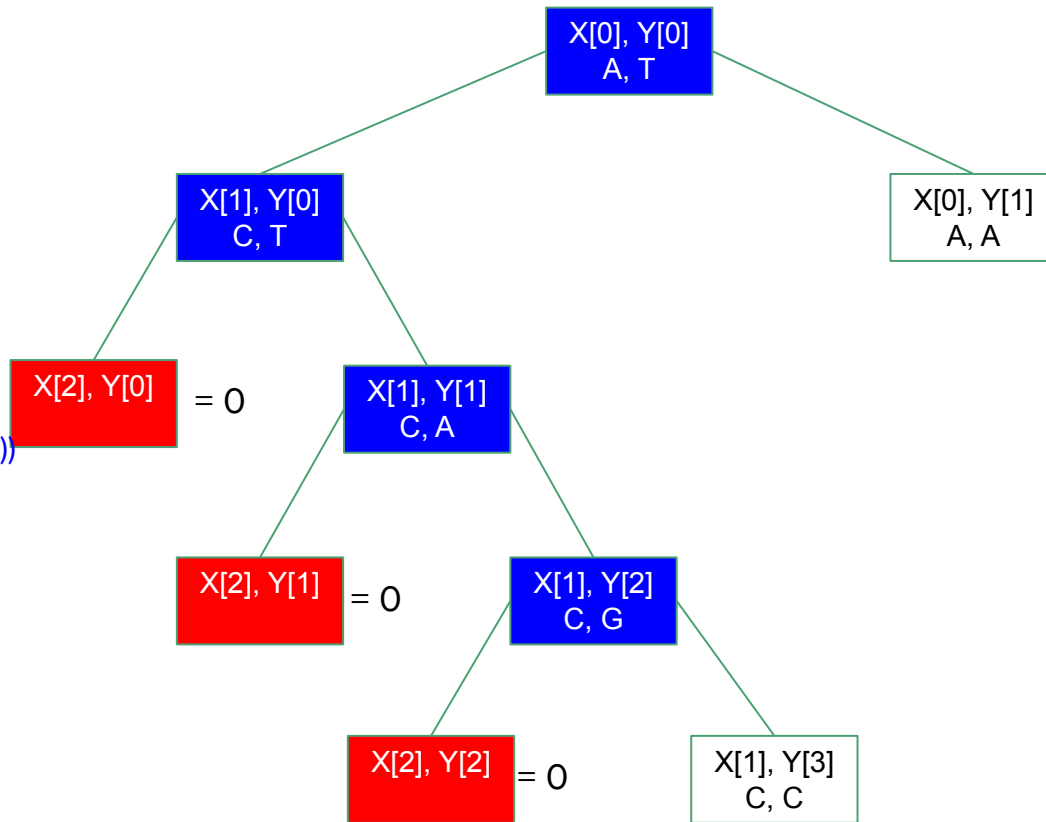
return $1 + \text{lcs}(X, Y, m+1, n+1)$

else:

return $\max(\text{lcs}(X, Y, m, n+1), \text{lcs}(X, Y, m+1, n))$

$X = \text{"AC"}$

$Y = \text{"TAGC"}$



LCS: recursion

if $m == \text{len}(X)$ or $n == \text{len}(Y)$:

return 0

elif $X[m] == Y[n]$:

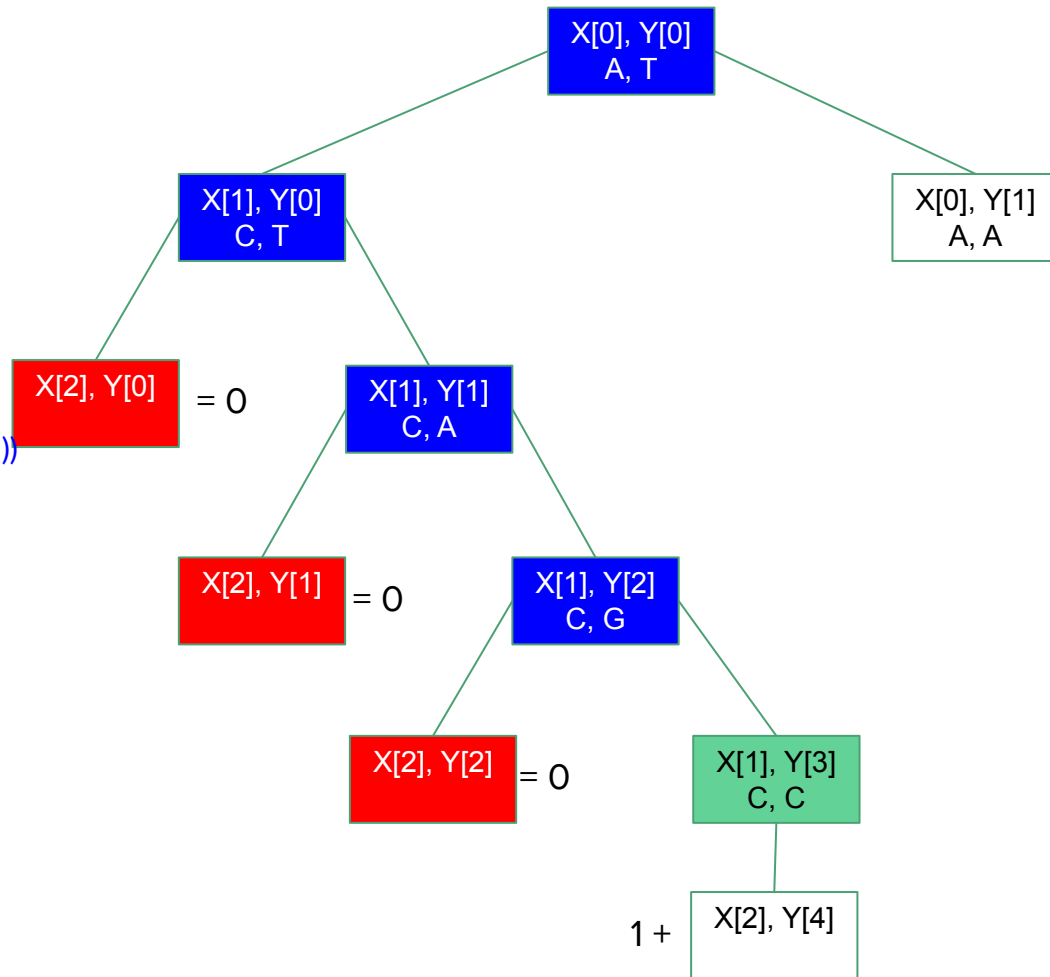
return $1 + \text{lcs}(X, Y, m+1, n+1)$

else:

return $\max(\text{lcs}(X, Y, m, n+1), \text{lcs}(X, Y, m+1, n))$

$X = \text{"AC"}$

$Y = \text{"TAGC"}$



LCS: recursion

if $m == \text{len}(X)$ or $n == \text{len}(Y)$:

return 0

elif $X[m] == Y[n]$:

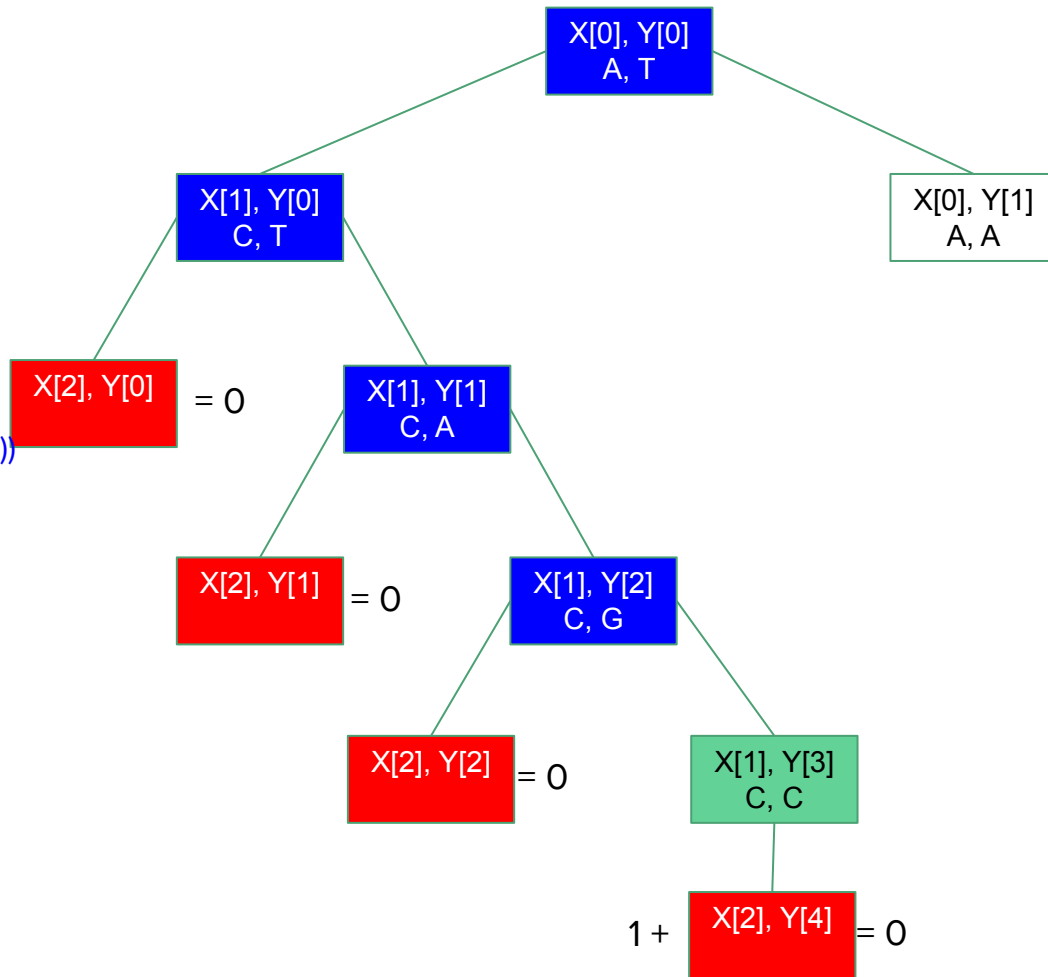
return $1 + \text{lcs}(X, Y, m+1, n+1)$

else:

return $\max(\text{lcs}(X, Y, m, n+1), \text{lcs}(X, Y, m+1, n))$

$X = \text{"AC"}$

$Y = \text{"TAGC"}$



LCS: recursion

if $m == \text{len}(X)$ or $n == \text{len}(Y)$:

return 0

elif $X[m] == Y[n]$:

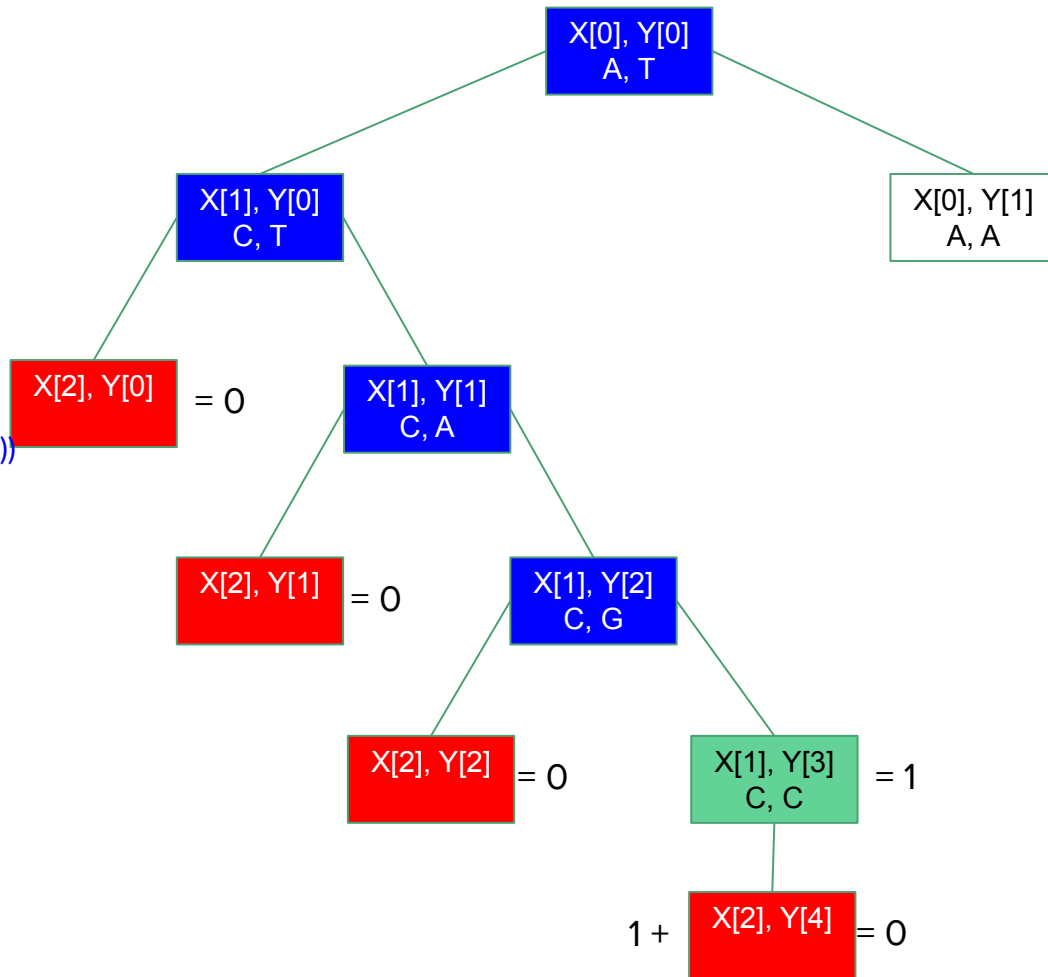
return $1 + \text{lcs}(X, Y, m+1, n+1)$

else:

return $\max(\text{lcs}(X, Y, m, n+1), \text{lcs}(X, Y, m+1, n))$

$X = \text{"AC"}$

$Y = \text{"TAGC"}$



LCS: recursion

if $m == \text{len}(X)$ or $n == \text{len}(Y)$:

return 0

elif $X[m] == Y[n]$:

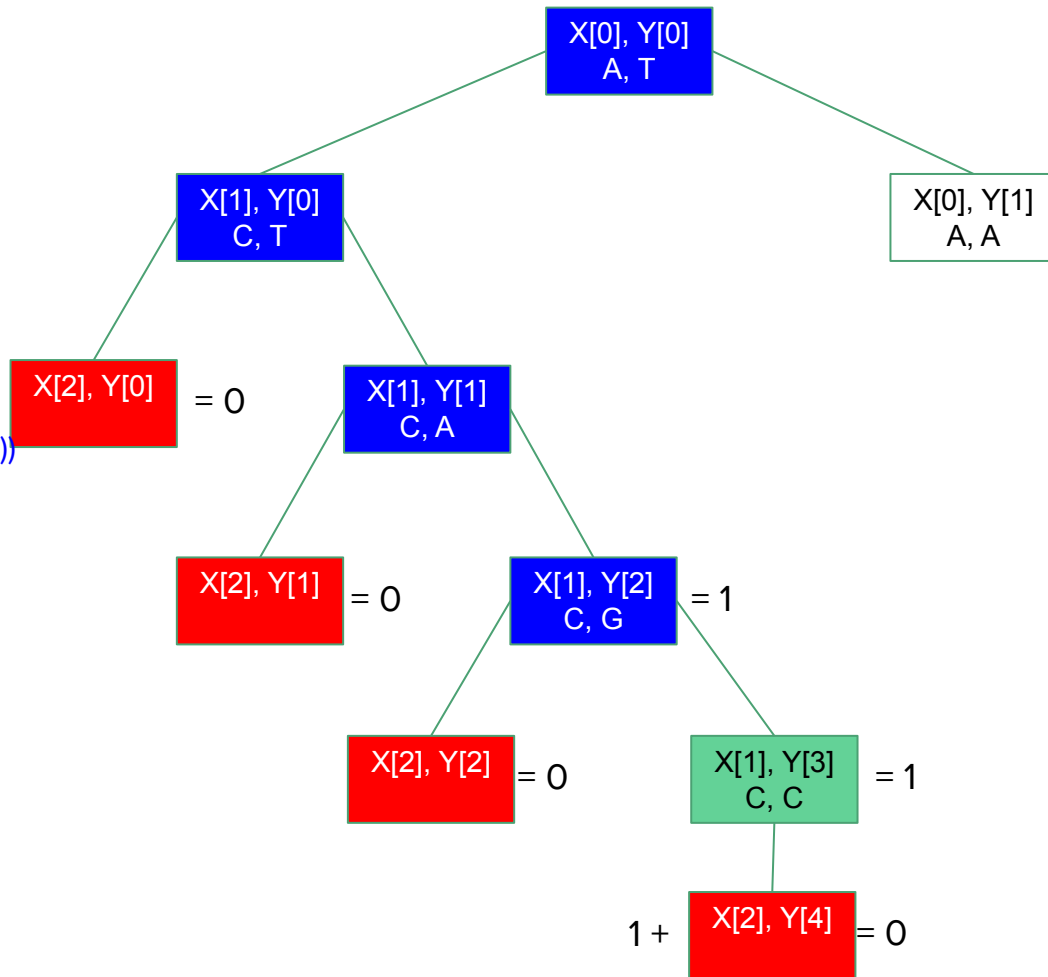
return $1 + \text{lcs}(X, Y, m+1, n+1)$

else:

return $\max(\text{lcs}(X, Y, m, n+1), \text{lcs}(X, Y, m+1, n))$

$X = \text{"AC"}$

$Y = \text{"TAGC"}$



LCS: recursion

if $m == \text{len}(X)$ or $n == \text{len}(Y)$:

return 0

elif $X[m] == Y[n]$:

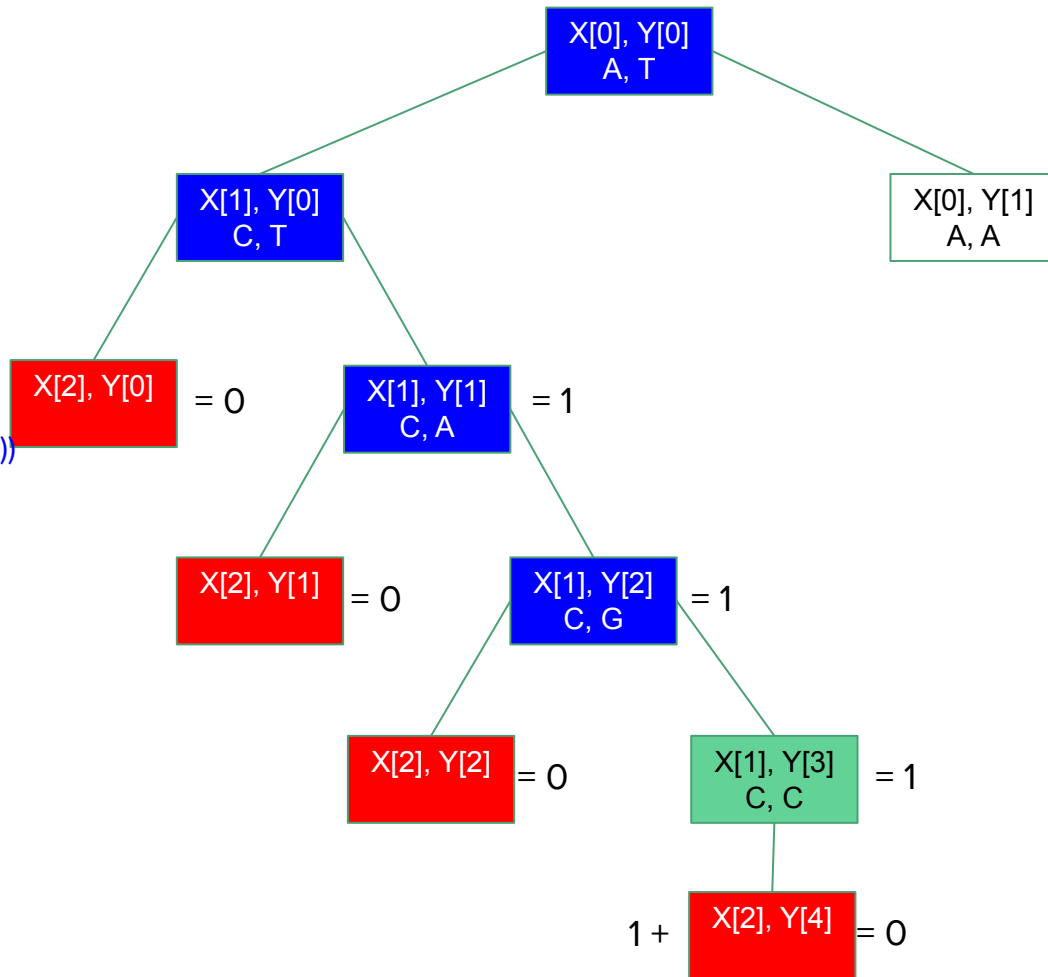
return $1 + \text{lcs}(X, Y, m+1, n+1)$

else:

return $\max(\text{lcs}(X, Y, m, n+1), \text{lcs}(X, Y, m+1, n))$

$X = \text{"AC"}$

$Y = \text{"TAGC"}$



LCS: recursion

if $m == \text{len}(X)$ or $n == \text{len}(Y)$:

return 0

elif $X[m] == Y[n]$:

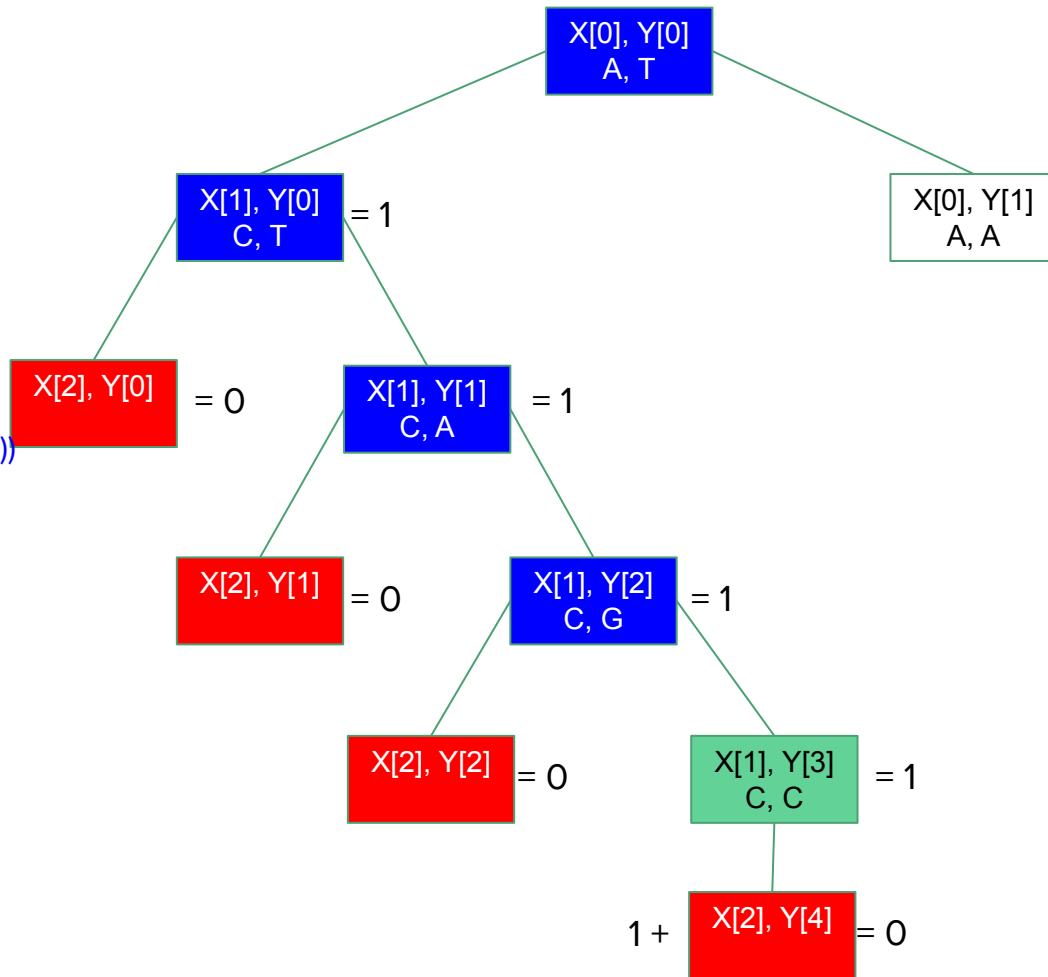
return $1 + \text{lcs}(X, Y, m+1, n+1)$

else:

return $\max(\text{lcs}(X, Y, m, n+1), \text{lcs}(X, Y, m+1, n))$

$X = \text{"AC"}$

$Y = \text{"TAGC"}$



LCS: recursion

if $m == \text{len}(X)$ or $n == \text{len}(Y)$:

return 0

elif $X[m] == Y[n]$:

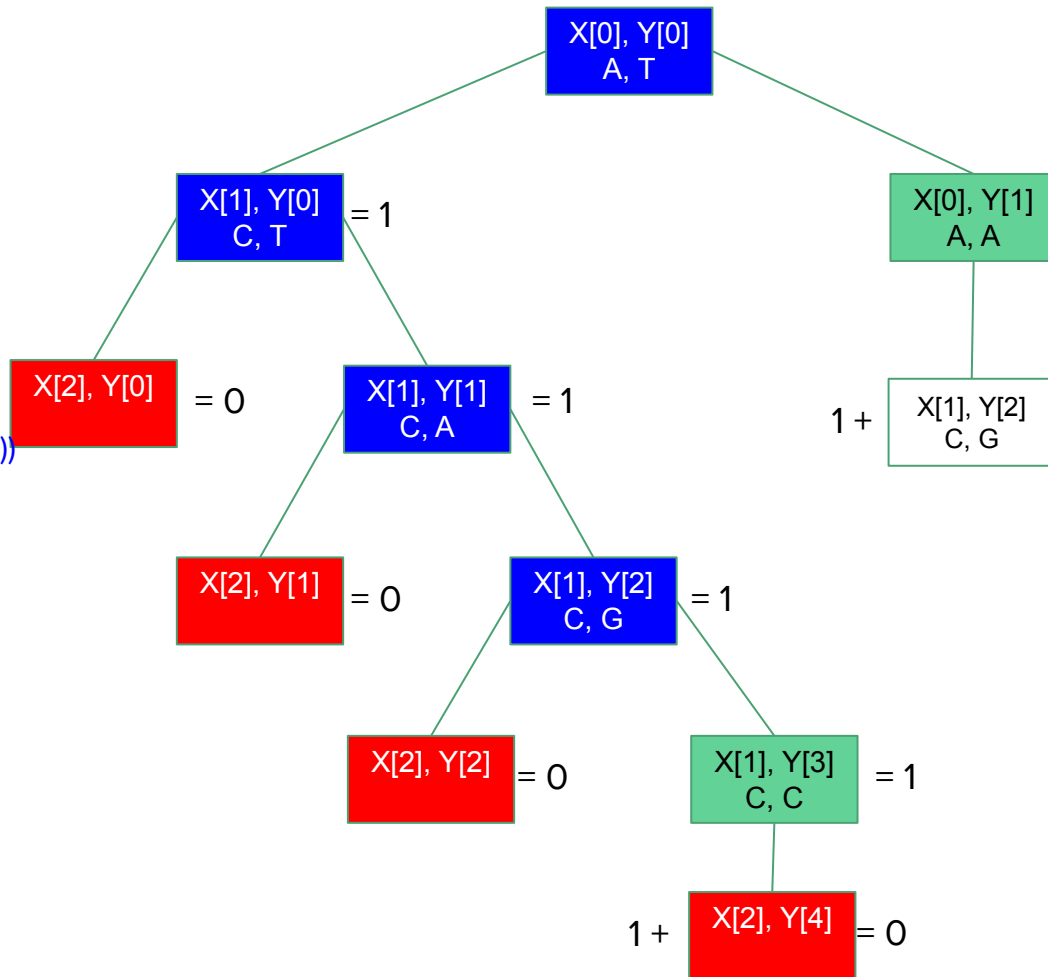
return $1 + \text{lcs}(X, Y, m+1, n+1)$

else:

return $\max(\text{lcs}(X, Y, m, n+1), \text{lcs}(X, Y, m+1, n))$

$X = \text{"AC"}$

$Y = \text{"TAGC"}$



LCS: recursion

if $m == \text{len}(X)$ or $n == \text{len}(Y)$:

return 0

elif $X[m] == Y[n]$:

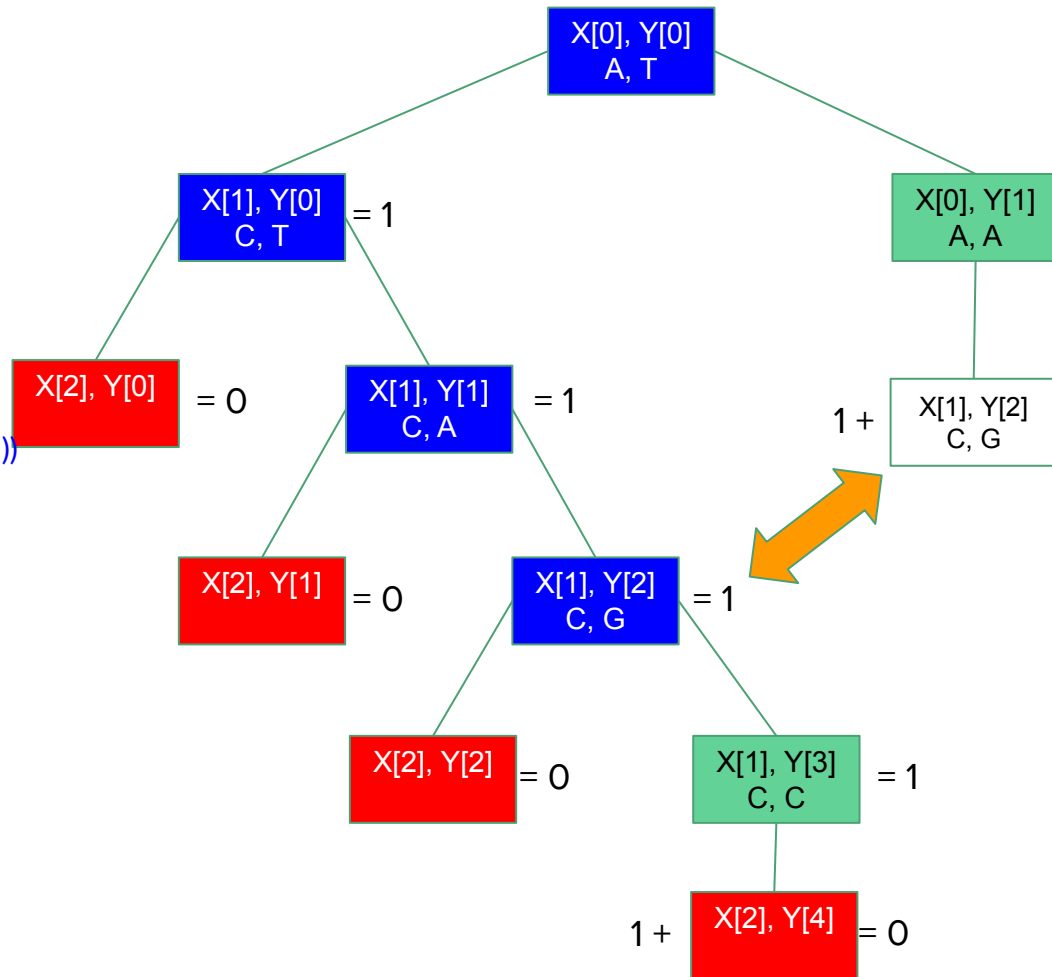
return $1 + \text{lcs}(X, Y, m+1, n+1)$

else:

return $\max(\text{lcs}(X, Y, m, n+1), \text{lcs}(X, Y, m+1, n))$

$X = \text{"AC"}$

$Y = \text{"TAGC"}$



LCS: recursion

if $m == \text{len}(X)$ or $n == \text{len}(Y)$:

return 0

elif $X[m] == Y[n]$:

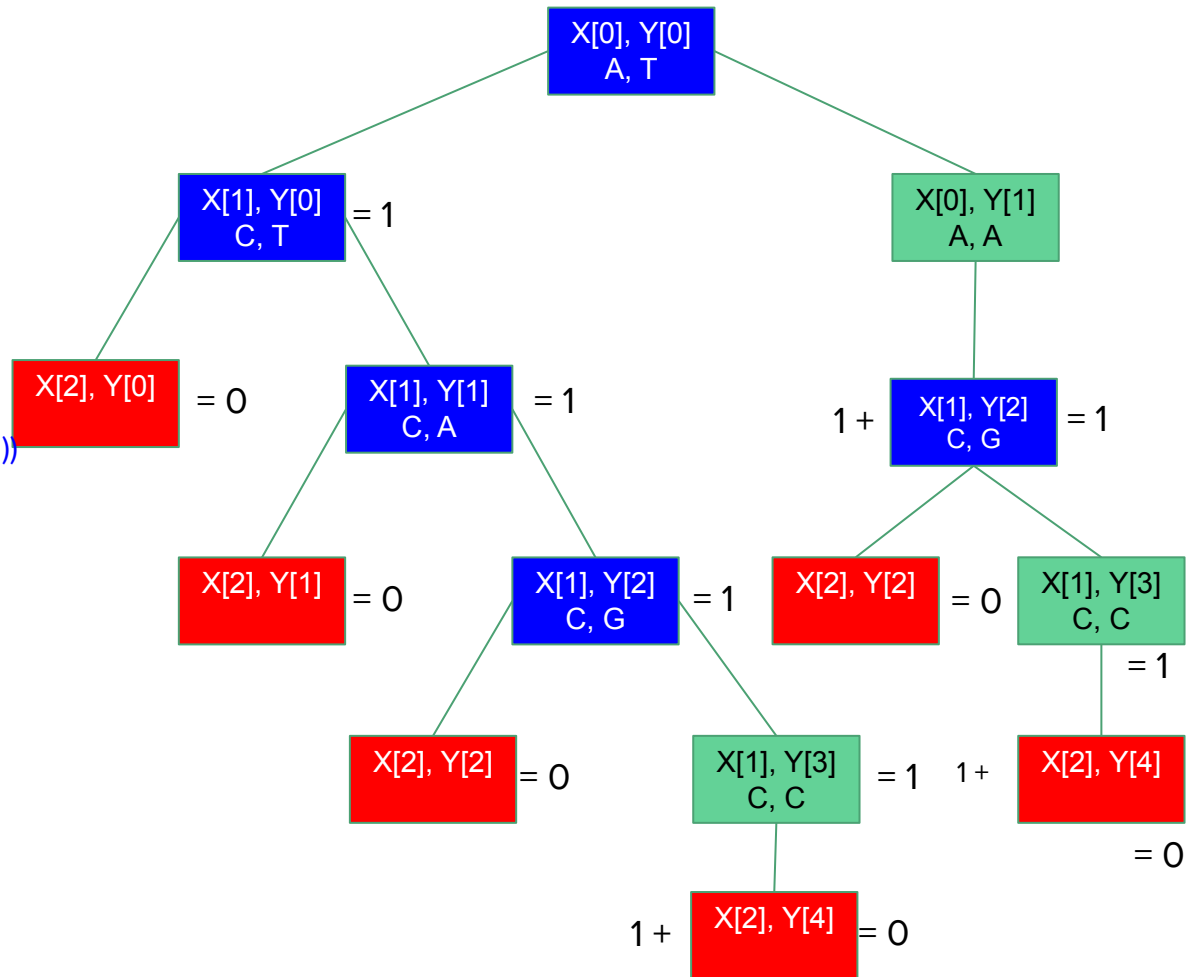
return $1 + \text{lcs}(X, Y, m+1, n+1)$

else:

return $\max(\text{lcs}(X, Y, m, n+1), \text{lcs}(X, Y, m+1, n))$

$X = \text{"AC"}$

$Y = \text{"TAGC"}$



LCS: recursion

if $m == \text{len}(X)$ or $n == \text{len}(Y)$:

return 0

elif $X[m] == Y[n]$:

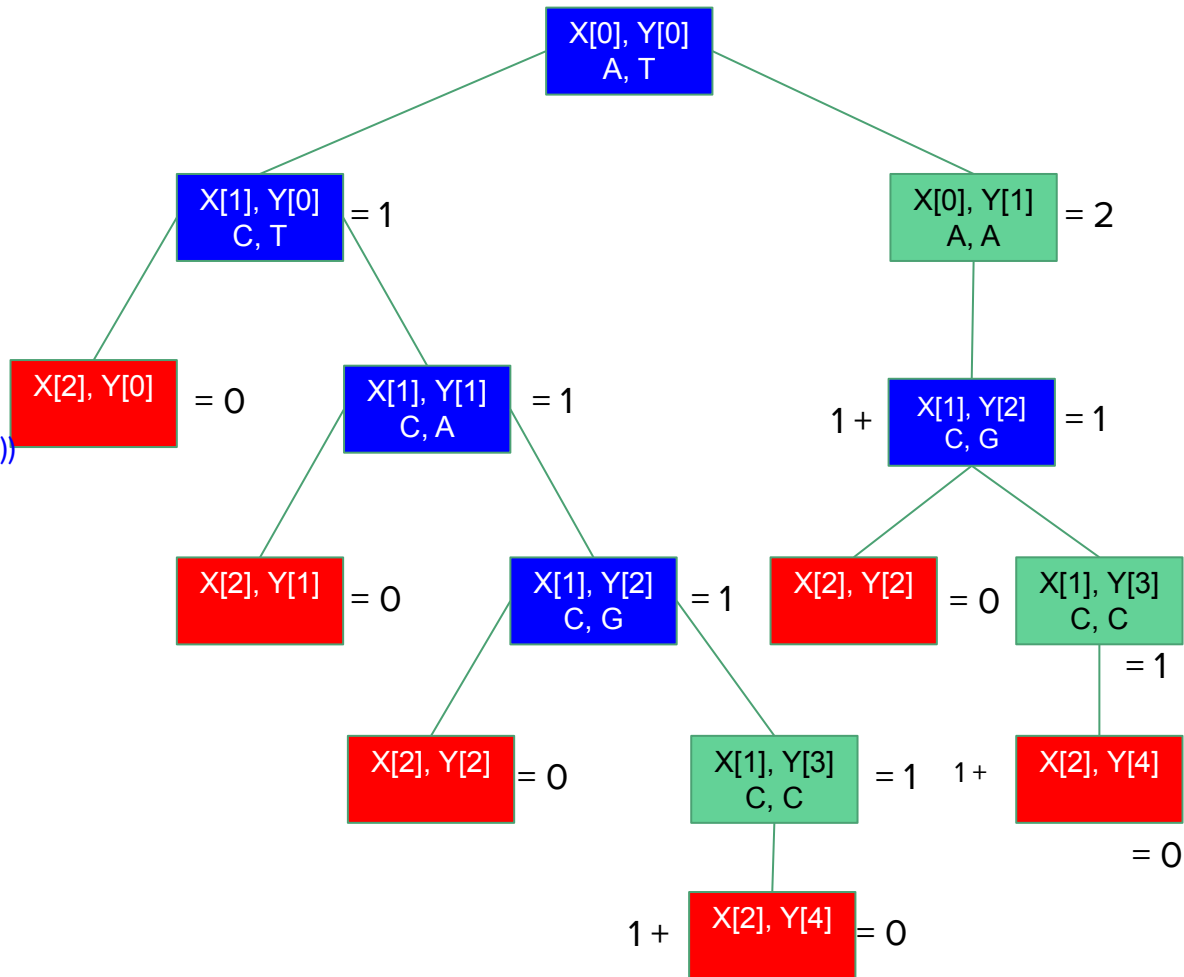
return $1 + \text{lcs}(X, Y, m+1, n+1)$

else:

return $\max(\text{lcs}(X, Y, m, n+1), \text{lcs}(X, Y, m+1, n))$

$X = \text{"AC"}$

$Y = \text{"TAGC"}$



LCS: recursion

if $m == \text{len}(X)$ or $n == \text{len}(Y)$:

return 0

elif $X[m] == Y[n]$:

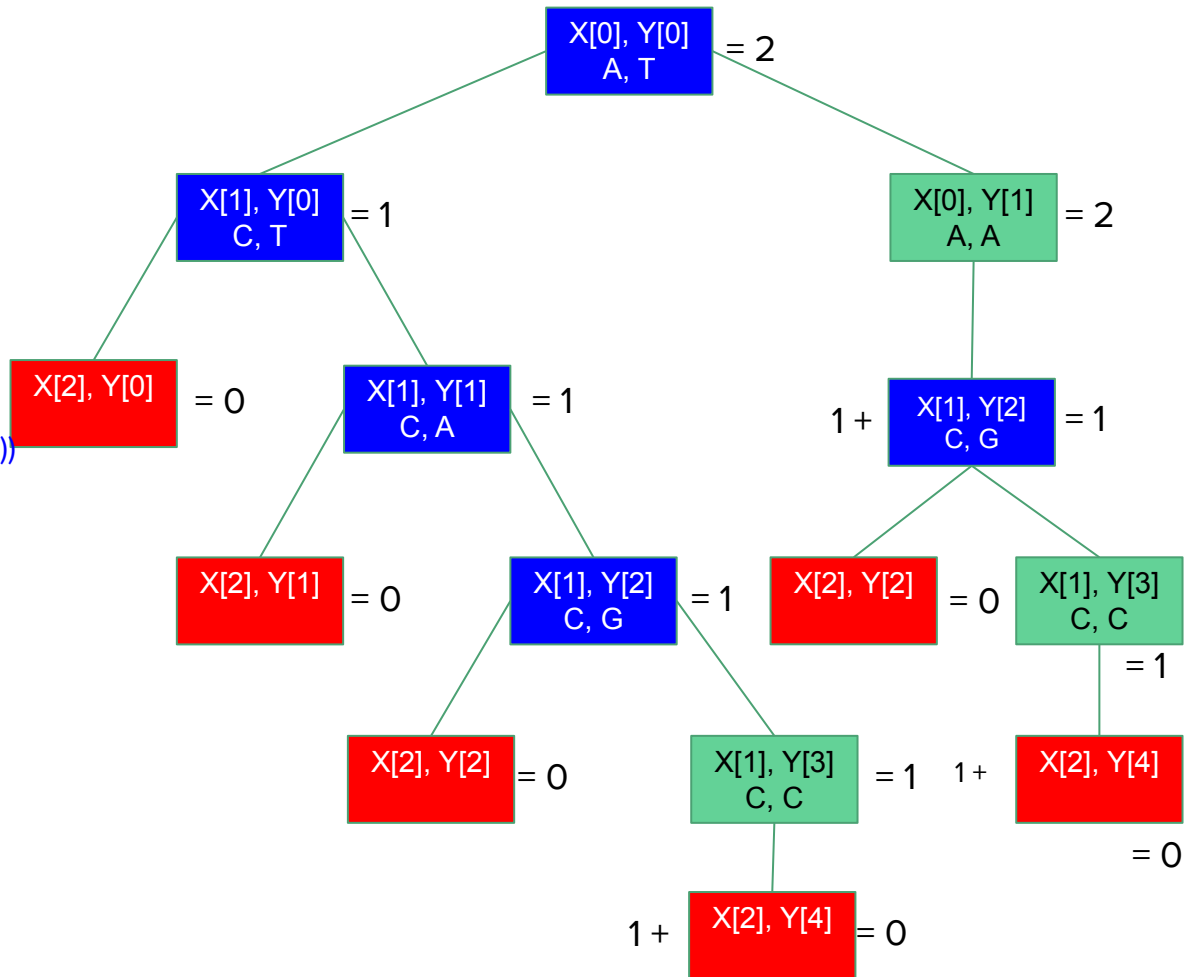
return $1 + \text{lcs}(X, Y, m+1, n+1)$

else:

return $\max(\text{lcs}(X, Y, m, n+1), \text{lcs}(X, Y, m+1, n))$

$X = \text{"AC"}$

$Y = \text{"TAGC"}$



LCS: memoization

Assignment:

Write an algorithm for computing longest common subsequence of two subsequences, A and B, using memoization.

Hint: Store the intermediate results in an $m \times n$ matrix in top-down order, where $m = \text{length of } A + 1$ and $n = \text{length of } B + 1$. For example, if $A = \text{"AC"}$ and $B = \text{"TAGC"}$, then store the intermediate results in the following array according to the previous tree

	0 (T)	1 (A)	2 (G)	3 (C)	4 (\0)
0 (A)					
1 (C)					
2 (\0)					

LCS: bottom-up approach

Let $A = a_1a_2...a_m$ and $B=b_1b_2...b_n$

$len(i, j)$: the length of an LCS between A and B

With proper initializations, $len(i, j)$ can be computed as follows:

$$len(i, j) = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0, \\ len(i-1, j-1) + 1 & \text{if } i, j > 0 \text{ and } a_i = b_j, \\ \max(len(i, j-1), len(i-1, j)) & \text{if } i, j > 0 \text{ and } a_i \neq b_j. \end{cases}$$

LCS: bottom-up approach

Example:

A = “president”,

B = “providence”

LCS: bottom-up approach

Example:

A = “president”,

B = “providence”

		p	r	o	v	i	d	e	n	c	e
p	0	0	0	0	0	0	0	0	0	0	0
r	0	1	1	1	1	1	1	1	1	1	1
e	0	1									
s	0										
i	0										
d	0										
e	0										
n	0										
t	0										

Here, $A[2] \neq B[1]$

Therefore,

$\text{len}(2, 1) = \max(\text{len}(1, 1), \text{len}(2, 0)) = 1$

LCS: bottom-up approach

Example:

A = “president”,

B = “providence”

		p	r	o	v	i	d	e	n	c	e
p	0	0	0	0	0	0	0	0	0	0	0
r	0	1	1	1	1	1	1	1	1	1	1
e	0	1	2	2	2	2	2	3	3	3	3
s	0	1	2	2	2	2	2	3	3	3	3
i	0	1	2	2	2	3	3	3	3	3	3
d	0	1	2	2	2	2	4	4	4	4	4
e	0	1	2	2	2	2	4	5	5	5	5
n	0	1	2	2	2	2	4	5	6	6	6
t	0	1	2	2	2	2	4	5	6	6	6

LCS: bottom-up approach

Example:

A = “president”,

B = “providence”

Length of LCS = $\text{len}(m, n) = 6$

		p	r	o	v	i	d	e	n	c	e
p	0	0	0	0	0	0	0	0	0	0	0
r	0	1	1	1	1	1	1	1	1	1	1
e	0	1	2	2	2	2	2	3	3	3	3
s	0	1	2	2	2	2	2	3	3	3	3
i	0	1	2	2	2	3	3	3	3	3	3
d	0	1	2	2	2	2	4	4	4	4	4
e	0	1	2	2	2	2	4	5	5	5	5
n	0	1	2	2	2	2	4	5	6	6	6
t	0	1	2	2	2	2	4	5	6	6	6

LCS: bottom-up approach

```
def lcs_bottom_up(X, Y):  
    m = len(X) + 1  
    n = len(Y) + 1  
    arr = [[0 for i in range(n)] for j in range(m)]  
    for i in range(1, m):  
        for j in range(1, n):  
            if (X[i-1] == Y[j-1]):  
                arr[i][j] = 1 + arr[i-1][j-1]  
            else:  
                arr[i][j] = max(arr[i-1][j], arr[i][j-1])  
    return arr[m-1][n-1]
```

$$T(n) = O(mn)$$

LCS: bottom-up approach

Example:

A = “president”,

B = “providence”

LCS = “priden”

		p	r	o	v	i	d	e	n	c	e
p	0	0	0	0	0	0	0	0	0	0	0
r	0	1	1	1	1	1	1	1	1	1	1
e	0	1	2	2	2	2	2	3	3	3	3
s	0	1	2	2	2	2	2	3	3	3	3
i	0	1	2	2	2	3	3	3	3	3	3
d	0	1	2	2	2	2	4	4	4	4	4
e	0	1	2	2	2	2	4	5	5	5	5
n	0	1	2	2	2	2	4	5	6	6	6
t	0	1	2	2	2	2	4	5	6	6	6

Matrix chain multiplication

Aka Matrix Product Parenthesization problem

Given : a chain/sequence of matrices $\{A_1, A_2, \dots, A_n\}$

Goal : find the most efficient way to multiply these matrices, i.e. determine an order for multiplying matrices that has the lowest cost

Matrix chain multiplication

Matrix multiplication is **associative**, i.e. no matter how the product is parenthesized, the result obtained will remain the same.

For example, for four matrices A_1 , A_2 , A_3 , and A_4 , we would have:

$(A_1 (A_2 (A_3 A_4)))$,

$(A_1 ((A_2 A_3) A_4))$,

$((A_1 A_2) (A_3 A_4))$,

$((A_1 (A_2 A_3)) A_4)$,

$((A_1 A_2) A_3) A_4$

Recall matrix multiplication

We can multiply two matrices A and B only if they are compatible: the number of columns of A must equal the number of rows of B

If A is a $p \times q$ matrix and B is a $q \times r$ matrix, the resulting matrix C is a $p \times r$ matrix

The time to compute C is dominated by the number of scalar multiplications, which is $p \cdot q \cdot r$.

Example: When multiplying $A_{3 \times 4}$ and $B_{4 \times 5}$ total number of elements in the resulting matrix will be 3×5 , and to get each of those elements, we need 4 multiplications. Thus, total number of multiplications will be $3 \times 5 \times 4 = 60$

Matrix chain multiplication

The way the chain is parenthesized can have a dramatic impact on the cost of evaluating the product.

For example, if A is a 10×30 matrix, B is a 30×5 matrix, and C is a 5×60 matrix, then

computing $(AB)C$ needs $(10 \times 30 \times 5) + (10 \times 5 \times 60) = 1500 + 3000 = 4500$ operations, while

computing $A(BC)$ needs $(30 \times 5 \times 60) + (10 \times 30 \times 60) = 9000 + 18000 = 27000$ operations

Matrix chain multiplication

How to optimize:

- Brute force – look at every possible way to parenthesize
- Dynamic programming

Matrix chain multiplication: Dynamic programming

Steps:

1. Characterize the structure of an optimal solution
2. Recursively define the value of an optimal solution
3. Compute the value of an optimal solution
4. Construct an optimal solution from computed information

1. The structure of an optimal solution

We can build an optimal solution into two subproblems finding optimal solutions to subproblem instances, and then combining these optimal subproblem solutions.

That is, to figure out how to best multiply $A_i \times \dots \times A_j$, we consider all possible middle points k and select the one that minimizes:

Optimal cost to multiply $A_i \dots A_k$
+ Optimal cost to multiply $A_{k+1} \dots A_j$
+ Cost to multiply the results

2. Recursive solution

Let $m[i, j]$ be the minimum number of scalar multiplications needed to compute the matrix $A_i \times \dots \times A_j$

We can define $m[i, j]$ recursively as follows:

$$m[i, j] = \begin{cases} 0 & \text{if } i = j, \\ \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j\} & \text{if } i < j. \end{cases}$$

Our recursive definition for the minimum cost of parenthesizing the product $A_i \times A_{i+1} \times \dots \times A_j$ is

3. Computing the optimal costs

Let us assume that matrix A_i has dimensions $p_{i-1} \times p_i$ for $i = 1, 2, \dots, n$.

Dimensions sequence = $\langle p_0, p_1, p_2, \dots, p_n \rangle$

We define two tables:

1. m of dimensions $n \times n$ for storing the $m[i,j]$ costs
2. s of dimensions $n-1 \times n-1$ for recording which index of k achieved the optimal cost in computing $m[i,j]$

3. Computing the optimal costs

Example:

Given the following matrices:

1. A_1 of dimension 5×4
2. A_2 of dimension 4×6
3. A_3 of dimension 6×2
4. A_4 of dimension 2×7

Dimensions sequence = $\langle 5, 4, 6, 2, 7 \rangle$

3. Computing the optimal costs

Example (Continued):

We define two tables, m and s as follows:

$m =$

	1	2	3	4
1				
2				
3				
4				

$s =$

	2	3	4
1			
2			
3			

Recall $m[i, j]$ = the minimum number of scalar multiplications needed to multiply $A_i \times \dots \times A_j$

3. Computing the optimal costs

Example (Continued):

$m[1, 1] = 0$ because the chain consists of just one matrix A_1 ,
so no multiplications

Similarly, $m[2, 2] = m[3, 3] = m[4, 4] = 0$

	1	2	3	4
1	0			
2		0		
3			0	
4				0

	2	3	4
1			
2			
3			

3. Computing the optimal costs

Example (Continued):

$m[1, 2]$ = the minimum number of multiplications to
compute $A_1 \times A_2$

$$= 5 \times 4 \times 6 = 120$$

Recall: Dimensions of $A_1 = 5 \times 4$
 $A_2 = 4 \times 6$

	1	2	3	4
1	0	120		
2		0		
3			0	
4				0

	2	3	4
1	1		
2			
3			

3. Computing the optimal costs

Example (Continued):

$m[2, 3]$ = the minimum number of multiplications to
compute $A_2 \times A_3$

$$= 4 \times 6 \times 2 = 48$$

Recall: Dimensions of $A_2 = 4 \times 6$
 $A_3 = 6 \times 2$

	1	2	3	4
1	0	120		
2		0	48	
3			0	
4				0

	2	3	4
1	1		
2		2	
3			

3. Computing the optimal costs

Example (Continued):

$m[3, 4]$ = the minimum number of multiplications to
compute $A_3 \times A_4$

$$= 6 \times 2 \times 7 = 84$$

Recall: Dimensions of $A_3 = 6 \times 2$
 $A_4 = 2 \times 7$

	1	2	3	4
1	0	120		
2		0	48	
3			0	84
4				0

	2	3	4
1	1		
2		2	
3			3

3. Computing the optimal costs

Example (Continued):

$m[1, 3]$ = the minimum number of multiplications to compute $A_1 \times A_2 \times A_3$

$$A_1 \times A_2 \times A_3 = (A_1 \times A_2) \times A_3 = A_1 \times (A_2 \times A_3)$$

$$\begin{aligned} \text{Dimensions of } (A_1 \times A_2) &= 5 \times 6 \\ (A_2 \times A_3) &= 4 \times 2 \end{aligned}$$

Number of multiplications in $(A_1 \times A_2) \times A_3 = m[1,2] + m[3,3] + 5 \times 6 \times 2$

Number of multiplications in $A_1 \times (A_2 \times A_3) = m[1,1] + m[2,3] + 5 \times 4 \times 2$

$$\therefore m[1, 3] = \min \{120 + 0 + 60, \mathbf{0 + 48 + 40}\} = 88$$

	1	2	3	4
1	0	120	88	
2		0	48	
3			0	84
4				0

	2	3	4
1	1	1	
2		2	
3			3

3. Computing the optimal costs

Example (Continued):

$m[2, 4]$ = the minimum number of multiplications to compute $A_2 \times A_3 \times A_4$

$$A_2 \times A_3 \times A_4 = (A_2 \times A_3) \times A_4 = A_2 \times (A_3 \times A_4)$$

$$\begin{aligned} \text{Dimensions of } (A_2 \times A_3) &= 4 \times 2 \\ (A_3 \times A_4) &= 6 \times 7 \end{aligned}$$

Number of multiplications in $(A_2 \times A_3) \times A_4 = m[2, 3] + m[4, 4] + 4 \times 2 \times 7$

Number of multiplications in $A_2 \times (A_3 \times A_4) = m[2, 2] + m[3, 4] + 4 \times 6 \times 7$

$$\therefore m[1, 3] = \min \{ \mathbf{48} + \mathbf{0} + \mathbf{56}, \quad 0 + 84 + 168 \} = 104$$

	1	2	3	4
1	0	120	88	
2		0	48	104
3			0	84
4				0

	2	3	4
1	1	1	
2		2	3
3			3

3. Computing the optimal costs

Example (Continued):

$m[1, 4]$ = the minimum number of multiplications to compute $A_1 \times A_2 \times A_3 \times A_4$

$$\begin{aligned} A_1 \times A_2 \times A_3 \times A_4 &= A_1 \times ((A_2 \times A_3) \times A_4) \\ &= (A_1 \times A_2) \times (A_3 \times A_4) \\ &= A_1 \times (A_2 \times (A_3 \times A_4)) \\ &= ((A_1 \times A_2) \times A_3) \times A_4 \\ &= (A_1 \times (A_2 \times A_3)) \times A_4 \end{aligned}$$

	1	2	3	4
1	0	120	88	
2		0	48	104
3			0	84
4				0

	2	3	4
1	1	1	
2		2	3
3			3

3. Computing the optimal costs

Example (Continued):

Since we have already computed $m[2, 4]$ and $m[1, 3]$, we consider the following three multiplications only

$$\begin{aligned} A_1 \times A_2 \times A_3 \times A_4 &= (A_1 \times A_2) \times (A_3 \times A_4) \\ &= A_1 \times (A_2 \times A_3 \times A_4) \\ &= (A_1 \times A_2 \times A_3) \times A_4 \end{aligned}$$

	1	2	3	4
1	0	120	88	
2		0	48	104
3			0	84
4				0

	2	3	4
1	1	1	
2		2	3
3			3

3. Computing the optimal costs

Example (Continued):

Dimensions of $(A_1 \times A_2 \times A_3) = 5 \times 2$
 $(A_2 \times A_3 \times A_4) = 4 \times 7$

Number of multiplications in $(A_1 \times A_2 \times A_3) \times A_4$
 $= m[1, 3] + m[4, 4] + 5 \times 2 \times 7$
 $= 88 + 0 + 70$

Number of multiplications in $A_1 \times (A_2 \times A_3 \times A_4)$
 $= m[1, 1] + m[2, 4] + 5 \times 4 \times 7$
 $= 0 + 104 + 140$

	1	2	3	4
1	0	120	88	
2		0	48	104
3			0	84
4				0

	2	3	4
1	1	1	
2		2	3
3			3

3. Computing the optimal costs

Example (Continued):

$$\begin{aligned} \text{Dimensions of } (A_1 \times A_2) &= 5 \times 6 \\ (A_3 \times A_4) &= 6 \times 7 \end{aligned}$$

$$\begin{aligned} \text{Number of multiplications in } (A_1 \times A_2) \times (A_3 \times A_4) \\ &= m[1, 2] + m[3, 4] + 5 \times 6 \times 7 \\ &= 120 + 84 + 210 \end{aligned}$$

$$\therefore m[1, 4] = \min \{ \mathbf{88} + \mathbf{0} + \mathbf{70}, 0 + 104 + 140, 120 + 84 + 210 \} = 158$$

That is the number of multiplications is minimum for $(A_1 \times A_2 \times A_3) \times A_4$

	1	2	3	4
1	0	120	88	158
2		0	48	104
3			0	84
4				0

	2	3	4
1	1	1	3
2		2	3
3			3

4. Constructing an optimal solution

The table $s[1...n-1, 2...n]$ gives us the information needed to construct an optimal solution

	2	3	4
1	1	1	3
2		2	3
3			3

4. Constructing an optimal solution

The table $s[1...n-1, 2...n]$ gives us the information needed to construct an optimal solution

$$A_1 \times A_2 \times A_3 \times A_4 = (A_1 \times A_2 \times A_3) \times A_4$$

	2	3	4
1	1	1	3
2		2	3
3			3

4. Constructing an optimal solution

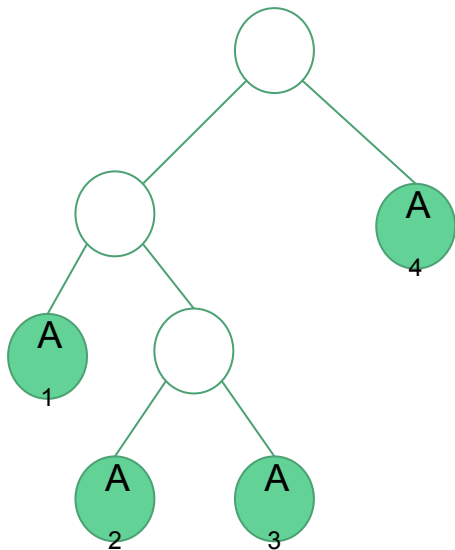
The table $s[1...n-1, 2...n]$ gives us the information needed to construct an optimal solution

$$A_1 \times A_2 \times A_3 \times A_4 = (A_1 \times (A_2 \times A_3)) \times A_4$$

	2	3	4
1	1	1	3
2		2	3
3			3

4. Constructing an optimal solution

$$A_1 \times A_2 \times A_3 \times A_4 = (A_1 \times (A_2 \times A_3)) \times A_4$$



Chapter 3:

Algorithm Design Strategies

(Part II)

Contents

- Brute-force algorithms
 - Greedy algorithms
 - Action-selection problem
 - Huffman coding
 - Minimum spanning tree algorithms
 - Kruskal's, Prim's
 - Shortest path algorithm - Dijkstra's
 - Flow networks - Ford Fulkerson algorithm
 - Divide and Conquer
 - Backtracking
 - N-queen problem
 - Branch-and-bound
 - 0/1 Knapsack problem
-

Traversal techniques

Graph traversal

Process of visiting each vertex in a graph

Given a graph, $G = (V, E)$, and a vertex, $v \in V(G)$, visit all vertices in G that are reachable from v

2 ways of doing this:

1. Depth-first search (DFS)
2. Breadth-first search (BFS)

Depth-first search (DFS)

Process all descendants of a vertex before we move to an adjacent vertex.

DFS in a graph is similar to DFS in a tree. Since graphs may contain cycles unlike trees, we may come to the same node again. To avoid processing a node more than once, we keep track of visited nodes.

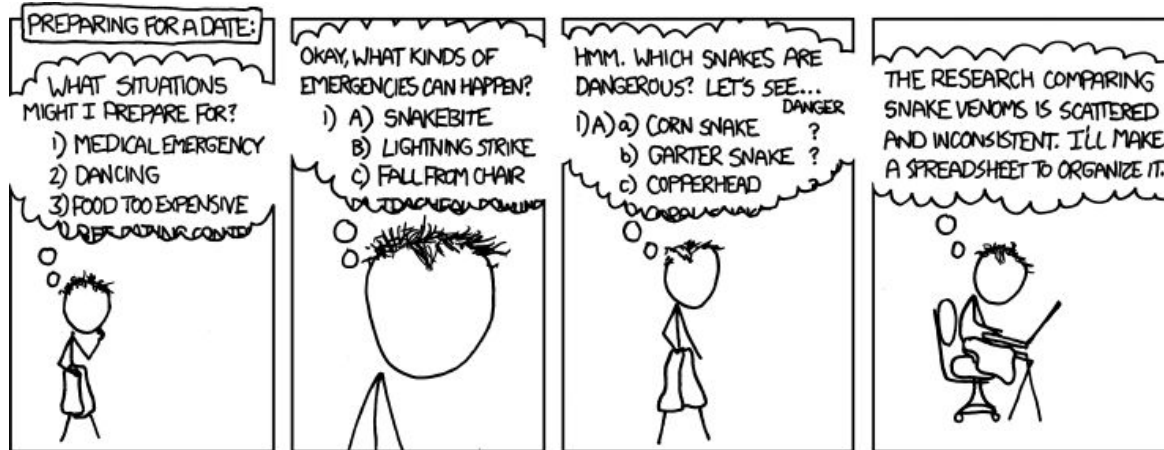
Uses a stack data structure to perform the search.

Depth-first search (DFS)

Basic idea:

1. Start by putting any one of the graph's vertices (starting vertex) on top of a stack.
2. Pop the topmost item from the stack and add it to the visited list.
3. Push the popped vertex's unvisited neighbors into the top of stack.
4. Keep repeating steps 2 and 3 until the stack is empty.

DFS



I REALLY NEED TO STOP USING DEPTH-FIRST SEARCHES.

<https://xkcd.com/761/>

Depth-first search (DFS)

Algorithm: (Recursive) **DFS(G, s)**

Input: A graph, G , and a starting vertex, s

Output: A sequence of processed vertices

Steps:

1. $\text{mark}(s);$ // Mark s as visited
2. $\forall (s, v) \in E(G)$
 - a. $\text{DFS}(G, v)$

Depth-first search (DFS)

Algorithm: (Iterative) **DFS**(**G**, **s**)

Input: A graph, **G**, and a starting vertex, **s**

Output: A sequence of processed vertices

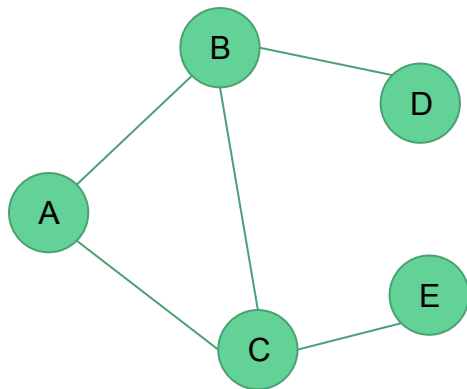
Steps:

1. `mark(s);` // Mark **s** as visited
2. `L := {s}` // Push **s** into the stack

3. **while** `L ≠ ∅` **do**
 - a. `u := last(L)` // Top of the stack
 - b. **if** `∃ (u, v)` such that **v** is unmarked **then** // Find neighbors of **u**
 - i. choose **v** of the smallest index;
 - ii. `mark(v); L := L ∪ {v}`
 - c. **else**
 - i. `L := L \ {u}` // Pop from the stack
 - d. **endif**
4. **endwhile**

Depth-first search (DFS)

Example: Perform DFS on the following graph starting from A



Stack

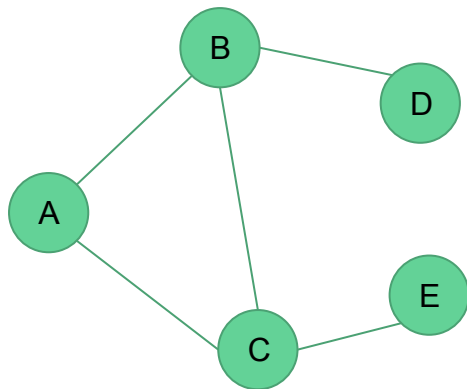


Visited vertices



Depth-first search (DFS)

Example: Perform DFS on the following graph starting from A



Stack



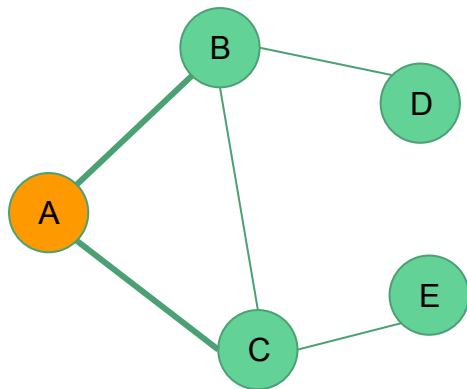
Visited vertices



1. `mark(s);` // Mark s as visited
2. `L = {s}` // Push s into the stack

Depth-first search (DFS)

Example: Perform DFS on the following graph starting from A



Stack



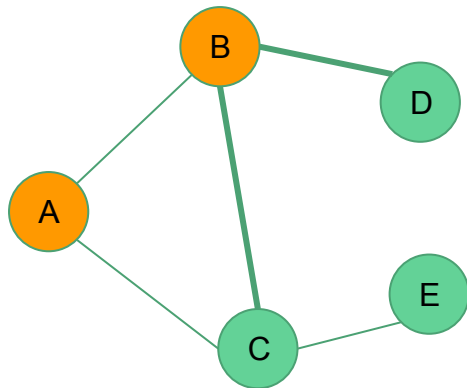
Visited vertices



```
3.  while L ≠ ∅ do
    a.  u ← last(L)    // Top of the stack
    b.  if ∃ (u, v) such that v is unmarked then
        // Find neighbors of u
        i.  choose v of the smallest index;
        ii. mark(v); L ← L ∪ {v}
    c.  else
        i.  L ← L \ {u} // Pop from the stack
    d.  endif
4.  endwhile
```

Depth-first search (DFS)

Example: Perform DFS on the following graph starting from A



Stack



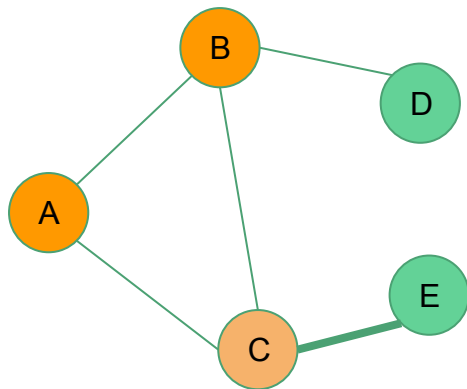
Visited vertices

A	B	C		
---	---	---	--	--

```
3.  while L ≠ ∅ do
    a.  u ← last(L)  // Top of the stack
    b.  if ∃ (u, v) such that v is unmarked then
        // Find neighbors of u
        i.  choose v of the smallest index;
        ii. mark(v); L ← L ∪ {v}
    c.  else
        i.  L ← L \ {u} // Pop from the stack
    d.  endif
4.  endwhile
```

Depth-first search (DFS)

Example: Perform DFS on the following graph starting from A



Stack



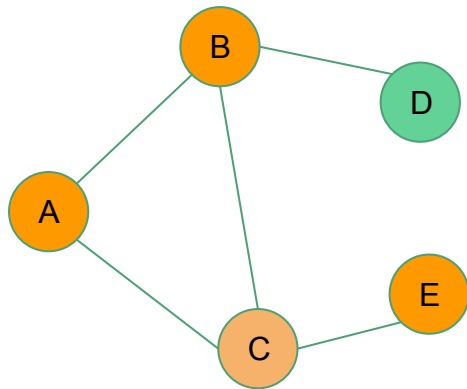
Visited vertices

A	B	C	E	
---	---	---	---	--

```
3.  while L ≠ ∅ do
    a.  u ← last(L)  // Top of the stack
    b.  if ∃ (u, v) such that v is unmarked then
        // Find neighbors of u
        i.  choose v of the smallest index;
        ii. mark(v); L ← L ∪ {v}
    c.  else
        i.  L ← L \ {u}  // Pop from the stack
    d.  endif
4.  endwhile
```


Depth-first search (DFS)

Example: Perform DFS on the following graph starting from A



Stack



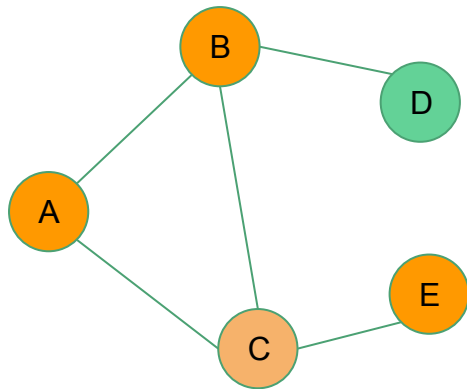
Visited vertices

A	B	C	E	
---	---	---	---	--

```
3.  while L ≠ ∅ do
    a.  u ← last(L)  // Top of the stack
    b.  if ∃ (u, v) such that v is unmarked then
        // Find neighbors of u
        i.  choose v of the smallest index;
        ii. mark(v); L ← L ∪ {v}
    c.  else
        i.  L ← L \ {u} // Pop from the stack
    d.  endif
4.  endwhile
```

Depth-first search (DFS)

Example: Perform DFS on the following graph starting from A



Stack



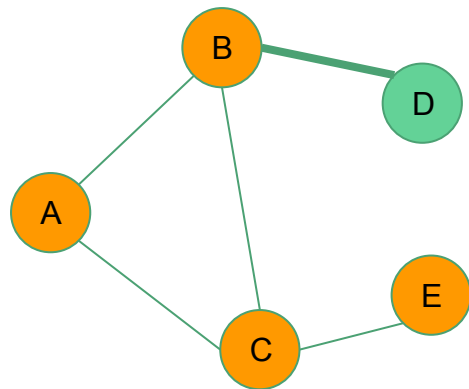
Visited vertices

A	B	C	E	
---	---	---	---	--

```
3.  while L ≠ ∅ do
    a.  u ← last(L)  // Top of the stack
    b.  if ∃ (u, v) such that v is unmarked then
        // Find neighbors of u
        i.  choose v of the smallest index;
        ii. mark(v); L ← L ∪ {v}
    c.  else
        i.  L ← L \ {u} // Pop from the stack
    d.  endif
4.  endwhile
```

Depth-first search (DFS)

Example: Perform DFS on the following graph starting from A



Stack



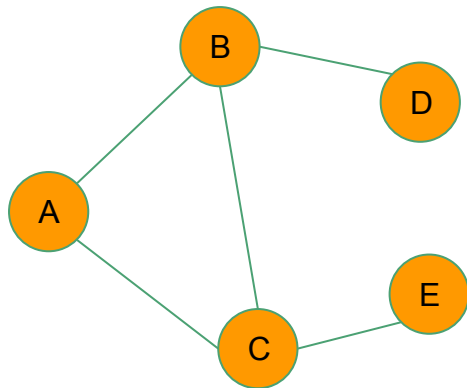
Visited vertices

A	B	C	E	D
---	---	---	---	---

```
3. while L ≠ ∅ do
  a. u ← last(L) // Top of the stack
  b. if ∃ (u, v) such that v is unmarked then
    // Find neighbors of u
    i. choose v of the smallest index;
    ii. mark(v); L ← L ∪ {v}
  c. else
    i. L ← L \ {u} // Pop from the stack
  d. endif
4. endwhile
```

Depth-first search (DFS)

Example: Perform DFS on the following graph starting from A



Stack



Visited vertices

A	B	C	E	D
---	---	---	---	---

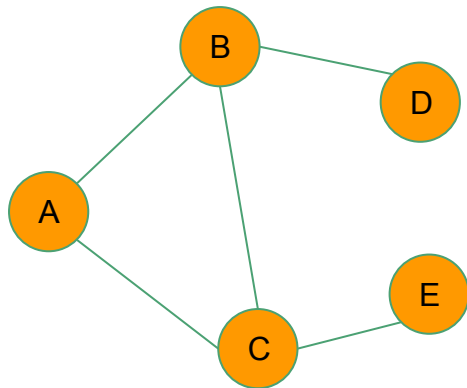
```

3. while  $L \neq \emptyset$  do
    a.  $u \leftarrow \text{last}(L)$  // Top of the stack
    b. if  $\exists (u, v)$  such that  $v$  is unmarked then
        // Find neighbors of  $u$ 
        i. choose  $v$  of the smallest index;
        ii.  $\text{mark}(v); L \leftarrow L \cup \{v\}$ 
    c. else
        i.  $L \leftarrow L \setminus \{u\}$  // Pop from the stack
    d. endif
4. endwhile

```

Depth-first search (DFS)

Example: Perform DFS on the following graph starting from A



Stack

A

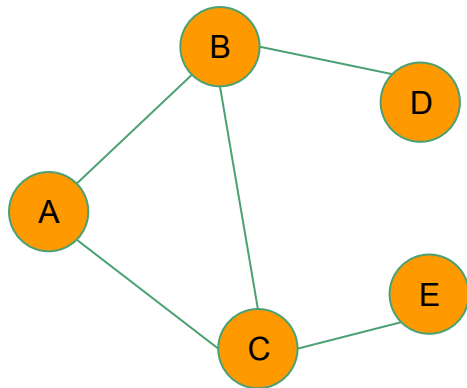
Visited vertices

A	B	C	E	D
---	---	---	---	---

```
3.  while L ≠ ∅ do
    a.  u ← last(L)  // Top of the stack
    b.  if ∃ (u, v) such that v is unmarked then
        // Find neighbors of u
        i.  choose v of the smallest index;
        ii. mark(v); L ← L ∪ {v}
    c.  else
        i.  L ← L \ {u} // Pop from the stack
    d.  endif
4.  endwhile
```

Depth-first search (DFS)

Example: Perform DFS on the following graph starting from A



Stack



Visited vertices

A	B	C	E	D
---	---	---	---	---

```
3.  while L ≠ ∅ do
    a.  u ← last(L)  // Top of the stack
    b.  if ∃ (u, v) such that v is unmarked then
        // Find neighbors of u
        i.  choose v of the smallest index;
        ii. mark(v); L ← L ∪ {v}
    c.  else
        i.  L ← L \ {u} // Pop from the stack
    d.  endif
4.  endwhile
```

Applications of DFS

- Finding a minimum spanning tree for unweighted graphs
- Detecting a cycle in the graph
- Finding a path from one node to another
- Topological ordering: determining the order of compilation tasks, resolving symbol dependencies in linkers etc.
- Solving problems with only one solution, such as maze
- etc.

Breadth-first search (BFS)

Process all adjacent vertices of a vertex before going to the next level.

Uses a queue data structure to perform the search.

Breadth-first search (BFS)

Basic idea:

1. Start by putting any one of the graph's vertices (starting vertex) at the back of a queue.
2. Dequeue the queue (take the vertex at the front of the queue) and add it to the visited list.
3. Enqueue the dequeued vertex's unvisited neighbours to the back of the queue.
4. Keep repeating steps 2 and 3 until the queue is empty.

Breadth-first search (BFS)

Algorithm: **BFS**(**G**, **s**)

Input: A graph, **G**, and a starting vertex, **s**

Output: A sequence of processed vertices

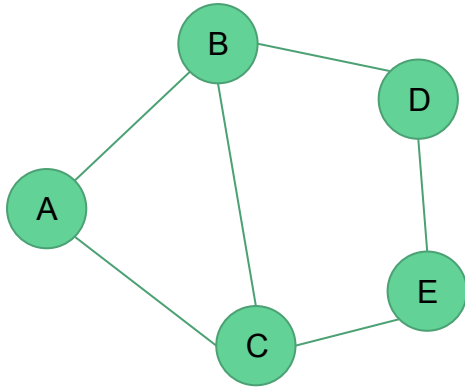
Steps:

1. `mark(s);` // Mark **s** as visited
2. `L := {s}` // Push **s** into the queue

3. **while** `L ≠ ∅` **do**
 - a. `u := first(L)` // Front of the queue
 - b. **if** `∃ (u, v)` such that **v** is unmarked **then** // Find neighbors of **u**
 - i. choose **v** of the smallest index;
 - ii. `mark(v); L := L ∪ {v}`
 - c. **else**
 - i. `L := L \ {u}` // Dequeue the queue
 - d. **endif**
4. **endwhile**

Breadth-first search (BFS)

Example: Perform BFS on the following graph starting from A



Visited vertices

A				
---	--	--	--	--

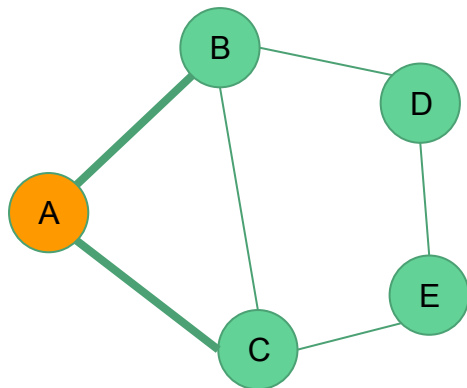
List

A				
---	--	--	--	--

1. `mark(s);` // Mark s as visited
2. `L = {s}` // Push s into the queue

Breadth-first search (BFS)

Example: Perform BFS on the following graph starting from A



Visited vertices

A	B			
---	---	--	--	--

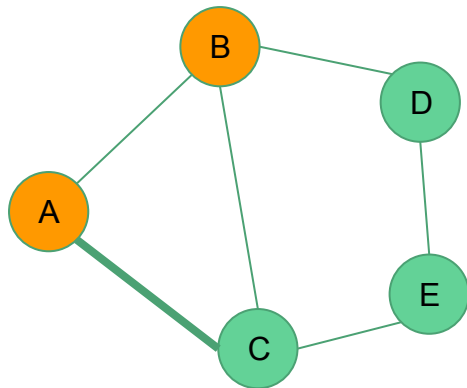
List

A	B			
---	---	--	--	--

```
3. while  $L \neq \emptyset$  do
  a.  $u = \text{first}(L)$  // Front of the queue
  b. if  $\exists (u, v)$  such that  $v$  is unmarked
    then // Find neighbors of  $u$ 
      i. choose  $v$  of the smallest index;
      ii. mark( $v$ );  $L = L \cup \{v\}$ 
  c. else
      i.  $L = L \setminus \{u\}$  // Dequeue the queue
  d. endif
4. endwhile
```

Breadth-first search (BFS)

Example: Perform BFS on the following graph starting from A



Visited vertices

A	B	C		
---	---	---	--	--

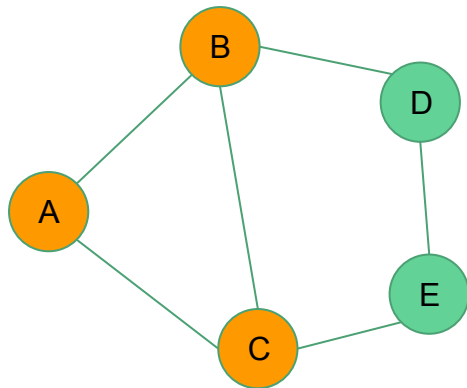
List

A	B	C		
---	---	---	--	--

```
3. while L ≠ ∅ do
  a. u = first(L) // Front of the queue
  b. if ∃ (u, v) such that v is unmarked
     then // Find neighbors of u
       i. choose v of the smallest index;
       ii. mark(v); L = L ∪ {v}
  c. else
       i. L = L \ {u} // Dequeue the queue
  d. endif
4. endwhile
```

Breadth-first search (BFS)

Example: Perform BFS on the following graph starting from A



Visited vertices

A	B	C		
---	---	---	--	--

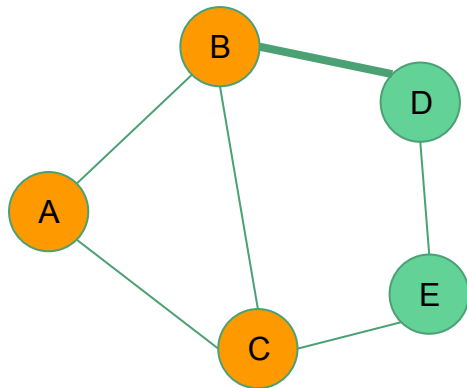
List

B	C			
---	---	--	--	--

```
3. while L ≠ ∅ do
  a. u = first(L) // Front of the queue
  b. if ∃ (u, v) such that v is unmarked
     then // Find neighbors of u
        i. choose v of the smallest index;
        ii. mark(v); L = L ∪ {v}
  c. else
        i. L = L \ {u} // Dequeue the queue
  d. endif
4. endwhile
```

Breadth-first search (BFS)

Example: Perform BFS on the following graph starting from A



Visited vertices

A	B	C	D	
---	---	---	---	--

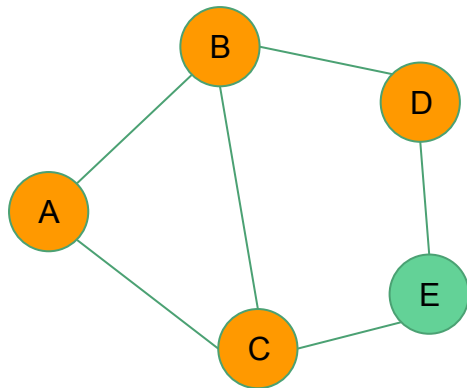
List

B	C	D		
---	---	---	--	--

```
3. while L ≠ ∅ do
  a. u = first(L) // Front of the queue
  b. if ∃ (u, v) such that v is unmarked
     then // Find neighbors of u
       i. choose v of the smallest index;
       ii. mark(v); L = L ∪ {v}
  c. else
       i. L = L \ {u} // Dequeue the queue
  d. endif
4. endwhile
```

Breadth-first search (BFS)

Example: Perform BFS on the following graph starting from A



Visited vertices

A	B	C	D	
---	---	---	---	--

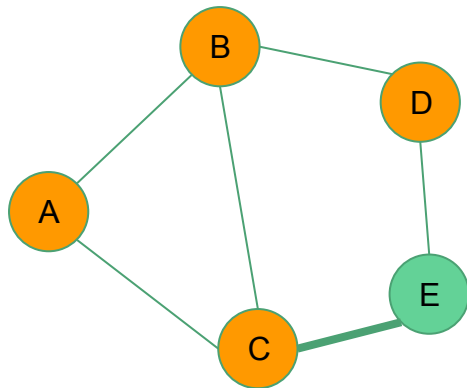
List

C	D			
---	---	--	--	--

```
3. while L ≠ ∅ do
  a. u = first(L) // Front of the queue
  b. if ∃ (u, v) such that v is unmarked
     then // Find neighbors of u
        i. choose v of the smallest index;
        ii. mark(v); L = L ∪ {v}
  c. else
        i. L = L \ {u} // Dequeue the queue
  d. endif
4. endwhile
```


Breadth-first search (BFS)

Example: Perform BFS on the following graph starting from A



Visited vertices

A	B	C	D	E
---	---	---	---	---

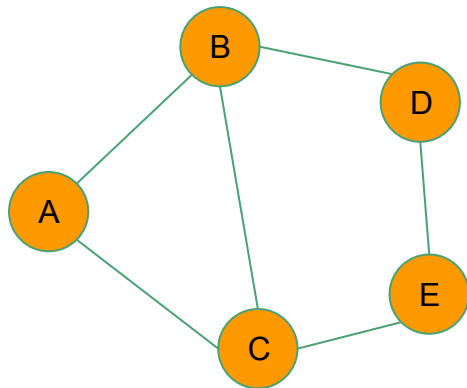
List

C	D	E		
---	---	---	--	--

```
3. while L ≠ ∅ do
  a. u = first(L) // Front of the queue
  b. if ∃ (u, v) such that v is unmarked
     then // Find neighbors of u
        i. choose v of the smallest index;
        ii. mark(v); L = L ∪ {v}
  c. else
        i. L = L \ {u} // Dequeue the queue
  d. endif
4. endwhile
```

Breadth-first search (BFS)

Example: Perform BFS on the following graph starting from A



Visited vertices

A	B	C	D	E
---	---	---	---	---

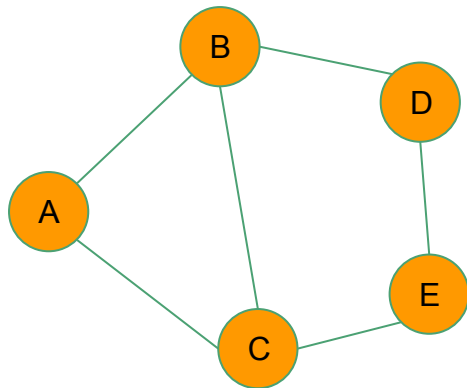
List

D	E			
---	---	--	--	--

```
3. while L ≠ ∅ do
  a. u = first(L) // Front of the queue
  b. if ∃ (u, v) such that v is unmarked
     then // Find neighbors of u
        i. choose v of the smallest index;
        ii. mark(v); L = L ∪ {v}
  c. else
        i. L = L \ {u} // Dequeue the queue
  d. endif
4. endwhile
```

Breadth-first search (BFS)

Example: Perform BFS on the following graph starting from A



Visited vertices

A	B	C	D	E
---	---	---	---	---

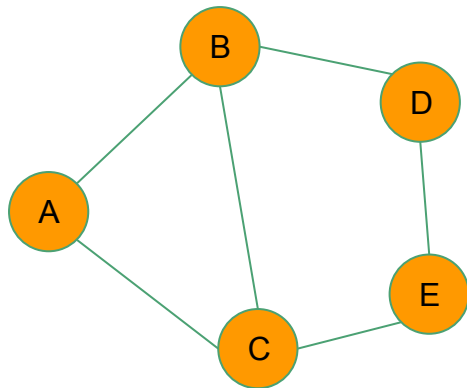
List

E				
---	--	--	--	--

```
3. while L ≠ ∅ do
  a. u = first(L) // Front of the queue
  b. if ∃ (u, v) such that v is unmarked
    then // Find neighbors of u
      i. choose v of the smallest index;
      ii. mark(v); L = L ∪ {v}
  c. else
    i. L = L \ {u} // Dequeue the queue
  d. endif
4. endwhile
```

Breadth-first search (BFS)

Example: Perform BFS on the following graph starting from A



Visited vertices

A	B	C	D	E
---	---	---	---	---

List

--	--	--	--	--

```
3. while L ≠ ∅ do
  a. u = first(L) // Front of the queue
  b. if ∃ (u, v) such that v is unmarked
     then // Find neighbors of u
        i. choose v of the smallest index;
        ii. mark(v); L = L ∪ {v}
  c. else
        i. L = L \ {u} // Dequeue the queue
  d. endif
4. endwhile
```

Applications of BFS

- Finding a minimum spanning tree for unweighted graphs
- Web crawler: Begin from a starting page and follow all links from this page and keep doing the same
- Social networks: Find people within a given distance 'k' from a person
- Finding the shortest path to another node
- GPS navigation systems: Finding the direction to reach from one place to another
- etc.

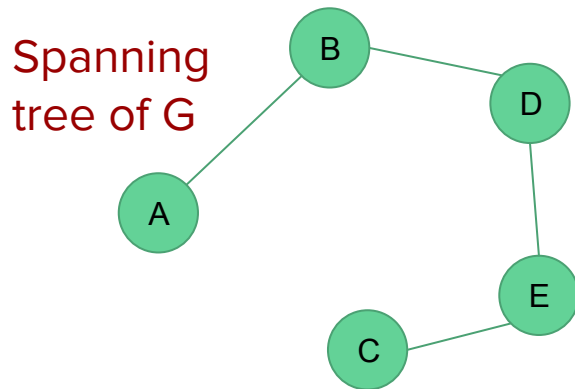
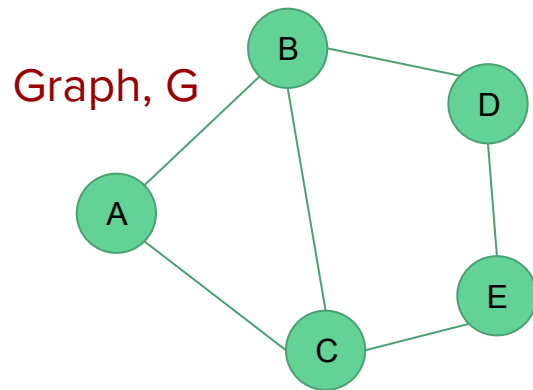
Minimum spanning tree

Spanning tree

A spanning tree of a connected graph G is a tree that consists solely of edges in G and that includes all of the vertices in G .

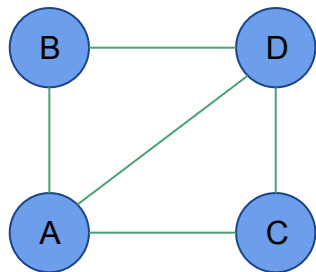
Our solution to generate a spanning tree must satisfy the following constraints:

1. We must use only edges within the graph
2. We must use exactly $n-1$ edges
3. We may not use edges that would produce a cycle

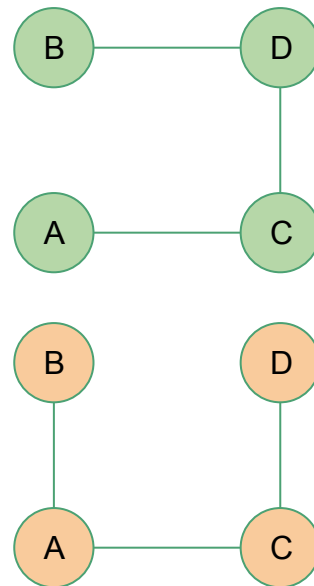
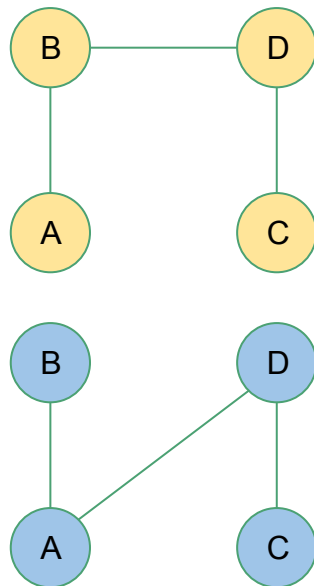
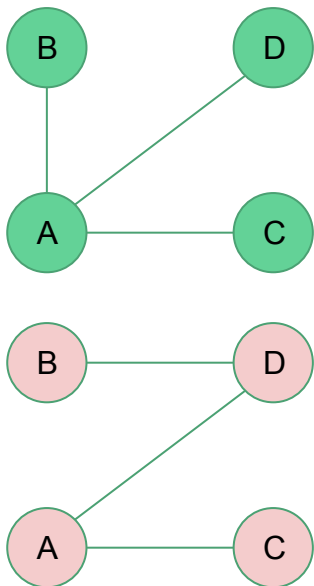


Spanning tree

A single graph can have many spanning trees.



Graph G



Spanning trees of G

Spanning tree

A spanning tree can be generated using a DFS or a BFS. The spanning tree is formed from those edges traversed during the search.

- If a breadth first search is used, the resulting spanning tree is called a **breadth first spanning tree**.
- If a depth first search is used, it is called **depth first spanning tree**.

For a disconnected / disjoint graph, a **spanning forest** is defined.

Minimum spanning tree (MST)

A **minimum spanning tree of a weighted graph** is a **spanning tree of least weight**, i.e. a spanning tree in which the total weight of the edges is guaranteed to be the minimum of all possible trees in the graph.

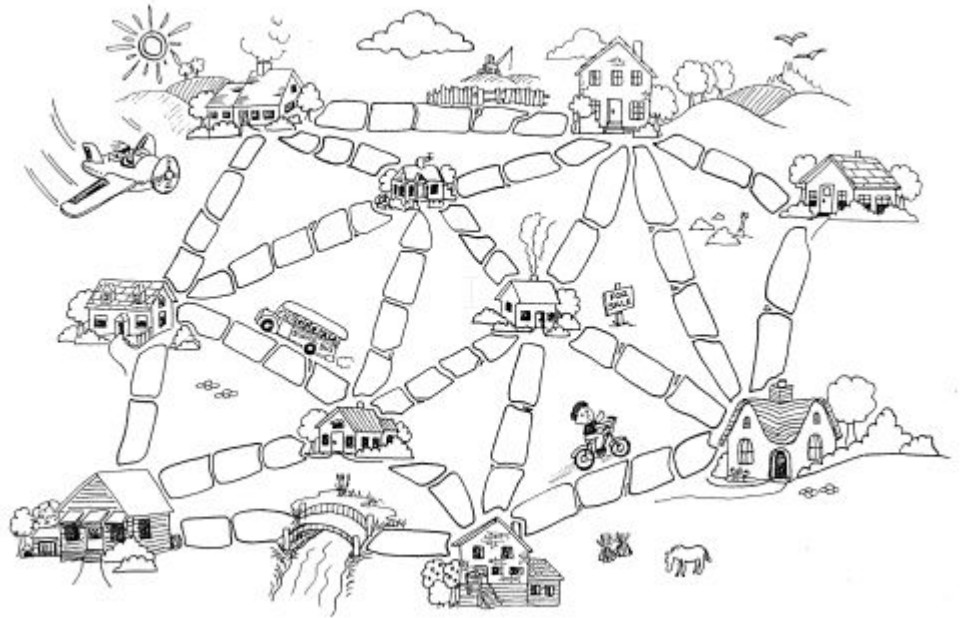
If the weights in the network/graph are unique, there is only one MST

If there are duplicate weights, there may be one or more MSTs

Application: Network design, Muddy city problem

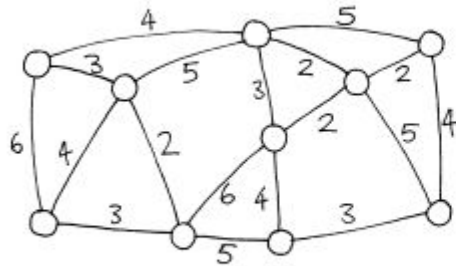
Muddy city problem

- A city with no paved road
- The mayor of the city decided to pave some of the streets with the following two conditions:
 1. Enough streets must be paved so that it is possible for everyone to travel from their house to anyone else's house only along paved roads, and
 2. The paving should cost as little as possible.



Muddy city problem

Solution: Minimum spanning tree



Growing a MST

Problem:

Given a connected, undirected graph $G = (V, E)$ with a weight function $w:E \rightarrow \mathbb{R}$, find a minimum spanning tree for G

A greedy strategy:

Grow the minimum spanning tree one edge at a time

- Kruskal's algorithm
- Prim's algorithm

Growing a MST

GENERIC-MST(G, w)

```
1   $A = \emptyset$ 
2  while  $A$  does not form a spanning tree
3      find an edge  $(u, v)$  that is safe for  $A$ 
4       $A = A \cup \{(u, v)\}$ 
5  return  $A$ 
```

Kruskal's algorithm: Finds a safe edge by finding, of all the edges that connect any two trees in the forest, an edge of least weight.

Prim's algorithm: Adds to the tree A a light edge that connects A to an isolated vertex

Disjoint set

A group of sets where no item can be in more than one set.

A disjoint-set data structure maintains a collection $S = \{S_1, S_2, \dots, S_k\}$ of disjoint dynamic sets.

Example:

$S = \{\{a\}, \{b\}, \{c,d\}, \{e, f, g, h\}\}$ is a disjoint-set.

Basic disjoint set operations

Make_set (x)

Creates a new set whose only member (and thus representative) is x . Since the sets are disjoint, we require that x not already be in some other set.

Union (x, y)

Unites the dynamic sets that contain x and y , say S_x and S_y , into a new set that is the union of these two sets.

Find_set(x)

Returns a pointer to the representative of the (unique) set containing x .

Kruskal's algorithm (using disjoint sets)

MST-KRUSKAL(G, w)

```
1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
```

Disjoint set operations

Make-set(x) :

Creates a new set whose only member is x

Union(x, y):

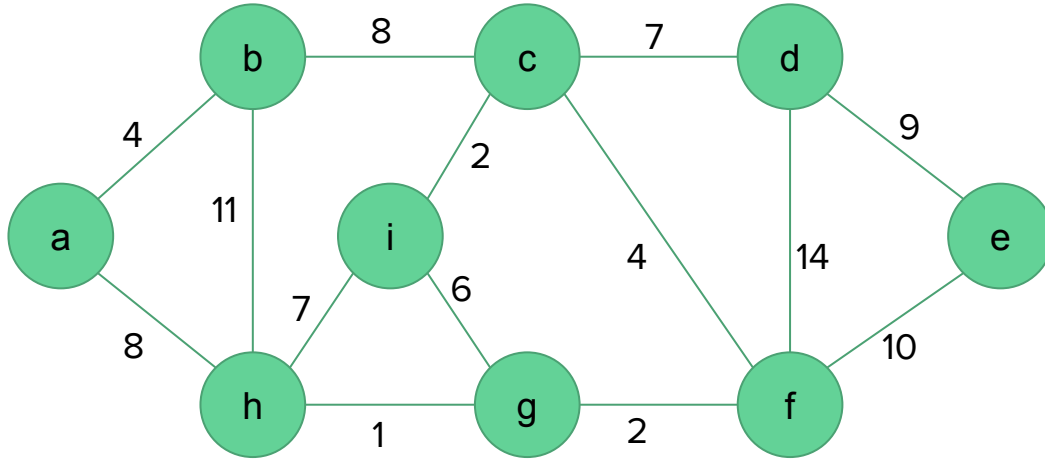
Unites the dynamic sets that contain x and y into a new set that is the union of these two sets

Find-Set(x):

Returns a pointer to the representative of the set containing x

Example

Find a minimum spanning tree of the following graph



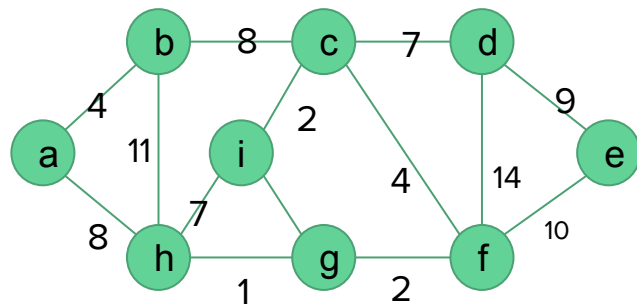
Example

MST-KRUSKAL(G, w)

```
1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
```

L1: $A = \{\}$

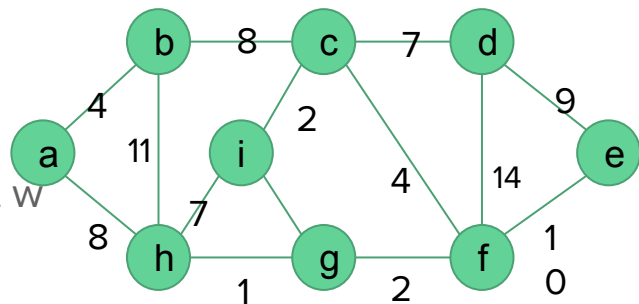
L2: $\{a\} \{b\} \{c\} \{d\} \{e\} \{f\} \{g\} \{h\} \{i\}$



Example

L4: sort the edges of $G:E$ into nondecreasing order by weight w

- 1 (h, g)
- 2 (g, f)
- 2 (i, c)
- 4 (a, b)
- 4 (c, f)
- 6 (i, g)
- 7 (h, i)
- 7 (c, d)
- 8 (b, c)
- 8 (a, h)
- 9 (d, e)
- 10 (e, f)
- 11 (b, h)
- 14 (d, f)



Example

w	Edge (u,v)	Disjoint sets	A
1	(h, g)	{a} {b} {c} {d} {e} {f} {g, h} {i}	{ (h, g) }
2	(g, f)		
2	(i, c)		
4	(a, b)		
4	(c, f)		
6	(i, g)		
7	(h, i)		
7	(c, d)		
8	(b, c)		
8	(a, h)		
9	(d, e)		
10	(e, f)		
11	(b, h)		
14	(d, f)		

```

5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )

```

w	Edge (u,v)	Disjoint sets	A
1	(h, g)	{a} {b} {c} {d} {e} {f} {g, h} {i}	{ (h, g) }
2	(g, f)		
2	(i, c)		
4	(a, b)		
4	(c, f)		
6	(i, g)		
7	(h, i)		
7	(c, d)		
8	(b, c)		
8	(a, h)		
9	(d, e)		
10	(e, f)		
11	(b, h)		
14	(d, f)		

```

5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )

```

w	Edge (u,v)	Disjoint sets	A
1	(h, g)	{a} {b} {c} {d} {e} {f} {g, h} {i}	{ (h, g) }
2	(g, f)	{a} {b} {c} {d} {e} {f, g, h} {i}	{ (h, g) , (g, f) }
2	(i, c)		
4	(a, b)		
4	(c, f)		
6	(i, g)		
7	(h, i)		
7	(c, d)		
8	(b, c)		
8	(a, h)		
9	(d, e)		
10	(e, f)		
11	(b, h)		
14	(d, f)		


```

5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )

```

w	Edge (u,v)	Disjoint sets	A
1	(h,g)	{a} {b} {c} {d} {e} {f} {g,h} {i}	{(h,g)}
2	(g,f)	{a} {b} {c} {d} {e} {f,g,h} {i}	{(h,g), (g,f)}
2	(i,c)	{a} {b} {c,i} {d} {e} {f,g,h}	{(h,g), (g,f), (c,i) }
4	(a,b)		
4	(c,f)		
6	(i,g)		
7	(h,i)		
7	(c,d)		
8	(b,c)		
8	(a,h)		
9	(d,e)		
10	(e,f)		
11	(b,h)		
14	(d,f)		

```

5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )

```

w	Edge (u,v)	Disjoint sets	A
1	(h,g)	{a} {b} {c} {d} {e} {f} {g,h} {i}	{(h,g)}
2	(g,f)	{a} {b} {c} {d} {e} {f,g,h} {i}	{(h,g), (g,f)}
2	(i,c)	{a} {b} {c,i} {d} {e} {f,g,h}	{(h,g), (g,f), (c,i)}
4	(a,b)	{a,b} {c,i} {d} {e} {f,g,h}	{(h,g), (g,f), (c,i), (a,b) }
4	(c,f)	{a,b} {c,f,g,h,i} {d} {e}	{(h,g), (g,f), (c,i), (a,b), (c,f) }
6	(i,g)	Find-set(i) == Find-set(g)	
7	(h,i)		
7	(c,d)		
8	(b,c)		
8	(a,h)		
9	(d,e)		
10	(e,f)		
11	(b,h)		
14	(d,f)		

```

5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )

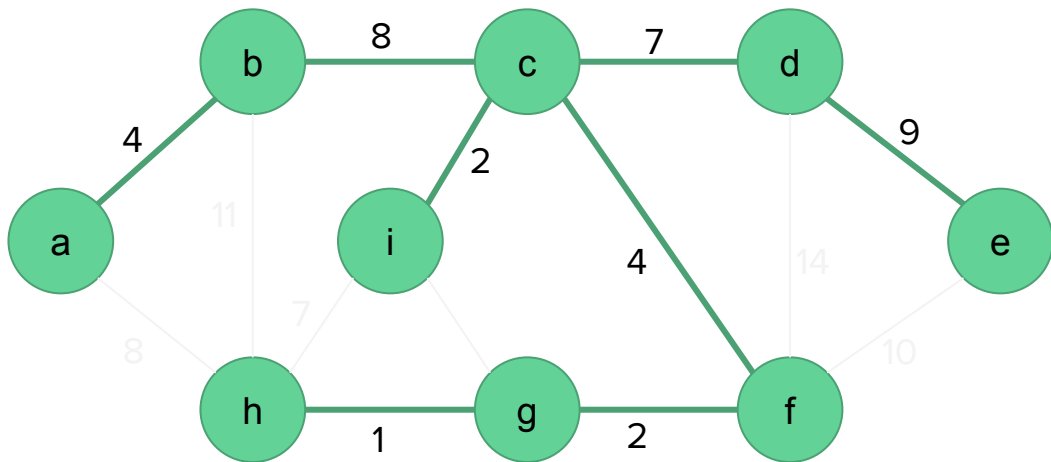
```

w	Edge (u,v)	Disjoint sets	A
1	(h,g)	{a} {b} {c} {d} {e} {f} {g,h} {i}	{ (h,g) }
2	(g,f)	{a} {b} {c} {d} {e} {f,g,h} {i}	{ (h,g) , (g,f) }
2	(i,c)	{a} {b} {c,i} {d} {e} {f,g,h}	{ (h,g) , (g,f) , (c,i) }
4	(a,b)	{a,b} {c,i} {d} {e} {f,g,h}	{ (h,g) , (g,f) , (c,i) , (a,b) }
4	(c,f)	{a,b} {c,f,g,h,i} {d} {e}	{ (h,g) , (g,f) , (c,i) , (a,b) , (c,f) }
6	(i,g)	Find-set(i) == Find-set(g)	
7	(h,i)	Find-set(h) == Find-set(i)	
7	(c,d)	{a,b} {c,d,f,g,h,i} {e}	{ (h,g) , (g,f) , (c,i) , (a,b) , (c,f) , (c,d) }
8	(b,c)	{a,b,c,d,f,g,h,i} {e}	{ (h,g) , (g,f) , (c,i) , (a,b) , (c,f) , (c,d) , (b,c) }
8	(a,h)	Find-set(a) == Find-set(h)	}
9	(d,e)	{a,b,c,d,e,f,g,h,i}	
10	(e,f)	Find-set(e) == Find-set(f)	{ (h,g) , (g,f) , (c,i) , (a,b) , (c,f) , (c,d) , (b,c) , (d,e) }
11	(b,h)	Find-set(b) == Find-set(h)	
14	(d,f)	Find-set(d) == Find-set(f)	

Example

The MST is the tree containing A.

$A = \{ (h, g), (g, f), (c, i), (a, b), (c, f), (c, d), (b, c), (d, e) \}$



Analysis of Kruskal's algorithm

MST-KRUSKAL(G, w)

```
1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
```

Analysis of Kruskal's algorithm

MST-KRUSKAL(G, w)

```
1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
```

Line 1: $O(1)$

Line 2 - 3: $O(|V|)$

Line 4: Sorting: $O(|E| \log |E|)$

Line 5: $O(|E|)$

Line 6 - 8: Depends on the implementation of Find-Set and Union operations. We assume that they are very fast ($O(1)$)

Overall complexity = $O(1 + V + E \log E + E)$
= $O(E \log E)$

Prim's algorithm

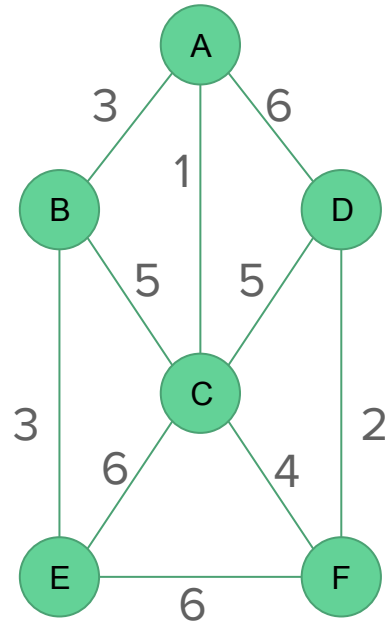
Grows a single tree and adds a light edge (edge with the lowest weight) in each iteration

Steps:

1. Start by picking any vertex to be the root of the tree.
2. While the tree does not contain all vertices in the graph, find shortest edge leaving the tree and add it to the tree

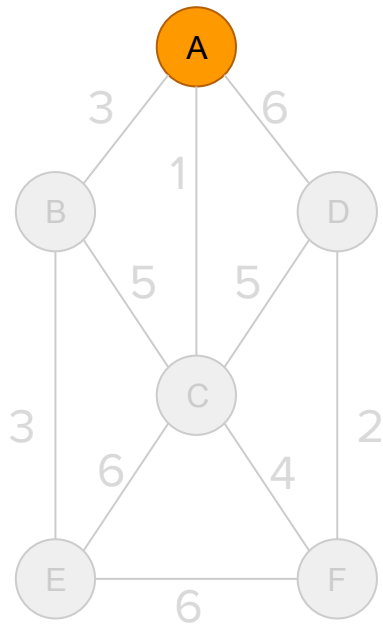
Prim's algorithm

Example: Find a minimum spanning tree of the following graph



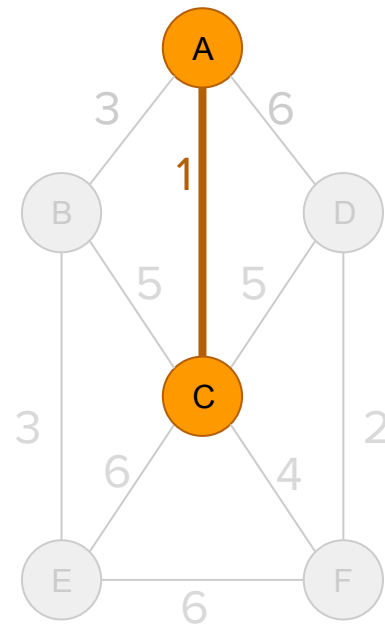
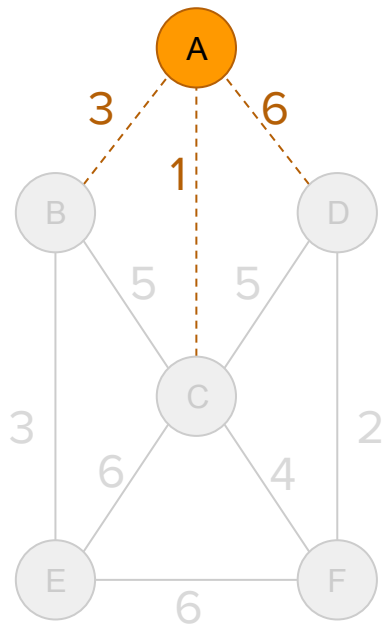
Prim's algorithm

Step 1: Pick any vertex to be the root of the tree. Let's say A will be the root



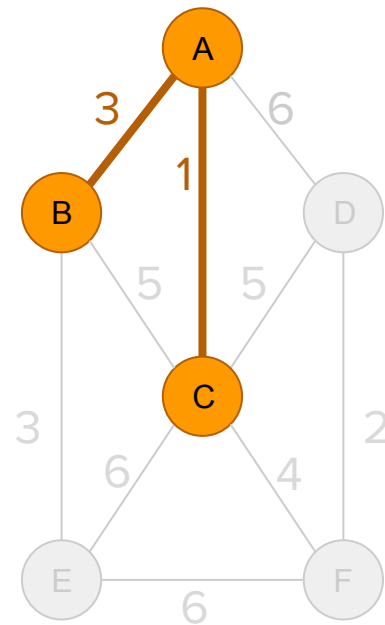
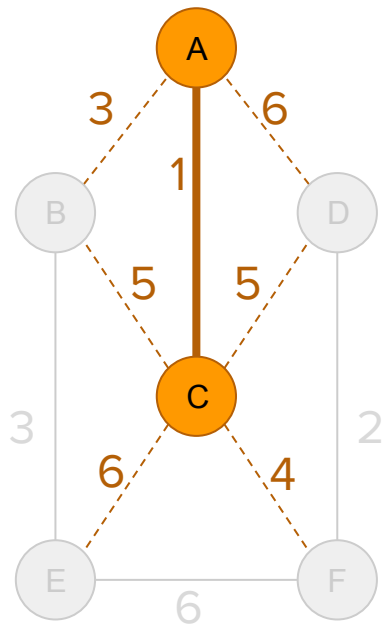
Prim's algorithm

Step 2: Find shortest edge leaving the tree and add it to the tree



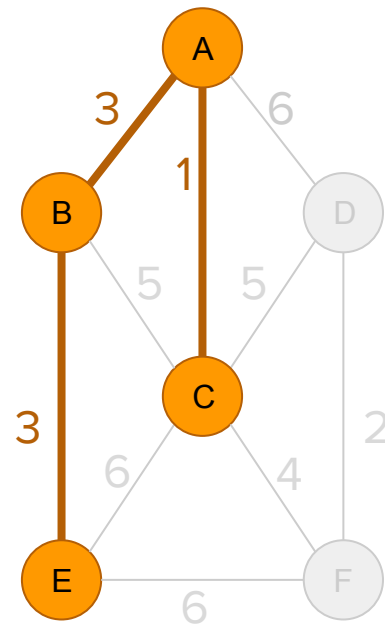
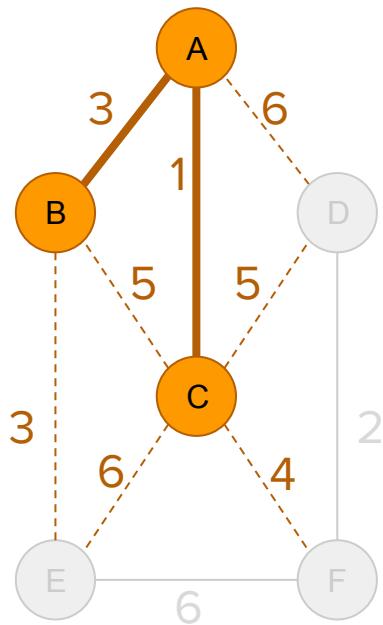
Prim's algorithm

Step 2: Find shortest edge leaving the tree and add it to the tree



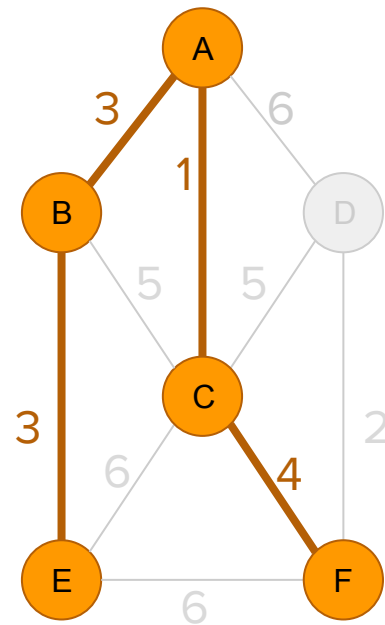
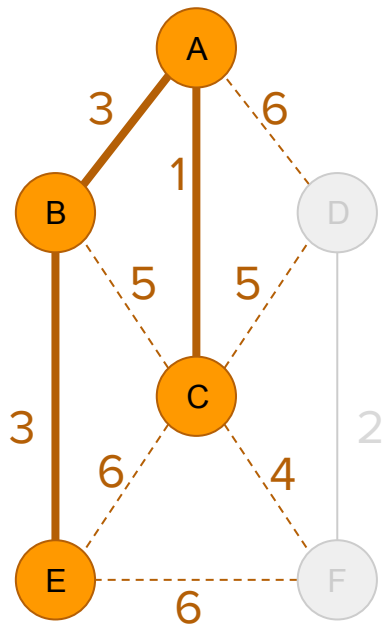
Prim's algorithm

Step 2: Find shortest edge leaving the tree and add it to the tree



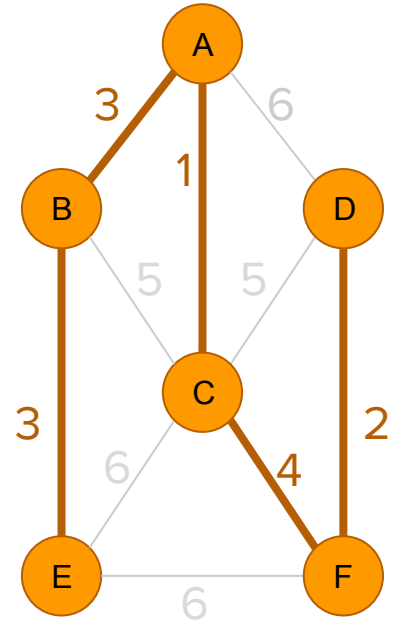
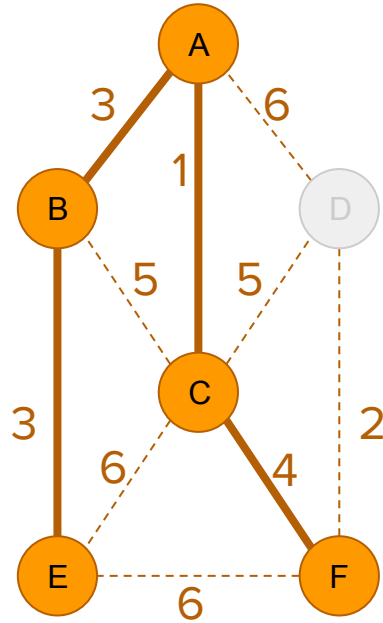
Prim's algorithm

Step 2: Find shortest edge leaving the tree and add it to the tree



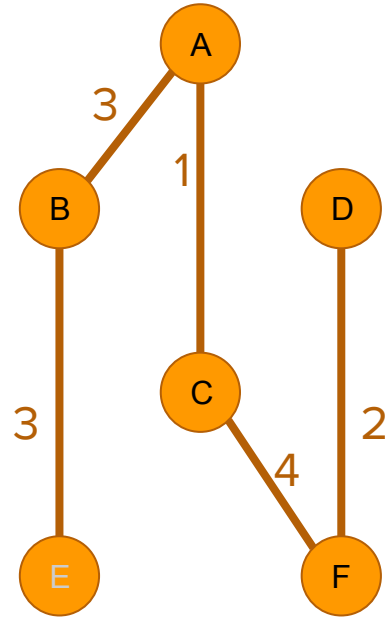
Prim's algorithm

Step 2: Find shortest edge leaving the tree and add it to the tree



Prim's algorithm

So the spanning tree is



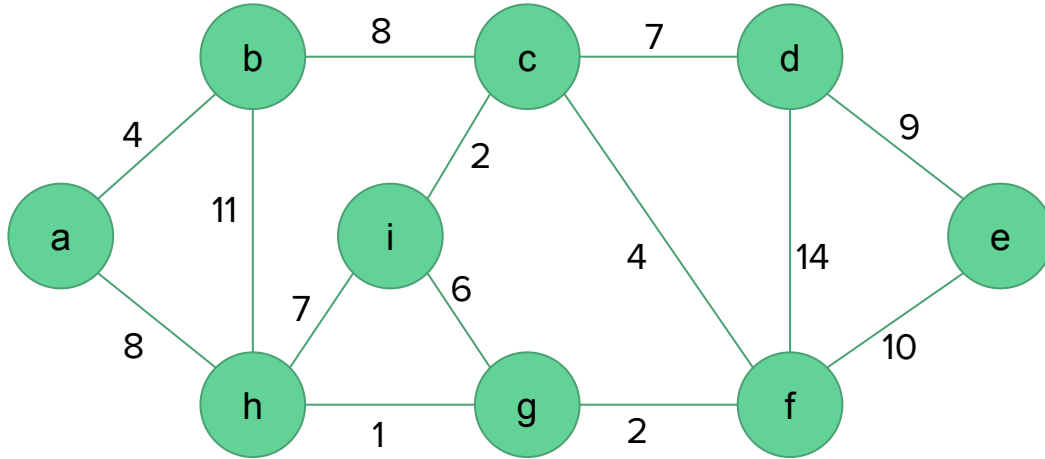
Prim's algorithm

MST-PRIM(G, w, r)

```
1  for each  $u \in G.V$ 
2       $u.key = \infty$ 
3       $u.\pi = \text{NIL}$ 
4   $r.key = 0$ 
5   $Q = G.V$ 
6  while  $Q \neq \emptyset$ 
7       $u = \text{EXTRACT-MIN}(Q)$ 
8      for each  $v \in G.Adj[u]$ 
9          if  $v \in Q$  and  $w(u, v) < v.key$ 
10               $v.\pi = u$ 
11               $v.key = w(u, v)$ 
```


Example

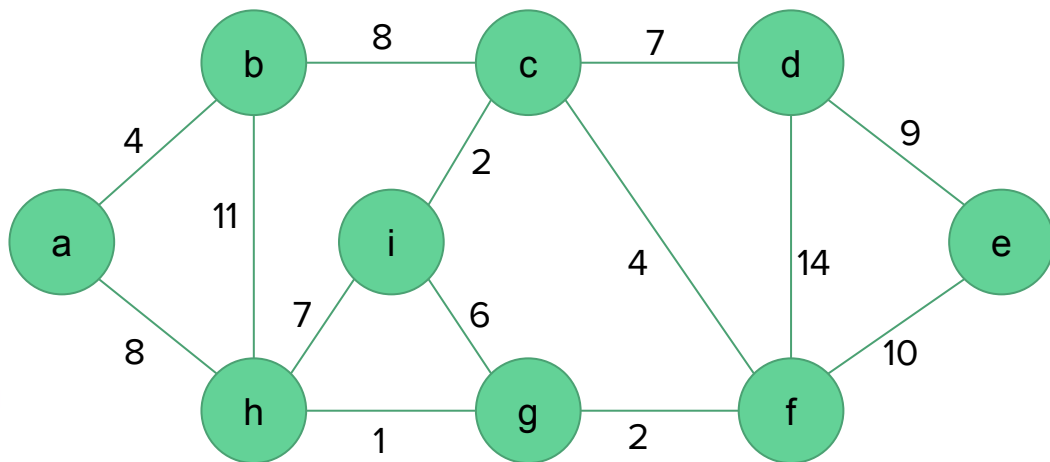
Find a minimum spanning tree of the following graph



Example

MST-PRIM(G, w, r)

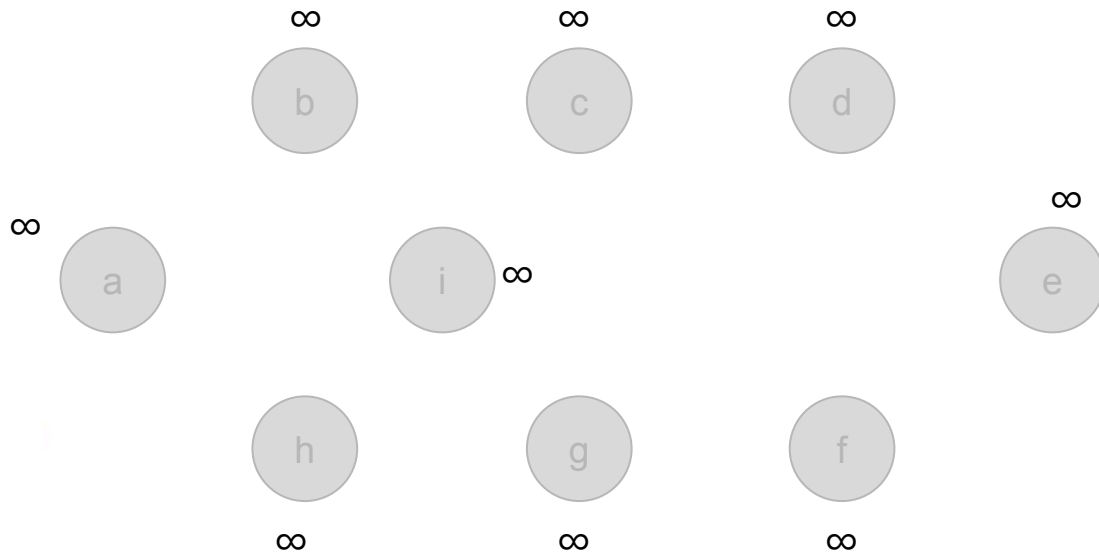
```
1  for each  $u \in G.V$ 
2     $u.key = \infty$ 
3     $u.\pi = \text{NIL}$ 
4   $r.key = 0$ 
5   $Q = G.V$ 
6  while  $Q \neq \emptyset$ 
7     $u = \text{EXTRACT-MIN}(Q)$ 
8    for each  $v \in G.Adj[u]$ 
9      if  $v \in Q$  and  $w(u, v) < v.key$ 
10          $v.\pi = u$ 
11          $v.key = w(u, v)$ 
```



Example

MST-PRIM(G, w, r)

- 1 **for** each $u \in G.V$
- 2 $u.key = \infty$
- 3 $u.\pi = \text{NIL}$



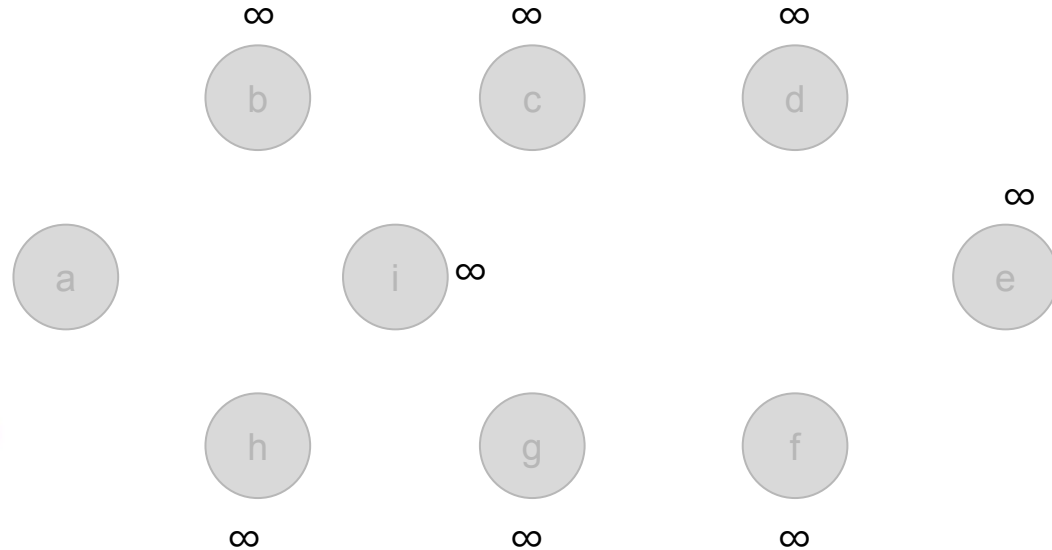
Example

MST-PRIM(G, w, r)

4 $r.key = 0$

5 $Q = G.V$ $Q = \{a, b, c, d, e, f, g, h, i\}$

0

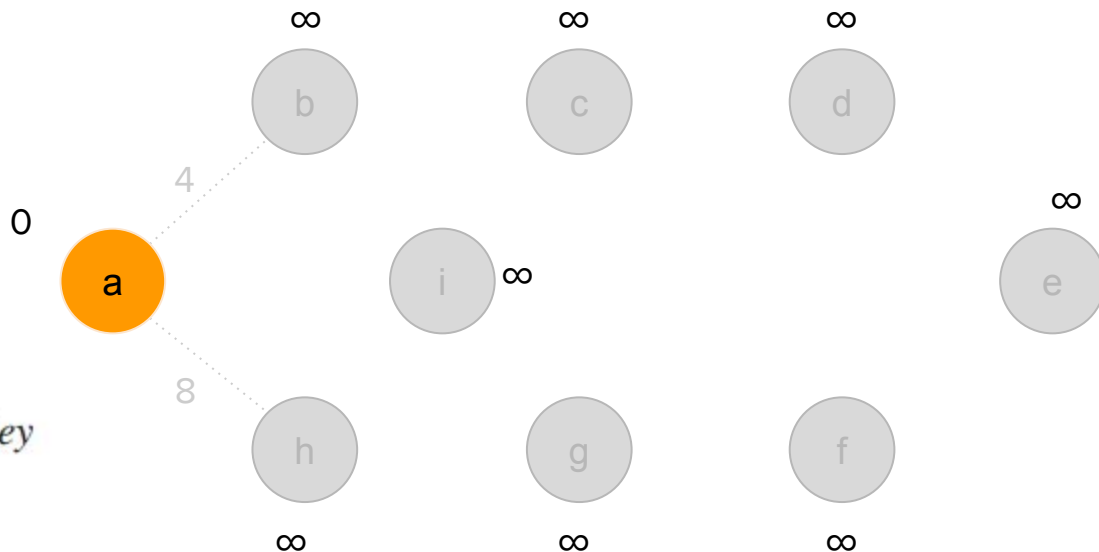


Example

MST-PRIM(G, w, r)

$Q = \{b, c, d, e, f, g, h, i\}$

```
6 while  $Q \neq \emptyset$ 
7    $u = \text{EXTRACT-MIN}(Q)$ 
8   for each  $v \in G.\text{Adj}[u]$ 
9     if  $v \in Q$  and  $w(u, v) < v.\text{key}$ 
10       $v.\pi = u$ 
11       $v.\text{key} = w(u, v)$ 
```



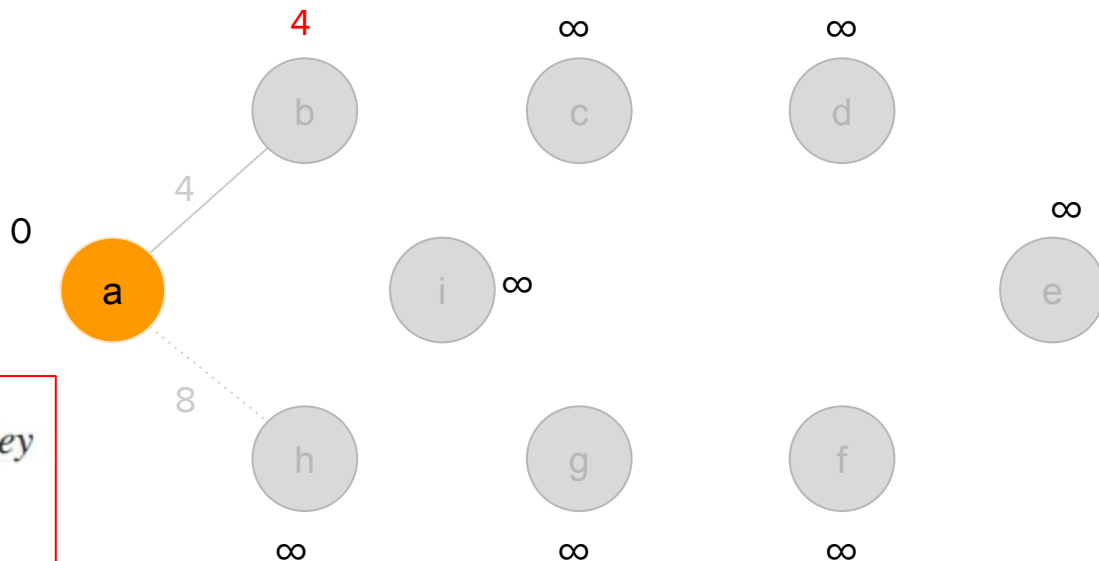
$u = a$
 $G.\text{Adj}[u] = \{b, h\}$

Example

MST-PRIM(G, w, r)

$Q = \{b, c, d, e, f, g, h, i\}$

```
6 while  $Q \neq \emptyset$ 
7    $u = \text{EXTRACT-MIN}(Q)$ 
8   for each  $v \in G.\text{Adj}[u]$ 
9     if  $v \in Q$  and  $w(u, v) < v.\text{key}$ 
10       $v.\pi = u$ 
11       $v.\text{key} = w(u, v)$ 
```



$G.\text{Adj}[u] = \{b, h\}$

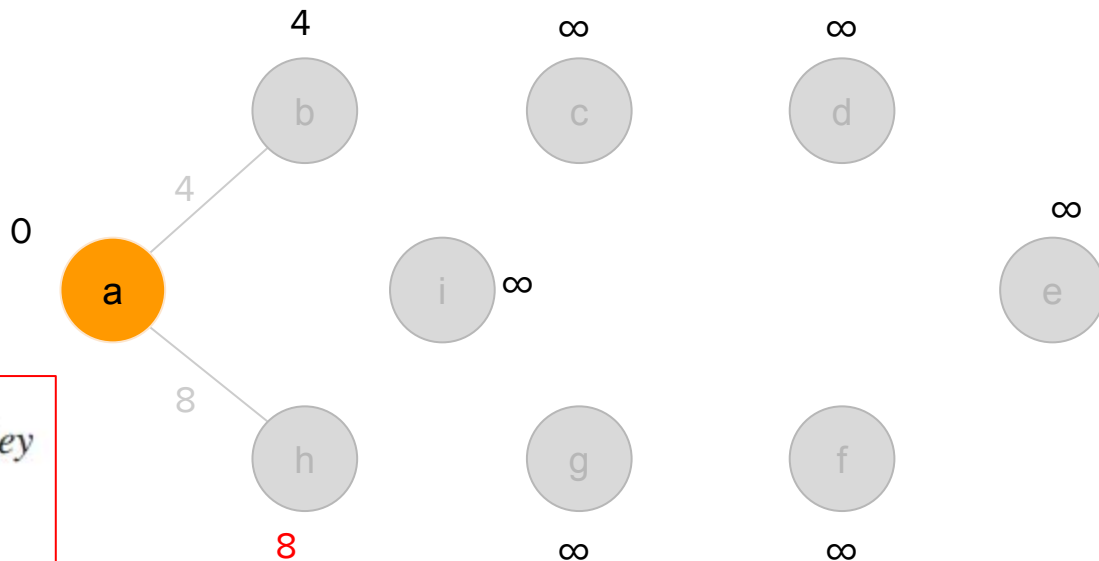
$b \in Q$ and $w(u, v) < b.\text{key}$? True

Example

MST-PRIM(G, w, r)

$Q = \{b, h, c, d, e, f, g, i\}$

```
6 while  $Q \neq \emptyset$ 
7    $u = \text{EXTRACT-MIN}(Q)$ 
8   for each  $v \in G.\text{Adj}[u]$ 
9     if  $v \in Q$  and  $w(u, v) < v.\text{key}$ 
10       $v.\pi = u$ 
11       $v.\text{key} = w(u, v)$ 
```



$G.\text{Adj}[u] = \{b, h\}$

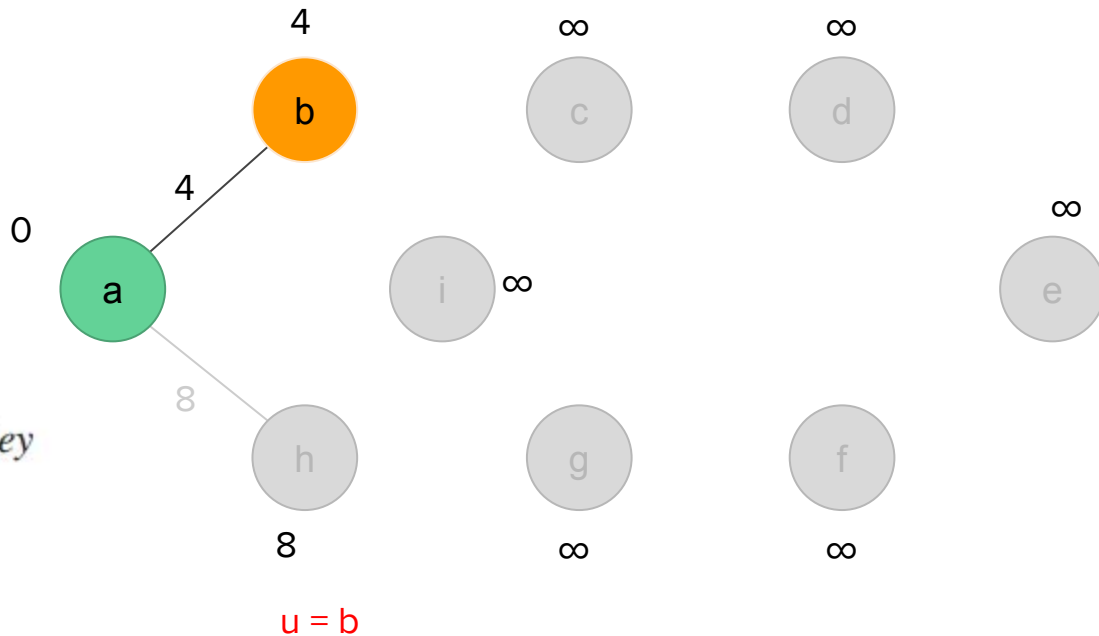
$h \in Q$ and $w(u, v) < h.\text{key}$? True

Example

MST-PRIM(G, w, r)

$Q = \{h, c, d, e, f, g, i\}$

```
6 while  $Q \neq \emptyset$ 
7    $u = \text{EXTRACT-MIN}(Q)$ 
8   for each  $v \in G.\text{Adj}[u]$ 
9     if  $v \in Q$  and  $w(u, v) < v.\text{key}$ 
10       $v.\pi = u$ 
11       $v.\text{key} = w(u, v)$ 
```

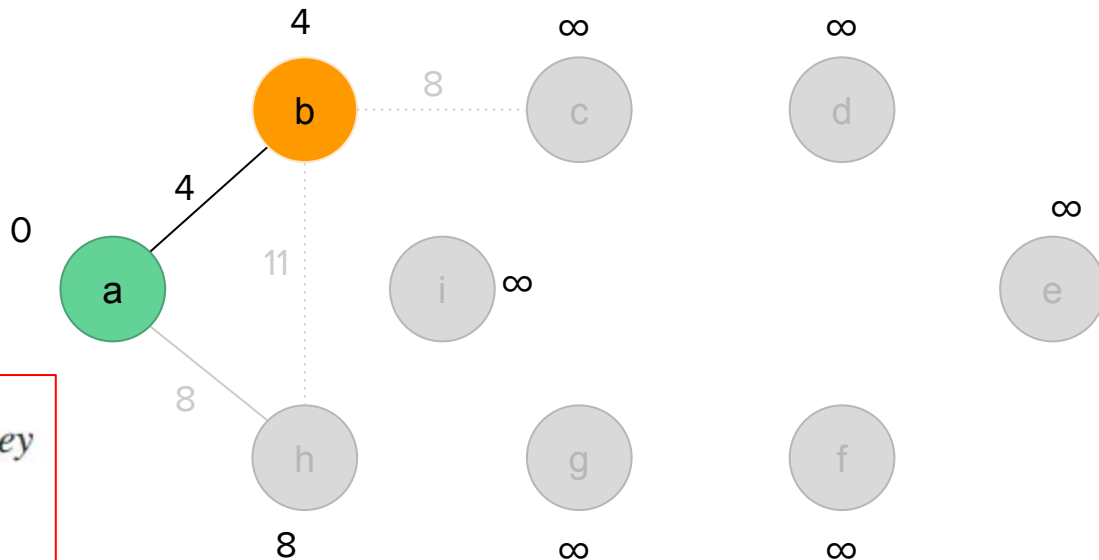


Example

MST-PRIM(G, w, r)

$Q = \{h, c, d, e, f, g, i\}$

```
6 while  $Q \neq \emptyset$ 
7    $u = \text{EXTRACT-MIN}(Q)$ 
8   for each  $v \in G.\text{Adj}[u]$ 
9     if  $v \in Q$  and  $w(u, v) < v.\text{key}$ 
10       $v.\pi = u$ 
11       $v.\text{key} = w(u, v)$ 
```



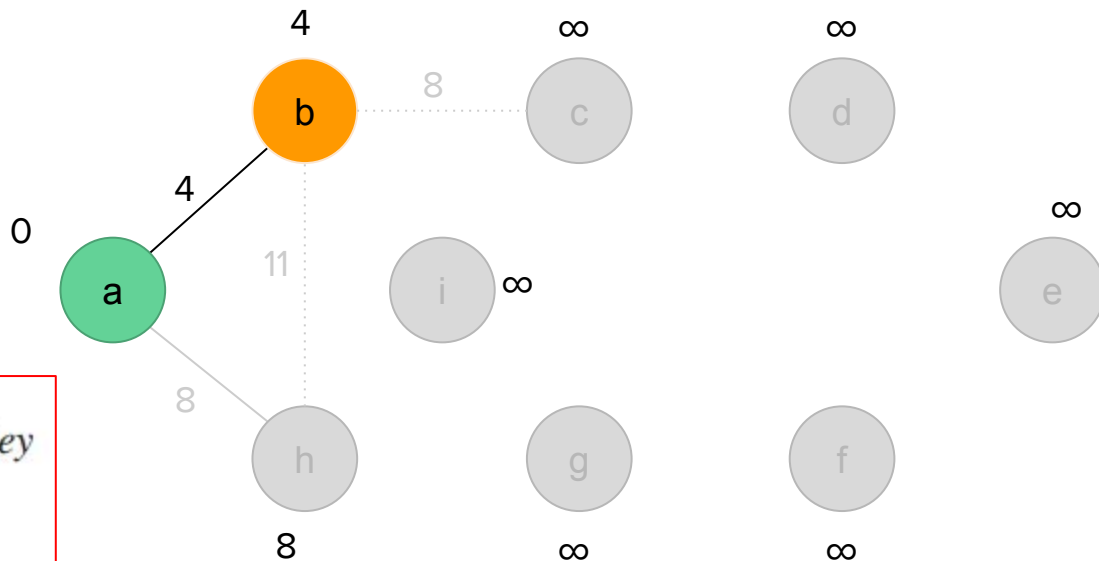
$G.\text{Adj}[u] = \{a, c, h\}$

Example

MST-PRIM(G, w, r)

$Q = \{h, c, d, e, f, g, i\}$

```
6 while  $Q \neq \emptyset$ 
7    $u = \text{EXTRACT-MIN}(Q)$ 
8   for each  $v \in G.\text{Adj}[u]$ 
9     if  $v \in Q$  and  $w(u, v) < v.\text{key}$ 
10       $v.\pi = u$ 
11       $v.\text{key} = w(u, v)$ 
```



$G.\text{Adj}[u] = \{a, c, h\}$

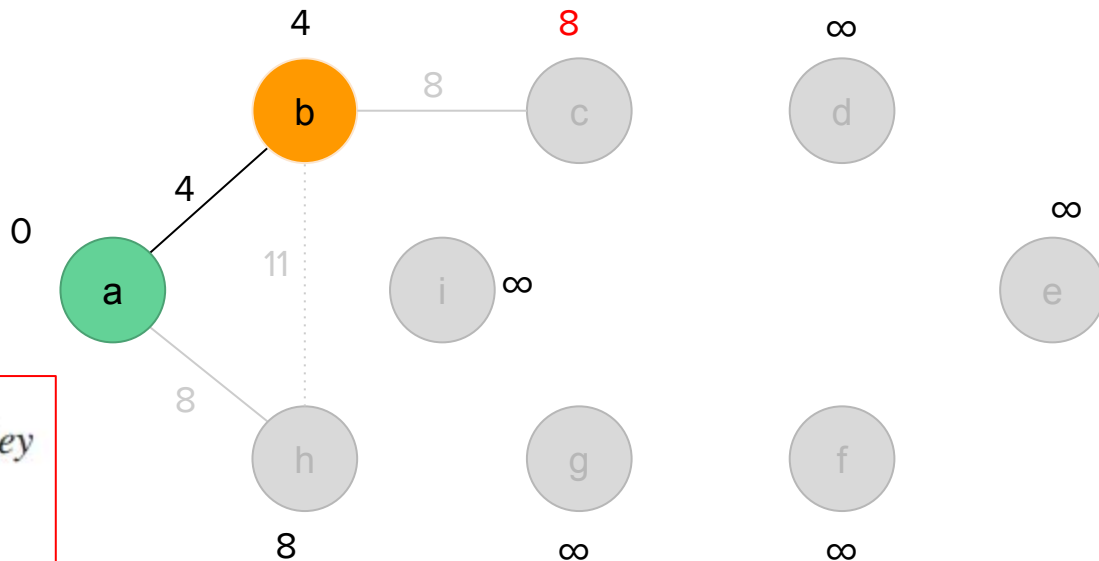
$a \in Q$ and $w(u,v) < a.\text{key}$? False

Example

MST-PRIM(G, w, r)

$Q = \{h, c, d, e, f, g, i\}$

```
6 while  $Q \neq \emptyset$ 
7    $u = \text{EXTRACT-MIN}(Q)$ 
8   for each  $v \in G.\text{Adj}[u]$ 
9     if  $v \in Q$  and  $w(u, v) < v.\text{key}$ 
10       $v.\pi = u$ 
11       $v.\text{key} = w(u, v)$ 
```



$G.\text{Adj}[u] = \{a, c, h\}$

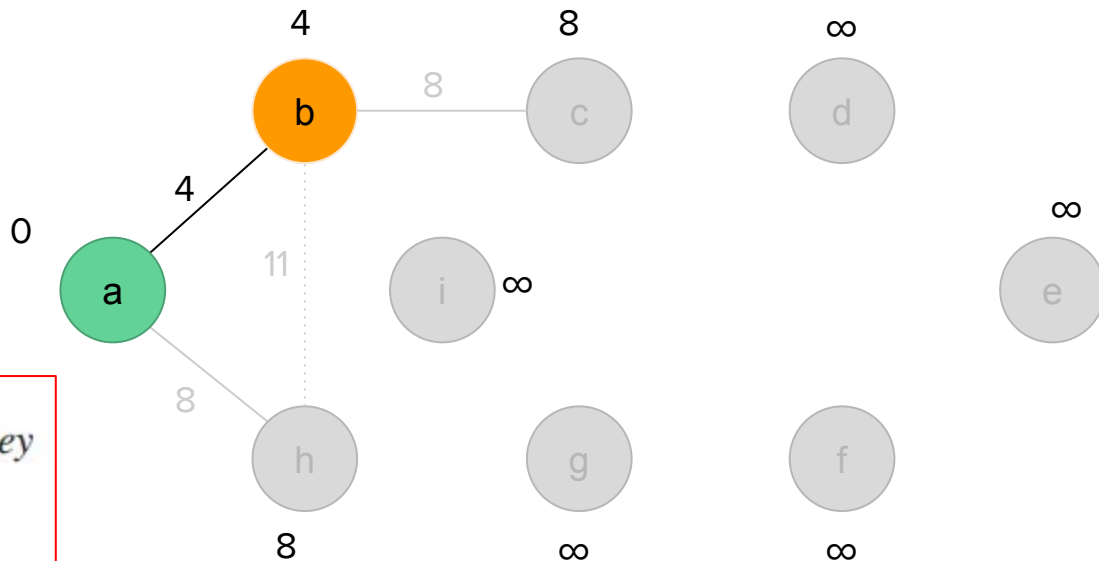
$c \in Q$ and $w(u, c) < c.\text{key}$? True

Example

MST-PRIM(G, w, r)

$Q = \{h, c, d, e, f, g, i\}$

```
6 while  $Q \neq \emptyset$ 
7    $u = \text{EXTRACT-MIN}(Q)$ 
8   for each  $v \in G.\text{Adj}[u]$ 
9     if  $v \in Q$  and  $w(u, v) < v.\text{key}$ 
10       $v.\pi = u$ 
11       $v.\text{key} = w(u, v)$ 
```



$G.\text{Adj}[u] = \{a, c, h\}$

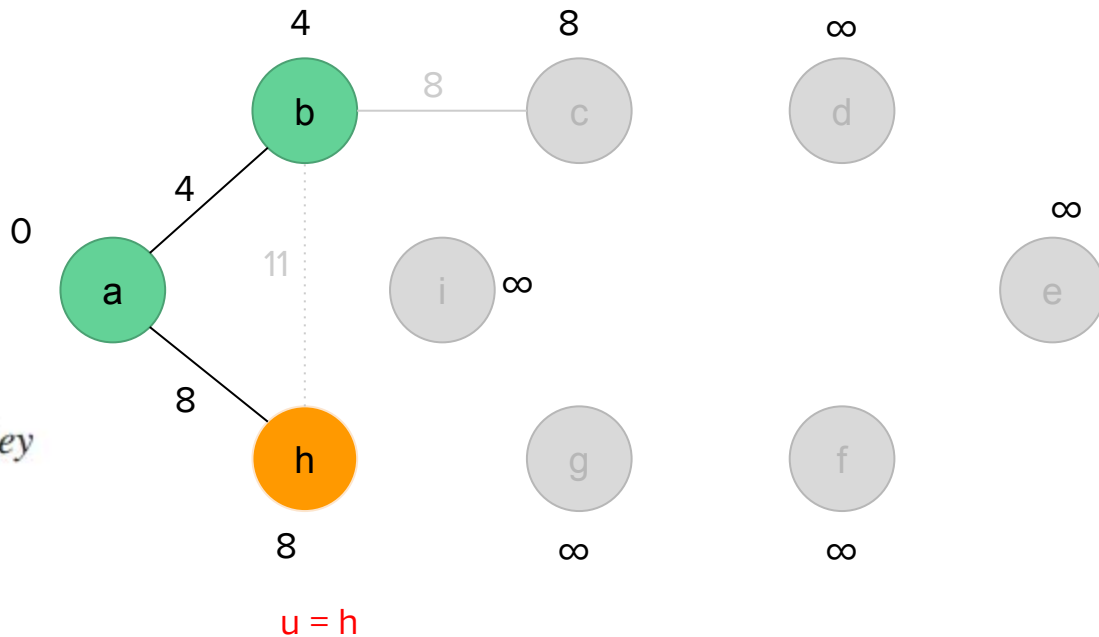
$h \in Q$ and $w(u, h) < h.\text{key}$? False

Example

MST-PRIM(G, w, r)

$Q = \{c, d, e, f, g, i\}$

```
6 while  $Q \neq \emptyset$ 
7    $u = \text{EXTRACT-MIN}(Q)$ 
8   for each  $v \in G.\text{Adj}[u]$ 
9     if  $v \in Q$  and  $w(u, v) < v.\text{key}$ 
10       $v.\pi = u$ 
11       $v.\text{key} = w(u, v)$ 
```



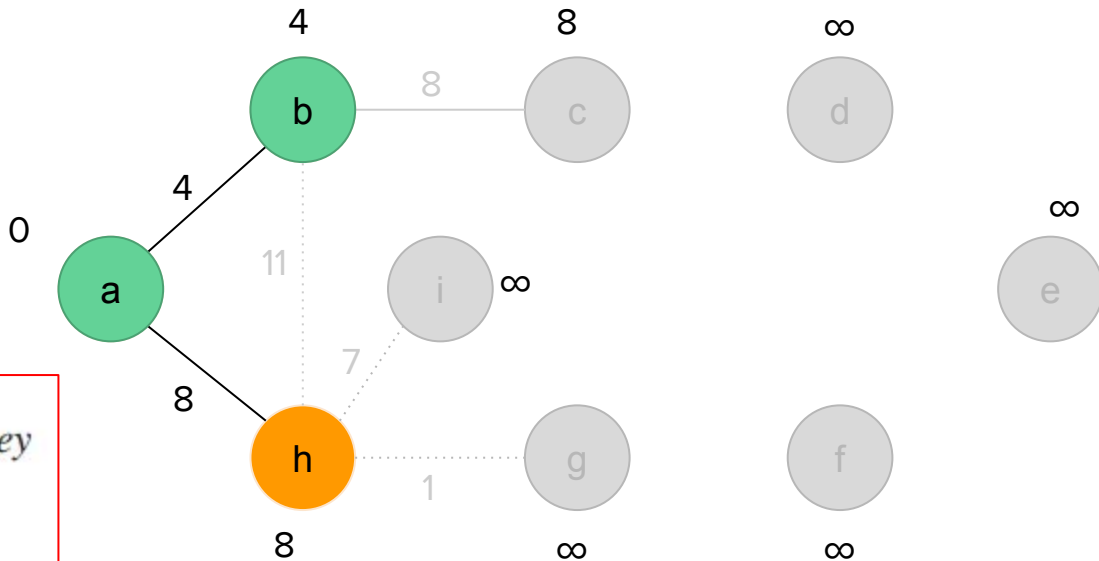
Example

$$\text{MST-PRIM}(G, w, r)$$
$$Q = \{c, d, e, f, g, i\}$$

```

6  while  $Q \neq \emptyset$ 
7       $u = \text{EXTRACT-MIN}(Q)$ 
8      for each  $v \in G.\text{Adj}[u]$ 
9          if  $v \in Q$  and  $w(u, v) < v.\text{key}$ 
10              $v.\pi = u$ 
11              $v.\text{key} = w(u, v)$ 

```

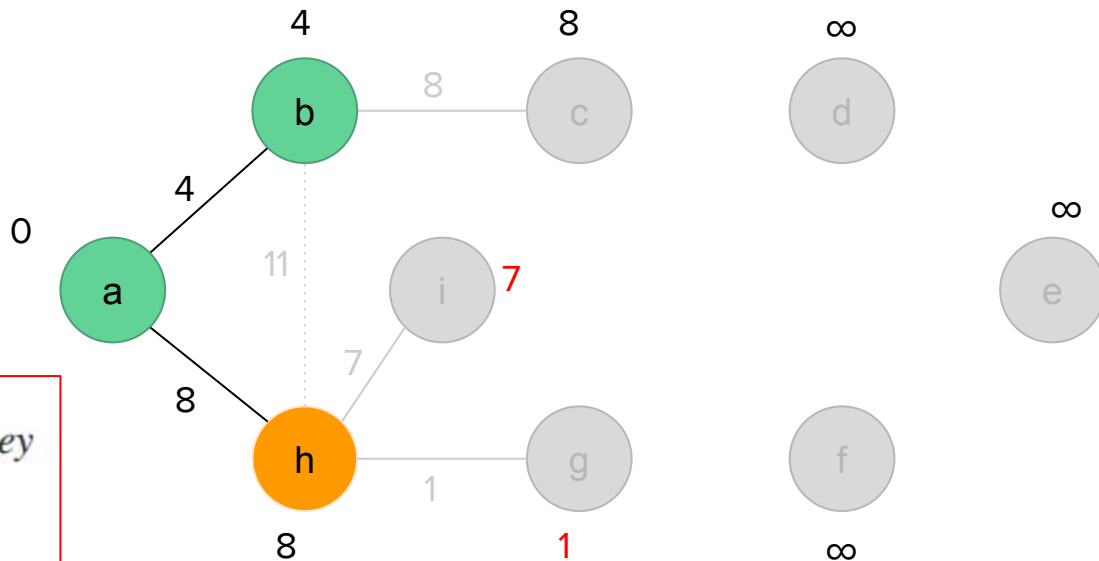

$$u = h$$
$$G.Adj[u] = \{a, b, g, i\}$$

Example

MST-PRIM(G, w, r)

$Q = \{g, i, c, d, e, f\}$

```
6 while  $Q \neq \emptyset$ 
7    $u = \text{EXTRACT-MIN}(Q)$ 
8   for each  $v \in G.\text{Adj}[u]$ 
9     if  $v \in Q$  and  $w(u, v) < v.\text{key}$ 
10       $v.\pi = u$ 
11       $v.\text{key} = w(u, v)$ 
```



$u = h$

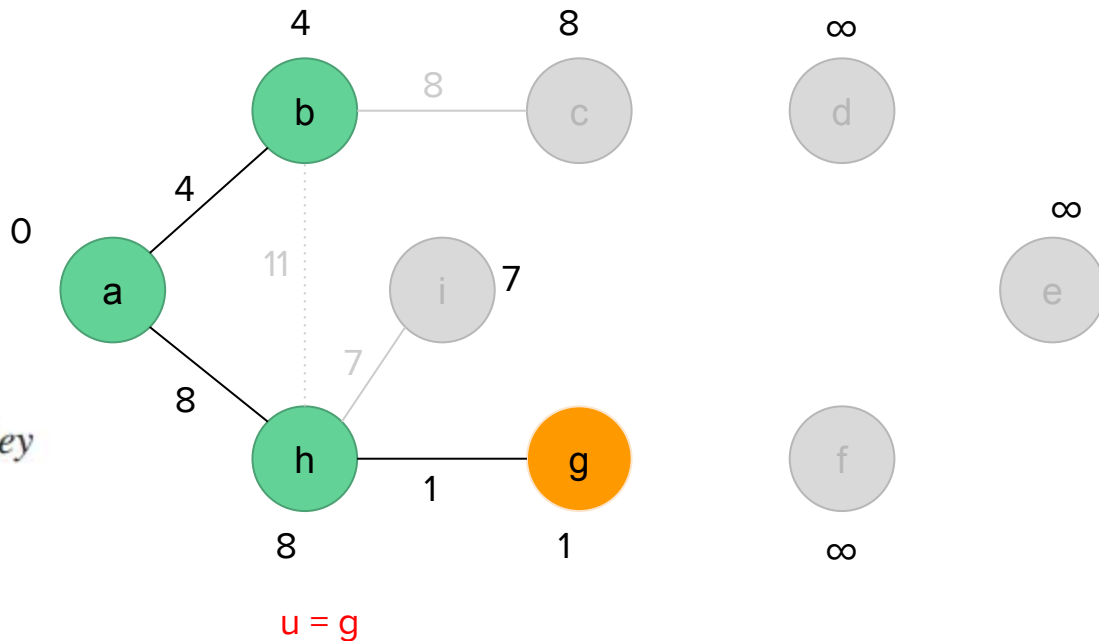
$G.\text{Adj}[u] = \{a, b, g, i\}$

Example

MST-PRIM(G, w, r)

$Q = \{i, c, d, e, f\}$

```
6 while  $Q \neq \emptyset$ 
7    $u = \text{EXTRACT-MIN}(Q)$ 
8   for each  $v \in G.\text{Adj}[u]$ 
9     if  $v \in Q$  and  $w(u, v) < v.\text{key}$ 
10        $v.\pi = u$ 
11        $v.\text{key} = w(u, v)$ 
```

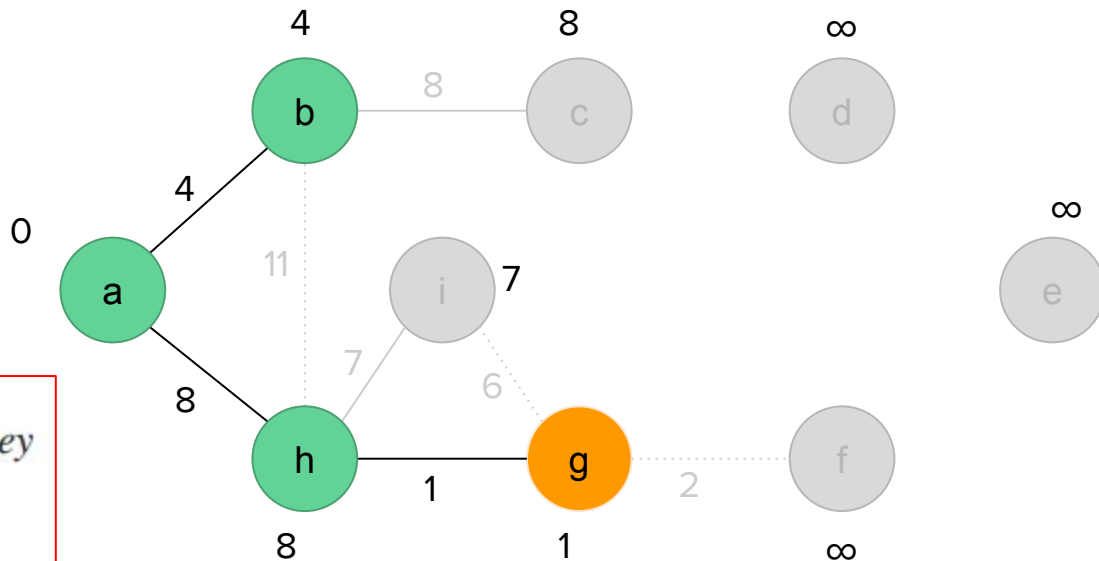


Example

MST-PRIM(G, w, r)

$Q = \{c, d, e, f, i\}$

```
6 while  $Q \neq \emptyset$ 
7    $u = \text{EXTRACT-MIN}(Q)$ 
8   for each  $v \in G.\text{Adj}[u]$ 
9     if  $v \in Q$  and  $w(u, v) < v.\text{key}$ 
10       $v.\pi = u$ 
11       $v.\text{key} = w(u, v)$ 
```



$u = g$

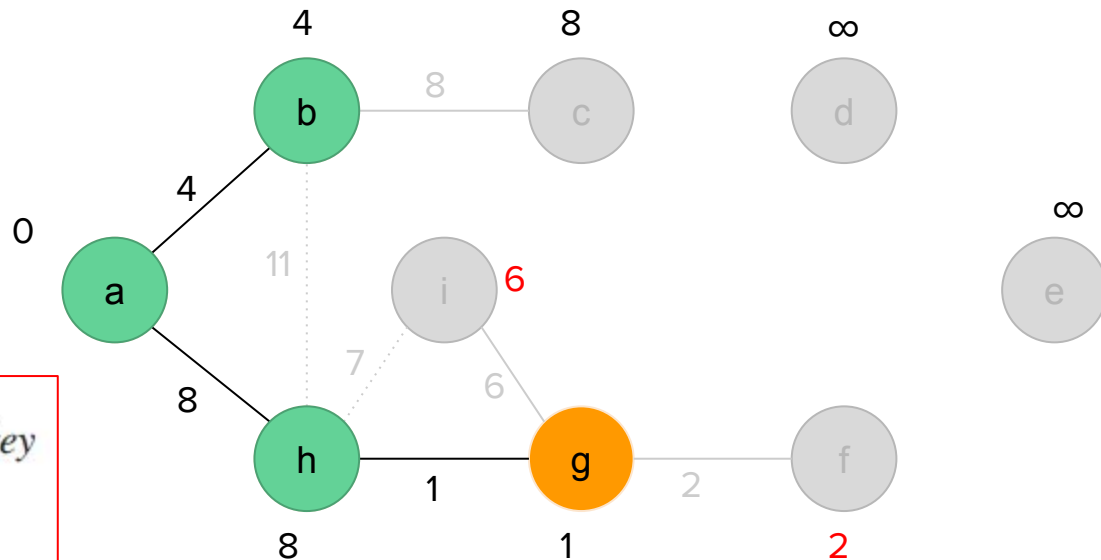
$G.\text{Adj}[u] = \{f, h, i\}$

Example

MST-PRIM(G, w, r)

$Q = \{f, i, c, d, e\}$

```
6 while  $Q \neq \emptyset$ 
7    $u = \text{EXTRACT-MIN}(Q)$ 
8   for each  $v \in G.\text{Adj}[u]$ 
9     if  $v \in Q$  and  $w(u, v) < v.\text{key}$ 
10       $v.\pi = u$ 
11       $v.\text{key} = w(u, v)$ 
```



$u = g$

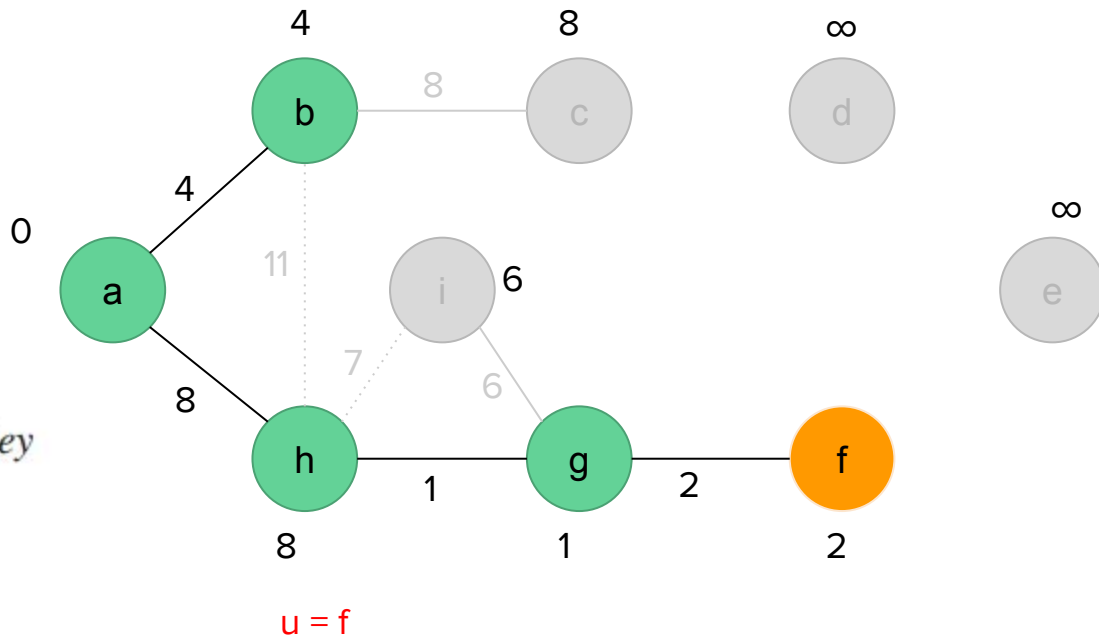
$G.\text{Adj}[u] = \{f, h, i\}$

Example

MST-PRIM(G, w, r)

$Q = \{i, c, d, e\}$

```
6 while  $Q \neq \emptyset$ 
7    $u = \text{EXTRACT-MIN}(Q)$ 
8   for each  $v \in G.\text{Adj}[u]$ 
9     if  $v \in Q$  and  $w(u, v) < v.\text{key}$ 
10        $v.\pi = u$ 
11        $v.\text{key} = w(u, v)$ 
```

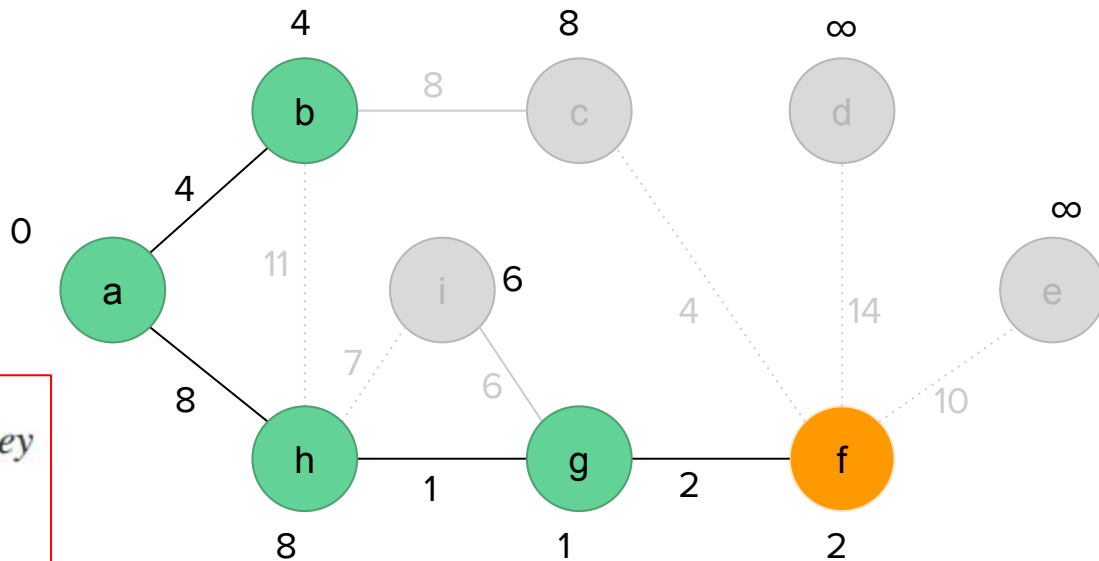


Example

MST-PRIM(G, w, r)

$Q = \{i, c, d, e\}$

```
6 while  $Q \neq \emptyset$ 
7    $u = \text{EXTRACT-MIN}(Q)$ 
8   for each  $v \in G.\text{Adj}[u]$ 
9     if  $v \in Q$  and  $w(u, v) < v.\text{key}$ 
10       $v.\pi = u$ 
11       $v.\text{key} = w(u, v)$ 
```



$u = f$

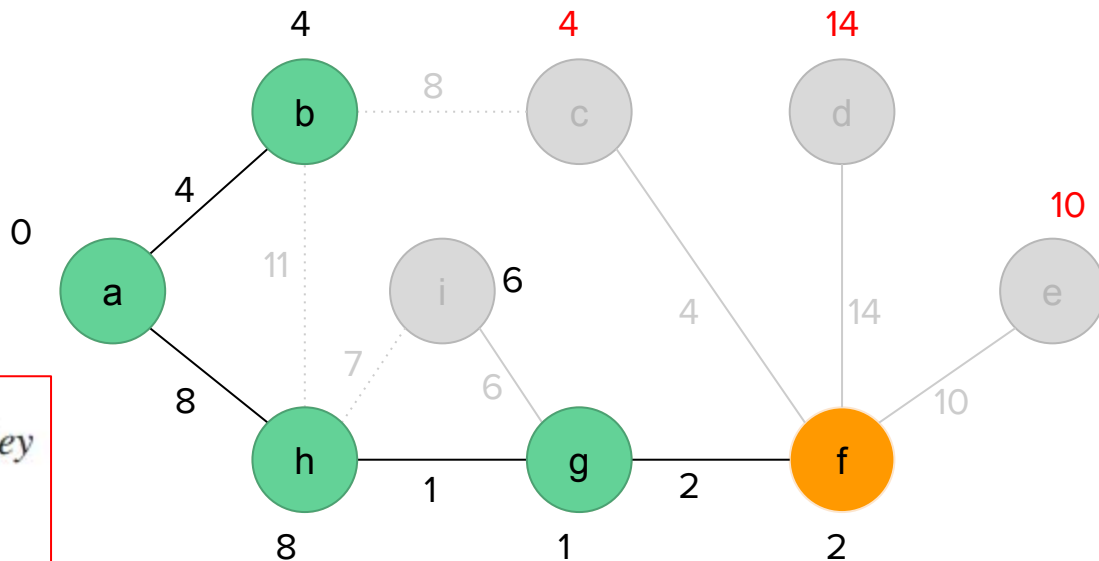
$G.\text{Adj}[u] = \{c, d, e, g\}$

Example

MST-PRIM(G, w, r)

$Q = \{c, i, e, d\}$

```
6 while  $Q \neq \emptyset$ 
7    $u = \text{EXTRACT-MIN}(Q)$ 
8   for each  $v \in G.\text{Adj}[u]$ 
9     if  $v \in Q$  and  $w(u, v) < v.\text{key}$ 
10       $v.\pi = u$ 
11       $v.\text{key} = w(u, v)$ 
```



$u = f$

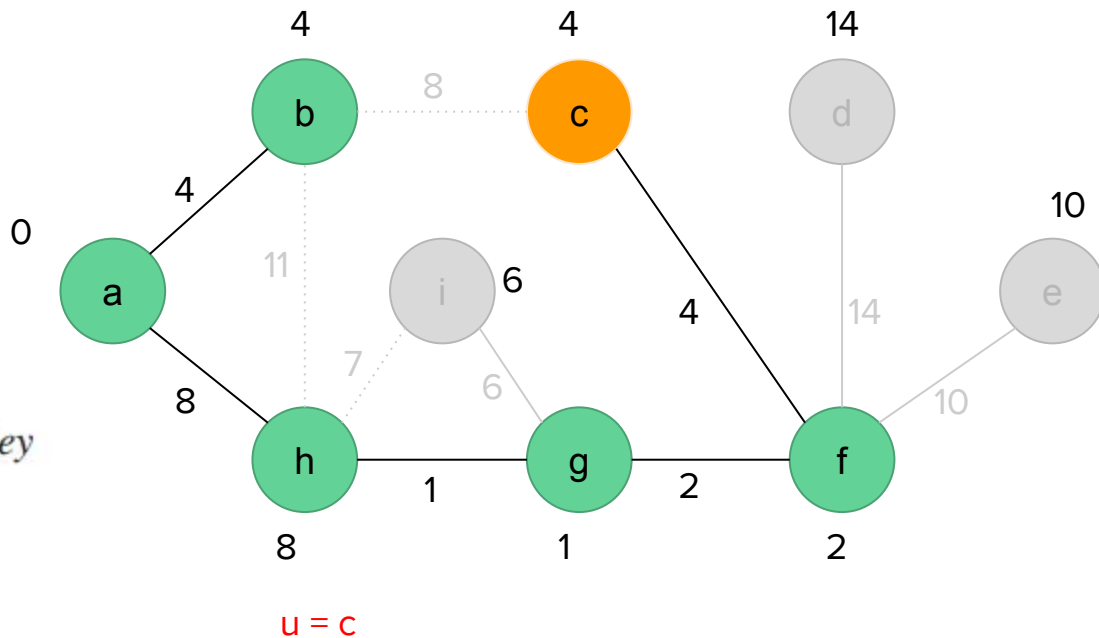
$G.\text{Adj}[u] = \{c, d, e, g\}$

Example

MST-PRIM(G, w, r)

$Q = \{i, e, d\}$

```
6 while  $Q \neq \emptyset$ 
7    $u = \text{EXTRACT-MIN}(Q)$ 
8   for each  $v \in G.\text{Adj}[u]$ 
9     if  $v \in Q$  and  $w(u, v) < v.\text{key}$ 
10        $v.\pi = u$ 
11        $v.\text{key} = w(u, v)$ 
```

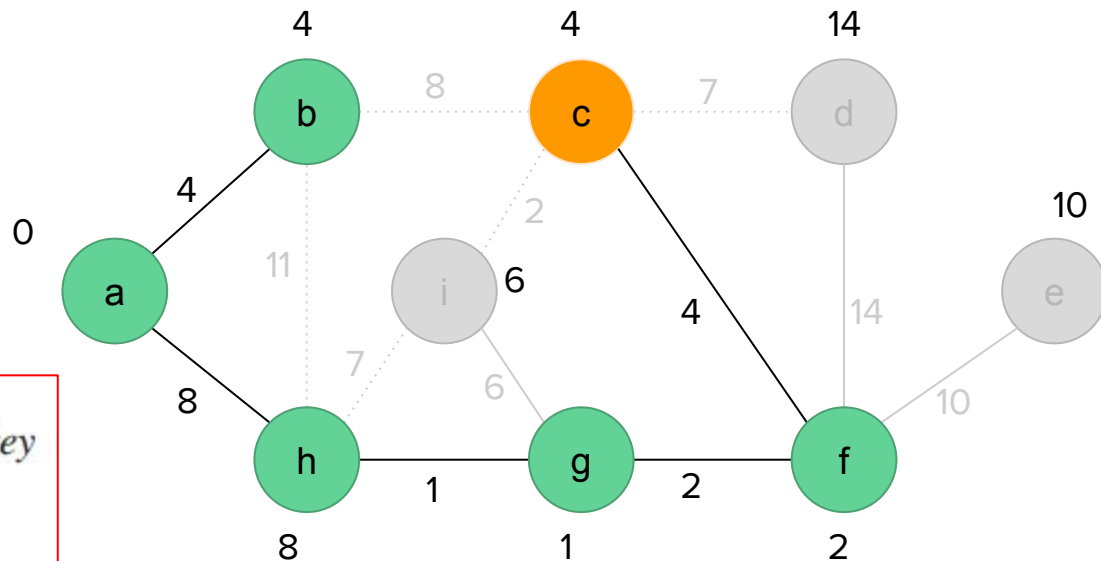


Example

MST-PRIM(G, w, r)

$Q = \{i, e, d\}$

```
6 while  $Q \neq \emptyset$ 
7    $u = \text{EXTRACT-MIN}(Q)$ 
8   for each  $v \in G.\text{Adj}[u]$ 
9     if  $v \in Q$  and  $w(u, v) < v.\text{key}$ 
10       $v.\pi = u$ 
11       $v.\text{key} = w(u, v)$ 
```



$u = c$

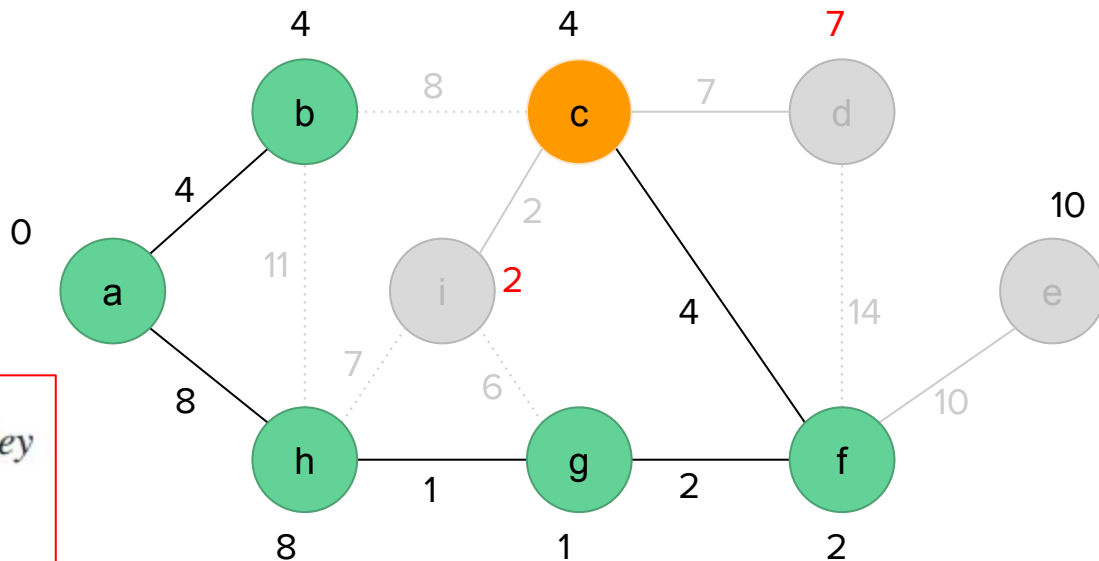
$G.\text{Adj}[u] = \{b, d, f, i\}$

Example

MST-PRIM(G, w, r)

$Q = \{i, d, e\}$

```
6 while  $Q \neq \emptyset$ 
7    $u = \text{EXTRACT-MIN}(Q)$ 
8   for each  $v \in G.\text{Adj}[u]$ 
9     if  $v \in Q$  and  $w(u, v) < v.\text{key}$ 
10        $v.\pi = u$ 
11        $v.\text{key} = w(u, v)$ 
```



$u = c$

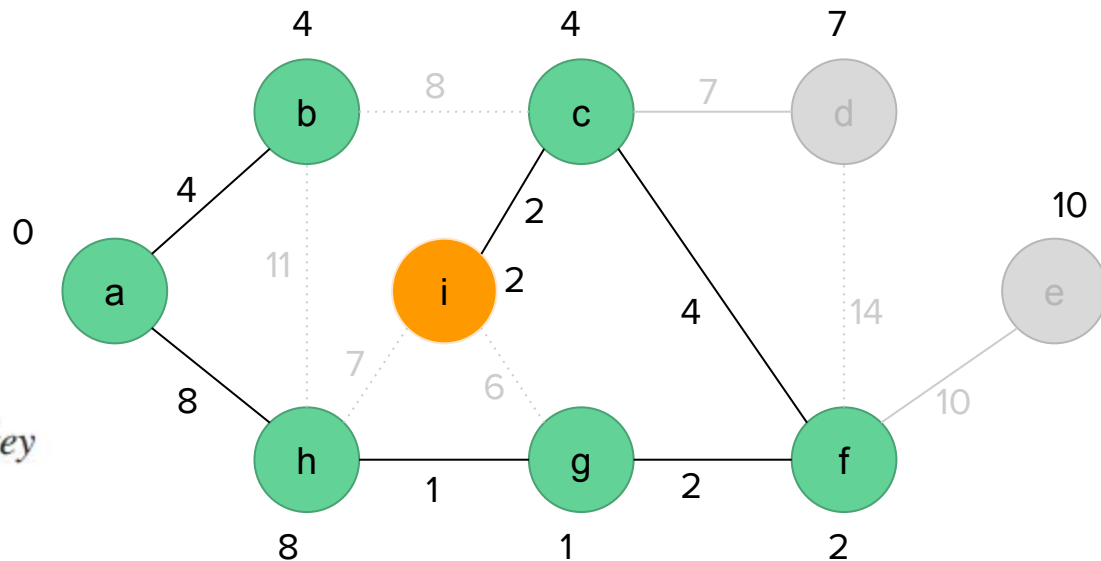
$G.\text{Adj}[u] = \{b, d, f, i\}$

Example

MST-PRIM(G, w, r)

$Q = \{d, e\}$

```
6 while  $Q \neq \emptyset$ 
7    $u = \text{EXTRACT-MIN}(Q)$ 
8   for each  $v \in G.\text{Adj}[u]$ 
9     if  $v \in Q$  and  $w(u, v) < v.\text{key}$ 
10       $v.\pi = u$ 
11       $v.\text{key} = w(u, v)$ 
```



$u = i$

$G.\text{Adj}[u] = \{b, c, g, h\}$

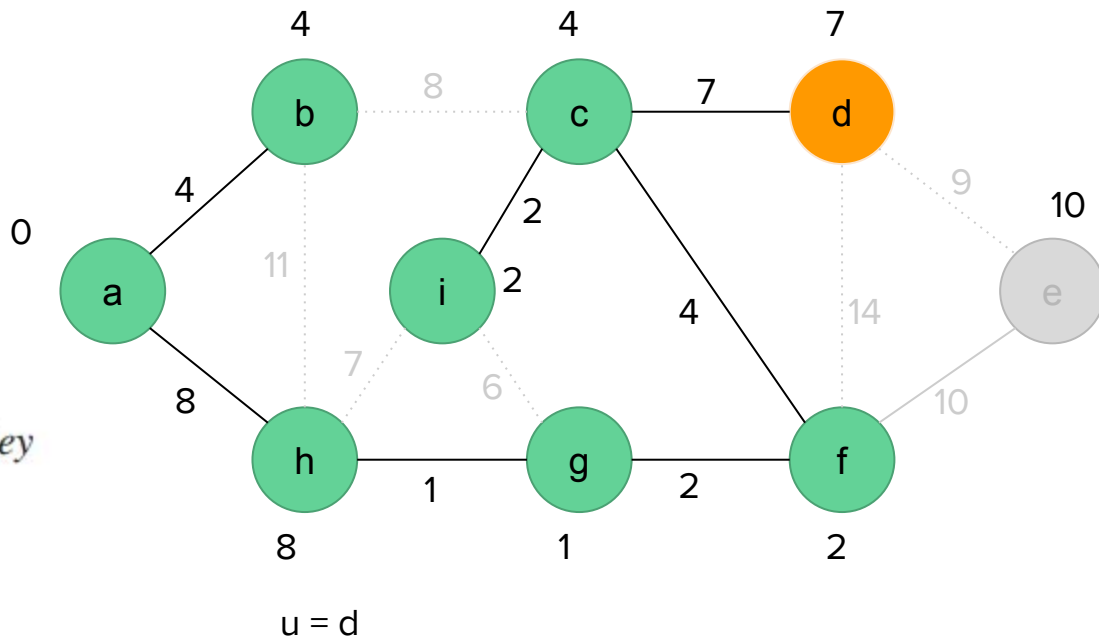
Example

$$\text{MST-PRIM}(G, w, r)$$
$$Q = \{e\}$$

```

6  while  $Q \neq \emptyset$ 
7       $u = \text{EXTRACT-MIN}(Q)$ 
8      for each  $v \in G.\text{Adj}[u]$ 
9          if  $v \in Q$  and  $w(u, v) < v.\text{key}$ 
10              $v.\pi = u$ 
11              $v.\text{key} = w(u, v)$ 

```

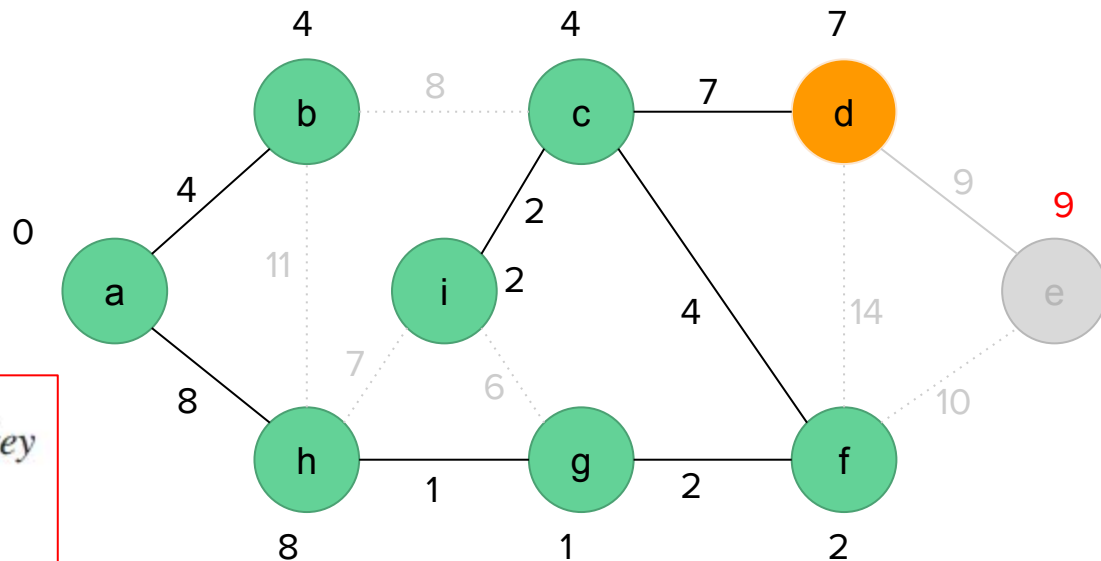


Example

MST-PRIM(G, w, r)

$Q = \{e\}$

```
6 while  $Q \neq \emptyset$ 
7    $u = \text{EXTRACT-MIN}(Q)$ 
8   for each  $v \in G.\text{Adj}[u]$ 
9     if  $v \in Q$  and  $w(u, v) < v.\text{key}$ 
10        $v.\pi = u$ 
11        $v.\text{key} = w(u, v)$ 
```



$u = d$

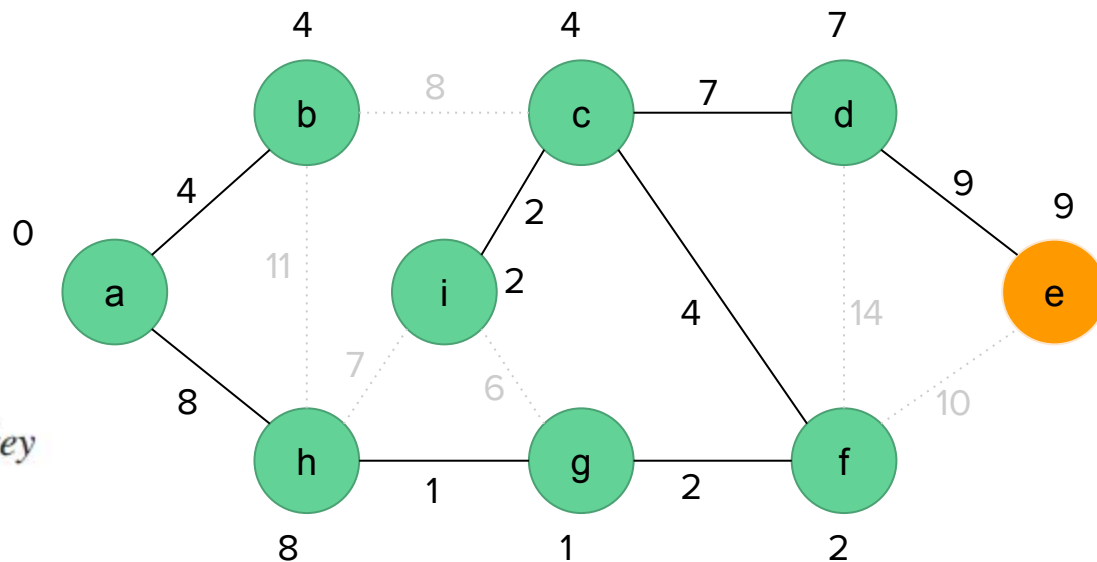
$G.\text{Adj}[u] = \{c, e, f\}$

Example

MST-PRIM(G, w, r)

$Q = \{\}$

```
6 while  $Q \neq \emptyset$ 
7    $u = \text{EXTRACT-MIN}(Q)$ 
8   for each  $v \in G.\text{Adj}[u]$ 
9     if  $v \in Q$  and  $w(u, v) < v.\text{key}$ 
10       $v.\pi = u$ 
11       $v.\text{key} = w(u, v)$ 
```



$u = e$
 $G.\text{Adj}[u] = \{d, f\}$

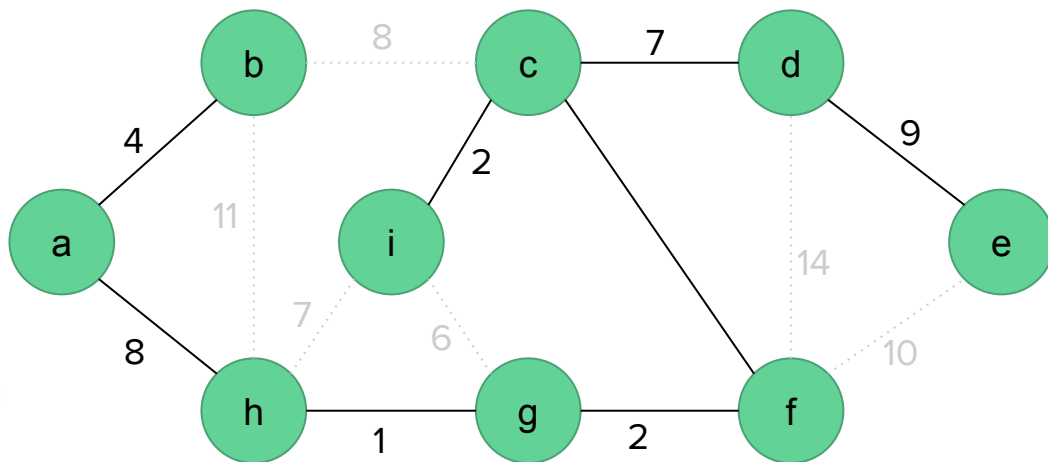
Example

$$\text{MST-PRIM}(G, w, r)$$

```

1  for each  $u \in G.V$ 
2       $u.key = \infty$ 
3       $u.\pi = \text{NIL}$ 
4   $r.key = 0$ 
5   $Q = G.V$ 
6  while  $Q \neq \emptyset$ 
7       $u = \text{EXTRACT-MIN}(Q)$ 
8      for each  $v \in G.Adj[u]$ 
9          if  $v \in Q$  and  $w(u, v) < v.key$ 
10              $v.\pi = u$ 
11              $v.key = w(u, v)$ 

```



Minimum spanning tree

Analysis of Prim's algorithm

MST-PRIM(G, w, r)

1 for each $u \in G.V$

2 $u.key = \infty$

3 $u.\pi = \text{NIL}$

4 $r.key = 0$

5 $Q = G.V$

6 while $Q \neq \emptyset$

7 $u = \text{EXTRACT-MIN}(Q)$

8 for each $v \in G.Adj[u]$

9 if $v \in Q$ and $w(u, v) < v.key$

10 $v.\pi = u$

11 $v.key = w(u, v)$

Line 1 - 3: $O(|V|)$

Line 4: $O(1)$

Line 5-11: ?

Analysis of Prim's algorithm

MST-PRIM(G, w, r)

```
1  for each  $u \in G.V$ 
2       $u.key = \infty$ 
3       $u.\pi = \text{NIL}$ 
4   $r.key = 0$ 
5   $Q = G.V$ 
6  while  $Q \neq \emptyset$ 
7       $u = \text{EXTRACT-MIN}(Q)$ 
8      for each  $v \in G.Adj[u]$ 
9          if  $v \in Q$  and  $w(u, v) < v.key$ 
10              $v.\pi = u$ 
11              $v.key = w(u, v)$ 
```

Line 1 - 3: $O(|V|)$

Line 4: $O(1)$

Line 5-11: Depends on how we implement the min-priority queue Q

Analysis of Prim's algorithm

- Depends on how we implement the min-priority queue Q
- If we implement Q as a binary min-heap

MST-PRIM(G, w, r)

```
1  for each  $u \in G.V$ 
2       $u.key = \infty$ 
3       $u.\pi = \text{NIL}$ 
4   $r.key = 0$ 
5   $Q = G.V$ 
6  while  $Q \neq \emptyset$ 
7       $u = \text{EXTRACT-MIN}(Q)$ 
8      for each  $v \in G.Adj[u]$ 
9          if  $v \in Q$  and  $w(u, v) < v.key$ 
10              $v.\pi = u$ 
11              $v.key = w(u, v)$ 
```

Line 1 - 3: $O(|V|)$

Line 4: $O(1)$

Line 5: $O(|V|)$

Line 6: The while loop executes $|V|$ times.

Line 7:

Extract-Min takes $O(\lg V)$.

Therefore total time for all calls to Extract-Min is $O(V \lg V)$

Analysis of Prim's algorithm

- Depends on how we implement the min-priority queue Q
- If we implement Q as a binary min-heap

MST-PRIM(G, w, r)

```
1  for each  $u \in G.V$ 
2       $u.key = \infty$ 
3       $u.\pi = \text{NIL}$ 
4   $r.key = 0$ 
5   $Q = G.V$ 
6  while  $Q \neq \emptyset$ 
7       $u = \text{EXTRACT-MIN}(Q)$ 
8      for each  $v \in G.Adj[u]$ 
9          if  $v \in Q$  and  $w(u, v) < v.key$ 
10              $v.\pi = u$ 
11              $v.key = w(u, v)$ 
```

Line 1 - 3: $O(|V|)$

Line 4: $O(1)$

Line 5: $O(|V|)$

Line 6: The while loop executes $|V|$ times.

Line 7:

Extract-Min takes $O(\lg V)$.

Therefore total time for all calls to Extract-Min is $O(V \lg V)$

Line 8: The for loop executes $O(E)$ since the sum of the lengths of all adjacency lists is $2|E|$

Line 11: It decreases the key of the node in the min-heap, so the heap needs to be adjusted, which a binary min-heap supports in $O(\lg V)$ time.

Analysis of Prim's algorithm

- Depends on how we implement the min-priority queue Q
- If we implement Q as a binary min-heap

MST-PRIM(G, w, r)

1 for each $u \in G.V$

2 $u.key = \infty$

3 $u.\pi = \text{NIL}$

4 $r.key = 0$

5 $Q = G.V$

6 while $Q \neq \emptyset$

7 $u = \text{EXTRACT-MIN}(Q)$

8 for each $v \in G.Adj[u]$

9 if $v \in Q$ and $w(u, v) < v.key$

10 $v.\pi = u$

11 $v.key = w(u, v)$

Line 1 - 3: $O(|V|)$

Line 4: $O(1)$

Line 5: $O(|V|)$

Line 6: $O(|V|)$.

Line 7: $O(V \lg V)$

Line 8: $O(E)$

Line 11: $O(E \lg V)$

Total time for Prim's algo = $O(V \lg V + E \lg V) = O(E \lg V)$

Analysis of Prim's algorithm

- Depends on how we implement the min-priority queue Q
- If we implement Q as a **Fibonacci heap**

MST-PRIM(G, w, r)

1 for each $u \in G.V$

2 $u.key = \infty$

3 $u.\pi = \text{NIL}$

4 $r.key = 0$

5 $Q = G.V$

6 while $Q \neq \emptyset$

7 $u = \text{EXTRACT-MIN}(Q)$

8 for each $v \in G.Adj[u]$

9 if $v \in Q$ and $w(u, v) < v.key$

10 $v.\pi = u$

11 $v.key = w(u, v)$

Line 1 - 3: $O(|V|)$

Line 4-5: $O(1)$

Line 6: $O(|V|)$.

Line 7: $O(V \log V)$

Line 8: $O(E)$

Line 11: $O(E)$

Total time for Prim's algo = $O(V \lg V + E)$

Shortest path algorithms

Shortest path algorithm

Finds the shortest path between two vertices in a graph

Applications:

- Finding the shortest path from one location to another in Google Maps, MapQuest, OpenStreetMap, (KTM Public Route) etc.
- Used by Telephone networks, Cellular networks for routing/connection in communication
- IP routing
- Word ladder problem

Single-source shortest path problem

The problem of finding shortest paths from a source vertex v to all other vertices in the graph.

Optimal substructure of a shortest path

Shortest-path algorithms typically rely on the property that a shortest path between two vertices contains other shortest paths within it.

Dijkstra's algorithm

A solution to the single-source shortest path problem in graph theory.

- **Input:** Weighted graph $G = (V, E)$ and source vertex $v \in V$, such that all edge weights are nonnegative
- **Output:** Lengths of shortest paths (or the shortest paths themselves) from a given source vertex $v \in V$ to all other vertices

Dijkstra's shortest path algorithm

Steps:

1. Insert the first vertex into the tree
2. From every vertex already in the tree, examine the total path length to all adjacent vertices not in the tree. Selected the edge with the minimum total path weight and insert it into the tree
3. Repeat step 2 until all vertices are in the tree

Dijkstra's shortest path algorithm

$$d(v) \leftarrow \begin{cases} \infty & \text{if } v \neq S \\ 0 & \text{if } v = S \end{cases}$$

$Q :=$ the set of nodes in V , sorted by $d(v)$ # Q is a min-priority queue

while Q not empty **do**

$v \leftarrow Q.pop()$

for all neighbours u of v **do**

if $d(v) + e(v, u) \leq d(u)$ **then**

$d(u) \leftarrow d(v) + e(v, u)$

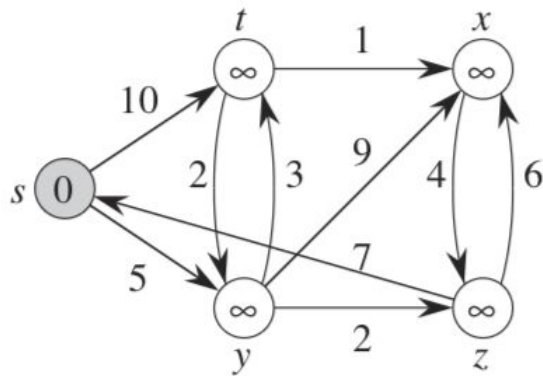
end if

end for

end while

Dijkstra's shortest path algorithm: Example

Find the shortest path from s to all other vertices.



Dijkstra's shortest path algorithm: Example

Find the shortest path distance from s to all other vertices.

$$d(v) \leftarrow \begin{cases} \infty & \text{if } v \neq S \\ 0 & \text{if } v = S \end{cases}$$

$Q :=$ the set of nodes in V , sorted by $d(v)$

while Q not empty **do**

$v \leftarrow Q.pop()$

for all neighbours u of v **do**

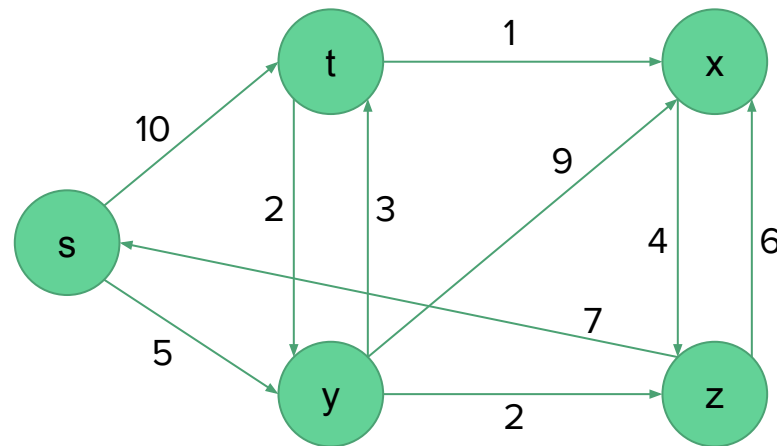
if $d(v) + e(v, u) \leq d(u)$ **then**

$d(u) \leftarrow d(v) + e(v, u)$

end if

end for

end while

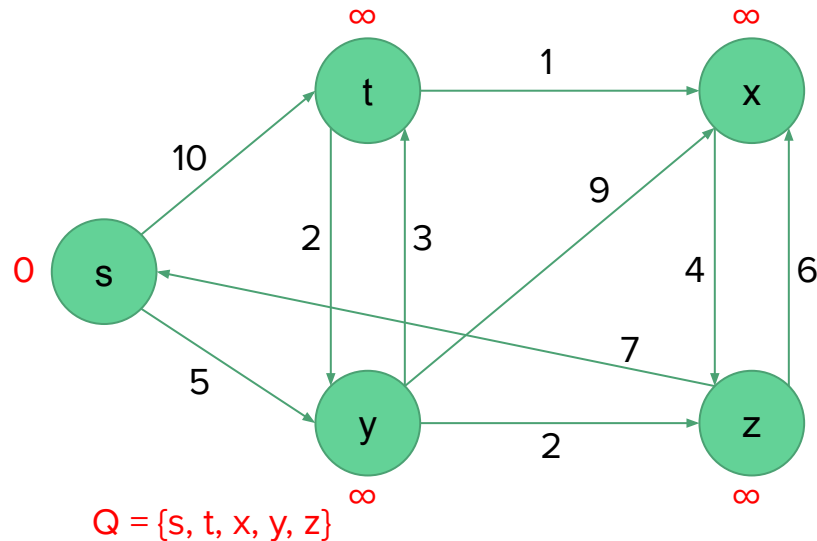


Dijkstra's shortest path algorithm: Example

Find the shortest path distance from s to all other vertices.

$$d(v) \leftarrow \begin{cases} \infty & \text{if } v \neq S \\ 0 & \text{if } v = S \end{cases}$$

$Q :=$ the set of nodes in V , sorted by $d(v)$



Dijkstra's shortest path algorithm: Example

Find the shortest path distance from s to all other vertices.

$$d(v) \leftarrow \begin{cases} \infty & \text{if } v \neq S \\ 0 & \text{if } v = S \end{cases}$$

$Q :=$ the set of nodes in V , sorted by $d(v)$

while Q not empty **do**

$v \leftarrow Q.pop()$

for all neighbours u of v **do**

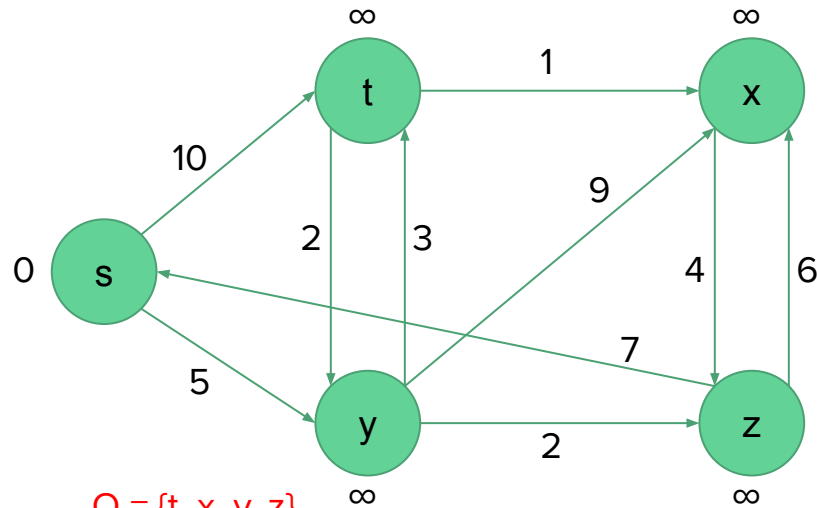
if $d(v) + e(v, u) \leq d(u)$ **then**

$d(u) \leftarrow d(v) + e(v, u)$

end if

end for

end while



Dijkstra's shortest path algorithm: Example

Find the shortest path distance from s to all other vertices.

$$d(v) \leftarrow \begin{cases} \infty & \text{if } v \neq S \\ 0 & \text{if } v = S \end{cases}$$

$Q :=$ the set of nodes in V , sorted by $d(v)$

while Q not empty **do**

$v \leftarrow Q.pop()$

for all neighbours u of v **do**

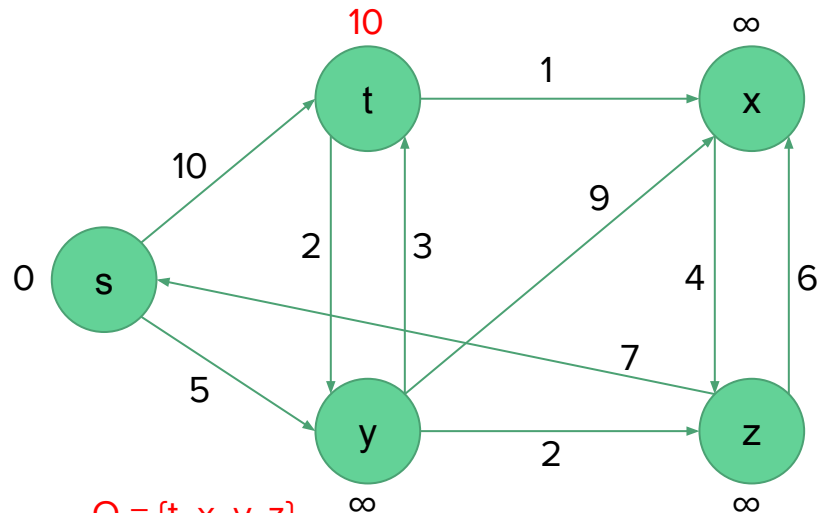
if $d(v) + e(v, u) \leq d(u)$ **then**

$d(u) \leftarrow d(v) + e(v, u)$

end if

end for

end while



$Q = \{t, x, y, z\}$

$v = s$

Neighbours of $v = \{t, y\}$

$d(s) + e(s, t) \leq d(t)$ True

Dijkstra's shortest path algorithm: Example

Find the shortest path distance from s to all other vertices.

$$d(v) \leftarrow \begin{cases} \infty & \text{if } v \neq S \\ 0 & \text{if } v = S \end{cases}$$

$Q :=$ the set of nodes in V , sorted by $d(v)$

while Q not empty **do**

$v \leftarrow Q.pop()$

for all neighbours u of v **do**

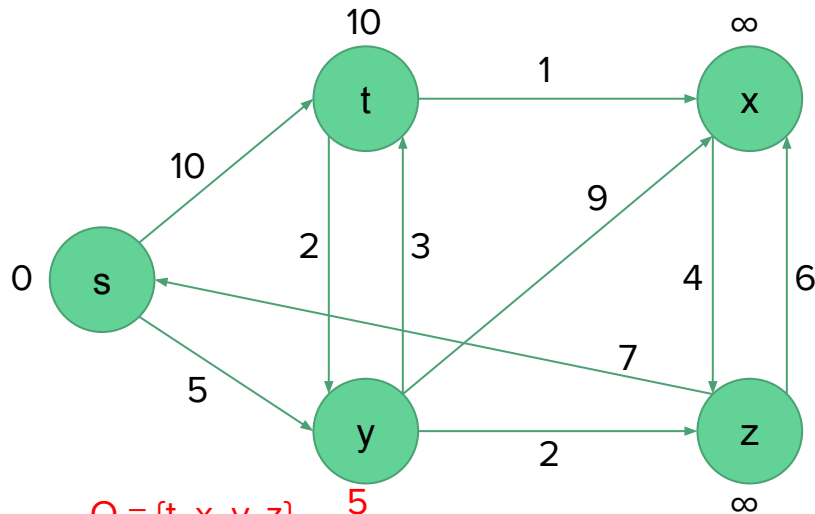
if $d(v) + e(v, u) \leq d(u)$ **then**

$d(u) \leftarrow d(v) + e(v, u)$

end if

end for

end while



$Q = \{t, x, y, z\}$

$v = s$

Neighbours of $v = \{t, y\}$

$d(s) + e(s, y) \leq d(y)$ True

Dijkstra's shortest path algorithm: Example

Find the shortest path distance from s to all other vertices.

$$d(v) \leftarrow \begin{cases} \infty & \text{if } v \neq S \\ 0 & \text{if } v = S \end{cases}$$

$Q :=$ the set of nodes in V , sorted by $d(v)$

while Q not empty **do**

$v \leftarrow Q.pop()$

for all neighbours u of v **do**

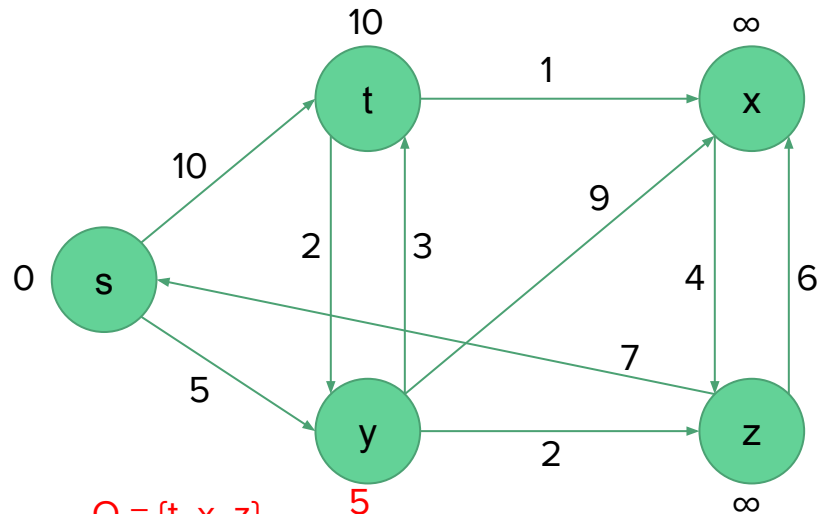
if $d(v) + e(v, u) \leq d(u)$ **then**

$d(u) \leftarrow d(v) + e(v, u)$

end if

end for

end while



Dijkstra's shortest path algorithm: Example

Find the shortest path distance from s to all other vertices.

$$d(v) \leftarrow \begin{cases} \infty & \text{if } v \neq S \\ 0 & \text{if } v = S \end{cases}$$

$Q :=$ the set of nodes in V , sorted by $d(v)$

while Q not empty **do**

$v \leftarrow Q.pop()$

for all neighbours u of v **do**

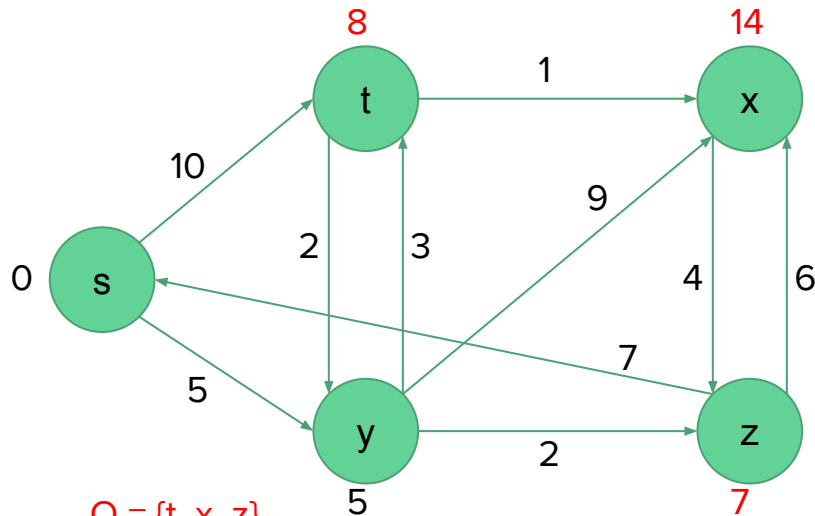
if $d(v) + e(v, u) \leq d(u)$ **then**

$d(u) \leftarrow d(v) + e(v, u)$

end if

end for

end while



$Q = \{t, x, z\}$

$v = y$

Neighbours of $v = \{t, x, z\}$

$d(y) + e(y, t) \leq d(t)$ True

$d(y) + e(y, x) \leq d(x)$ True

$d(y) + e(y, z) \leq d(z)$ True

Dijkstra's shortest path algorithm: Example

Find the shortest path distance from s to all other vertices.

$$d(v) \leftarrow \begin{cases} \infty & \text{if } v \neq S \\ 0 & \text{if } v = S \end{cases}$$

$Q :=$ the set of nodes in V , sorted by $d(v)$

while Q not empty **do**

$v \leftarrow Q.pop()$

for all neighbours u of v **do**

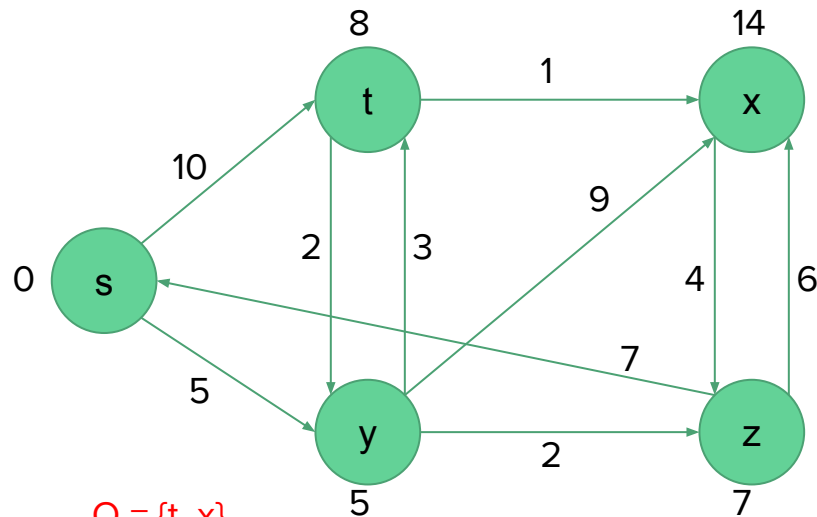
if $d(v) + e(v, u) \leq d(u)$ **then**

$d(u) \leftarrow d(v) + e(v, u)$

end if

end for

end while



$Q = \{t, x\}$

$v = z$

Neighbours of $v = \{x, s\}$

Dijkstra's shortest path algorithm: Example

Find the shortest path distance from s to all other vertices.

$$d(v) \leftarrow \begin{cases} \infty & \text{if } v \neq S \\ 0 & \text{if } v = S \end{cases}$$

$Q :=$ the set of nodes in V , sorted by $d(v)$

while Q not empty **do**

$v \leftarrow Q.pop()$

for all neighbours u of v **do**

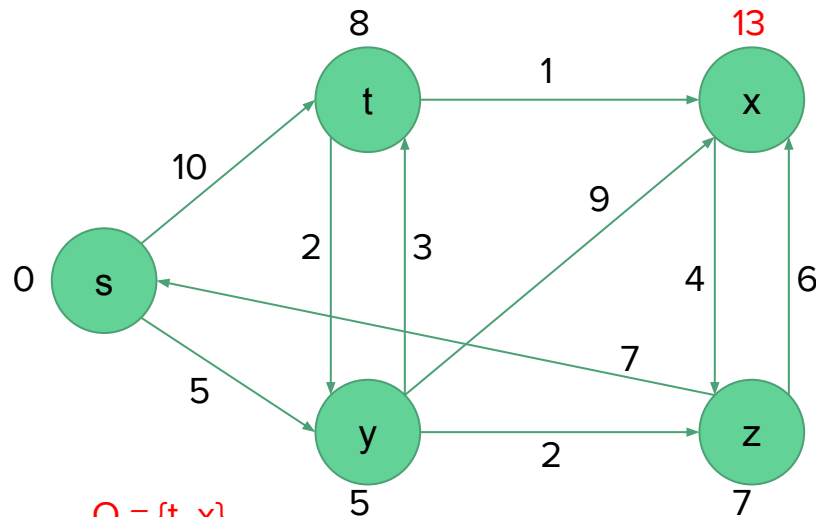
if $d(v) + e(v, u) \leq d(u)$ **then**

$d(u) \leftarrow d(v) + e(v, u)$

end if

end for

end while



$Q = \{t, x\}$

$v = z$

Neighbours of $v = \{x, s\}$

$d(z) + e(z, x) \leq d(x)$ True

$d(z) + e(z, s) \leq d(s)$ False

Dijkstra's shortest path algorithm: Example

Find the shortest path distance from s to all other vertices.

$$d(v) \leftarrow \begin{cases} \infty & \text{if } v \neq S \\ 0 & \text{if } v = S \end{cases}$$

$Q :=$ the set of nodes in V , sorted by $d(v)$

while Q not empty **do**

$v \leftarrow Q.pop()$

for all neighbours u of v **do**

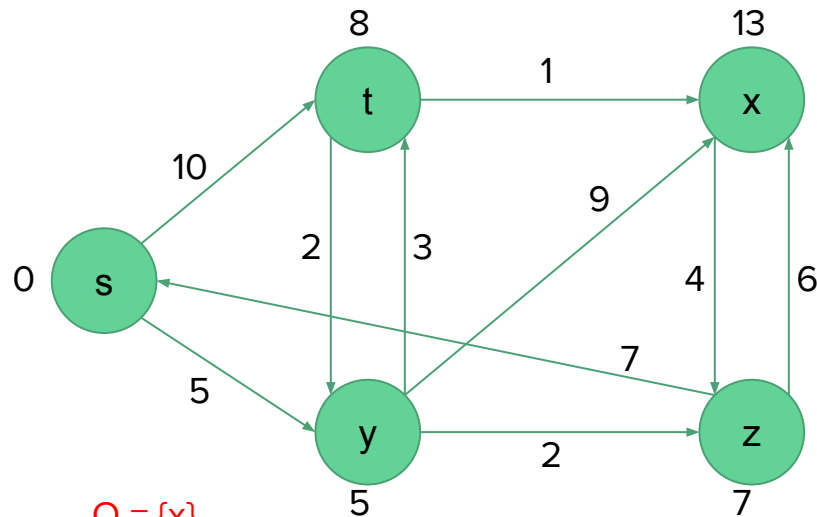
if $d(v) + e(v, u) \leq d(u)$ **then**

$d(u) \leftarrow d(v) + e(v, u)$

end if

end for

end while



$Q = \{x\}$

$v = t$

Neighbours of $v = \{x, y\}$

Dijkstra's shortest path algorithm: Example

Find the shortest path distance from s to all other vertices.

$$d(v) \leftarrow \begin{cases} \infty & \text{if } v \neq S \\ 0 & \text{if } v = S \end{cases}$$

$Q :=$ the set of nodes in V , sorted by $d(v)$

while Q not empty **do**

$v \leftarrow Q.pop()$

for all neighbours u of v **do**

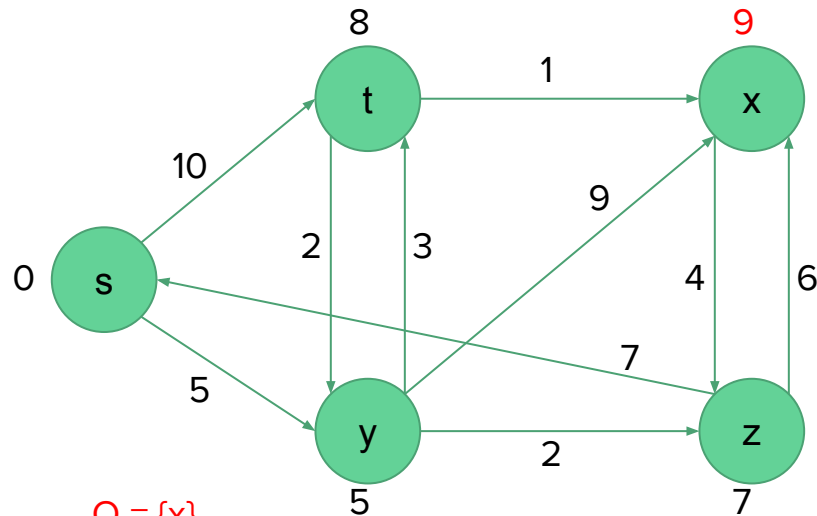
if $d(v) + e(v, u) \leq d(u)$ **then**

$d(u) \leftarrow d(v) + e(v, u)$

end if

end for

end while



$Q = \{x\}$

$v = t$

Neighbours of $v = \{x, y\}$

$d(t) + e(t, x) \leq d(x)$ True

$d(t) + e(t, y) \leq d(y)$ False

Running time

- Depends on the implementation
- The simplest implementation is to store vertices in an array or linked list. This will produce a running time of $O(|V|^2 + |E|)$
- For sparse graphs, or graphs with very few edges and many nodes, it can be implemented more efficiently storing the graph in an adjacency list using a binary heap or priority queue. This will produce a running time of $O((|E| + |V|) \log |V|)$

A* search algorithm

A* (pronounced '**A-star**') is a search algorithm that finds the shortest path between some nodes S and T in a graph

It is a **generalization of Dijkstra's algorithm** that cuts down on the size of the subgraph that must be explored using a **heuristic function**

Suppose we want to get to node T, and we are currently at node v. Informally, a heuristic function $h(v)$ is a function that 'estimates' how v is away from T

A* search algorithm

$$d(v) \leftarrow \begin{cases} \infty & \text{if } v \neq S \\ 0 & \text{if } v = S \end{cases}$$

$Q :=$ the set of nodes in V , sorted by $d(v) + h(v)$

while Q not empty **do**

$v \leftarrow Q.pop()$

for all neighbours u of v **do**

if $d(v) + e(v, u) \leq d(u)$ **then**

$d(u) \leftarrow d(v) + e(v, u)$

end if

end for

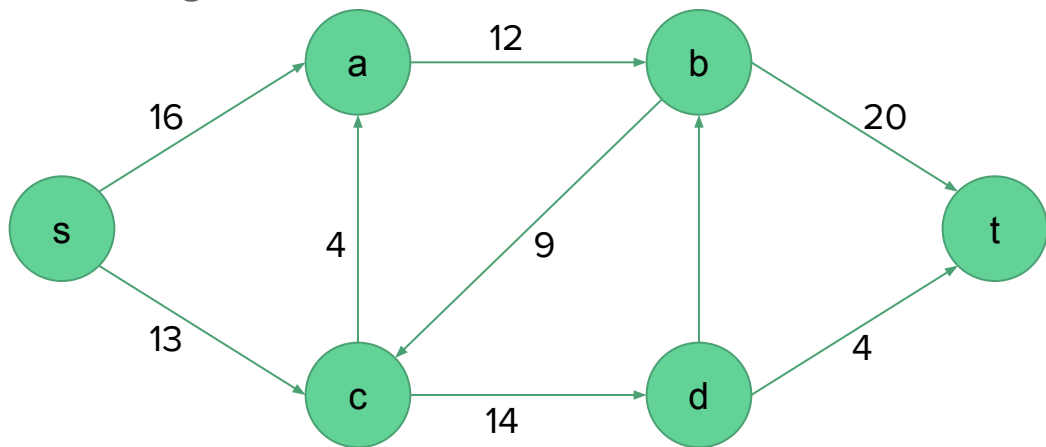
end while

Dijkstra's algorithm is a special case of A*, when we set $h(v) = 0$ for all v

Flow networks

Flow network

- A directed connected graph $G = (V, E)$, where each edge e is associated with its capacity $c(e) > 0$.
- If E contains an edge (u, v) , then there is no edge (v, u) in the reverse direction
- We distinguish two vertices: a source s and a sink t ($s \neq t$)



Flow network

A **flow** in G is a real-valued function $f : V \times V \rightarrow \mathbb{R}$ that satisfies the following two properties:

Capacity constraint: Flow on edge e does not exceed its capacity $c(e)$

That is, for all $u, v \in V$, we require $0 \leq f(u, v) \leq c(u, v)$

Flow conservation: For every node $v \neq s, t$ (nodes other than s and t), incoming flow is equal to the outgoing flow.

That is, for all $u \in V - \{s, t\}$, we require

$$\sum_{v \in V} f(v, u) = \sum_{v \in V} f(u, v)$$

Network flow problem (maximum-flow)

Given: A flow network G with source s and sink t .

Problem: Find a flow of maximum value

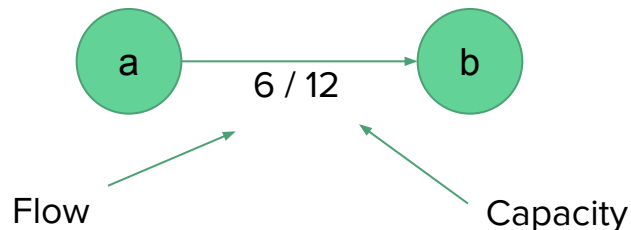
Network flow problem

Alternative formulation: Minimum cut

- Remove some edges from the graph such that after removing the edges, there is no path from s to t
- The cost of removing e is equal to its capacity $c(e)$
- The minimum cut problem is to find a cut with total minimum cut

Terminologies

Residual capacity of an edge



Formally, given a flow network $G = (V, E)$ with capacity c and flow f , the residual capacity $c_f(u, v)$, where $u, v \in V$, is defined by

$$c_f(u, v) = \begin{cases} c(u, v) - f(u, v) & \text{if } (u, v) \in E, \\ f(v, u) & \text{if } (v, u) \in E, \\ 0 & \text{otherwise.} \end{cases}$$

Terminologies

A **Residual graph/network** contains all the edges that have strictly positive residual capacity.

Formally, given a flow network $G = (V, E)$ and a flow f , the residual network of G induced by f is $G_f = (V, E_f)$, where

$$E_f = \{(u, v) \in V \times V : c_f(u, v) > 0\}$$

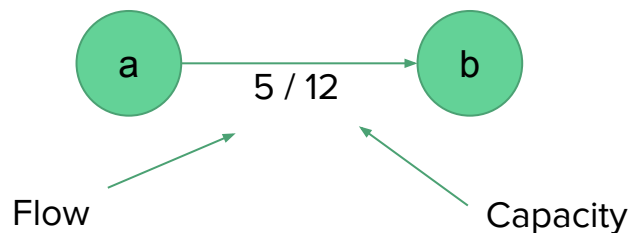
The edges in E_f are either edges in E or their reversals.

Terminologies

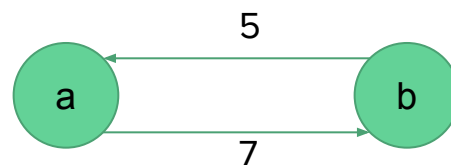
Given a flow network G and a flow f , the **residual network** G_f is constructed by placing

1. Edges with residual capacity $c_f(u, v) = c(u, v) - f(u, v)$
2. Edges with residual capacity $c_f(v, u) = f(u, v)$ (representing a decrease of a flow)

Example:



An edge in a flow network



Corresponding edge in the induced residual network

Terminologies

Augmenting path

Given a flow network $G = (V, E)$ and a flow f , an **augmenting path** p is a simple path from s to t in the residual network G_f .

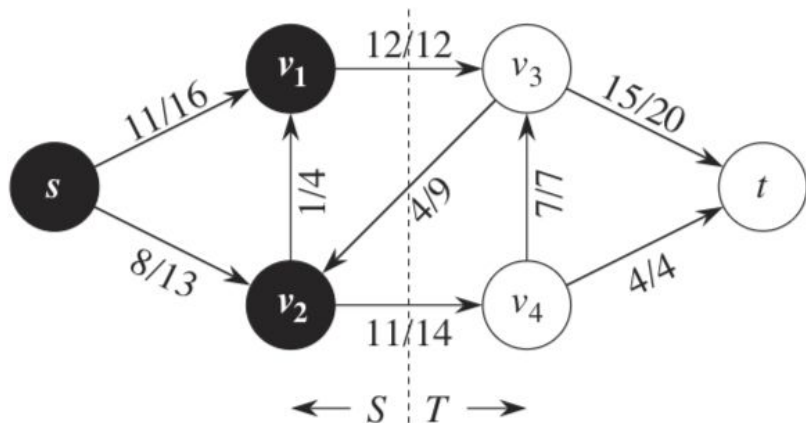
By the definition of the residual network, we may increase the flow on an edge (u,v) of an augmenting path by up to $c_f(u, v)$ without violating the capacity constraint on whichever of (u, v) and (v, u) is in the original flow network G .

If there are augmenting paths, then the flow is not maximum yet.

Terminologies

Cuts of flow networks

A cut (S, T) of flow network $G = (V, E)$ is a partition of V into S and $T = V - S$ such that $s \in S$ and $t \in T$.



If f is a flow, then the **net flow** $f(S, T)$ across the cut (S, T) is

$$f(S, T) = \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{u \in S} \sum_{v \in T} f(v, u)$$

The **capacity** of the cut (S, T) is

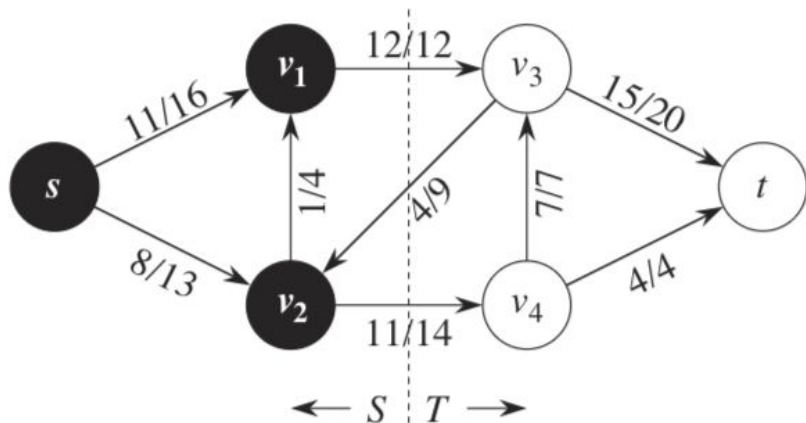
$$c(S, T) = \sum_{u \in S} \sum_{v \in T} c(u, v)$$

A minimum cut of a network is a cut whose capacity is minimum over all cuts of the network.

Terminologies

Cuts of flow networks

A cut (S, T) of flow network $G = (V, E)$ is a partition of V into S and $T = V - S$ such that $s \in S$ and $t \in T$.



Example:

Here the cut is $(\{s, v_1, v_2\}, \{v_3, v_4, t\})$.

$$\begin{aligned}\text{Net flow} &= f(v_1, v_3) + f(v_2, v_4) - f(v_3, v_2) \\ &= 12 + 11 - 4 \\ &= 19\end{aligned}$$

$$\text{Capacity} = c(v_1, v_3) + c(v_2, v_4) = 12 + 14 = 26$$

Max-flow min-cut theorem

Theorem 26.6 (Max-flow min-cut theorem)

If f is a flow in a flow network $G = (V, E)$ with source s and sink t , then the following conditions are equivalent:

1. f is a maximum flow in G .
2. The residual network G_f contains no augmenting paths.
3. $|f| = c(S, T)$ for some cut (S, T) of G .

Ford-Fulkerson method

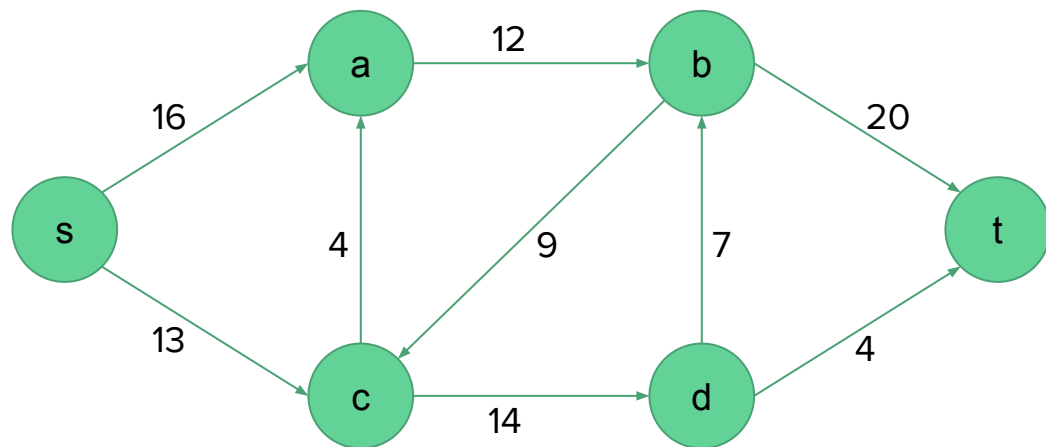
Main idea:

Find valid flow paths until there is none left, and add them up.

Steps:

1. Set $f_{\text{total}} = 0$
2. Repeat until there is no path from s to t
 - a. Run DFS from s to find a flow path to t
 - b. Let f be the minimum capacity value on the path (aka bottleneck capacity)
 - c. Add f to f_{total}
 - d. For each edge $u \rightarrow v$ on the path
 - i. Decrease $c(u \rightarrow v)$ by f
 - ii. Increase $c(v \rightarrow u)$ by f

Example

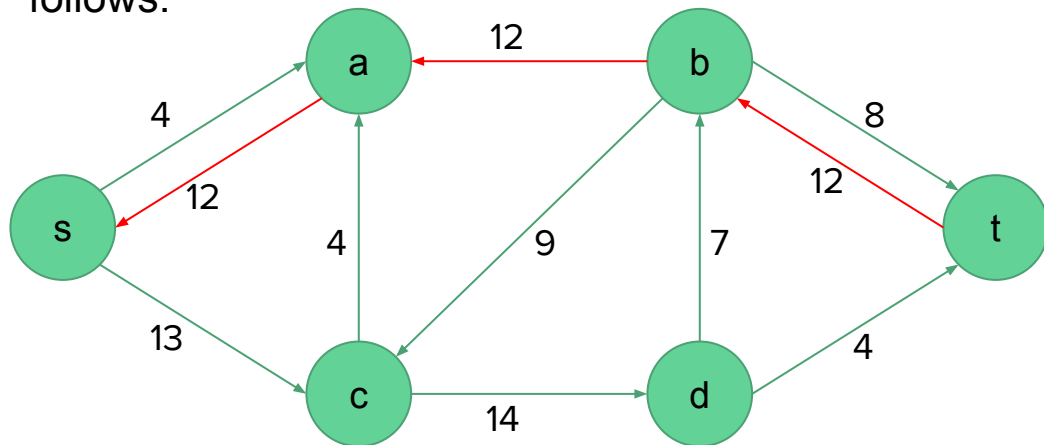


Augmenting paths:

	Min. capacity
$s \rightarrow a \rightarrow b \rightarrow t$	12
$s \rightarrow c \rightarrow d \rightarrow t$	4
$s \rightarrow c \rightarrow a \rightarrow b \rightarrow t$	4
$s \rightarrow c \rightarrow d \rightarrow b \rightarrow t$	7
$s \rightarrow a \rightarrow b \rightarrow c \rightarrow d \rightarrow t$	4

Example

If we choose the augmenting path $s \rightarrow a \rightarrow b \rightarrow t$, the induced residual graph would be as follows:



Augmenting paths:

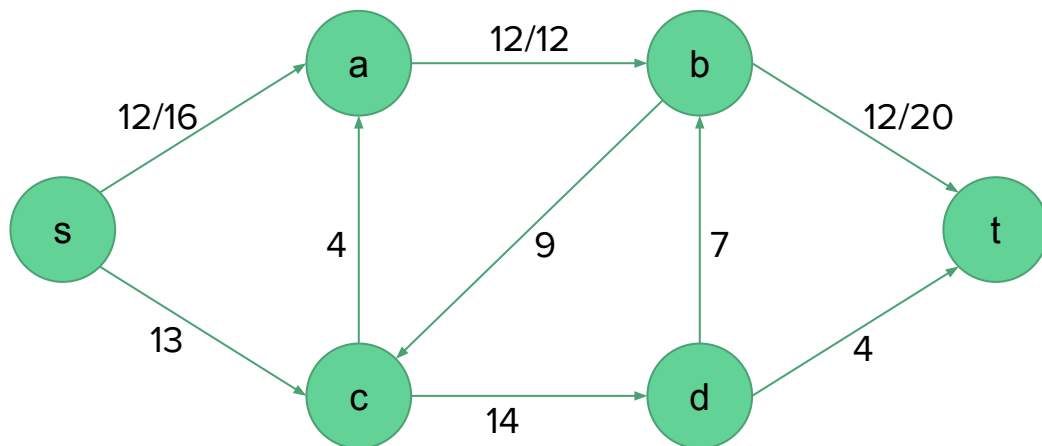
	Min. capacity
$s \rightarrow c \rightarrow d \rightarrow t$	4
$s \rightarrow c \rightarrow d \rightarrow b \rightarrow t$	7

Selected augmenting paths:

	Flow
$s \rightarrow a \rightarrow b \rightarrow t$	12

Example

And the corresponding flow network would be as follows:



Augmenting paths:

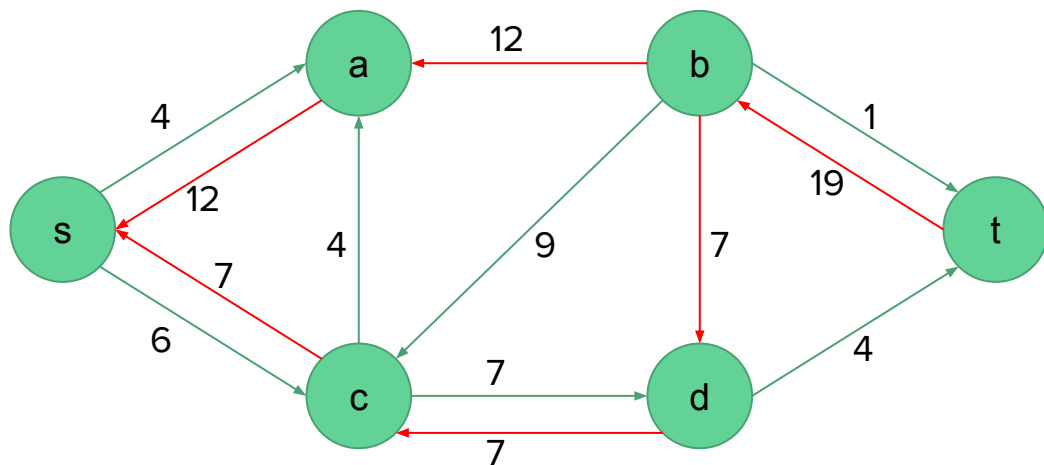
	Min. capacity
$s \rightarrow c \rightarrow d \rightarrow t$	4
$s \rightarrow c \rightarrow d \rightarrow b \rightarrow t$	7

Selected augmenting paths:

	Flow
$s \rightarrow a \rightarrow b \rightarrow t$	12

Example

If we choose the augmenting path $s \rightarrow c \rightarrow d \rightarrow b \rightarrow t$, the residual graph would be as follows:



Augmenting paths:

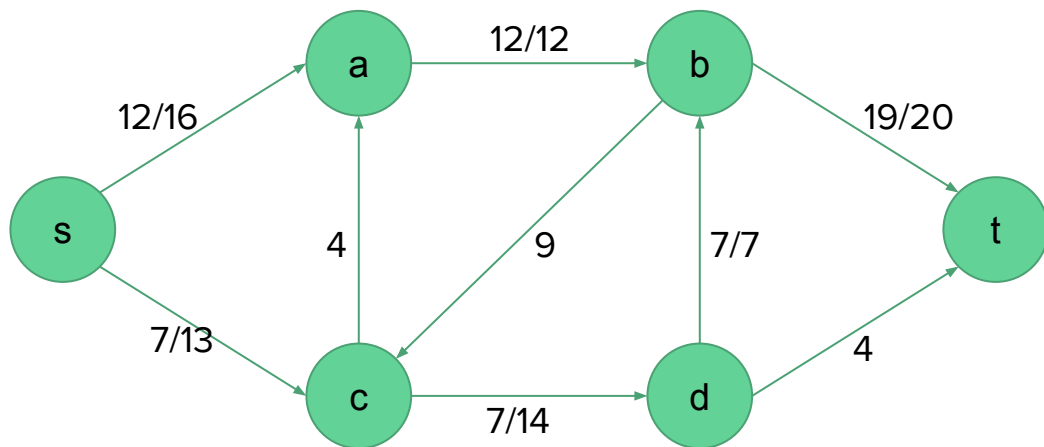
	Min. capacity
$s \rightarrow c \rightarrow d \rightarrow t$	4

Selected augmenting paths:

	Flow
$s \rightarrow a \rightarrow b \rightarrow t$	12
$s \rightarrow c \rightarrow d \rightarrow b \rightarrow t$	7

Example

And the corresponding flow network would be as follows:



Augmenting paths:

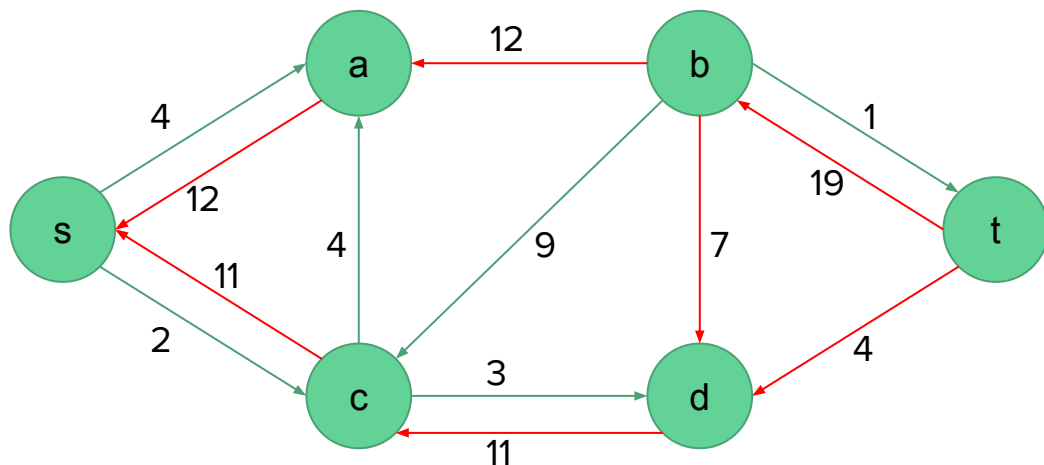
	Min. capacity
$s \rightarrow c \rightarrow d \rightarrow t$	4

Selected augmenting paths:

	Flow
$s \rightarrow a \rightarrow b \rightarrow t$	12
$s \rightarrow c \rightarrow d \rightarrow b \rightarrow t$	7

Example

If we choose the augmenting path $s \rightarrow c \rightarrow d \rightarrow t$, the residual graph would be as follows:



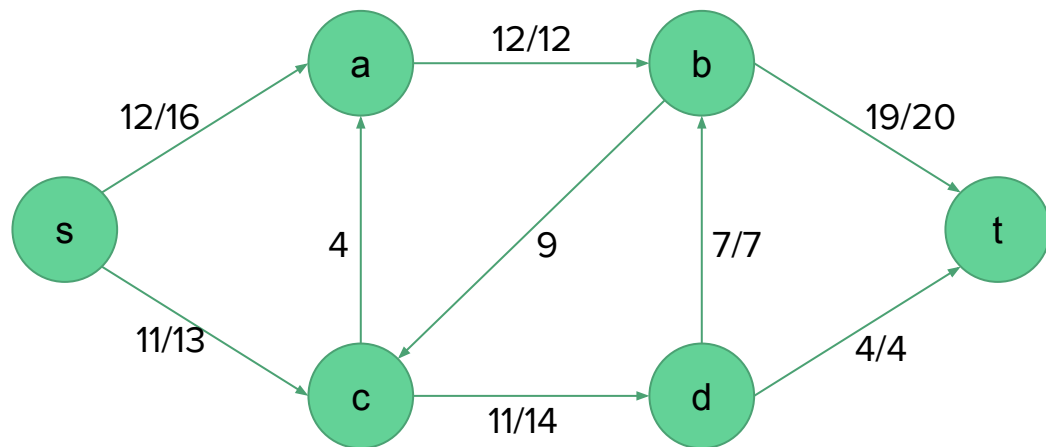
No more augmenting paths

Selected augmenting paths:

	Flow
$s \rightarrow a \rightarrow b \rightarrow t$	12
$s \rightarrow c \rightarrow d \rightarrow b \rightarrow t$	7
$s \rightarrow c \rightarrow d \rightarrow t$	4

Example

There is no path from s to t, thus the max flow network is as below:



No more augmenting paths

Selected augmenting paths:

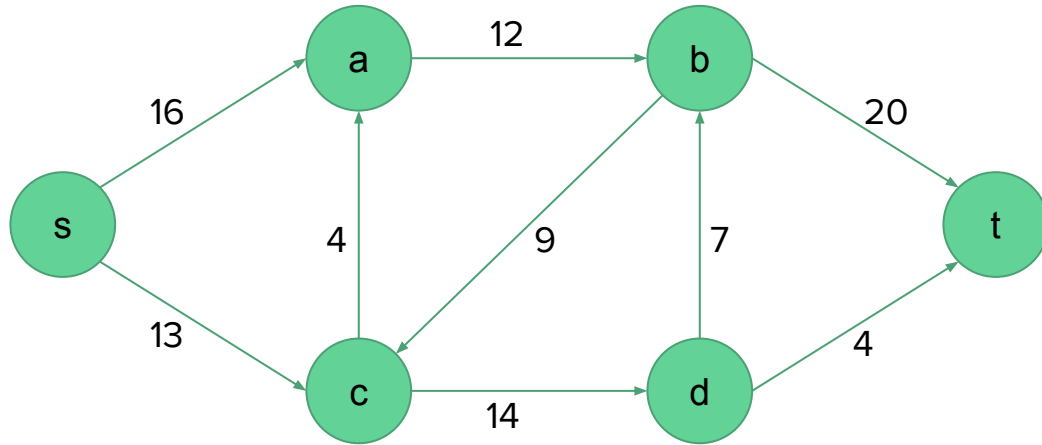
	Flow
$s \rightarrow a \rightarrow b \rightarrow t$	12
$s \rightarrow c \rightarrow d \rightarrow b \rightarrow t$	7
$s \rightarrow c \rightarrow d \rightarrow t$	4
Total flow	23

Computing minimum cut

- To compute minimum cut, we use the residual graph induced by the max flow
- Mark all nodes reachable from s
 - Call this set of reachable nodes A
- Now separate these nodes from the others
 - Cut edges from A to $V-A$

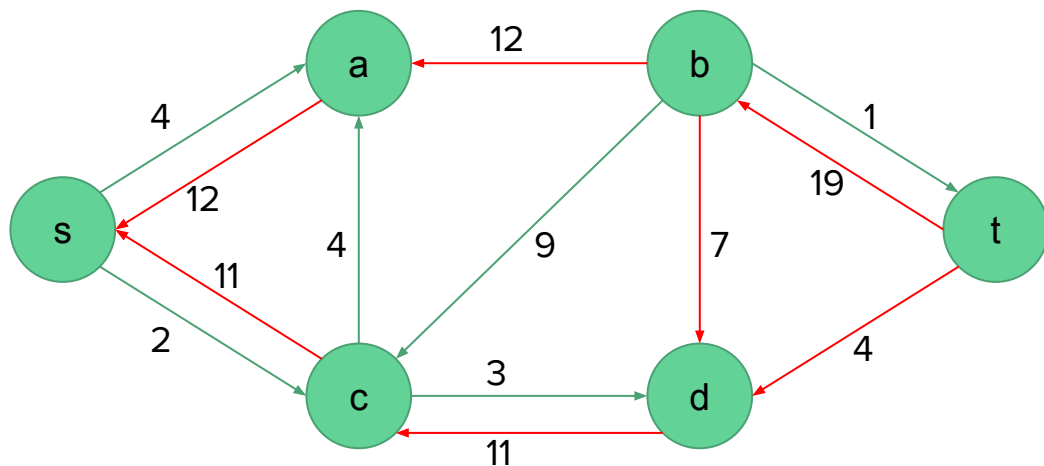
Computing minimum cut

Example: Compute the minimum cut in the following flow network



Computing minimum cut: Example

The residual graph induced by the max flow is as below:



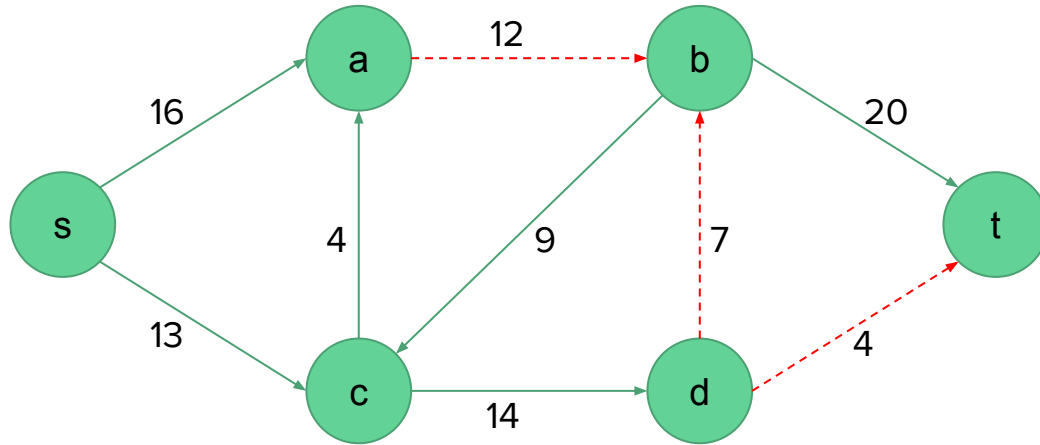
Nodes reachable from s

$A = \{a, c, d\}$

$V-A = \{b, t\}$

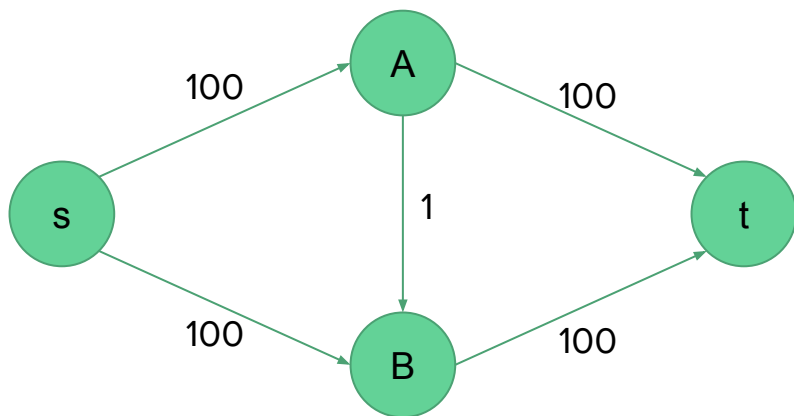
Computing minimum cut: Example

Cut edges from A to V-A



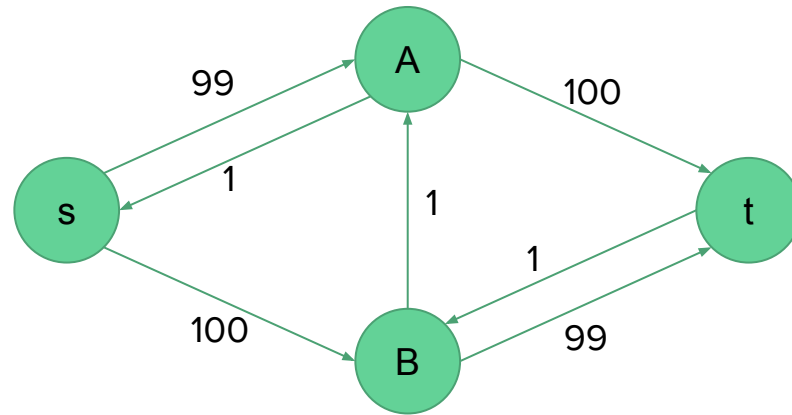
Analysis of Ford-Fulkerson

- Running time depends on how we find augmenting paths
- If we choose it poorly, the algorithm might not even terminate
- For example, consider the following network



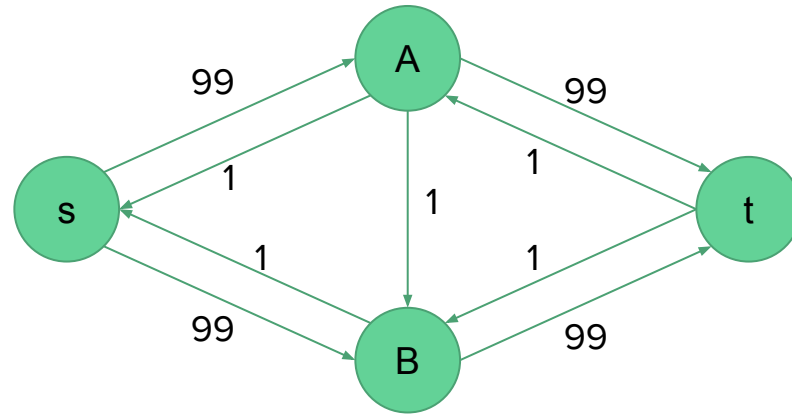
Analysis of Ford-Fulkerson

Augmenting path: $s \rightarrow A \rightarrow B \rightarrow t$



Analysis of Ford-Fulkerson

Augmenting path: $s \rightarrow B \rightarrow A \rightarrow t$



Analysis of Ford-Fulkerson

If we find the augmenting path by using a BFS/DFS, the algorithm runs in polynomial time.

Analysis of Ford-Fulkerson

FORD-FULKERSON-METHOD(G, s, t)

- 1 initialize flow f to 0
- 2 **while** there exists an augmenting path p in the residual network G_f
- 3 augment flow f along p
- 4 **return** f

If we perform DFS/BFS to find augmenting paths, it will take $O(V+E)$ time to find an augmenting path.

The while loop of line 2-4 executes for at most $|f|$ times since we may add 1 unit flow in each iteration.

Thus, total time of Ford-Fulkerson algorithm is $O(|f| (V + E))$.

Applications

- Airline scheduling
 - Managing flight crews by reusing them over multiple flights
- Network connectivity
 - Routing as many packets as possible on a given network
 - Finding min number of connections (edges) whose removal disconnects t from s
- Project selection
 - Choosing a feasible subset of projects to maximize revenue
- Bipartite matching etc.
 - Finding an assignment of jobs to applicants in such that as many applicants as possible get jobs.